```
import rabbitpy

def producer():
  connection = rabbitpy.Connection() # Connect to RabbitMQ server
  channel = connection.channel()     # Create new channel on the connection

  exchange = rabbitpy.Exchange(channel, 'exchange') # Create an exchange
  exchange.declare()

  queue1 = rabbitpy.Queue(channel, 'example1') # Create 1st queue
  queue1.declare()

  queue2 = rabbitpy.Queue(channel, 'example2') # Create 2nd queue
  queue2.declare()

  queue1.bind(exchange, 'example-key') # Bind queue1 to a single key
  queue2.bind(exchange, 'example-key') # Bind queue2 to the same key

  message = rabbitpy.Message(channel, 'Test message')
  message.publish(exchange, 'example-key') # Publish the message using the key
  exchange.delete()
```

(a)

```
import rabbitpy

def consumer():
  connection = rabbitpy.Connection()
  channel = connection.channel()

  queue = rabbitpy.Queue(channel, 'example1')

  # While there are messages in the queue, fetch them using Basic.Get
  while len(queue) > 0:
    message = queue.get()
    print('Message Q1: %s' % message.body.decode())
    message.ack()

  queue = rabbitpy.Queue(channel, 'example2')

  while len(queue) > 0:
    message = queue.get()
    print('Message Q2: %s' % message.body.decode())
    message.ack()
```

(b)

Figure 4.30: A simple (a) producer and (b) consumer for RabbitMQ, adapted from [Roy, 2018].

supports fragmentation and assembly of large messages, for which additional notifications are sent.

Of course, an important aspect of AMQP is its support for **persistent messaging**. Achieving persistence is handled through several mechanisms. First, a message can be marked as durable, indicating that the source expects any intermediate node, such as a queue, to be able to recover in the case of a failure. An intermediate node that cannot guarantee such durability will have to reject a message. Second, a source or target node can also indicate its durability: if durable, will it maintain its state, or will it also maintain the (unsettled) state of durable messages? Combining the latter with durable messages effectively establishes reliable message transfer and persistent messaging.

AMQP is truly a messaging protocol in the sense that it does not by itself support, for example, publish-subscribe primitives. It expects that such issues are handled by more advanced, proprietary queue managers, akin to the message brokers discussed in Section 4.3.3.

Finally, there is no reason why a queue manager cannot be connected to another queue manager. In fact, it is quite common to organize queue managers into an overlay network in which messages are routed from producers to their consumers. AMQP does not specify how the overlay network should be constructed and managed, and, indeed, different providers of AMQP-based systems offer different solutions. Of particular importance is specifying how messages should be routed through the network. The bottom line is that administrators will need to do a lot of this specification manually. Only in cases where overlays have regular structures, such as cycles or trees, it becomes easier to provide the necessary routing details.

AMQP in practice

One of the more popular implementations of message brokering is the RabbitMQ server, extensively described by Roy [2018]. The RabbitMQ server is based on the 0.9 version of AMQP, while also fully supporting version 1.0. An important difference between these two versions is the use of **exchanges**. Instead of directly manipulating queues, producers contact an exchange, which, in turn, places messages in one or several queues. In essence, an exchange allows a producer to use a synchronous or asynchronous RPC to a queue manager. An exchange never stores messages; it simply makes sure that a message is placed in an appropriate queue.

To illustrate this concept, consider Figure 4.30(a) showing a simple producer. As mentioned previously, AMQP embeds channels as part of more durable connections, and the first we do is create a means for communicating with the server (we are omitting various details; in this case we establish a default connection with the server). In the following lines, an exchange is created within the realm of the server, along with two queues. In Lines 16

and 17, both queues are bound to the same exchange by a simple key named `example-key`. After creating a message, the producer tells the exchange that it should publish that message to any queue bound by the specified key.

As to be expected, the exchange will place the message in both queues, which is verified by the simple consumer shown in Figure 4.30(b). In this case, the consumer simply fetches messages from either queue and prints their content.

Exchanges and queues have many options, for which we refer the interested reader to Roy [2018]. It is important to note that using systems such as RabbitMQ, but also other messaging brokers, one can generally set up advanced overlay networks for application-level message routing. At the same time, maintenance of those networks can easily become a nightmare.

4.4 Multicast communication

An important topic in communication in distributed systems is the support for sending data to multiple receivers, also known as **multicast communication**. For many years, this topic has belonged to the domain of network protocols, where numerous proposals for network-level and transport-level solutions have been implemented and evaluated [Janic, 2005; Obraczka, 1998]. A major issue in all solutions was setting up the communication paths for information dissemination. In practice, this involved a huge management effort, often requiring human intervention. In addition, as long as there is no convergence of proposals, ISPs have shown to be reluctant to support multicasting [Diot et al., 2000].

With the advent of peer-to-peer technology, and notably structured overlay management, it became easier to set up communication paths. As peer-to-peer solutions are typically deployed at the application layer, various application-level multicasting techniques have been introduced. In this section, we will take a brief look at these techniques.

Multicast communication can also be accomplished in other ways than setting up explicit communication paths. As we also explore in this section, gossip-based information dissemination provides simple (yet often less efficient) ways for multicasting.

4.4.1 Application-level tree-based multicasting

The basic idea in **application-level multicasting** is that nodes are organized into an overlay network, which is then used to disseminate information to its members. An important observation is that network routers are not involved in group membership. As a consequence, the connections between nodes in the overlay network may cross several physical links, and as such, routing messages within the overlay may not be optimal in comparison to what could have been achieved by network-level routing.

A crucial design issue is the construction of the overlay network. In essence, there are two approaches [El-Sayed et al., 2003; Hosseini et al., 2007; Allani et al., 2009]. First, nodes may organize themselves directly into a tree, meaning that there is a unique (overlay) path between every pair of nodes. An alternative approach is that nodes organize into a mesh network in which every node will have multiple neighbors and, in general, there exist multiple paths between every pair of nodes. The main difference between the two is that the latter generally provides higher robustness: if a connection breaks (e.g., because a node fails), there will still be an opportunity to disseminate information without having to immediately reorganize the entire network.

Note 4.13 (Advanced: Constructing a multicast tree in Chord)
To make matters concrete, let us consider a relatively simple scheme for constructing a multicast tree in Chord, which we described in Note 2.6. This scheme was originally proposed for Scribe [Castro et al., 2002b] which is an application-level multicasting scheme built on top of Pastry [Rowstron and Druschel, 2001]. The latter is also a DHT-based peer-to-peer system.

Assume a node wants to start a multicast session. To this end, it simply generates a multicast identifier, say, mid which is just a randomly chosen 160-bit key. It then looks up succ(mid), which is the node responsible for that key, and promotes it to become the root of the multicast tree that will be used for sending data to interested nodes. To join the tree, a node P simply executes the operation lookup(mid) having the effect that a lookup message with the request to join the multicast group mid will be routed from P to succ(mid). The routing algorithm itself will be explained in detail in Chapter 6.

On its way toward the root, the join request will pass several nodes. Assume it first reaches node Q. If Q had never seen a join request for mid before, it will become a **forwarder** for that group. At that point, P will become a child of Q whereas the latter will continue to forward the join request to the root. If the next node on the root, say, R is also not yet a forwarder, it will become one and record Q as its child and continue to send the join request.

On the other hand, if Q (or R) is already a forwarder for mid, it will also record the previous sender as its child (i.e., P or Q, respectively), but there will not be a need to send the join request to the root anymore, as Q (or R) will already be a member of the multicast tree.

Nodes such as P that have explicitly requested to join the multicast tree are, by definition, also forwarders. The result of this scheme is that we construct a multicast tree across the overlay network with two types of nodes: pure forwarders that act as helpers, and nodes that are also forwarders, but have explicitly requested to join the tree. Multicasting is now simple: a node merely sends a multicast message toward the root of the tree by again executing the lookup(mid) operation, after which that message can be sent along the tree.

We note that this high-level description of multicasting in Scribe does not do complete justice to its original design. The interested reader is encouraged to take a look at the details, which can be found in [Castro et al., 2002b].

Performance issues in overlays

From the high-level description given above, it should be clear that although building a tree by itself is not that difficult once we have organized the nodes into an overlay, building an efficient tree may be a different story. Note that in our description so far, the selection of nodes that participate in the tree does not take into account any performance metrics: it is purely based on the (logical) routing of messages through the overlay.

To understand the problem at hand, take a look at Figure 4.31 which shows a small set of five nodes that are organized in a simple overlay network, with the node A forming the root of a multicast tree. The costs for traversing a physical link are also shown. Now, whenever A multicasts a message to the other nodes, the *logical* route, i.e., at the level of the overlay, is simply $A \rightarrow B \rightarrow E \rightarrow D \rightarrow C$. The actual route crosses the network-level links in the following order: $A \rightarrow Ra \rightarrow Rb \rightarrow B \rightarrow Rb \rightarrow Ra \rightarrow Re \rightarrow E \rightarrow Re \rightarrow Rc \rightarrow Rd \rightarrow D \rightarrow Rd \rightarrow Rc \rightarrow C$. It is thus seen that this message will traverse each of the links $\langle B, Rb\rangle$, $\langle Ra, Rb\rangle$, $\langle E, Re\rangle$, $\langle Rc, Rd\rangle$, and $\langle D, Rd\rangle$ twice. The overlay network would have been more efficient if we had not constructed *overlay links* $\langle B, E\rangle$, and $\langle D, E\rangle$, but instead $\langle A, E\rangle$ and $\langle C, E\rangle$. Such a configuration would have saved the double traversal across physical links $\langle Ra, Rb\rangle$ and $\langle Rc, Rd\rangle$.

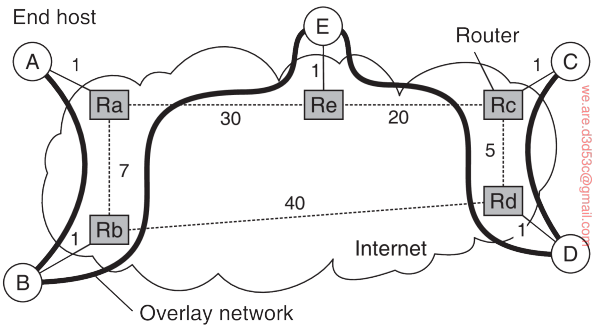


Figure 4.31: The relation between links in an overlay and network-level routes.

The quality of an application-level multicast tree is generally measured by three different metrics: link stress, stretch, and tree cost. **Link stress** is defined per link and counts how often a packet crosses the same link [Chu et al., 2002]. A link stress greater than 1 comes from the fact that although at a logical level a packet may be forwarded along two different connections, part of those connections may actually correspond to the same physical link, as we showed in Figure 4.31.

The **stretch** or **relative delay penalty** (**RDP**) measures the ratio of the delay between two nodes in the overlay, and the delay that those two nodes would experience in the underlying network. For example, messages from

B to C follow the route B → Rb → Ra → Re → E → Re → Rc → Rd → D → Rd → Rc → C in the overlay network, having a total cost of 73 units. However, messages would have been routed in the underlying network along the path B → Rb → Rd → Rc → C, with a total cost of 47 units, leading to a stretch of 1.55. Obviously, when constructing an overlay network, the goal is to minimize the aggregated stretch, or similarly, the average RDP measured over all node pairs.

Finally, the **tree cost** is a global metric, generally related to minimizing the aggregated link costs. For example, if the cost of a link is taken to be the delay between its two end nodes, then optimizing the tree cost boils down to finding a minimal spanning tree in which the total time for disseminating information to all nodes is minimal.

To simplify matters somewhat, assume that a multicast group has an associated and well-known node that keeps track of the nodes that have joined the tree. When a new node issues a join request, it contacts this **rendezvous node** to obtain a (potentially partial) list of members. The goal is to select the best member that can operate as the new node's parent in the tree. Who should it select? There are many alternatives, and different proposals often follow very different solutions.

Consider, for example, a multicast group with only a single source. In this case, the selection of the best node is obvious: it should be the source (because in that case, we can be assured that the stretch will be equal to 1). However, in doing so, we would introduce a star topology with the source in the middle. Although simple, it is not difficult to imagine the source may easily become overloaded. In other words, selection of a node will generally be constrained in such a way that only those nodes may be chosen who have k or fewer neighbors, with k being a design parameter. This constraint severely complicates the tree-establishment algorithm, as a suitable solution may require that part of the existing tree is reconfigured. Tan et al. [2003] provide an extensive overview and evaluation of various solutions to this problem.

Note 4.14 (Advanced: Switch-trees)
As an illustration, let us take a closer look at one specific family, known as **switch-trees** [Helder and Jamin, 2002]. The basic idea is simple. Assume we already have a multicast tree with a single source as root. In this tree, a node P can switch parents by dropping the link to its current parent in favor of a link to another node. The only constraints imposed on switching links is that the new parent can never be a member of the subtree rooted at P (as this would partition the tree and create a loop), and that the new parent will not have too many immediate children. This last requirement is needed to limit the load of forwarding messages by any single node.

There are different criteria for deciding to switch parents. A simple one is to

optimize the route to the source, effectively minimizing the delay when a message is to be multicasted. To this end, each node regularly receives information on other nodes (we will explain one specific way of doing this below). At that point, the node can evaluate whether another node would be a better parent in terms of delay along the route to the source, and if so, initiates a switch.

Another criterion could be whether the delay to the potential other parent is lower than to the current parent. If every node takes this as a criterion, then the aggregated delays of the resulting tree should ideally be minimal. In other words, this is an example of optimizing the cost of the tree, as we explained above. However, more information would be needed to construct such a tree, but as it turns out, this simple scheme is a reasonable heuristic leading to a good approximation of a minimal spanning tree.

As an example, consider the case where a node P receives information on the neighbors of its parent. Note that the neighbors consist of P's grandparent, along with the other siblings of P's parent. Node P can then evaluate the delays to each of these nodes and subsequently choose the one with the lowest delay, say Q, as its new parent. To that end, it sends a switch request to Q. To prevent loops from being formed due to concurrent switching requests, a node that has an outstanding switch request will simply refuse to process any incoming requests. In effect, this leads to a situation where only completely independent switches can be carried out simultaneously. Furthermore, P will provide Q with enough information to allow the latter to conclude that both nodes have the same parent, or that Q is the grandparent.

An important problem that we have not yet addressed is node failure. In the case of switch-trees, a simple solution is proposed: whenever a node notices that its parent has failed, it attaches itself to the root. At that point, the optimization protocol can proceed as usual and will eventually place the node at a good point in the multicast tree. Experiments described in [Helder and Jamin, 2002] show that the resulting tree is indeed close to a minimal spanning one.

4.4.2 Flooding-based multicasting

So far, we have assumed that when a message is to be multicasted, it is to be received by every node in the overlay network. Strictly speaking, this corresponds to **broadcasting**. In general, multicasting refers to sending a message to a subset of all the nodes, that is, a specific *group of nodes*. A key design issue when it comes to multicasting is to minimize the use of intermediate nodes for which the message is not intended. To make this clear, if the overlay is organized as a multi-level tree, yet only the leaf nodes are the ones who should receive a multicast message, then clearly there may be quite some nodes who need to store and subsequently forward a message that is not meant for them.

One simple way to avoid such inefficiency, is to construct an overlay network *per multicast group*. As a consequence, multicasting a message m to a

group G is the same as broadcasting m to G. The drawback of this solution is that a node belonging to several groups, will, in principle, need to maintain a separate list of its neighbors for each group of which it is a member.

If we assume that an overlay corresponds to a multicast group, and thus that we need to broadcast a message, a naive way of doing so is to apply **flooding**. In this case, each node simply forwards a message m to each of its neighbors, except to the one from which it received m. Furthermore, if a node keeps track of the messages it received and forwarded, it can simply ignore duplicates.

To understand the performance of flooding, we model an overlay network as a connected undirected graph $G = (V, E)$ with nodes V and links E. Let v_0 be the node that initiates flooding. In principle, every node other node than v_0 will send out the message to its neighbors once, except to the neighbor from which it had received the message. We will thus see a total of $\delta(v_0) + \sum_{v \in V - v_0}(\delta(v) - 1)$ messages, with $\delta(v)$ the number of neighbors of the node v. For any undirected graph, $\sum_{v \in V} \delta(v) = 2 \cdot |E|$, with $|E|$ denoting the total number of edges. In other words, we will see $2|E| - |V| + 1$ messages being sent, making flooding quite inefficient. Only if G is a tree, will flooding be optimal, for in that case, $|E| = |V| - 1$. In the worst case, when G is fully connected, we have $|E| = \binom{|V|}{2}$ leading to an order of $|V|^2$ messages.

Suppose now that we have no information on the structure of the overlay network and that the best we can assume is that it can be represented as a **random graph**, which (to keep it simple) is a graph having a probability p_{edge} that two vertices are joined by an edge, also known as an **Erdös-Rényi graph** [Erdös and Rényi, 1959]. Note that we are actually considering our overlay network to be an unstructured peer-to-peer network, and that we do not have any information on how it is being constructed. With a probability p_{edge} that two nodes are joined, and a total of $\binom{|V|}{2}$ edges, it is not difficult to see that we can expect our overlay to have $|E| = \frac{1}{2} \cdot p_{edge} \cdot |V| \cdot (|V| - 1)$ edges. To give an impression of what we are dealing with, Figure 4.32 shows the relationship between the number of nodes and edges for different values of p_{edge}. As can be seen, the number of edges can easily become large, even for small values of p_{edge}.

To reduce the number of messages, we can also use **probabilistic flooding** as introduced by Banaei-Kashani and Shahab [2003] and formally analyzed by Oikonomou and Stavrakakis [2007]. The idea is simple: when a node is flooding a message m and needs to forward m to a specific neighbor, it will do so with a probability p_{flood}. The effect can be dramatic: the total number of messages sent will drop linearly in p_{flood}. However, there is also a risk: the lower p_{flood}, the higher the chance that not all nodes in the network will be reached. This risk is caused by the simple fact that all neighbors of a specific node Q may have decided *not* to forward m to Q. If Q has n neighbors, then this can happen roughly with a probability of $(1 - p_{flood})^n$. Clearly, the

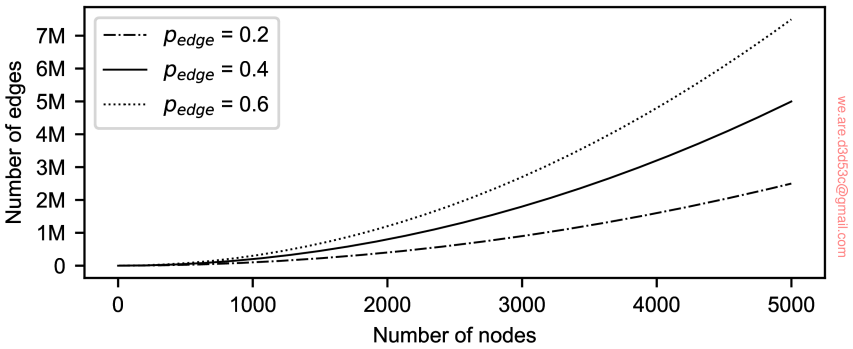


Figure 4.32: The size of a random overlay as function of the number of nodes.

number of neighbors plays an important role in deciding whether to forward a message, and, indeed, we can replace the static probability of forwarding with one that takes the degree of the neighbor into account. This has been further developed and analyzed by Sereno and Gaeta [2011]. To give an idea of the efficiency of probabilistic broadcasting: in a random network of 10,000 nodes and $p_{edge} = 0.1$, we need only set $p_{flood} = 0.01$ to establish a more than 50-fold reduction in the number of messages sent in comparison to full flooding.

When dealing with a structured overlay, that is, one having a more or less deterministic topology, designing efficient flooding schemes is simpler. As an example, consider an n-dimensional hypercube, shown in Figure 4.33 for the case $n = 4$, as also discussed in Section 2.4.1.

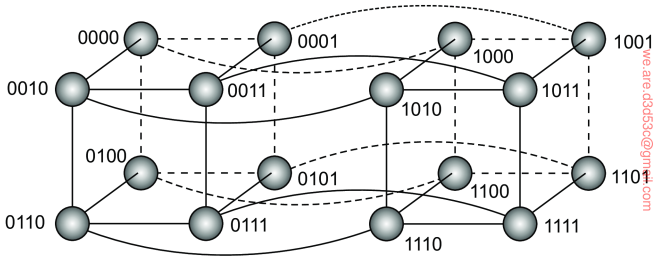


Figure 4.33: A simple peer-to-peer, four-dimensional hypercube.

A simple and efficient broadcast scheme has been designed by Schlosser et al. [2002] and relies on keeping track of neighbors *per dimension*. This is best explained by considering that every node in an n-dimensional hypercube is represented by a bit string of length n. Each edge in the overlay is labeled with its dimension. For the case $n = 4$, node 0000 will have as its neighbors the set $\{0001, 0010, 0100, 1000\}$. The edge between 0000 and 0001 is labeled

"4" corresponding to changing the 4th bit when comparing 0000 to 0001 and *vice versa*. Likewise, the edge $\langle 0000, 0100 \rangle$ is labeled "2," and so forth. A node initially broadcasts a message m to all of its neighbors, and tags m with the label of the edge over which it sends the message. In our example, if node 1001 broadcasts a message, it will send the following:

- (m,1) to 0001
- (m,2) to 1101
- (m,3) to 1011
- (m,4) to 1000

When a node receives a broadcast message, it will forward it only along edges that have a higher dimension. In other words, in our example, node 1101 will forward m only to nodes 1111 (joined to 1101 by an edge labeled "3") and 1100 (joined by an edge with label "4"). Using this scheme, it can be shown that every broadcast requires precisely $N - 1$ messages, where $N = 2^n$, that is the number of nodes in a n-dimensional hypercube. This broadcasting scheme is therefore optimal in terms of the number of messages sent.

Note 4.15 (Advanced: Ring-based flooding)

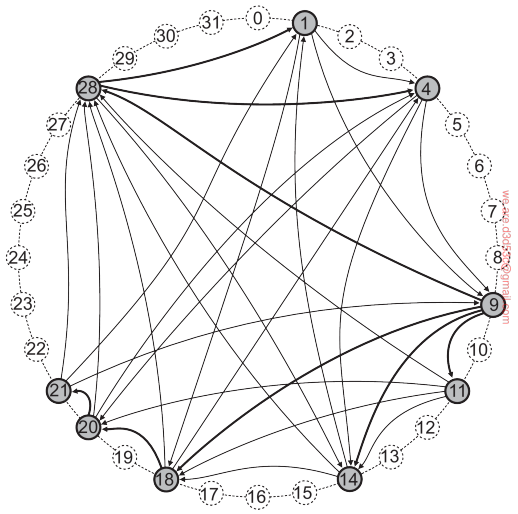


Figure 4.34: A Chord ring in which node 9 broadcasts a message.

A hypercube is a straightforward example of how we can effectively use knowledge of the structure of an overlay network to establish efficient flooding. In the case of Chord, we can follow an approach proposed by Ghodsi [2010]. Recall that in Chord each node is identified by a number p, and each resource (typically

a file), is assigned a key k from the same space as used for node identifiers. The successor succ(k) of a key k is the node with the smallest identifier $p \geq k$. Consider the small Chord ring shown in Figure 4.34 and assume that node 9 wants to flood a message to all other nodes.

In our example, node 9 divides the identifier space into four segments (one for each of its neighbors). Node 28 is requested to make sure that the message reaches all nodes with identifiers $28 \leq k < 9$ (recall that we are applying modulo arithmetic); node 18 takes care of nodes with identifiers $18 \leq k < 28$; node 14 for $14 \leq k < 18$; and 11 for identifiers $11 \leq k < 14$.

Node 28 will subsequently divide the part of the identifier space it is requested to handle into two subsegments: one for its neighboring node 1 and another for 4. Likewise, node 18, responsible for segment $[18, 28)$ will "split" that segment into only one part: it has only one neighbor to delegate the flood to, and forwards the message to node 20, telling it that it should handle segment $[20, 28)$.

In the last step, only node 20 has work to do. It forwards the message to node 21, telling it to forward it to nodes known to it in the segment $[21, 28)$. As there are no such nodes anymore, the broadcast completes.

As in the case of our hypercube example, we see that flooding is done with $N - 1$ messages, with N being the number of nodes in the system.

4.4.3 Gossip-based data dissemination

An important technique for disseminating information is to rely on **epidemic behavior**, also referred to as **gossiping**. Observing how diseases spread among people, researchers have since long investigated whether simple techniques could be developed for spreading information in very large-scale distributed systems. The main goal of these **epidemic protocols** is to rapidly propagate information among a large collection of nodes using only local information. In other words, there is no central component by which information dissemination is coordinated.

To explain the general principles of these algorithms, we assume that all updates for a specific data item are initiated at a single node. In this way, we simply avoid write-write conflicts. The following presentation is based on the classical paper by Demers et al. [1987] on epidemic algorithms. An overview of epidemic information dissemination can be found in [Eugster et al., 2004].

Information dissemination models

As the name suggests, epidemic algorithms are based on the theory of epidemics, which studies the spreading of infectious diseases. In the case of large-scale distributed systems, instead of spreading diseases, they spread information. Research on epidemics for distributed systems also aims at an entirely different goal: whereas health organizations will do their best to prevent infectious diseases from spreading across large groups of people,

designers of epidemic algorithms for distributed systems will try to "infect" all nodes with new information as fast as possible.

Using the terminology from epidemics, a node that is part of a distributed system is called **infected** if it holds data that it is willing to spread to other nodes. A node that has not yet seen this data is called **susceptible**. Finally, an updated node that is not willing or able to spread its data is said to have been **removed**. Note that we assume we can distinguish old from new data, for example because it has been timestamped or versioned. In this light, nodes are also said to spread updates.

A popular propagation model is that of **anti-entropy**. In this model, a node P picks another node Q at random, and subsequently exchanges updates with Q. There are three approaches to exchanging updates:

1. P only pulls in new updates from Q
2. P only pushes its own updates to Q
3. P and Q send updates to each other (i.e., a push-pull approach)

When it comes to rapidly spreading updates, only pushing updates turns out to be a bad choice. Intuitively, this can be understood as follows. First, note that in a pure push-based approach, updates can be propagated only by infected nodes. However, if many nodes are infected, the probability of each one selecting a susceptible node is relatively small. Consequently, chances are that a particular node remains susceptible for a long period simply because an infected node does not select it.

In contrast, the pull-based approach works much better when many nodes are infected. In that case, spreading updates is essentially triggered by susceptible nodes. Chances are big that such a node will contact an infected one to subsequently pull in the updates and become infected as well.

If only a single node is infected, updates will rapidly spread across all nodes using either form of anti-entropy, although push-pull remains the best strategy [Jelasity et al., 2007]. Define a **round** as spanning a period in which every node will have taken the initiative once to exchange updates with a randomly chosen other node. It can then be shown that the number of rounds to propagate a single update to all nodes is of the order $\mathcal{O}(log(N))$, where N is the number of nodes in the system. This indicates indeed that propagating updates is fast, but above all scalable.

Note 4.16 (Advanced: An analysis of anti-entropy)
A simple and straightforward analysis will give some idea on how well anti-entropy works. Consider a system with N nodes. One of these nodes initiates the spreading of a message m to all other nodes. Let p_i denote the probability that a node P has not yet received m after the ith round. We distinguish the following three cases:

- With a pure pull-based approach, $p_{i+1} = (p_i)^2$: not only had P not yet been updated in the previous round, also P contacts a node that had not yet received m.
- With a pure push-based approach, $p_{i+1} = p_i \cdot (1 - \frac{1}{N-1})^{N(1-p_i)}$: again, P should not have been updated in the previous round, but also none of the updated nodes should contact P. The probability that not a single node contacts P is $1 - \frac{1}{N-1}$; we can expect that there are $N(1 - p_i)$ updated nodes in round i.
- In a push-pull approach, we can simply combine the two: P should not contact an updated node, and should not be contacted by one.

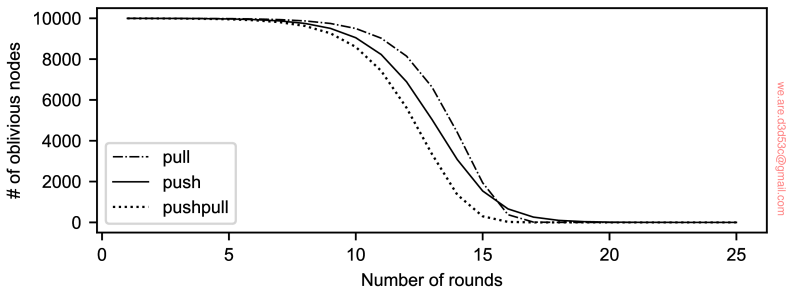


Figure 4.35: (a) The number of nodes that have not been updated as a function of the number of dissemination rounds.

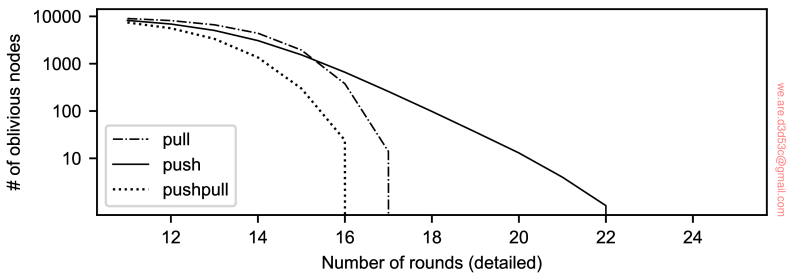


Figure 4.35: (b) Zooming in the differences between the three anti-entropy approaches when almost all nodes have been updated.

Figure 4.35(a) shows how quickly the probability of not yet being updated drops as a function of the number of rounds in a network of 10,000 nodes. After a slow start, the information is rapidly disseminated. In Figure 4.35(b) we have zoomed into what happens in the last number of rounds while also using a log scale for the y-axis. Indeed, assuming that nodes are up-and-running all the time,

it turns out that an anti-entropy push-pull strategy can be an extremely effective dissemination protocol.

One specific variant of epidemic protocols is called **rumor spreading**. It works as follows. If node P has just been updated for a data item x, it contacts an arbitrary other node Q and tries to push the update to Q. However, it is possible that Q was already updated by another node. In that case, P may lose interest in spreading the update any further, say with probability p_{stop}. In other words, it then becomes removed.

Rumor spreading is gossiping as we mostly experience it in real life. When Bob has some hot news to spread around, he may phone his friend Alice, telling her all about it. Alice, like Bob, will be really excited to spread the rumor to her friends as well. However, she will become disappointed when phoning a friend, say Chuck, only to hear that the news has already reached him. Chances are that she will stop phoning other friends, for what good is it if they already know?

Rumor spreading turns out to be an excellent way of rapidly spreading news. However, it cannot guarantee that all nodes will actually be updated [Demers et al., 1987]. In fact, when there are many nodes that participate in the epidemics, the fraction s of nodes that will remain ignorant of an update, that is, remain susceptible, satisfies the equation:

$$s = e^{-(1/p_{stop}+1)(1-s)}$$

To get an idea of what this means, take a look at Figure 4.36, which shows s as a function of p_{stop}. Even for high values of p_{stop} we see that the fraction of nodes that remains ignorant is relatively low, and always less than approximately 0.2. For $p_{stop} = 0.20$ it can be shown that $s = 0.0025$. However, in those cases when p_{stop} is relatively high, additional measures will need to be taken to ensure that *all* nodes are updated.

Note 4.17 (Advanced: Analysis of rumor spreading)
To formally analyze the situation for rumor spreading, we let s denote the fraction of nodes that have not yet been updated, i.e., the fraction of susceptible nodes. Likewise, let i denote the fraction of infected nodes: the ones that have been updated and are still contacting other nodes to spread information. Finally, r is the fraction of nodes that have been updated, but have given up, i.e., they are passive and no longer play a role in disseminating information. Obviously, $s + i + r = 1$. Using theory from epidemics, it is not difficult to see the following:

$$\begin{array}{lllll} (1) & ds/dt & = & -s \cdot i \\ (2) & di/dt & = & s \cdot i - p_{stop} \cdot (1-s) \cdot i \\ \Rightarrow & di/ds & = & -(1+p_{stop}) + \frac{p_{stop}}{s} \\ \Rightarrow & i(s) & = & -(1+p_{stop}) \cdot s + p_{stop} \cdot \ln(s) + C \end{array}$$

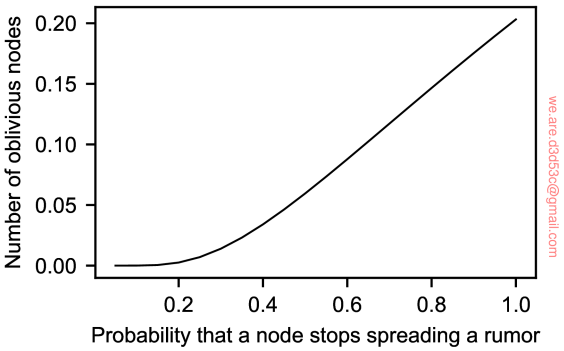


Figure 4.36: The relation between the fraction s of update-ignorant nodes and the probability p_{stop} that a node will stop gossiping once it contacts a node that has already been updated.

where we use the notation $i(s)$ to express i as a function of s. When $s = 1$, no nodes have yet been infected, meaning that $i(1) = 0$. This allows us to derive that $C = 1 + p_{stop}$, and thus

$$i(s) = (1 + p_{stop}) \cdot (1 - s) + p_{stop} \cdot \ln(s)$$

We are looking for the situation that there is no more rumor spreading, i.e., when $i(s) = 0$. Having a closed expression for $i(s)$ then leads to

$$s = e^{-(1/p_{stop}+1)(1-s)}$$

One of the main advantages of epidemic algorithms is their scalability since the number of synchronizations between processes is relatively small compared to other propagation methods. For wide-area systems, Lin and Marzullo [1999] have shown that it makes sense to take the actual network topology into account to achieve better results. In that case, nodes that are connected to only a few other nodes are contacted with a relatively high probability. The underlying assumption is that such nodes form a bridge to other remote parts of the network; therefore, they should be contacted as soon as possible. This approach is referred to as **directional gossiping** and comes in different variants.

This problem touches upon an important assumption that most epidemic solutions make, namely that a node can randomly select any other node to gossip with. This implies that, in principle, the complete set of nodes should be known to each member. In a large system, this assumption can never hold, and special measures will need to be taken to mimic such properties. We return to this issue in Section 5.5.2 when we discuss a **peer-sampling service**.

Removing data

Epidemic algorithms are fantastic for spreading updates. However, they have a rather strange side effect: spreading the *deletion* of a data item is hard. The essence of the problem lies in the fact that deletion of a data item destroys all information on that item. Consequently, when a data item is simply removed from a node, that node will eventually receive old copies of the data item and interpret those as updates on something it did not have before.

The trick is to record the deletion of a data item as just another update, and keep a record of that deletion. In this way, old copies will not be interpreted as something new, but merely treated as versions that have been updated by a delete operation. The recording of a deletion is done by spreading **death certificates**.

Of course, the problem with death certificates is that they should eventually be cleaned up, or otherwise each node will gradually build a huge local database of historical information on deleted data items that is otherwise not used. Demers et al. [1987] propose to use what are called **dormant death certificates**. Each death certificate is timestamped when it is created. If it can be assumed that updates propagate to all nodes within a known finite time, then death certificates can be removed after this maximum propagation time has elapsed.

However, to provide hard guarantees that deletions are indeed spread to all nodes, only a very few nodes maintain dormant death certificates that are never thrown away. Assume node P has such a certificate for a data item x. If by any chance an obsolete update for x reaches P, P will react by simply spreading the death certificate for x again.

4.5 Summary

Having powerful and flexible facilities for communication between processes is essential for any distributed system. In traditional network applications, communication is often based on the low-level message-passing primitives offered by the transport layer. An important issue in middleware systems is to offer a higher level of abstraction that will make it easier to express communication between processes than the support offered by the interface to the transport layer.

One of the most widely used abstractions is the Remote Procedure Call (RPC). The essence of an RPC is that a service is implemented through a procedure, of which the body is executed at a server. The client is offered only the signature of the procedure, that is, the procedure's name along with its parameters. When the client calls the procedure, the client-side implementation, called a stub, takes care of wrapping the parameter values into a message and sending that to the server. The latter calls the actual procedure and returns the results, again in a message. The client's stub

extracts the result values from the return message and passes it back to the calling client application.

RPCs offer synchronous communication facilities, by which a client is blocked until the server has sent a reply. Although variations of either mechanism exist by which this strict synchronous model is relaxed, it turns out that general-purpose, high-level message-oriented models are often more convenient.

In message-oriented models, the issues are whether communication is persistent, and whether communication is synchronous. The essence of persistent communication is that a message that is submitted for transmission, is stored by the communication system as long as it takes to deliver it. In other words, neither the sender nor the receiver needs to be up and running for message transmission to take place. In transient communication, no storage facilities are offered so that the receiver must be prepared to accept the message when it is sent.

In asynchronous communication, the sender is allowed to continue immediately after the message has been submitted for transmission, possibly before it has even been sent. In synchronous communication, the sender is blocked at least until a message has been received. Alternatively, the sender may be blocked until message delivery has taken place or even until the receiver has responded, as with RPCs.

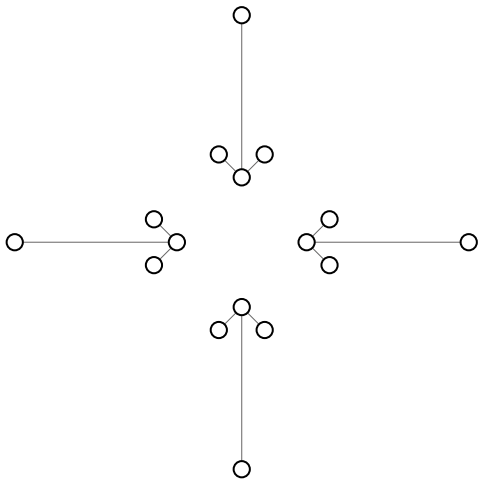
Message-oriented middleware models generally offer persistent asynchronous communication, and are used where RPCs are not appropriate. They are often used to assist the integration of (widely dispersed) collections of databases into large-scale information systems.

Finally, an important class of communication protocols in distributed systems is multicasting. The basic idea is to disseminate information from one sender to multiple receivers. We have discussed two different approaches. First, multicasting can be achieved by setting up a tree from the sender to the receivers. Considering that it is now well understood how nodes can self-organize into peer-to-peer system, solutions have also appeared to dynamically set up trees in a decentralized fashion. Second, flooding messages across the network is extremely robust, yet requires special attention if we want to avoid severe waste of resources as nodes may see messages multiple times. Probabilistic flooding, by which a node forwards a message with a certain probability, often proves to combine simplicity and efficiency, while being highly effective.

Another important class of dissemination solutions deploys epidemic protocols. These protocols have proven to be simple and extremely robust.

05

COORDINATION



In the previous chapters, we have looked at processes and communication between processes. While communication is important, it is not the entire story. Closely related is how processes cooperate and synchronize with one another. Cooperation is partly supported by naming, which allows processes to at least share resources, or entities in general.

In this chapter, we mainly concentrate on how processes can synchronize and coordinate their actions. For example, it is important that multiple processes do not simultaneously access a shared resource, such as a file, but instead cooperate in granting each other temporary exclusive access. Another example is that multiple processes may sometimes need to agree on the ordering of events, such as whether message m_1 from process P was sent before or after message m_2 from process Q.

Synchronization and coordination are two closely related phenomena. In **process synchronization** we make sure that one process waits for another to complete its operation. When dealing with **data synchronization**, the problem is to ensure that two sets of data are the same. When it comes to **coordination**, the goal is to manage the interactions and dependencies between activities in a distributed system [Malone and Crowston, 1994]. From this perspective, one could state that coordination encapsulates synchronization.

As it turns out, coordination in distributed systems is often much more difficult compared to that in uniprocessor or multiprocessor systems. The problems and solutions that are discussed in this chapter are, by their nature, rather general, and occur in many situations in distributed systems.

We start with a discussion of the issue of synchronization based on actual time, followed by synchronization in which only relative ordering matters rather than ordering in absolute time.

Often, it is important that a group of processes can appoint one process as a coordinator, which can be done through election algorithms. We discuss various election algorithms in a separate section. Before that, we look into several algorithms for coordinating mutual exclusion to a shared resource. As a special class of coordination problems, we also dive into location systems, by which we place a process in a multidimensional plane. Such placements are useful when dealing with very large distributed systems.

We also consider three different gossip-based coordination problems: aggregation, peer sampling, and overlay construction.

Finally, we already came across publish-subscribe systems, but have not yet discussed in any detail how we actually match subscriptions to publications. There are many ways to do this, and we look at centralized as well as decentralized implementations.

Distributed algorithms come in all sorts and flavors and have been developed for different types of distributed systems. Many examples (and further references) can be found in Andrews [2000], Cachin et al. [2011], and Fokkink [2018]. More formal approaches to a wealth of algorithms can be found in

textbooks from Attiya and Welch [2004], Lynch [1996], Santoro [2007], and Tel [2000].

5.1 Clock synchronization

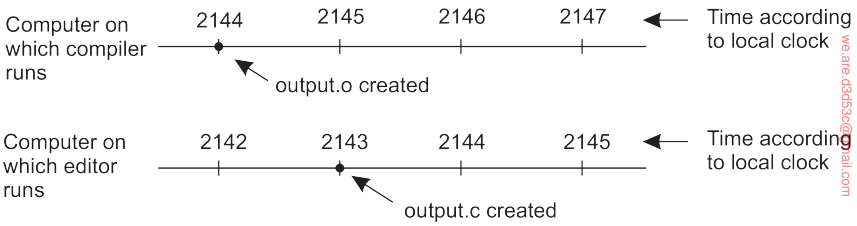
In a centralized system, time is unambiguous. When a process wants to know the time, it simply makes a call to the operating system. If process A asks for the time, and then a little later process B asks for the time, the value that B gets will be higher than (or possibly equal to) the value A got. It will certainly not be lower. In a distributed system, achieving agreement on time is not trivial.

Just think, for a moment, about the implications of the lack of global time on the Unix `make` program, as a simple example. Normally, in Unix, large programs are split up into multiple source files, so that a change to one source file requires only one file to be recompiled, not all the files. If a program consists of 100 files, not having to recompile everything because one file has been changed greatly increases the speed at which programmers can work.

The way `make` normally works is simple. When the programmer has finished changing all the source files, she runs `make`, which examines the times at which all the source and object files were last modified. If the source file `input.c` has time 2151 and the corresponding object file `input.o` has time 2150, `make` knows that `input.c` has been changed since `input.o` was created, and thus `input.c` must be recompiled. On the other hand, if `output.c` has time 2144 and `output.o` has time 2145, no compilation is needed. Thus `make` goes through all the source files to find out which ones need to be recompiled and calls the compiler to recompile them.

Now imagine what could happen in a distributed system in which there was no global agreement on time. Suppose that `output.o` has time 2144 as above, and shortly thereafter `output.c` is modified but is assigned time 2143 because the clock on its machine is slightly behind, as shown in Figure 5.1. `Make` will not call the compiler. The resulting executable binary program will then contain a mixture of object files from the old sources and the new sources. It may crash, and the programmer will go crazy trying to understand what is wrong with the code.

There are many more examples where an accurate account of time is needed. The example above can easily be reformulated to file timestamps in general. In addition, think of application domains such as financial brokerage, security auditing, and collaborative sensing, and it will become clear that accurate timing is important. From a different perspective, Najafi et al. [2021] argue that accurate timing is essential for most systems research, if only to ensure that performance measurements can be properly compared. Since time is so basic to the way people think and the effect of not having all the clocks synchronized can be so dramatic, it is fitting that we begin our study of



we are.d3d53c@gmail.com

Figure 5.1: When each machine has its own clock, an event that occurred after another event may nevertheless be assigned an earlier time.

synchronization with the simple question: Would it be possible to synchronize all the clocks in a distributed system? The answer is surprisingly complicated.

5.1.1 Physical clocks

Nearly all computers often have several circuits for keeping track of time. Despite the widespread use of the word "clock" to refer to these devices, they are not actually clocks in the usual sense. **Timer** is perhaps a better word. A computer timer is usually a precisely machined quartz crystal. When kept under tension, quartz crystals oscillate at a well-defined frequency that depends on the kind of crystal, how it is cut, and the amount of tension. Associated with each crystal are two registers, a **counter** and a **holding register**. Each oscillation of the crystal decrements the counter by one. When the counter gets to zero, an interrupt is generated and the counter is reloaded from the holding register. In this way, it is possible to program a timer to generate an interrupt 60 times a second, or at any other desired frequency. Each interrupt is called one **clock tick**.

When the system is booted and initialized for the very first time, the user is asked for the current time zone, or even time, to convert time to the number of ticks after some known starting date and stored in memory. Most computers have a special battery-backed up CMOS RAM so that the date and time need not be entered on subsequent boots. At every clock tick, the interrupt service procedure adds one to the time stored in memory. In this way, the (software) clock is kept up to date.

With a single computer and a single clock, it does not matter much if this clock is off by a small amount. Since all processes on the machine use the same clock, they will still be internally consistent. For example, if the file `input.c` has time 2151 and file `input.o` has time 2150, `make` will recompile the source file, even if the clock is off by 2 and the true times are 2153 and 2152, respectively. All that really matters are the relative times.

As soon as multiple CPUs are introduced, each with its own clock, the situation changes radically. Although the frequency at which a crystal oscillator runs is usually fairly stable, it is impossible to guarantee that the crystals in

different computers all run at the same frequency. In practice, when a system has n computers, all n crystals will run at slightly different rates, causing the (software) clocks gradually to get out of sync and give different values when read out. This difference in time values is called **clock skew**. As a consequence of this clock skew, programs that expect the time associated with a file, object, process, or message to be correct and independent of the machine on which it was generated (i.e., which clock it used) can fail, as we saw in the `make` example above.

In some systems (e.g., real-time systems), the actual clock time is important. Under these circumstances, external physical clocks are needed. For reasons of efficiency and redundancy, multiple physical clocks are generally considered desirable, which yields two problems: (1) how do we synchronize them with real-world clocks, and (2) how do we synchronize the clocks with each other?

Note 5.1 (More information: Determining real time)
Before answering these questions, let us digress slightly to see how time is actually measured. It is not nearly as easy as one might think, especially when high accuracy is required. Since the invention of mechanical clocks in the 17th century, time has been measured astronomically. Every day, the sun appears to rise on the eastern horizon, then climbs to a maximum height in the sky, and finally sinks in the west. The event of the sun's reaching its highest apparent point in the sky is called the **transit of the sun**. This event occurs at about noon each day. The interval between two consecutive transits of the sun is called the **solar day**. Since there are 24 hours in a day, each containing 3600 seconds, the **solar second** is defined as exactly $1/86400^{th}$ of a solar day. The geometry of the mean solar day calculation is shown in Figure 5.2.

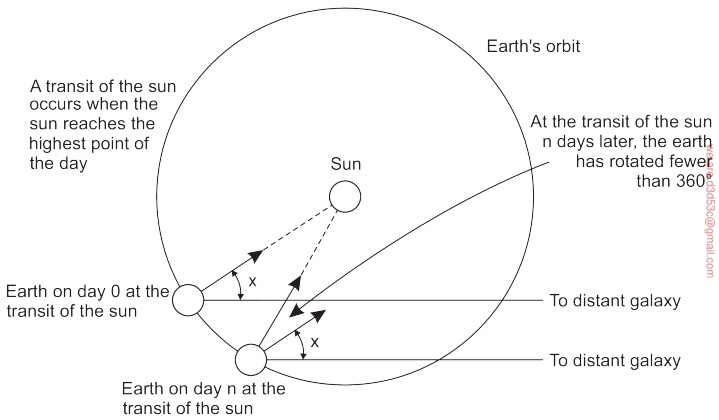


Figure 5.2: Computation of the mean solar day.

In the 1940s, it was established that the period of the earth's rotation is not constant. The earth is slowing down due to tidal friction and atmospheric drag. Based on studies of growth patterns in ancient coral, geologists now believe that 300 million years ago there were about 400 days per year. The length of the year (the time for one trip around the sun) is not thought to have changed; the day has simply become longer. In addition to this long-term trend, short-term variations in the length of the day also occur, probably caused by turbulence deep in the earth's core of molten iron. These revelations lead astronomers to compute the length of the day by measuring a large number of days and taking the average before dividing by 86,400. The resulting quantity was called the **mean solar second**.

With the invention of the atomic clock in 1948, it became possible to measure time much more accurately, and independent of the wiggling and wobbling of the earth, by counting transitions of the cesium 133 atom. The physicists took over the job of timekeeping from the astronomers and defined the second to be the time it takes the cesium 133 atom to make exactly 9,192,631,770 transitions. The choice of 9,192,631,770 was made to make the atomic second equal to the mean solar second in the year of its introduction. Currently, several laboratories around the world have cesium 133 clocks. Periodically, each laboratory tells the Bureau International de l'Heure (BIH) in Paris how many times its clock has ticked. The BIH averages these to produce **International Atomic Time**, which is abbreviated to **TAI**. Thus TAI is just the mean number of ticks of the cesium 133 clocks since midnight on Jan. 1, 1958 (the beginning of time) divided by 9,192,631,770.

Although TAI is highly stable and available to anyone who wants to go to the trouble of buying a cesium clock, there is a serious problem with it; 86,400 TAI seconds is now about 3 msec less than a mean solar day (because the mean solar day is getting longer all the time). Using TAI for keeping time would mean that over the course of the years, noon would get earlier and earlier, until it would eventually occur in the wee hours of the morning. People might notice this and we could have the same kind of situation as occurred in 1582 when Pope Gregory XIII decreed that 10 days be omitted from the calendar. This event caused riots in the streets because landlords demanded a full month's rent and bankers a full month's interest, while employers refused to pay workers for the 10 days they did not work, to mention only a few of the conflicts. The Protestant countries, as a matter of principle, refused to have anything to do with papal decrees and did not accept the Gregorian calendar for 170 years.

BIH solves the problem by introducing **leap seconds** whenever the discrepancy between TAI and solar time grows to 800 msec. The use of leap seconds is illustrated in Figure 5.3. This correction gives rise to a time system based on constant TAI seconds but which stays in phase with the apparent motion of the sun. This time system is known as **Coordinated Universal Time** abbreviated to **UTC**.

Most electric power companies synchronize the timing of their 60-Hz or 50-Hz clocks to UTC, so when BIH announces a leap second, the power companies raise their frequency to 61 Hz or 51 Hz for 60 or 50 sec, to advance all the clocks in their distribution area. Since 1 sec is a noticeable interval for a computer, an operating system that needs to keep accurate time over a period of years must have special

software to account for leap seconds as they are announced (unless they use the power line for time, which is usually too crude). The total number of leap seconds introduced into UTC so far is about 30.

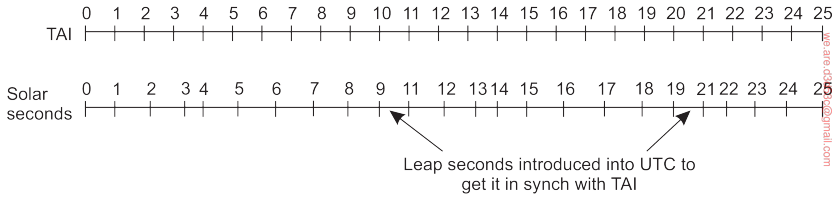


Figure 5.3: TAI seconds are of constant length, unlike solar seconds. Leap seconds are introduced when necessary to keep in phase with the sun.

The basis for keeping global time is a called **Coordinated Universal Time**, but is abbreviated as **UTC**. UTC is the basis of all modern civil timekeeping and is a worldwide standard. To provide UTC to people who need precise time, some 40 shortwave radio stations around the world broadcast a short pulse at the start of each UTC second. The accuracy of these stations is about $\pm$ 1 msec, but due to random atmospheric fluctuations that can affect the length of the signal path, in practice the accuracy is no better than $\pm$ 10 milliseconds.

Several earth satellites also offer a UTC service. The Geostationary Operational Environment Satellite can provide UTC accurately to 0.5 msec, and some other satellites do even better. By combining receptions from several satellites, ground timeservers can be built, offering an accuracy of 50 nanoseconds. UTC receivers are commercially available, and many computers are equipped with one.

5.1.2 Clock synchronization algorithms

If one machine has a UTC receiver, the goal becomes keeping all the other machines synchronized to it. If no machines have UTC receivers, each machine keeps track of its own time, and the goal is to keep all the machines together as well as possible. Many algorithms have been proposed for doing this synchronization. Surveys are provided by Ramanathan et al. [1990], Horauer [2004], Shin et al. [2011], and Levesque and Tipper [2016].

All clocks are based on some harmonic oscillator: an object that resonates at a certain frequency and from which we can subsequently derive time. Atomic clocks are based on the transitions of the cesium 133 atom, which is not only very high, but also very constant. Hardware clocks in most computers use a crystal oscillator based on quartz, which is also capable of producing

a very high, stable frequency, although not as stable as that of atomic clocks. A software clock in a computer is derived from that computer's hardware clock. In particular, the hardware clock is assumed to cause an interrupt f times per second. When this timer goes off, the interrupt handler adds 1 to a counter that keeps track of the number of ticks (interrupts) since some agreed-upon time in the past. This counter acts as a software clock C, resonating at frequency F.

When the UTC time is t, denote by $C_p(t)$ the value of the software clock on machine p. The goal of **clock synchronization algorithms** is to keep the deviation between the respective clocks of any two machines in a distributed system, within a specified bound, known as the **precision** π:

$$\forall t, \forall \mathsf{p}, \mathsf{q} : |C_p(t) - C_q(t)| \leq \pi$$

Note that precision refers to the deviation of clocks only *between machines* that are part of a distributed system. When considering an external reference point, like UTC, we speak of **accuracy**, aiming to keep it bound to a value α:

$$\forall t, \forall \mathsf{p} : |C_p(t) - t| \leq \alpha$$

The whole idea of clock synchronization is that we keep clocks *precise*, referred to as **internal synchronization** or *accurate*, known as **external synchronization**. A set of clocks that are accurate within bound α, will be precise within bound $\pi = 2\alpha$. However, being precise does not allow us to conclude anything about the accuracy of clocks.

In a perfect world, we would have $C_p(t) = t$ for all p and all t, and thus $\alpha = \pi = 0$. Unfortunately, hardware clocks, and thus also software clocks, are subject to **clock drift**: because their frequency is not perfect and affected by external sources such as temperature, clocks on different machines will gradually start showing different values for time. This is known as the **clock drift rate**: the difference per unit of time from a perfect reference clock. A typical quartz-based hardware clock has a clock drift rate of some 10^{-6} seconds per second, or approximately 31.5 seconds per year. Computer hardware clocks exist that have much lower drift rates.

The specifications of a hardware clock include its maximum clock drift rate ρ. If $F(t)$ denotes the actual oscillator frequency of the hardware clock at time t and F its ideal (constant) frequency, then a hardware clock is living up to its specifications if

$$\forall t : (1 - \rho) \leq \frac{F(t)}{F} \leq (1 + \rho)$$

By using hardware interrupts we are directly coupling a software clock to the hardware clock, and thus also its clock drift rate. In particular, we have that

$$C_p(t) = \frac{1}{F} \int_0^t F(t)dt, \text{ and thus: } \frac{dC_p(t)}{dt} = \frac{F(t)}{F}$$

which brings us to our goal, namely keeping the software clock drift rate also bounded to ρ:

$$\forall t : 1 - \rho \leq \frac{dC_p(t)}{dt} \leq 1 + \rho$$

Slow, perfect, and fast clocks are shown in Figure 5.4.

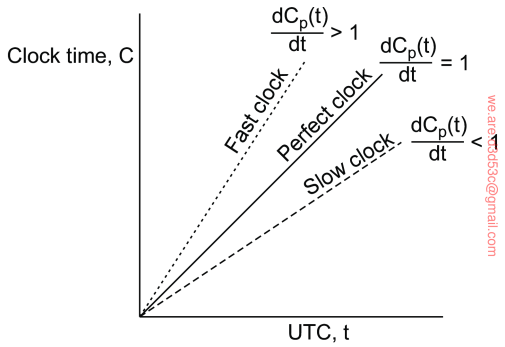


Figure 5.4: The relation between clock time and UTC when clocks tick at different rates.

If two clocks are drifting from UTC in the opposite direction, at a time Δt after they were synchronized, they may be as much as $2\rho \cdot \Delta t$ apart. If the system designers want to guarantee a precision π, that is, that no two clocks ever differ by more than π seconds, clocks must be resynchronized (in software) at least every $\pi/(2\rho)$ seconds. The various algorithms differ in precisely how this resynchronization is done.

Network Time Protocol

A common approach in many protocols, and originally proposed by Cristian [1989], is to let clients contact a timeserver. The latter can accurately provide the current time, for example because it is equipped with a UTC receiver or an accurate clock. The problem, of course, is that when contacting the server, message delays will have outdated the reported time. The trick is to find a good estimation for these delays. Consider the situation sketched in Figure 5.5.

In this case, A will send a request to B, timestamped with value T_1. B, in turn, will record the time of receipt T_2 (taken from its own local clock), and returns a response timestamped with value T_3, and piggybacking the previously recorded value T_2. Finally, A records the time of the response's arrival, T_4. Let us assume that the propagation delays from A to B is roughly the same as B to A, meaning that $\delta T_{req} = T_2 - T_1 \approx T_4 - T_3 = \delta T_{res}$. In that

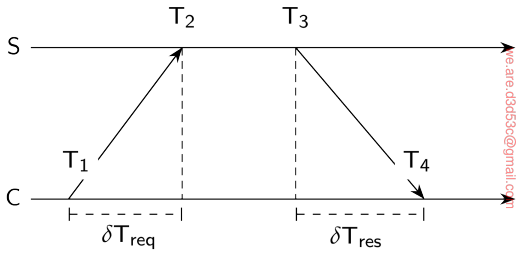


Figure 5.5: Getting the current time from a time server.

case, A can estimate its offset relative to B as

$$\theta = T_3 + \frac{(T_2 - T_1) + (T_4 - T_3)}{2} - T_4 = \frac{(T_2 - T_1) + (T_3 - T_4)}{2}$$

Of course, time is not allowed to run backward. If A's clock is fast, $\theta < 0$, meaning that A should, in principle, set its clock backward. This is not allowed as it could cause serious problems, such as an object file compiled just after the clock change having a time earlier than the source which was modified just before the clock change.

Such a change must be introduced gradually. One way is as follows. Suppose that the timer is set to generate 100 interrupts per second. Normally, each interrupt would add 10 msec to the time. When slowing down, the interrupt routine adds only 9 msec each time until the correction has been made. Similarly, the clock can be advanced gradually by adding 11 msec at each interrupt instead of jumping it forward all at once.

In the case of the **Network Time Protocol** (**NTP**), this protocol is set up pairwise between servers. In other words, B will also probe A for its current time. The offset θ is computed as given above, along with the estimation δ for the delay:

$$\delta = \frac{(T_4 - T_1) - (T_3 - T_2)}{2}$$

Eight pairs of (θ, δ) values are buffered, finally taking the minimal value found for δ as the best estimation for the delay between the two servers, and subsequently the associated value θ as the most reliable estimation of the offset.

Applying NTP symmetrically should, in principle, also let B adjust its clock to that of A. However, if B's clock is known to be more accurate, then such an adjustment would be foolish. To solve this problem, NTP divides servers into strata. A server with a **reference clock** such as a UTC receiver or an atomic clock, is known to be a **stratum-1 server** (the clock itself is said to operate at stratum 0). When A contacts B, it will adjust only its time if its own stratum level is higher than that of B. Moreover, after the synchronization, A's

stratum level will become one higher than that of B. In other words, if B is a stratum-k server, then A will become a stratum-$(k+1)$ server if its original stratum level was already larger than k. Due to the symmetry of NTP, if A's stratum level was *lower* than that of B, B will adjust itself to A.

There are many important features about NTP, of which many relate to identifying and masking errors, but also security attacks (see, e.g., Malhotra et al. [2016]). Many security issues, such as also described by Levesque and Tipper [2016], can be dealt with by establishing a secure channel between a time server and its client. NTP was originally described in [Mills, 1992] and is known to achieve (worldwide) accuracy in the range of 1–50 msec. A detailed description of NTP can be found in [Mills, 2011].

Clock synchronization in wireless networks

An important advantage of more traditional distributed systems is that we can easily and efficiently deploy timeservers. Moreover, most machines can contact each other, allowing for a relatively simple dissemination of information. These assumptions are no longer valid in many wireless networks, notably sensor networks. Nodes are resource constrained, and multihop routing is expensive. In addition, it is often important to optimize algorithms for energy consumption. These and other observations have led to the design of very different clock synchronization algorithms for wireless networks. In the following, we consider one specific solution. Sivrikaya and Yener [2004] provide a brief overview of other solutions. An extensive survey can be found in [Sundararaman et al., 2005].

Reference broadcast synchronization (**RBS**) is a clock synchronization protocol that is quite different from other proposals [Elson et al., 2002]. First, the protocol does not assume that there is a single node with an accurate account of the actual time available. Instead of aiming to provide all nodes UTC time, it aims at merely internally synchronizing the clocks. Second, the solutions we have discussed so far are designed to bring the sender and receiver into sync, essentially following a two-way protocol. RBS deviates from this pattern by letting only the receivers synchronize, keeping the sender out of the loop.

In RBS, a sender broadcasts a reference message that allows its receivers to adjust their clocks. A key observation is that in a sensor network the time to propagate a signal to other nodes is roughly constant, provided no multi-hop routing is assumed. Propagation time in this case is measured from the moment that a message leaves the network interface of the sender. As a consequence, two important sources of variation in message transfer no longer play a role in estimating delays: the time spent to construct a message, and the time spent to access the network. This principle is shown in Figure 5.6.

Note that in protocols such as NTP, a timestamp is added to the message before it is passed on to the network interface. Furthermore, as wireless

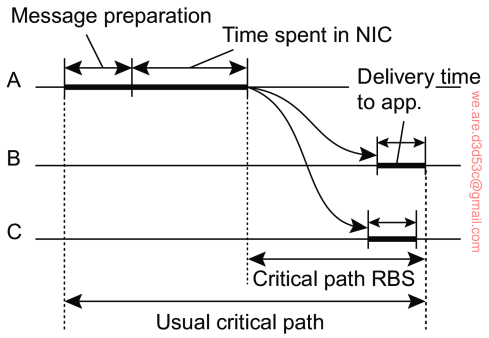


Figure 5.6: The usual critical path and the one used in RBS in determining network delays.

networks are based on a contention protocol, there is generally no saying how long it will take before a message can actually be transmitted. These factors of nondeterminism are eliminated in RBS. What remains is the delivery time at the receiver, but this time varies considerably less than the network-access time.

The idea underlying RBS is simple: when a node broadcasts a reference message m, each node p simply records the time $T_{p,m}$ that it received m. Note that $T_{p,m}$ is read from p's local clock. Ignoring clock skew, two nodes p and q can exchange each other's delivery times to estimate their mutual, relative offset:

$$Offset[p,q] = \frac{\sum_{k=1}^{M}(T_{p,k} - T_{q,k})}{M}$$

where M is the total number of reference messages sent. This information is important: node p will know the value of q's clock relative to its own value. Moreover, if it simply stores these offsets, there is no need to adjust its own clock, which saves energy.

Unfortunately, clocks can drift apart. The effect is that simply computing the average offset as done above will not work: the last values sent are simply less accurate than the first ones. Moreover, as time goes by, the offset will presumably increase. Elson et al. [2002] use a simple algorithm to compensate for this: instead of computing an average, they apply standard linear regression to compute the offset as a function:

$$Offset[p,q](t) = \alpha t + \beta$$

The constants α and β are computed from the pairs $(T_{p,k}, T_{q,k})$. This new form allows a much more accurate computation of q's current clock value by node p, and vice versa.

Note 5.2 (More information: How important is an accurate account of time?)
So why is time such a big deal for distributed systems? As we shall discuss in the remainder of this chapter, reaching consensus on a global ordering of events is what we really want, and this can be achieved without any notion of global absolute time. However, as will become clear, alternative methods for distributed coordination do not come easy.

Life would be much simpler if processes in a distributed system could time-stamp their events with infinite precision. Although infinite precision is asking too much, we can come practically close. Researchers at Google were confronted with the fact that their customers would really like to make use of a globally distributed database that supported transactions. Such a database would need to serve massive numbers of clients, rendering the use of, for example, a central transaction processing monitor as we discussed in Section 1.3.2, infeasible. Instead, for their Spanner system, Google decided to implement a true-time service, called TrueTime [Corbett et al., 2013]. This service provides three operations:

Operation	Result
`TT.now()`	A time interval $[T_{lwb}, T_{upb}]$ with $T_{lwb} < T_{upb}$
`TT.after(t)`	True if timestamp `t` has definitely passed
`TT.before(t)`	True if timestamp `t` has definitely not arrived

The most important aspect is that T_{lwb} and T_{upb} are *guaranteed* bounds. Of course, if $\epsilon = T_{upb} - T_{lwb}$ is large, say 1 hour, then implementing the service is relatively easy. Impressively enough, $\epsilon = 6ms$. To achieve this accuracy, the TrueTime service makes use of *time-master* machines of which there are several per data center. Time-slave daemons run on every machine in a data center and query multiple time masters, including ones from other data centers, very similar to what we described for NTP. Many time masters are equipped with accurate GPS receivers, while many others are independently equipped with atomic clocks. The result is a collection of time sources with a high degree of mutual independence (which is important for reasons of fault tolerance). Using a version of an algorithm developed by Marzullo and Owicki [1983], outliers are kept out of the computations. Meanwhile, the performance of TrueTime is continuously monitored and "bad" time machines are (manually) removed to give at least very high guarantees for the accuracy of the TrueTime service.

With a guaranteed accuracy of 6 milliseconds, building a transactional system becomes much easier: transactions can actually be timestamped, even by different servers, with the restriction that timestamping may need to be delayed for ϵ time units. More precisely, to know for sure that a transaction has committed, reading the resulting data may impose a wait for ϵ units. This is achieved by pessimistically assigning a timestamp to a transaction that writes data to the global database and making sure that clients never see any changes before the assigned timestamp (which is relatively easy to implement).

There are many details to this approach, which can be found in [Corbett et al., 2013]. As we are still dealing with a time interval, taking more traditional

ordering mechanisms into account it is possible to improve results, as explained by Demirbas and Kulkarni [2013].

5.2 Logical clocks

Clock synchronization is naturally related to time, although it may not be necessary to have an accurate account of the real time: it may be sufficient that every node in a distributed system agrees on *a* current time. We can go one step further. For running `make` it is adequate that two nodes agree that `input.o` is outdated by a new version of `input.c`, for example. In this case, keeping track of each other's events (such as a producing a new version of `input.c`) is what matters. For these algorithms, it is conventional to speak of the clocks as **logical clocks**.

In a seminal paper, Lamport [1978] showed that although clock synchronization is possible, it need not be absolute. If two processes do not interact, it is not necessary that their clocks be synchronized because the lack of synchronization would not be observable and thus could not cause problems. Furthermore, he pointed out that what usually matters is not that all processes agree on exactly what time it is, but rather that they agree on the *order in which events occur*. In the `make` example, what counts is whether `input.c` is older or newer than `input.o`, not their respective absolute creation times.

5.2.1 Lamport's logical clocks

To synchronize logical clocks, Lamport defined a relation called **happens-before**. The expression $a \rightarrow b$ is read "event a happens before event b" and means that all processes agree that first event a occurs, then afterward, event b occurs. The happens-before relation can be observed directly in two situations:

1. If a and b are events in the same process, and a occurs before b, then $a \rightarrow b$ is true.
2. If a is the event of a message being sent by one process, and b is the event of the message being received by another process, then $a \rightarrow b$ is also true. A message cannot be received before it is sent, or even at the same time it is sent, since it takes a finite, nonzero amount of time to arrive.

Happens-before is a transitive relation, so if $a \rightarrow b$ and $b \rightarrow c$, then $a \rightarrow c$. If two events, x and y, happen in different processes that do not exchange messages (not even indirectly via third parties), then $x \rightarrow y$ is not true, but neither is $y \rightarrow x$. These events are said to be **concurrent**, which simply means

that nothing can be said (or need be said) about when the events happened or which event happened first.

What we need is a way of measuring a notion of time such that for every event, a, we can assign it a time value $C(a)$ on which all processes agree. These time values must have the property that if $a \rightarrow b$, then $C(a) < C(b)$. To rephrase the conditions we stated earlier, if a and b are two events within the same process and a occurs before b, then $C(a) < C(b)$. Similarly, if a is the sending of a message by one process and b is the reception of that message by another process, then $C(a)$ and $C(b)$ must be assigned in such a way that everyone agrees on the values of $C(a)$ and $C(b)$ with $C(a) < C(b)$. In addition, the clock time, C, must always go forward (increasing), never backward (decreasing). Corrections to time can be made by adding a positive value, never by subtracting one.

Now let us look at the algorithm Lamport proposed for assigning times to events. Consider the three processes depicted in Figure 5.7. The processes run on different machines, each with its own clock. For the sake of argument, we assume that a clock is implemented as a software counter: the counter is incremented by a specific value every T time units. However, the value by which a clock is incremented differs per process. The clock in process P_1 is incremented by 6 units, 8 units in process P_2, and 10 units in process P_3, respectively. (Below, we explain that Lamport clocks are, in fact, **event counters**, which explains why their value may differ between processes.)

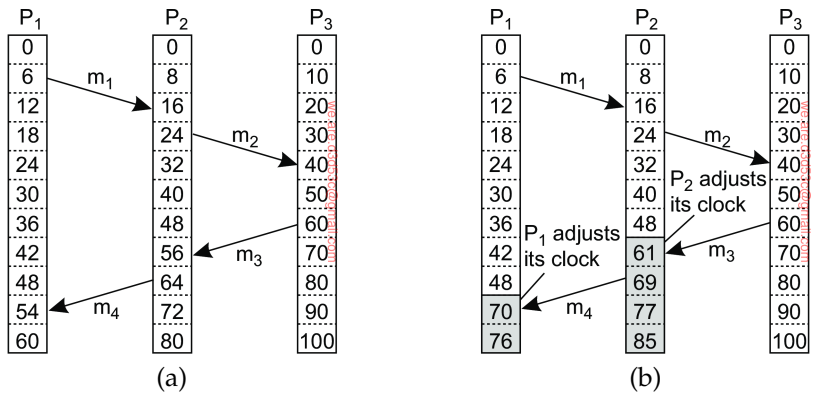


Figure 5.7: (a) Three processes, each with its own (logical) clock. The clocks run at different rates. (b) Lamport's algorithm corrects their values.

At time 6, process P_1 sends message m_1 to process P_2. How long this message takes to arrive depends on whose clock you believe. In any event, the clock in process P_2 reads 16 when it arrives. If the message carries the starting time, 6, in it, process P_2 will conclude that it took 10 ticks to make the journey. This value is certainly possible. According to this reasoning, message m_2 from P_2 to P_3 takes 16 ticks, again a plausible value.

Now consider message m_3. It leaves process P_3 at 60 and arrives at P_2 at 56. Similarly, message m_4 from P_2 to P_1 leaves at 64 and arrives at 54. These values are clearly impossible. It is this situation that must be prevented.

Lamport's solution follows directly from the happens-before relation. Since m_3 left at 60, it must arrive at 61 or later. Therefore, each message carries the sending time according to the sender's clock. When a message arrives and the receiver's clock shows a value before the time the message was sent, the receiver fast forwards its clock to be one more than the sending time. In Figure 5.7, we see that m_3 now arrives at 61. Similarly, m_4 arrives at 70.

Let us formulate this procedure more precisely. At this point, it is important to distinguish three different layers of software, as we already encountered in Section 2.2: the network, a middleware layer, and an application layer, as shown in Figure 5.8. What follows is typically part of the middleware layer.

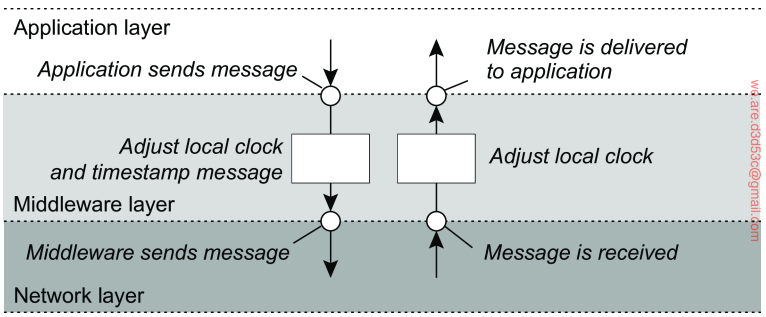


Figure 5.8: The positioning of Lamport's logical clocks in distributed systems.

To implement Lamport's logical clocks, each process P_i maintains a *local* counter C_i. These counters are updated according to the following steps [Raynal and Singhal, 1996]:

1. Before executing an event (i.e., sending a message over the network, delivering a message to an application, or some other internal event), P_i increments C_i: $C_i \leftarrow C_i + 1$.
2. When process P_i sends a message m to process P_j, it sets m's timestamp $ts(m)$ equal to C_i after having executed the previous step.
3. Upon the receipt of a message m, process P_j adjusts its own local counter as $C_j \leftarrow \max\{C_j, ts(m)\}$ after which it then executes the first step and delivers the message to the application.

In some situations, an additional requirement is desirable: no two events ever occur at the same time. To achieve this goal, we also use the unique process identifier to break ties and use tuples instead of only the counter's values. For example, an event at time 40 at process P_i will be timestamped as $\langle 40, i\rangle$. If we also have an event $\langle 40, j\rangle$ and $i < j$, then $\langle 40, i\rangle < \langle 40, j\rangle$.

Note that by assigning the event time $C(a) \leftarrow C_i(a)$ if a happened at process P_i at time $C_i(a)$, we have a distributed implementation of the global time value we were initially seeking for; we have thus constructed a **logical clock**.

Example: Totally ordered multicasting

As an application of Lamport's logical clocks, consider the situation in which a database has been replicated across several sites. For example, to improve query performance, a bank may place copies of an account database in two different cities, say New York and San Francisco. A query is always forwarded to the nearest copy. The price for a fast response to a query is partly paid in higher update costs because each update operation must be carried out at each replica.

In fact, there is a more stringent requirement regarding updates. Assume a customer in San Francisco wants to add $100 to her account, which currently contains $1,000. At the same time, a bank employee in New York initiates an update by which the customer's account is to be increased with 1 percent interest. Both updates should be carried out at both copies of the database. However, due to communication delays in the underlying network, the updates may arrive in the order as shown in Figure 5.9.

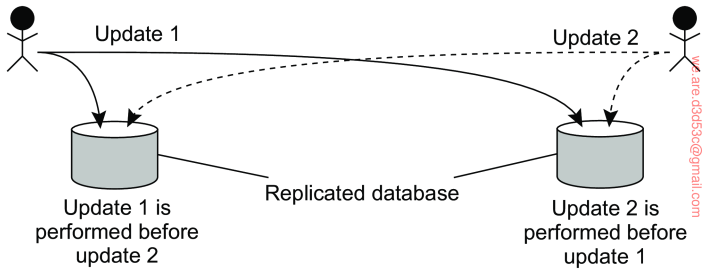


Figure 5.9: Updating a replicated database and leaving it in an inconsistent state.

The customer's update operation is performed in San Francisco before the interest update. In contrast, the copy of the account in the New York replica is first updated with the 1 percent interest, and after that with the $100 deposit. Consequently, the San Francisco database will record a total amount of $1,111, whereas the New York database records $1,110.

The problem that we are faced with is that the two update operations should have been performed in the same order at each copy. Although it makes a difference whether the deposit is processed before the interest update or the other way around, which order is followed is not essential from a consistency perspective. The important issue is that both copies should be the

same after the updates have taken place. In general, situations such as these require a **totally ordered multicast**, that is, a multicast operation by which all messages are delivered in the same order to each receiver. Lamport's logical clocks can be used to implement totally ordered multicasts in a completely distributed fashion.

Consider a group of processes multicasting messages to each other. Each message is always timestamped with the current (logical) time of its sender. When a message is multicasted, it is conceptually also sent to the sender. In addition, we assume that messages from the same sender are received in the order they were sent, and that no messages are lost.

When a process receives a message, it is put into a local queue, ordered according to its timestamp. The receiver multicasts an acknowledgment to the other processes. Note that if we follow Lamport's algorithm for adjusting local clocks, the timestamp of the received message is lower than the timestamp of the acknowledgment. The interesting aspect of this approach is that all processes will eventually have the same copy of the local queue (provided no messages are removed).

A process can deliver a queued message to the application it is running only when that message is at the head of the queue and has been acknowledged by each other process. At that point, the message is removed from the queue and handed over to the application; the associated acknowledgments can simply be removed. Because each process has the same copy of the queue, all messages are delivered in the same order everywhere. In other words, we have established totally ordered multicasting. We leave it as an exercise to the reader to figure out that it is not strictly necessary that each multicast message has been explicitly acknowledged. It is sufficient that a process *reacts* to an incoming message, either by returning an acknowledgment or sending its own multicast message.

Totally ordered multicasting is an important vehicle for replicated services, where the replicas are kept consistent by letting them execute the same operations in the same order everywhere. As the replicas essentially follow the same transitions in the same finite state machine, it is also known as **state machine replication** [Schneider, 1990].

Note 5.3 (Advanced: Using Lamport clocks to achieve mutual exclusion)
To further illustrate the usage of Lamport's clocks, let us see how we can use the previous algorithm for totally ordered multicasting to establish access to what is commonly known as a **critical region**: a section of code that can be executed by at most one process at a time. This algorithm is very similar to the one for multicasting, as essentially all processes need to agree on the order by which processes are allowed to enter their critical region.

Figure 5.10(a) shows the code that each process executes when requesting, releasing, or allowing access to the critical region (again, omitting details). Each

process maintains a request queue as well as a logical clock. To enter the critical region, a call to `requestToEnter` is made, which results in inserting an `ENTER` message with timestamp `(clock,procID)` into the local queue and sending that message to the other processes. The operation `cleanupQ` essentially sorts the queue. We return to it shortly.

```
1  class Process:
2    def __init__(self, chanID, procID, procIDSet):
3      self.chan.join(procID)
4      self.procID     = int(procID)
5      self.otherProcs.remove(self.procID)
6      self.queue      = []                    # The request queue
7      self.clock      = 0                     # The current logical clock
8
9    def requestToEnter(self):
10     self.clock = self.clock + 1                         # Increment clock value
11     self.queue.append((self.clock, self.procID, ENTER)) # Append request to q
12     self.cleanupQ()                                     # Sort the queue
13     self.chan.sendTo(self.otherProcs, (self.clock, self.procID, ENTER)) # Send request
14
15   def ackToEnter(self, requester):
16     self.clock = self.clock + 1                         # Increment clock value
17     self.chan.sendTo(requester, (self.clock, self.procID, ACK)) # Permit other
18
19   def release(self):
20     tmp = [r for r in self.queue[1:] if r[2] == ENTER]  # Remove all ACKs
21     self.queue = tmp                                    # and copy to new queue
22     self.clock = self.clock + 1                         # Increment clock value
23     self.chan.sendTo(self.otherProcs, (self.clock, self.procID, RELEASE)) # Release
24
25   def allowedToEnter(self):
26     commProcs = set([req[1] for req in self.queue[1:]]) # See who has sent a message
27     return (self.queue[0][1] == self.procID and len(self.otherProcs) == len(commProcs))
```

Figure 5.10: (a) Using Lamport's logical clocks for mutual exclusion.

When a process P receives an `ENTER` message from process Q, it can simply acknowledge that Q can enter its critical region, even if P wants to do so as well. In the latter case, P's `ENTER` request will have a lower logical timestamp than the `ACK` message sent by P to Q, meaning that P's request will have been inserted into Q's queue *before* P's `ACK` message.

Finally, when a process leaves its critical region, it calls `release`. It cleans up its local queue by removing all received `ACK` messages, leaving only the `ENTER` requests from other processes. It then multicasts a `RELEASE` message.

To actually enter a critical region, a process will have to repeatedly call `allowedToEnter` and when returned `False`, will have to block on a next incoming message. The operation `allowedToEnter` does what is to be expected: it checks if the calling process's `ENTER` message is at the head of the queue, and sees if all other processes have sent a message as well. The latter is encoded through the set

commProcs, which contains the procIDs of all processes having sent a message by inspecting all messages in the local queue from the second position and onwards.

```
1   def receive(self):
2     msg = self.chan.recvFrom(self.otherProcs)[1]       # Pick up any message
3     self.clock = max(self.clock, msg[0])               # Adjust clock value...
4     self.clock = self.clock + 1                        # ...and increment
5     if msg[2] == ENTER:
6       self.queue.append(msg)                           # Append an ENTER request
7       self.ackToEnter(msg[1])                          # and unconditionally allow
8     elif msg[2] == ACK:
9       self.queue.append(msg)                           # Append a received ACK
10    elif msg[2] == RELEASE:
11      del(self.queue[0])                               # Just remove first message
12    self.cleanupQ()                                    # And sort and cleanup
```

Figure 5.10: (b) Using Lamport's logical clocks for mutual exclusion: handling incoming requests.

What to do when a message is received is shown in Figure 5.10(b). First, the local clock is adjusted according to the rules for Lamport's logical clocks explained above. When receiving an ENTER or ALLOW message, that message is simply inserted into the queue. An entry request is always acknowledged, as we just explained. When a RELEASE message is received, the original ENTER request is removed. Note that this request is at the head of the queue. Thereafter, the queue is cleaned up again.

At this point, note that if we would clean up the queue by only sorting it, we may get into trouble. Suppose that processes P and Q want to enter their respective critical regions at roughly the same time, but that P is allowed to go first based on logical-clock values. P may find Q's request in its queue, along with ENTER or ALLOW messages from other processes. If its own request is at the head of its queue, P will proceed and enter its critical region. However, Q will also send an ALLOW message to P as well, in addition to its original ENTER message. That ALLOW message may arrive *after* P had already entered its critical region, but *before* ENTER messages from other processes. When Q eventually enters, and leaves its critical region, Q's RELEASE message would result in removing Q's original ENTER message, but not the ALLOW message it had previously sent to P. By now, that message is at the head of P's queue, effectively blocking the entrance to the critical region of other processes in P's queue. Cleaning up the queue thus also involves removing old ALLOW messages.

5.2.2 Vector clocks

Lamport's logical clocks lead to a situation where all events in a distributed system are totally ordered with the property that if event a happened before event b, then a will also be positioned in that ordering before b, that is, $C(a) < C(b)$.

However, with Lamport clocks, nothing can be said about the relationship between two events a and b by merely comparing their time values $C(a)$ and $C(b)$, respectively. In other words, if $C(a) < C(b)$, then this does not necessarily imply that a indeed happened before b. Something more is needed for that.

To explain, consider the messages as sent by the three processes shown in Figure 5.11. Denote by $T_{snd}(\mathsf{m_i})$ the logical time at which message $\mathsf{m_i}$ was sent, and likewise, by $T_{rcv}(\mathsf{m_i})$ the time of its receipt. By construction, we know that for each message $T_{snd}(\mathsf{m_i}) < T_{rcv}(\mathsf{m_i})$. But what can we conclude in general from $T_{rcv}(\mathsf{m_i}) < T_{snd}(\mathsf{m_j})$ for different messages $\mathsf{m_i}$ and $\mathsf{m_j}$?

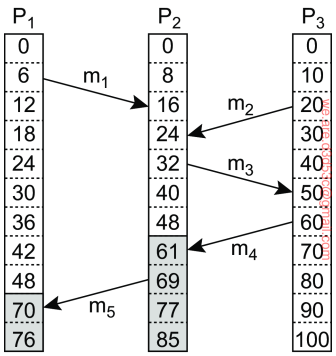


Figure 5.11: Concurrent message transmission using logical clocks.

In the case for which $\mathsf{m_i} = \mathsf{m_1}$ and $\mathsf{m_j} = \mathsf{m_3}$, we know that these values correspond to events that took place at process $\mathsf{P_2}$, meaning that $\mathsf{m_3}$ was indeed sent after the receipt of message $\mathsf{m_1}$. This may indicate that the sending of message $\mathsf{m_3}$ depended on what was received through message $\mathsf{m_1}$. At the same time, we also know that $T_{rcv}(\mathsf{m_1}) < T_{snd}(\mathsf{m_2})$. However, as far as we can tell from Figure 5.11, the sending of $\mathsf{m_2}$ has nothing to do with the receipt of $\mathsf{m_1}$.

The problem is that Lamport clocks do not capture **causality**. In practice, causality is captured by means of **vector clocks**. To better understand where these come from, we follow the explanation as given by Baquero and Preguica [2016]. In fact, tracking causality is simple if we assign each event a unique name such as the combination of a process ID and a locally incrementing counter: $\mathsf{p_k}$ is the kth event that happened at process P. The problem then boils down to keeping track of **causal histories**. For example, if two local events happened at process P, then the causal history $H(\mathsf{p_2})$ of event $\mathsf{p_2}$ is $\{\mathsf{p_1}, \mathsf{p_2}\}$.

Now assume that process P sends a message to process Q (which is an event at P and thus recorded as $\mathsf{p_k}$ for some k), and that at the time of arrival (an event for Q), the most recent causal history of Q was $\{\mathsf{q_1}\}$. To track

causality, P also sends its most recent causal history (assume it was $\{p_1, p_2\}$, extended with p_3 expressing the sending of the message). Upon arrival, Q records the event (q_2), and merges the two causal histories into a new one: $\{p_1, p_2, p_3, q_1, q_2\}$.

Checking whether an event p causally precedes an event q can be done by checking whether $H(p) \subset H(q)$ (i.e., it should be a *proper* subset). In fact, with our notation, it even suffices to check whether $p \in H(q)$, assuming that q is always the last local event in $H(q)$.

The problem with causal histories, is that their representation is not very efficient. However, there is no need to keep track of all successive events from the same process: the last one will do. If we subsequently assign an index to each process, we can represent a causal history as a vector, in which the jth entry represents the number of events that happened at process P_j. Causality can then be captured by means of **vector clocks**, which are constructed by letting each process P_i maintain a vector VC_i with the following two properties:

1. $VC_i[i]$ is the number of events that have occurred so far at P_i. In other words, $VC_i[i]$ is the local logical clock at process P_i.
2. If $VC_i[j] = k$ then P_i knows that k events have occurred at P_j. It is thus P_i's knowledge of the local time at P_j.

The first property is maintained by incrementing $VC_i[i]$ at the occurrence of each new event that happens at process P_i. The second property is maintained by piggybacking vectors along with messages that are sent. In particular, the following steps are performed:

1. Before executing an event (i.e., sending a message over the network, delivering a message to an application, or some other internal event), P_i executes $VC_i[i] \leftarrow VC_i[i] + 1$. This is equivalent to recording a new event that happened at P_i.
2. When process P_i sends a message m to P_j, it sets m's (vector) timestamp $ts(m)$ equal to VC_i after having executed the previous step (i.e., it also records the sending of the message as an event that takes place at P_i).
3. Upon the receipt of a message m, process P_j adjusts its own vector by setting $VC_j[k] \leftarrow \max\{VC_j[k], ts(m)[k]\}$ for each k (which is equivalent to merging causal histories), after which it executes the first step (recording the receipt of the message) and then delivers the message to the application.

Note that if an event a has timestamp $ts(a)$, then $ts(a)[i] - 1$ denotes the number of events processed at P_i that causally precede a. As a consequence, when P_j receives a message m from P_i with timestamp $ts(m)$, it knows about the number of events that have occurred at P_i that causally preceded the

sending of m. More important, however, is that P_j is also told how many events at *other* processes have taken place, known to P_i, before P_i sent message m. In other words, timestamp $ts(m)$ tells the receiver how many events in other processes have preceded the sending of m, and on which m may causally depend.

To see what this means, consider Figure 5.12 which shows three processes. In Figure 5.12(a), P_2 sends a message m_1 at logical time $VC_2 = (0,1,0)$ to process P_1. Message m_1 thus receives timestamp $ts(m_1) = (0,1,0)$. Upon its receipt, P_1 adjusts its logical time to $VC_1 \leftarrow (1,1,0)$ and delivers it. Message m_2 is sent by P_1 to P_3 with timestamp $ts(m_2) = (2,1,0)$. Before P_1 sends another message, m_3, an event happens at P_1, eventually leading to timestamping m_3 with value $(4,1,0)$. After receiving m_3, process P_2 sends message m_4 to P_3, with timestamp $ts(m_4) = (4,3,0)$.

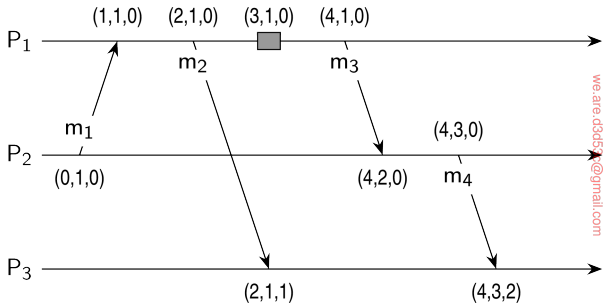


Figure 5.12: Capturing potential causality when exchanging messages.

Now consider the situation shown in Figure 5.12(b). Here, we have delayed sending message m_2 until after message m_3 has been sent, and after the event had taken place. It is not difficult to see that $ts(m_2) = (4,1,0)$, while $ts(m_4) = (2,3,0)$. Compared to Figure 5.12(a), we have the following situation:

Situation	$ts(m_2)$	$ts(m_4)$	$ts(m_2) < ts(m_4)$	$ts(m_2) > ts(m_4)$	Conclusion
Figure 5.12(a)	$(2,1,0)$	$(4,3,0)$	Yes	No	m_2 may causally precede m_4
Figure 5.12(b)	$(4,1,0)$	$(2,3,0)$	No	No	m_2 and m_4 may conflict

We use the notation $ts(a) < ts(b)$ if and only if for all k, $ts(a)[k] \leq ts(b)[k]$ and there is at least one index k' for which $ts(a)[k'] < ts(b)[k']$. Thus, by using vector clocks, process P_3 can detect whether m_4 may be causally dependent on m_2, or whether there may be a potential conflict. Note, by the way, that without knowing the actual information contained in messages, it is not possible to state with certainty that there is indeed a causal relationship, or perhaps a conflict.

Note 5.4 (Advanced: Enforcing causal communication)
Using vector clocks, it is now possible to ensure that a message is delivered only if all messages that may have causally preceded it have been received as well. To enable such a scheme, we will assume that messages are multicast within a group of processes. Note that this **causally ordered multicasting** is weaker than totally ordered multicasting. Specifically, if two messages are not in any way related to each other, we do not care in which order they are delivered to applications. They may even be delivered in different order at different locations.

For enforcing causal message delivery, we assume that clocks are adjusted only when sending and *delivering* messages (note, again, that messages are not adjusted when they are *received* by a process, but only when they are delivered to an application). In particular, only upon sending a message, will process P_i increment $VC_i[i]$ by 1. Only when it delivers a message m with timestamp $ts(m)$, will it adjust $VC_i[k]$ to $\max\{VC_i[k], ts(m)[k]\}$ for each k.

Now suppose that P_j receives a message m from P_i with (vector) timestamp $ts(m)$. The delivery of the message to the application layer will then be delayed until the following two conditions are met:

1. $ts(m)[i] = VC_j[i] + 1$
2. $ts(m)[k] \leq VC_j[k]$ for all $k \neq i$

The first condition states that m is the next message that P_j was expecting from process P_i. The second condition states that P_j has delivered all the messages that have been delivered by P_i when it sent message m. Note that there is no need for process P_j to delay the delivery of its own messages.

As an example, consider three processes P_1, P_2, and P_3 as shown in Figure 5.13. At local time $(1,0,0)$, P_1 multicasts message m to the other two processes. Note that $ts(m) = (1,0,0)$. Its receipt and subsequent delivery by P_2, will bring the logical clock at P_2 to $(1,0,0)$, effectively indicating that it has received one message from P_1, has itself sent no message so far, and has not yet delivered a message from P_3. P_2 then decides to multicast m^*, at updated time $(1,1,0)$, which arrives at P_3 *sooner* than m.

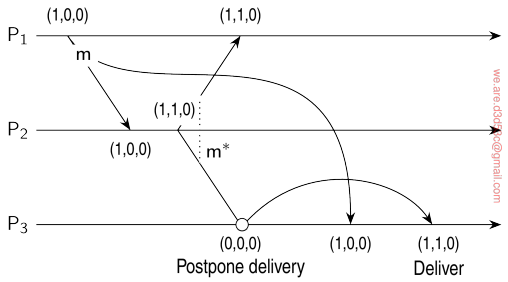


Figure 5.13: Enforcing causal communication.

When comparing the timestamp of m with its current time, which is $(0,0,0)$, P_3 concludes that it is still missing a message from P_1 which P_2 apparently had delivered before sending m^*. P_3 therefore, decides to postpone the delivery of m^* (and will also not adjust its local, logical clock). Later, after m has been received and delivered by P_3, which brings its local clock to $(1,0,0)$, P_3 can deliver message m^* and also update its clock.

A note on ordered message delivery Some middleware systems, notably ISIS and its successor Horus [Birman and van Renesse, 1994], provide support for totally ordered and causally ordered (reliable) multicasting. There has been some controversy whether such support should be provided as part of the message-communication layer, or whether applications should handle ordering (see, e.g., Cheriton and Skeen [1993]; Birman [1994]). Matters have not been settled, but more important is that the arguments still hold today.

There are two main problems with letting the middleware deal with message ordering. First, because the middleware cannot tell what a message actually contains, only *potential* causality is captured. For example, two messages from the same sender that are completely independent will always be marked as causally related by the middleware layer. This approach is overly restrictive and may lead to efficiency problems.

A second problem is that not all causality may be captured. Consider some digital chat room. Suppose Alice posts a message. If she then phones Bob telling about what she just wrote, Bob may post another message as a reaction without having seen Alice's posting. In other words, there is a causality between Bob's posting and that of Alice due to *external* communication. This causality is not captured by the chat room system.

In essence, ordering issues, like many other application-specific communication issues, can be adequately solved by looking at the application for which communication is taking place. This is also known as the **end-to-end principle** in systems design [Saltzer et al., 1984]. A drawback of having only application-level solutions is that a developer is forced to concentrate on issues that do not immediately relate to the core functionality of the application. For example, ordering may

not be the most important problem when developing a messaging system such as the one for a chat room. In that case, having an underlying communication layer handle ordering may turn out to be convenient. We will come across the end-to-end argument several times.

5.3 Mutual exclusion

Fundamental to distributed systems is the concurrency and collaboration among multiple processes. In many cases, this also means that processes will need to simultaneously access the same resources. To prevent that such concurrent accesses corrupt the resource, or make it inconsistent, solutions are needed to grant mutual exclusive access by processes. In this section, we take a look at some important and representative distributed algorithms that have been proposed. Surveys of distributed algorithms for mutual exclusion are provided by Saxena and Rai [2003] and Velazquez [1993]. Various algorithms are also presented in [Kshemkalyani and Singhal, 2008].

5.3.1 Overview

Distributed mutual exclusion algorithms can be classified into two different categories. In **token-based solutions** mutual exclusion is achieved by passing a special message between the processes, known as a **token**. There is only one token available, and who ever has that token is allowed to access the shared resource. When finished, the token is passed on to a next process. If a process having the token is not interested in accessing the resource, it passes it on.

Token-based solutions have a few important properties. First, depending on how the processes are organized, they can fairly easily ensure that every process will get a chance at accessing the resource. In other words, they provide guarantees for **safety** by avoiding what is known as **starvation**. Second, **deadlocks** by which several processes are indefinitely waiting for each other to proceed, can easily be avoided, contributing to their simplicity. The main drawback of token-based solutions is a rather serious one: when the token is lost (e.g. because the process holding it crashed), an intricate distributed procedure needs to be started to ensure that a new token is created, but above all, that it is also the only token.

As an alternative, many distributed mutual exclusion algorithms follow a **permission-based approach**. In this case, a process wanting to access the resource first requires the permission from other processes. There are many ways toward granting such permission and in the sections that follow we will consider a few of them.

5.3.2 A centralized algorithm

A straightforward way to achieve mutual exclusion in a distributed system is to simulate how it is done in a one-processor system. One process is elected as the coordinator. Whenever a process wants to access a shared resource, it sends a request message to the coordinator stating which resource it wants to access and asking for permission. If no other process is currently accessing that resource, the coordinator sends back a reply granting permission, as shown in Figure 5.14(a). When the reply arrives, the requester can go ahead.

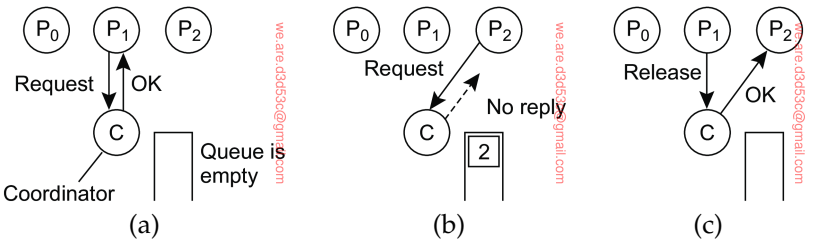


Figure 5.14: (a) Process P_1 asks for permission to access a shared resource. Permission is granted. (b) Process P_2 asks permission to access the same resource, but receives no reply. (c) When P_1 releases the resource, the coordinator replies to P_2.

Now suppose that another process, P_2 in Figure 5.14(b) asks for permission to access the resource. The coordinator knows that a different process is already at the resource, so it cannot grant permission. The exact method used to deny permission is system dependent. In Figure 5.14(b) the coordinator just refrains from replying, thus blocking process P_2, which is waiting for a reply. Alternatively, it could send a reply saying "permission denied." Either way, it queues the request from P_2 for the time being and waits for more messages.

When process P_1 is finished with the resource, it sends a message to the coordinator releasing its exclusive access, as shown in Figure 5.14(c). The coordinator takes the first item off the queue of deferred requests and sends that process a grant message. If the process was still blocked (i.e., this is the first message to it), it unblocks and accesses the resource. If an explicit message has already been sent denying permission, the process will have to poll for incoming traffic or block later. Either way, when it sees the grant, it can go ahead as well.

It is easy to see that the algorithm guarantees mutual exclusion: the coordinator lets only one process at a time access the resource. It is also fair, since requests are granted in the order in which they are received. No process ever waits forever (no starvation). The scheme is easy to implement, too, and requires only three messages per use of resource (request, grant, release). Its simplicity makes it an attractive solution for many practical situations.

The centralized approach also has shortcomings. The coordinator is a single point of failure, so if it crashes, the entire system may go down. If processes normally block after making a request, they cannot distinguish a dead coordinator from "permission denied" since in both cases no message comes back. In addition, in a large system, a single coordinator can become a performance bottleneck. Nevertheless, the benefits coming from its simplicity often outweigh the potential drawbacks. Moreover, distributed solutions are not necessarily better, as we illustrate next.

5.3.3 A distributed algorithm

Using Lamport's logical clocks, and inspired by Lamport's original solution for distributed mutual exclusion (which we discussed in Note 5.3), Ricart and Agrawala [1981] provided the following algorithm. Their solution requires a total ordering of all events in the system. That is, for any pair of events, such as messages, it must be unambiguous which one actually happened first.

The algorithm works as follows. When a process wants to access a shared resource, it builds a message containing the name of the resource, its process number, and the current (logical) time. It then sends the message to all other processes, conceptually including itself. The sending of messages is assumed to be reliable; that is, no message is lost.

When a process receives a request message from another process, the action it takes depends on its own state regarding the resource named in the message. Three different cases have to be clearly distinguished:

- If the receiver is not accessing the resource and does not want to access it, it sends back an OK message to the sender.
- If the receiver already has access to the resource, it simply does not reply. Instead, it queues the request.
- If the receiver wants to access the resource as well but has not yet done so, it compares the timestamp of the incoming message with the one contained in the message that it has sent everyone. The lowest one wins. If the incoming message has a lower timestamp, the receiver sends back an OK message. If its own message has a lower timestamp, the receiver queues the incoming request and sends nothing.

After sending out requests asking permission, a process sits back and waits until everyone else has given permission. As soon as all the permissions are in, it may go ahead. When it is finished, it sends OK messages to all processes in its queue and deletes them all from the queue. If there is no conflict, it clearly works. However, suppose that two processes try to access the resource simultaneously, as shown in Figure 5.15(a).

Process P_0 sends everyone a request with timestamp 8, while at the same time, process P_2 sends everyone a request with timestamp 12. P_1 is not

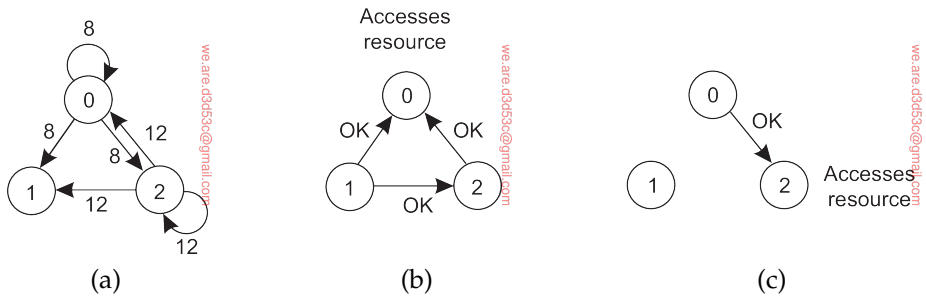


Figure 5.15: (a) Two processes want to access a shared resource at the same moment. (b) P_0 has the lowest timestamp, so it wins. (c) When process P_0 is done, it sends an OK also, so P_2 can now go ahead.

interested in the resource, so it sends OK to both senders. Processes P_0 and P_2 both see the conflict and compare timestamps. P_2 sees that it has lost, so it grants permission to P_0 by sending OK. Process P_0 now queues the request from P_2 for later processing and accesses the resource, as shown in Figure 5.15(b). When it is finished, it removes the request from P_2 from its queue and sends an OK message to P_2, allowing the latter to go ahead, as shown in Figure 5.15(c). The algorithm works because in the case of a conflict, the lowest timestamp wins and everyone agrees on the ordering of the timestamps.

With this algorithm, mutual exclusion is guaranteed without deadlock or starvation. If the total number of processes is N, then the number of messages that a process needs to send and receive before it can enter its **critical region** is $2 \cdot (N-1)$: $N-1$ request messages to all other processes, and subsequently $N-1$ OK messages, one from each other process. A critical region is a series of instructions to be executed by a process, which requires mutually exclusive access.

Unfortunately, this algorithm has N points of failure. If any process crashes, it will fail to respond to requests. This silence will be interpreted (incorrectly) as denial of permission, thus blocking all subsequent attempts by all processes to enter any of their respective critical regions. The algorithm can be patched up as follows. When a request comes in, the receiver always sends a reply, either granting or denying permission. Whenever either a request or a reply is lost, the sender times out and keeps trying until either a reply comes back or the sender concludes that the destination is dead. After a request is denied, the sender should block, waiting for a subsequent OK message.

Another problem with this algorithm is that either a multicast communication primitive must be used, or each process must maintain the group membership list itself, including processes entering the group, leaving the group, and crashing. The method works best with small groups of processes

that never change their group memberships. Finally, note that *all* processes are involved in *all* decisions concerning accessing the shared resource, which may impose a burden on processes running on resource-constrained machines.

Various minor improvements are possible to this algorithm. For example, getting permission from everyone is overkill. All that is needed is a method to prevent two processes from accessing the resource at the same time. The algorithm can be modified to grant permission when it has collected permission from a simple majority of the other processes, rather than from all of them.

5.3.4 A token-ring algorithm

An entirely different approach to deterministically achieving mutual exclusion in a distributed system is illustrated in Figure 5.16. In software, we construct an overlay network in the form of a logical ring in which each process is assigned a position in the ring. All that matters is that each process knows who is next in line after itself.

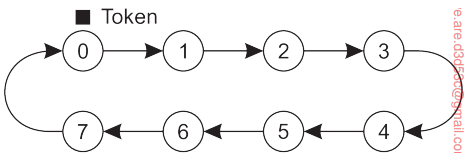


Figure 5.16: An overlay network constructed as a logical ring with a token circulating between its members.

When the ring is initialized, process P_0 is given a **token**. The token circulates around the ring. Assuming there are N processes, the token is passed from process P_k to process $P_{(k+1) \bmod N}$ in point-to-point messages. When a process acquires the token from its neighbor, it checks to see if it needs to access the shared resource. If so, the process goes ahead, does all the work it needs to, and releases the resources. After it has finished, it passes the token along the ring. It is not permitted to enter the resource immediately again using the same token.

If a process is handed the token by its neighbor and is not interested in the resource, it just passes the token along. As a consequence, when no processes need the resource, the token just circulates around the ring.

The correctness of this algorithm is easy to see. Only one process has the token at any instant, so only one process can actually get to the resource. Since the token circulates among the processes in a well-defined order, starvation cannot occur. Once a process decides it wants to have access to the resource, at worst it will have to wait for every other process to use the resource.

This algorithm has its own problems. If the token is ever lost, for example because its holder crashes or due to a lost message containing the token, it must be regenerated. In fact, detecting that it is lost may be difficult, since the

amount of time between successive appearances of the token on the network is unbounded. The fact that the token has not been spotted for an hour does not mean that it has been lost; somebody may still be using it.

The algorithm also runs into trouble if a process crashes, but recovery is relatively easy. If we require a process receiving the token to acknowledge receipt, a dead process will be detected when its neighbor tries to give it the token and fails. At that point, the dead process can be removed from the group, and the token holder can throw the token over the head of the dead process to the next member down the line, or the one after that, if necessary. Of course, doing so requires that everyone maintains the current ring configuration.

5.3.5 A decentralized algorithm

Let us take a look at a fully decentralized solution. Lin et al. [2004] propose to use a voting algorithm. Each resource is assumed to be replicated N times. Every replica has its own coordinator for controlling the access by concurrent processes.

However, whenever a process wants to access the resource, it will simply need to get a majority vote from $m > N/2$ coordinators. We assume that when a coordinator does not give permission to access a resource (which it will do when it had granted permission to another process), it will tell the requester.

The assumption is that when a coordinator crashes, it recovers quickly but will have forgotten any vote it gave before it crashed. Another way of viewing this is that a coordinator resets itself at arbitrary moments. The risk that we are taking is that a reset will make the coordinator forget that it had previously granted permission to some process to access the resource. As a consequence, it may incorrectly grant this permission again to another process after its recovery.

Let $p = \Delta t/T$ be the probability that a coordinator resets during a time interval Δt, while having a lifetime of T. The probability $\mathbb{P}[k]$ that k out of m coordinators reset during the same interval is then

$$\mathbb{P}[k] = \binom{m}{k} p^k (1-p)^{m-k}$$

If f coordinators reset, then the correctness of the voting mechanism will be violated when we have more than m other coordinators think it is okay to allocate the resource, that is, when $N - (m - f) \geq m$, or, in other words, when $f \geq 2m - N$. The probability that such a violation occurs is $\sum_{k=2m-N}^{m} \mathbb{P}[k]$. To give an impression of what this could mean, Figure 5.17 shows the probability of violating correctness for different values of N, m, and p. Note that we compute p by considering the number of seconds per hour that a coordinator resets, and also taking this value to be the average time needed to access

a resource. Our values for p are considered to be (very) conservative. The conclusion is that, in general, the probability of violating correctness can be so low that it can be neglected in comparison to other types of failure.

N	m	p	Violation
8	5	3 sec/hour	$< 10^{-5}$
8	6	3 sec/hour	$< 10^{-11}$
16	9	3 sec/hour	$< 10^{-4}$
16	12	3 sec/hour	$< 10^{-21}$
32	17	3 sec/hour	$< 10^{-4}$
32	24	3 sec/hour	$< 10^{-43}$

N	m	p	Violation
8	5	30 sec/hour	$< 10^{-3}$
8	6	30 sec/hour	$< 10^{-7}$
16	9	30 sec/hour	$< 10^{-2}$
16	12	30 sec/hour	$< 10^{-13}$
32	17	30 sec/hour	$< 10^{-2}$
32	24	30 sec/hour	$< 10^{-27}$

Figure 5.17: Violation probabilities for various parameter values of decentralized mutual exclusion.

To implement this scheme, we can use a system in which a resource is replicated N times. Assume that the resource is known under its unique name rname. We can then assume that the ith replica is named rname_i which is then used to compute a unique key using a known hash function. As a consequence, every process can generate the N keys given a resource's name, and subsequently look up each node responsible for a replica (and controlling access to that replica) using some commonly used naming system.

If permission to access the resource is denied (i.e., a process gets less than m votes), it is assumed that it will back off for some randomly chosen time, and make a next attempt later. The problem with this scheme is that if many nodes want to access the same resource, it turns out that the utilization rapidly drops. In that case, there are so many nodes competing to get access that eventually no one can get enough votes, leaving the resource unused. A solution to solve this problem can be found in [Lin et al., 2004].

Note 5.5 (More information: A comparison of the mutual-exclusion algorithms)
A brief comparison of the mutual exclusion algorithms we have looked at is instructive. In Figure 5.18 we have listed the algorithms and two performance properties: the number of messages required for a process to access and release a shared resource, and the delay before access can occur (assuming messages are passed sequentially over a network).

In the following, we assume only point-to-point messages (or, equivalently, count a multicast to N processes as N messages).

- The centralized algorithm is simplest and also most efficient. It requires only three messages to enter and leave a critical region: a request, a grant to enter, and a release to exit.

- The distributed algorithm requires $N-1$ request messages, one to each of the other processes, and an additional $N-1$ grant messages, for a total of $2(N-1)$.
- With the token ring algorithm, the number is variable. If every process constantly wants to enter a critical region, then each token pass will result in one entry and exit, for an average of one message per critical region entered. At the other extreme, the token may sometimes circulate for hours without anyone being interested in it. In this case, the number of messages per entry into a critical region is unbounded.
- The decentralized case requires sending N messages to coordinators, and another N responses. If it does not get a majority, it will have to release (at most) $N/2$ votes. If it did get enough votes, it will have to send an additional N release messages later on. A process may need to go through $k \geq 1$ attempts.

Algorithm	Messages per entry/exit	Delay before entry (in message times)
Centralized	3	2
Distributed	$2(N-1)$	$2(N-1)$
Token ring	$1, \ldots, \infty$	$0, \ldots, N-1$
Decentralized	$2kN + (k-1)N/2 + N, k = 1, 2, \ldots$	$2kN + (k-1)N/2$

Figure 5.18: A comparison of four mutual exclusion algorithms.

The delay from the moment a process needs to enter a critical region until its actual entry also varies. For a worst-case analysis, we assume that messages are sent one after the other (i.e., there are never two or more messages in transit at the same time), and that message transfer time is roughly the same everywhere. Delay can then be expressed in **message transfer time units**, or simply **MTTU**. Under these assumptions, when the time using a resource is short, the dominant factor in the delay is determined by the total number of messages sent through the system before access can be granted. When resources are used for a long period of time, the dominant factor is waiting for everyone else to take their turn. In Figure 5.18 we show the former case.

- It takes only two MTTUs to enter a critical region in the centralized case, caused by a request message and the subsequent grant message sent by the coordinator.
- The distributed algorithm requires sending $N-1$ request messages, and receiving another $N-1$ grant messages, adding up to $2(N-1)$ MTTUs.
- For the token ring, the delay varies from 0 MTTU (in case the token had just arrived) to $N-1$ (for when the token had just departed).
- In the case of decentralized, the delay is dependent on the number of times a process needed to return the (minority of) votes. With having to go through $k \geq 1$ attempts, a process may see $2kN + (k-1)N/2$ MTTUs.

> Virtually all algorithms suffer badly in the event of crashes. Special measures and additional complexity must be introduced to avoid having a crash bring down the entire system. It is somewhat ironic that distributed algorithms are generally more sensitive to crashes than centralized ones. In this sense, it should not come as a surprise that, indeed, centralized mutual exclusion is widely applied: it is simple to understand the behavior, and relatively easy to increase the fault tolerance of the centralized server.

5.3.6 Example: Simple locking with ZooKeeper

For many practical reasons, mutual exclusion in distributed systems is often done with a centralized coordinator, not in the least because the behavior of these solutions is much easier to understand than many other noncentralized versions. Let us briefly look at a system that is designed for coordination tasks in distributed systems, and which is by now also widely deployed. **ZooKeeper** was developed to provide facilities for supporting various coordination tasks, including locking, leader election, monitoring, to name but just a few. It has been designed for scalability and fault tolerance. For our purposes now, we concentrate only on a relatively simple setup, namely to support locking through a single server. ZooKeeper's fault tolerance aspects are discussed in Chapter 8. The system was first described in [Hunt et al., 2010], to be later followed up with a practical description by Junqueira and Reed [2014].

ZooKeeper basics

As mentioned, ZooKeeper is designed to facilitate different coordination functions. An important design principle is its lack of blocking primitives: clients send messages to the ZooKeeper service and, in principle, are always immediately returned a response. In the case of locking, this means that a client will, for example, be informed whether it was able to grab a lock. If the lock could not be acquired, a new attempt will be necessary, as we discuss shortly.

To facilitate a range of coordination functions, ZooKeeper maintains a namespace, organized as a tree. Operations on the tree are simple: creating and deleting nodes, as well as reading and updating the data contained in a node (if any). A partial update of a node is not possible: all the node's data will be overwritten in the case of an update. In addition, a client can check with the server whether a node exists. At this point, it should be become clear how one could implement a locking service in ZooKeeper: to acquire a lock, simply let a process create a special node, say `lock`, but have that operation fail if the node already exists. Releasing a lock is done by merely deleting the node `lock`. We return to some important details below.

There are two types of nodes in ZooKeeper. Persistent nodes need to be created and deleted explicitly. In contrast, ephemeral nodes are created explicitly, but are automatically removed if the connection with the creating client closes (perhaps because of a failure), or expires. Of course, an ephemeral node can also be explicitly deleted.

An important consequence of not supporting blocking operations, is that a client will have to regularly check the status of the namespace: have nodes been added or removed, have nodes in the namespace been changed? However, polling is generally not considered to be very efficient. To this end, ZooKeeper supports a notification mechanism by which a client can subscribe to a change in a node or a branch in the tree. When a change happens, the client receives a message.

There are a few things that need to be considered to make this all work. First, suppose a client subscribes to changes at a specific node after having read the state of that node. If the node is updated twice in a row, we want to prevent that the client sees the second update before being notified, as the notification dealt with changes to the *previous* state, i.e., the state after the first update took place.

Second, it should be clear that there can be a lot of concurrency among competing clients. As we shall discuss in detail in Chapter 7, problematic are, in particular, the situations in which one client C_1 reads data to subsequently decide to update that data. If, between the read and intended update another client C_2 performs an update, client C_1 may now request an update based on out-of-date information. Such intended updates can be prevented by using version numbers, as shown in Figure 5.19.

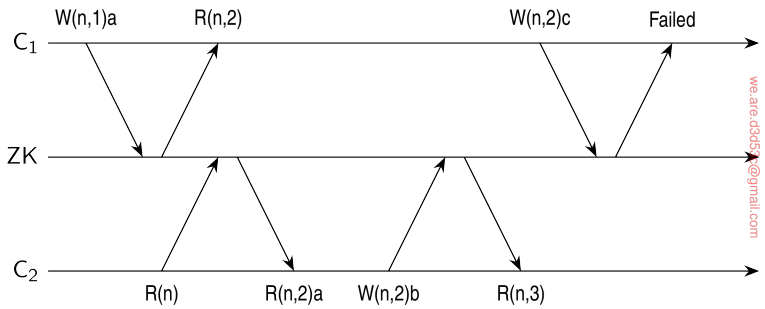


Figure 5.19: Preventing updates based on out-of-date information through versions (based on [Junqueira and Reed, 2014].

We use the notation $W(n,k)a$ to denote the request to write the value a to node n, under the assumption that its current version is k. $R(n,k)$ denotes that the current version of node n is k. The operation $R(n)$ tells that a client wants to read the current value of node n, and $R(n,k)a$ means that the value a from

node n is returned with its current version k. In Figure 5.19 we see that client C_1 writes value a to node n, assuming its version is 1. If no write has ever taken place, this assumption is correct. The ZooKeeper service (ZK) returns the new version number, namely 2.

Meanwhile, client C_2 issues a read request for node n and is returned the previously written value a having version number 2. C_2 decides to update n, assuming its current version is still 2, by means of the write operation W(n, 2)b. ZooKeeper accepts the write, and returns the new version number 3. This concurrent update places C_1 in the position that its intended write operation W(n, 2)c will fail: ZooKeeper states that its assumption about the version number is incorrect. The best what C_1 could do is read n again to see if it still wants to update the node.

Clearly, each client may need to try several times before an update actually takes place. This may not always seem so efficient, yet this approach ensures that data maintained by ZooKeeper is at least consistent with what clients expect it to be. As mentioned, we will return to consistency in much more detail in Chapter 7.

A ZooKeeper locking protocol

Implementing a lock using ZooKeeper is now fairly straightforward. ZooKeeper uses the same path notation for nodes as in Unix file systems. That means that we can create a node /lock at the root of the tree. The existence of that node means that a client has successfully acquired the lock. Releasing the lock is simply done by deleting node /lock.

Any client wanting to create the same node will receive the message that the node already exists and thus that the operation failed. Capturing such exceptions is at the core of blocking a client until /lock is removed. When the node already exists, the client will request ZooKeeper to send a notification when the node is deleted. Until that moment, the client simply waits (i.e., locally blocks) until notified, after which it tries to create the node again.

There are many subtleties to deal with. For example, a decent locking mechanism should allow a client to bail out after several attempts. Likewise, if a client who has created /lock crashes before deleting it again, we need to make sure that ZooKeeper will delete the node. Furthermore, we will find ourselves in an unfortunate race when the following happens:

1. A client C_1 creates a node /lock.
2. A client C_2 wants to acquire the lock but is notified that the associated node already exists.
3. Before C_2 subscribes to a notification, C_1 releases the lock, i.e., deletes /lock.
4. Client C_2 subscribes to changes to /lock and blocks locally.

This is analogous to the situation we described before: C_2 should be able to subscribe to changes based on the *existence* of /lock. In other words, when performing the fourth step, C_2 should be immediately notified that the situation has already changed since it last visited the tree. These subtleties are handled by ZooKeeper.

5.4 Election algorithms

Many distributed algorithms require one process to act as coordinator, initiator, or otherwise perform some special role. In general, it does not matter which process takes on this special responsibility, but one of them has to do it. In this section, we will look at algorithms for **electing a coordinator**. We will often also speak of a leader election.

If all processes are the same, with no distinguishing characteristics, there is no way to select one of them to be special. Consequently, we will assume that each process P has a unique identifier $id(P)$. In general, election algorithms attempt to locate the process with the highest identifier and designate it as coordinator. The algorithms differ in the way they locate the coordinator.

Furthermore, we also assume that every process knows the identifier of every other process. In other words, each process has complete knowledge of the process group in which a coordinator must be elected. What the processes do not know is which ones are currently up and which ones are currently down. The goal of an election algorithm is to ensure that when an election starts, it concludes with all processes agreeing on whom the new coordinator is to be. There are many algorithms and variations, of which several important ones are discussed in the textbooks by Tel [2000] and Lynch [1996].

5.4.1 The bully algorithm

A well-known solution for electing a coordinator is the **bully algorithm** devised by Garcia-Molina [1982]. In the following, we consider N processes $\{P_0, \ldots, P_{N-1}\}$ and let $id(P_k) = k$. When any process notices that the coordinator is no longer responding to requests, it initiates an election. A process, P_k, holds an election as follows:

1. P_k sends an ELECTION message to all processes with higher identifiers: $P_{k+1}, P_{k+2}, \ldots, P_{N-1}$.
2. If no one responds, P_k wins the election and becomes coordinator.
3. If one of the higher-ups answers, it takes over and P_k's job is done.

At any moment, a process can get an ELECTION message from one of its lower-numbered colleagues. When such a message arrives, the receiver sends an OK message back to the sender to indicate that it is alive and will take

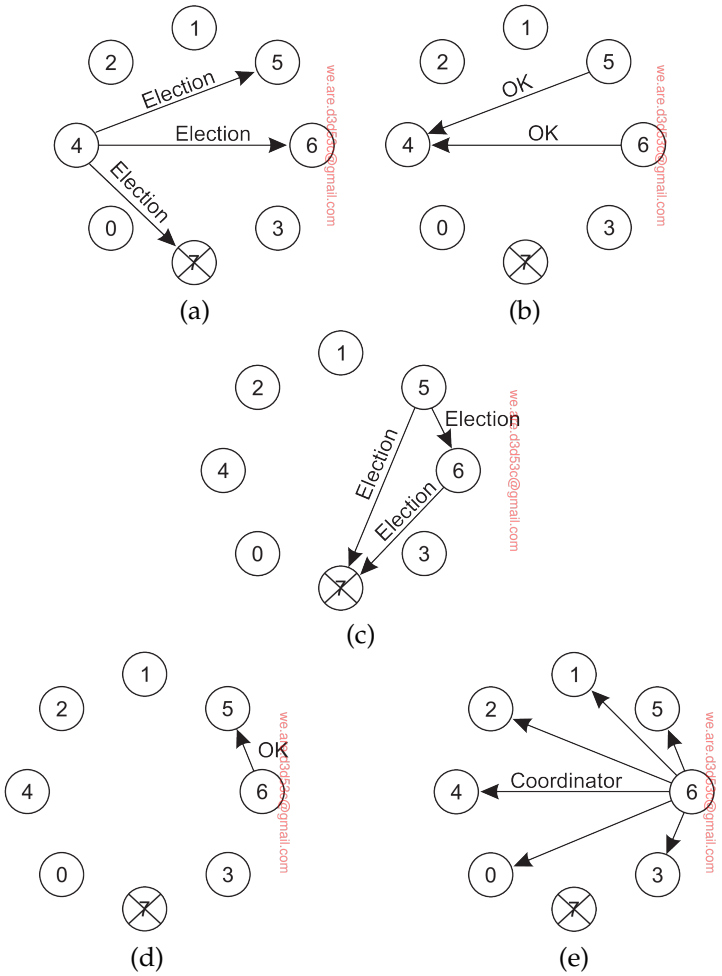


Figure 5.20: The bully election algorithm. (a) Process 4 holds an election. (b) Processes 5 and 6 respond, telling 4 to stop. (c) Now 5 and 6 each hold an election. (d) Process 6 tells 5 to stop. (e) Process 6 wins and tells everyone.

over. The receiver then holds an election, unless it is already holding one. Eventually, all processes give up but one, and that one is the new coordinator. It announces its victory by sending all processes a message telling them that starting immediately it is the new coordinator.

If a process that was previously down comes back up, it holds an election. If it happens to be the highest-numbered process currently running, it will win the election and take over the coordinator's job. Thus the biggest guy in town always wins, hence the name "bully algorithm."

In Figure 5.20 we see an example of how the bully algorithm works. The group consists of eight processes, with identifiers numbered from 0 to 7.

Previously, process P_7 was the coordinator, but it has just crashed. Process P_4 is the first one to notice this, so it sends ELECTION messages to all the processes higher than it, namely P_5, P_6, and P_7, as shown in Figure 5.20(a). Processes P_5 and P_6 both respond with OK, as shown in Figure 5.20(b). Upon getting the first of these responses, P_4 knows that its job is over, knowing that either one of P_5 or P_6 will take over and become coordinator. Process P_4 just sits back and waits to see who the winner will be (although at this point it can make a pretty good guess).

In Figure 5.20(c) both P_5 and P_6 hold elections, each one sending messages only to those processes with identifiers higher than itself. In Figure 5.20(d), P_6 tells P_5 that it will take over. At this point, P_6 knows that P_7 is dead and that it (P_6) is the winner. If there is state information to be collected from disk or elsewhere to pick up where the old coordinator left off, P_6 must now do what is needed. When it is ready to take over, it announces the takeover by sending a COORDINATOR message to all running processes. When P_4 gets this message, it can now continue with the operation it was trying to do when it discovered that P_7 was dead, but using P_6 as the coordinator this time. In this way, the failure of P_7 is handled, and the work can continue.

If process P_7 is ever restarted, it will send all the others a COORDINATOR message and bully them into submission.

5.4.2 A ring algorithm

Consider the following election algorithm that is based on the use of a **(logical) ring**. Unlike some ring algorithms, this one does not use a token. We assume that each process knows who its successor is. When any process notices that the coordinator is not functioning, it builds an ELECTION message containing its own process identifier and sends the message to its successor. If the successor is down, the sender skips over the successor and goes to the next member along the ring, or the one after that, until a running process is located. At each step along the way, the sender adds its own identifier to the list in the message, effectively making itself a candidate to be elected as coordinator.

Eventually, the message gets back to the process that started it all. That process recognizes this event when it receives an incoming message containing its own identifier. At that point, the message type is changed to COORDINATOR and circulated once again, this time to inform everyone else who the coordinator is (the list member with the highest identifier) and who the members of the new ring are. When this message has circulated once, it is removed and everyone goes back to work.

In Figure 5.21 we see what happens if two processes, P_3 and P_6, discover simultaneously that the previous coordinator, process P_7, has crashed. Each of these builds an ELECTION message and each of them starts circulating its message, independent of the other one. Eventually, both messages will go all the way around, and both P_3 and P_6 will convert them into COORDINATOR

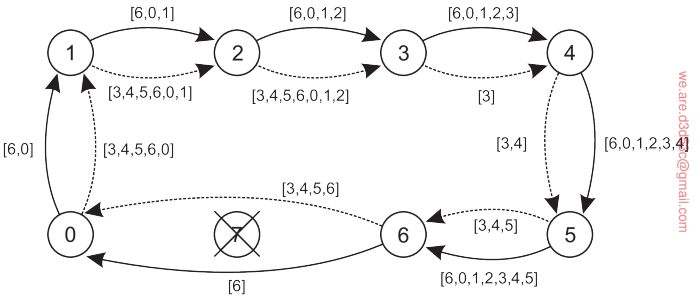


Figure 5.21: Election algorithm using a ring. The solid line shows the election messages initiated by P_6; the dashed one those by P_3.

messages, with exactly the same members and in the same order. When both have gone around again, both will be removed. It does no harm to have extra messages circulating; at worst it consumes a little bandwidth, but this is not considered wasteful.

5.4.3 Example: Leader election in ZooKeeper

Let us look at a practical example of how leader election takes place in a distributed system. As mentioned, ZooKeeper is logically centralized coordination service. In Section 5.3.6 we considered a simple setup with just a single ZooKeeper server. In practice, we see that ZooKeeper maintains a (relatively small) set of servers, forming what is called an **ensemble**. For a client, an ensemble appears as just a single server. For ZooKeeper, this ensemble is also coordinated by a single server, called the leader. The other servers are called followers and essentially act as up-to-date standbys for whenever the leader malfunctions. In that case, one of the followers will be elected as the new leader. From the client's perspective, a malfunctioning leader is mostly invisible: as soon as one of the standbys has taken over the role of leader, everything proceeds as if nothing happened. We return to many details of such fault-tolerant behavior in Chapter 8, but for now, let us dive a bit deeper into how the server in a ZooKeeper ensemble choose their leader.

It is important to note that a leader-election algorithm *within* a ZooKeeper ensemble is fundamentally different from devising a leader-election algorithm *by means of* ZooKeeper. The first deals with the *implementation of ZooKeeper* as a coordination service; the second with *using ZooKeeper* as a coordination service, perhaps as part of the implementation of a larger distributed system.

Which leader-election algorithm is used within a ZooKeeper ensemble is not that important, as long as, in the end, a single leader is chosen and enough followers are running as up-to-date standbys. Enough in this case means that a majority of the servers that form an ensemble are operating properly. For now, we will simply assume this to be the case.

The default election algorithm in an implementation of ZooKeeper is a simple version of bullying and works as follows [Junqueira and Reed, 2014]. Each server s in the ensemble has an identifier *id*(s), as well as a monotonically increasing counter *tx*(s) of the latest transaction it handled. Think of a transaction as a series of operations the server has executed, such as the combination of adding a node and registering that it should send a client a notification when anything changes. Typically, the leader of the ensemble will have performed the transaction on its own, locally stored, namespace and instructed its followers to do so as well on their respective local copy of that namespace.

When a follower s believes something is wrong with the leader (e.g., it suspects that the leader crashed), it sends out an ELECTION message to all other servers, along with the pair (voteID,voteTX). For this first message, it sets voteID to *id*(s) and voteTX to *tx*(s). During an election, each server s maintains two variables. The first, *leader*(s), records the identifier of the server that s believes may turn out to be the final leader, and is initialized to *id*(s). The second, *lastTX*(s) is what s has learned to be the most recent transaction, initially being its own value, namely *id*(s).

When a server s^* receives (voteID,voteTX), it proceeds as follows:

- If $lastTX(s^*) <$ voteTX, then s^* just received more up-to-date information on the most recent transaction. In that case, it sets
  - $leader(s^*) \leftarrow$ voteID
  - $lastTX(s^*) \leftarrow$ voteTX
- If $lastTX(s^*) =$ voteTX *and* $leader(s^*) <$ voteID, then s^* knows as much about the most recent transaction as what it was just sent, but its perspective on which server will be the next leader needs to be update:
  - $leader(s^*) \leftarrow$ voteID

Each time a server s^* receives a (voteID,voteTX) message, it may update its own information on whom it suspects to be the next leader. If s^* believes it should be the next leader, and has not sent out a message stating this, it will broadcast the pair ($id(s^*)$,$tx(s^*)$). Under the assumption that communication is reliable, this broadcast alone should do the job. Typically, s^* will send out a message stating that it is the leader, when either initiating an election, or when receiving a message with a lower voteTX than its own $tx(s^*)$, or receiving a message with an up-to-date value for voteTX, but with a smaller voteID than $id(s^*)$. Once a leader has been elected, all servers will make sure that they synchronize on the latest transaction.

Note 5.6 (Advanced: ZooKeeper leader election in Python)

```
class Process:
  def __init__(self, chanID, procID, procIDSet, initTX):
    self.txID       = initTX       # Your own most recent transaction
    self.leader     = self.procID  # Who you believe may become leader
    self.lastTX     = self.txID    # What is the most recent transaction
    self.noleader   = False        # Are you still in the race for leader?

  def receive(self):
    while True:
      msg             = self.chan.recvFrom(self.otherProcs)
      sender, payload = msg[0], msg[1]
      if payload[0] == ELECTION: # A process started an election
        voteID, voteTX = payload[1], payload[2]

        if self.lastTX < voteTX: # You're not up to date on most recent transaction
          self.leader = voteID   # Record the suspected leader
          self.lastTX = voteTX   # As well as the likely most recent transaction

        elif (self.lastTX == voteTX) and (self.leader < voteID): # Wrong leader
          self.leader = voteID   # Update your suspected leader

        elif (self.procID > voteID) and (self.txID >= voteTX) and (not self.noleader):
          # At this point, you may very well be the new leader (having a sufficiently
          # high process identifier as well as perhaps the most recent transaction).
          # No one has told you so far that you could not be leader. Tell the others.
          self.chan.sendTo(self.otherProcs, (LEADER, self.procID, self.txID))

      if payload[0] == LEADER:
        # Check if the sender should indeed be leader
        if ((self.lastTX < payload[2]) or
            ((self.lastTX == payload[2]) and (self.leader <= payload[1]))):
          # The sender is more up-to-date than you, or is equally up-to-date but
          # has a higher process identifier. Declare yourself follower.
          self.chan.sendTo(sender, (FOLLOWER, self.procID))
        else:
          # Sender is wrong: you have information that the sender based its decision
          # on outdated information
          self.chan.sendTo(sender, (NOLEADER))
```

Figure 5.22: A simplified version of ZooKeeper's leader election for an ensemble of servers.

Coming to the conclusion that one of the servers in a ZooKeeper ensemble is now indeed the new leader, can be a bit tricky. One way is to let a server come to the conclusion that it may never become a leader in the current round, in which case it tells the alleged leader that it will become a follower. This means that as soon as a server has collected enough followers, it can promote itself to leader. In ZooKeeper, this happens when a server has a majority of the other servers in the ensemble as followers. If we take an oversimplified approach by assuming

that messages are never lost, nor do servers crash during an election, a leader is elected when all other servers are its followers. The code is shown in Figure 5.22 (again, omitting many details).

5.4.4 Example: Leader election in Raft

As another example of a leader-election algorithm that is in practical use, let us take a look at **Raft** [Ongaro and Ousterhout, 2014]. We will return to Raft in Chapter 8 when discussing consensus algorithms and focus here only on how it selects a leader.

Raft operates in a setting in which a handful of known replicated servers (typically five) collaborate by ensuring that each server executes the same set of operations, and all in the same order. To this end, one of the servers is elected as leader to tell the others what the exact order is of those operations. The protocol assumes that messages may be lost and that servers may crash.

Each server can be in one of three states: follower, candidate, or leader. Furthermore, the protocol operates in **terms**, where during each term there is exactly one leader, although it could have perhaps crashed. Each term is numbered, starting with 0. The leader is assumed to regularly send out a message, either containing information on an operation that should be carried out, or otherwise a heartbeat message to tell the other servers that their leader is still up and running.

Each server initially starts in the follower state (that is, there is initially no leader). Adopting the terminology from Howard et al. [2015], after a *follower timeout*, a following server concludes that the leader may have crashed (which, by the way, may be a false conclusion). As a result, it enters the candidate state and starts an election, volunteering to be the new leader. An election starts with a broadcast to all other servers, along with increasing the term number by 1. At that point, three situations may happen.

1. A candidate may receive a message from an alleged leader. If that server indicates it is operating in the same term as the candidate server, the latter will become follower again for the current term.
2. When a following server receives an election message for the first time (in a new term), it simply votes for the candidate and ignores any other election messages (for that new term). Therefore, a candidate server can also receive a vote. If it has a majority of votes (i.e., more than half of the servers, including itself), it promotes itself as leader for the new term.
3. As long as there is no alleged leader, or not enough votes have been received, the candidate server waits until a *candidate timeout* happens. At that point, the candidate server will simply start a new election (and again, for a next term).

Of course, if all servers enter the candidate state, we bear the risk of indefinitely not being able to cast enough votes for a single server. A simple solution is to slightly vary the *follower timeout* and *candidate timeout* on a per-server basis. The result is that, generally, there is only a single candidate for the new term, allowing each other server to vote for that candidate and become a follower. As soon as a server becomes leader, it will send out a heartbeat message to the rest.

5.4.5 Elections in large-scale systems

Many leader-election algorithms apply to only relatively small distributed systems. In such cases, the various nodes exchange information among each other, often through several rounds, to eventually collectively decide on the new leader. In Chapter 8, we shall see that this collective decision-making can take place even when a fraction of the participating nodes exhibits nonconforming behavior, as in the case of security attacks or faults. The problem of electing a leader becomes rather nasty when there are potentially many processes to choose from. This is the case, for example, in **permissionless blockchains**.

Proof of work

A relatively simple, yet by now heavily criticized solution, is to have the candidates run a computational race, referred to as **proof of work**. Running such a race, which is done by solving a computational puzzle, should be known to be possible, yet difficult. The first process solving the puzzle, wins, and may proceed as leader to append a block of validated transactions to the existing blockchain.

The type of problem used for blockchains is based on what is known as **hashing**. As introduced in Section 1.2.5, a hash function takes as input a (possibly large) data set and produces a fixed-length string of bits, typically of length 1024 or 2048, called a *hash*. A cryptographically well-defined hash function has several important properties [Ferguson and Schneier, 2003]:

1. Computing the hash of a given data set is relatively easy, i.e., it does not require significant computational efforts.
2. However, given a specific hash, it is computationally very difficult to find a corresponding data set with the same associated hash.
3. With very high probability, any two different input data sets will lead to two seemingly unrelated, different hashes. Even if the two data sets are minimally different, their associated hashes will most likely be very different.

The third property is used to securely protect blocks of validated transactions against adversarial modifications: even the change of a single bit will not go unnoticed. Combined with the second property, it becomes virtually impossible to modify a block meaningfully, such that the modified block's hash is the same as the original block's hash.

To set up a race between validators, each validator computes the hash of its block of validated transactions. Note again that a hash is technically just a fixed-length string of bits. We denote the computed hash over a block a **digest**. The validator is then required to find an *additional* bit string, called a **nonce**, such that the hash computed over the digest and the nonce when taken together produces a bit string with a predetermined number of leading zeroes. Given property 2 of hash functions, we know that it is computationally very difficult to find a data set that matches a given hash. In our case, we have as input the hash of the block of validated transactions (i.e., the digest), and are now required to find a bit string (i.e., nonce) that, taken together with the digest, has an associated hash that starts with a given number of zeroes. This is computationally very difficult. In essence, each validator simply needs to go through a most likely lengthy trial-and-error process of generating a nonce and checking whether that nonce, combined with the digest, will lead to the required result.

By controlling how many leading zeroes that the outcome should have, we essentially control the difficulty of the computations. For example, with just 1 leading zero, there is a 50% chance that a generated nonce will lead to the desired result. Demanding 2 leading zeroes reduces this probability to 25%, 3 leading zeroes to 12.5%, and so forth. With 64 leading zeroes, which is common practice, the chance that an arbitrarily chosen nonce will do the trick is a mere 0.0000 0000 0000 0000 05% (that is, 17 zeroes after the decimal dot). Put differently, a validator will on average need to check about 18 billion billion nonces to find one that leads to the desired result. Using dedicated supercomputers, this takes about 10 minutes. It would take an average laptop about 100 years.

With increasing hardware capacity, the time it takes to find a nonce will drop. For this reason, the difficulty of finding a nonce is deliberately controlled in such a way that a race will have an expected duration. For Bitcoin systems, a popular application of blockchains, the duration is approximately 10 minutes. If races tend to become shorter, then the difficulty for finding a nonce is increased. If races turn out to be too lengthy, the difficulty is lowered. Adjusting the difficulty is done by regularly computing the average time it took to run the race over the last 2000 blocks of transactions, or so. The adjustment of the difficulty is done by all validators using the same globally known method, and is therefore done in a completely decentralized manner.

There is a reason for properly setting the expected duration of a race. Suppose we have very lengthy races. This means that validators would have

to do a lot of work to find a proper nonce. The good part of the story is that the chance that two validators find a nonce at more or less the same time is relatively small (although this depends, of course, on how many validators participate in the race). As a consequence, the chance that there will be only one winner who will have enough time to successfully broadcast its victory to all others is relatively high.

The downside of the story, however, is that with lengthy races, the transaction processing capacity may be too low. Suppose that we have a race duration of 10 minutes and an average number of 2500 transactions per block (which are common values for Bitcoin). This means that, effectively, the system can handle about 4 transactions per second. Increasing the number of transactions per block is an obvious path to follow, yet even such improvements still lead to low rates in comparison to many centralized solutions with a fixed, trusted third party validating transactions.

On the other hand, assume we would have very short-lasting races. In that case, two validators may more easily concurrently find a proper nonce, both will declare victory, append their block to the blockchain (which was already massively replicated), and before other validators had the chance to stop their work, we would find ourselves with different copies of the same blockchain. There are different ways to correct for these concurrent versions of what is supposed to be a single blockchain, but obviously, it comes at the price of invalidating potentially many transactions, and rolling back to a valid state. Of course, the advantage of a smaller race duration is that, in principle, we can handle more transactions per time unit.

Proof of stake

As a reaction to the waste of computational resources found in proof-of-work systems, much effort has been spent on alternative leader-election algorithms for permissionless blockchains. An important class is formed by so-called **proof-of-stake** systems. Leaving out many details, as well as the many variants of such systems, the basic principle is as follows. An easy-to-read overview of the principles of proof-of-stake leader election is provided by Nguyen et al. [2019].

First, we need to make the assumption that each transaction as recorded in a blockchain has one or more associated tokens. Moreover, at each moment, each token has exactly one owner. This is typically the case with monetary transactions, where a token is associated with a digital coin. Because blockchains are fully readable by any participant, we may also assume that tokens may be *passed* between owners, and that, indeed, *copying* a token and associating it with multiple owners cannot go undetected. How this can be realized is discussed in Section 9.4.3. Each token will, therefore, have only a single owner. Second, we make also make the (realistic) assumption that for

a given blockchain, there are a total of N tokens, which may vary over time depending on the application for which a blockchain is being used.

A simple leader-election process for blockchains then consists of a function that generates a random number k between 1 and N. That number is used as an index of the kth token, of which the owner becomes the next leader. The function used to generate a random number is public, meaning that any participant can execute it to determine k. Obviously, the more tokens a process P owns, the higher the probability that P will be elected as leader.

The simplicity of this algorithm and the fact that there are still so many race-based blockchains, indicates that there are many issues to deal with before proof-of-stake solutions can actually work. To understand, we need to realize that with proof of work, there is a price to pay to become leader, namely using considerable computational resources. If that price is compensated by a reward (namely a transaction fee), we have some sort of balance. In many proof-of-stake systems, there may also be a reward, but there is no price to be paid in advance for becoming a leader. As a result, these systems are generally more vulnerable to security attacks, for example, by letting participants generate invalid transactions that will disrupt the work of honest leaders.

Leader election in blockchains is part of reaching consensus on the status of a blockchain. We return to these matters in Section 8.2.6.

Note 5.7 (Advanced: Selecting superpeers)
Many leader-election algorithms often concentrate on the selection of only a single node. There are situations when several nodes should actually be selected, such as in the case of **super peers** in peer-to-peer networks, which we discussed in Section 2.4.3.

The following requirements need to be met for super-peer selection (see also [Lo et al., 2005]):

1. Normal nodes should have low-latency access to super peers.
2. Super peers should be evenly distributed across the overlay network.
3. There should be a predefined portion of super peers relative to the total number of nodes in the overlay network.
4. Each super peer should not need to serve more than a fixed number of normal nodes.

Fortunately, these requirements are relatively easy to meet in most peer-to-peer systems, given the fact that the overlay network is either structured (as in DHT-based systems), or randomly unstructured (as, for example, can be realized with gossip-based solutions). Let us take a look at solutions proposed by Lo et al. [2005].

In the case of DHT-based systems, the basic idea is to reserve a fraction of the identifier space for super peers. In a DHT-based system, each node receives a random and uniformly assigned m-bit identifier. Now suppose we reserve the

first (i.e., leftmost) k bits to identify super peers. For example, if we need N superpeers, then the first $\lceil log_2(N) \rceil$ bits of any *key* can be used to identify these nodes.

To explain, assume we have a (small) Chord system with $m = 8$ and $k = 3$. When looking up the node responsible for a specific key K, we can first decide to route the lookup request to the node responsible for the pattern K $\wedge$ 11100000 which is then treated as the superpeer. (We use the binary operator "$\wedge$" to denote a bitwise and.) Note that each node with identifier ID can check whether it is a super peer by looking up ID $\wedge$ 11100000 to see if this request is routed to itself. Provided node identifiers are uniformly assigned to nodes, it can be seen that with a total of N nodes the number of super peers is, on average, equal to $2^{k-m}N$.

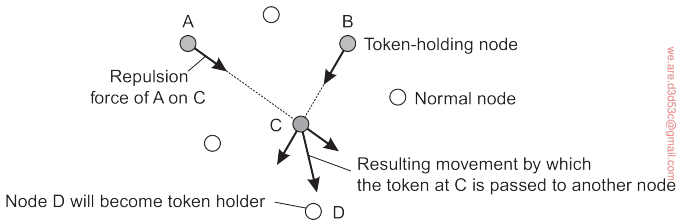


Figure 5.23: Moving tokens in a two-dimensional space using repulsion forces.

A different approach is based on positioning nodes in an m-dimensional geometric space. In this case, assume we need to place N super peers *evenly* throughout the overlay. The basic idea is simple: a total of N tokens are spread across N randomly chosen nodes. No node can hold more than one token. Each token represents a repelling force by which another token is inclined to move away. The net effect is that if all tokens exert the same repulsion force, they will move away from each other and spread themselves evenly in the geometric space.

This approach requires that nodes holding a token learn about other tokens. To this end, we can use a gossiping protocol, by which a token's force is disseminated throughout the network. If a node discovers that the total forces that are acting on it exceed a threshold, it will move the token in the direction of the combined forces, as shown in Figure 5.23. When a token is held by a node for a given amount of time, that node will promote itself to superpeer.

5.4.6 Elections in wireless environments

Traditional election algorithms are generally based on assumptions that are not realistic in wireless environments. For example, they assume that message passing is reliable and that the topology of the network does not change. These assumptions are false in most wireless environments, especially when dealing with mobile devices.

Only few protocols for elections have been developed that work in mobile environments. Vasudevan et al. [2004] propose a solution that can handle

failing nodes and partitioning networks. An important property of their solution is that the *best* leader can be elected rather than just a random one as was more or less the case in the previously discussed solutions. Their protocol works as follows. To simplify our discussion, we concentrate only on ad hoc networks and ignore that nodes can actually move. This may be the case, for example, in sensor networks.

Consider such a wireless ad hoc network. To elect a leader, any node in the network, called the source, can initiate an election by sending an ELECTION message to its immediate neighbors (i.e., the nodes in its range). When a node receives an ELECTION for the first time, it designates the sender as its parent, and subsequently sends out an ELECTION message to all its immediate neighbors, except for the parent. When a node receives an ELECTION message from a node besides its parent, it merely acknowledges the receipt.

When node R has designated node Q as its parent, it forwards the ELECTION message to its immediate neighbors (excluding Q) and waits for acknowledgments to come in before acknowledging the ELECTION message from Q. This waiting has an important consequence. First, note that neighbors who have already selected a parent will immediately respond to R. More specifically, if all neighbors already have a parent, R is a leaf node and will be able to report back to Q quickly. In doing so, it will also report information such as its battery lifetime and other resource capacities.

This information will later allow Q to compare R's capacities to that of other downstream nodes, and select the best eligible node for leadership. Of course, Q had sent an ELECTION message only because its own parent P had done so as well. In turn, when Q eventually acknowledges the ELECTION message previously sent by P, it will pass the most eligible node to P as well. In this way, the source will eventually get to know which node is best to be selected as leader, after which it will broadcast this information to the rest.

This process is illustrated in Figure 5.24. Nodes have been labeled a to j, along with their capacity. Node a initiates an election by broadcasting an ELECTION message to nodes b and j, as shown in Figure 5.24(b) After that step, ELECTION messages are propagated to all nodes, ending with the situation shown in Figure 5.24(e), where we have omitted the last broadcast by nodes f and i. From there on, each node reports to its parent the node with the best capacity, as shown in Figure 5.24(f). For example, when node g receives the acknowledgments from its children e and h, it will notice that h is the best node, propagating $[h, 8]$ to its own parent, node b. In the end, the source will note that h is the best leader and will broadcast this information to all other nodes.

When multiple elections are initiated, each node will decide to join only one election. To this end, each source tags its ELECTION message with a unique identifier. Nodes will participate only in the election with the highest identifier, stopping any running participation in other elections.

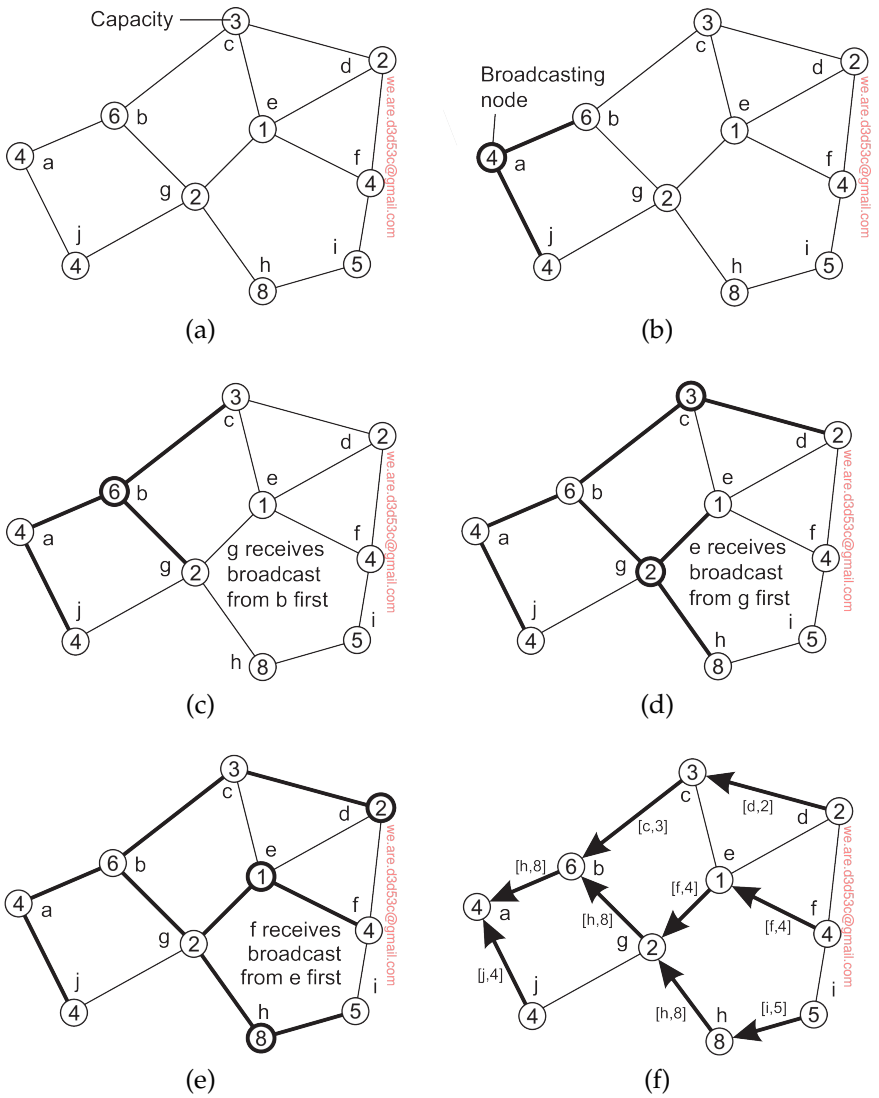


Figure 5.24: Election algorithm in a wireless network, with node a as the source. (a) Initial network. (b)–(e) The build-tree phase (last broadcast step by nodes f and i not shown). (f) Reporting of best node to source.

With some minor adjustments, the protocol can be shown to operate also when the network partitions, and when nodes join and leave. The details can be found in Vasudevan et al. [2004].

5.5 Gossip-based coordination

As a next topic in coordination, we take a look at a few important examples in which gossiping is deployed. In the following, we look at aggregation, large-scale peer sampling, and overlay construction, respectively.

5.5.1 Aggregation

Let us take a look at some interesting applications of **epidemic protocols**. We already mentioned spreading updates, which is perhaps the most widely deployed application. In the same light, gossiping can be used to discover nodes that have a few outgoing wide-area links, to subsequently apply directional gossiping.

Another interesting application area is simply collecting, or actually aggregating information [Jelasity et al., 2005]. Consider the following information exchange. Every node P_i initially chooses an arbitrary number, say v_i. When node P_i contacts node P_j, they each update their value as:

$$v_i, v_j \leftarrow (v_i + v_j)/2$$

Obviously, after this exchange, both P_i and P_j will have the same value. In fact, it is not difficult to see that eventually all nodes will have the same value, namely the average of all initial values. Propagation speed is again exponential.

What use does computing the average have? Consider the situation that all nodes P_i have set v_i to zero, except for P_1 who has set v_1 to 1:

$$v_i \leftarrow \begin{cases} 1 & \text{if } i = 1 \\ 0 & \textit{otherwise} \end{cases}$$

If there are N nodes, then eventually each node will compute the average, which is $1/N$. As a consequence, every node P_i can estimate the size of the system as being $1/v_i$.

Computing the average may be difficult when nodes regularly join and leave the system. One practical solution to this problem is to introduce epochs. Assuming that node P_1 is stable, it simply starts a new epoch now and then. When node P_i sees a new epoch for the first time, it resets its own variable v_i to zero and starts computing the average again.

Of course, other results can also be computed. For example, instead of having a fixed node such as P_1 start the computation of the average, we

can easily pick a random node as follows. Every node P_i initially sets v_i to a random number from the same interval, say $(0, 1]$, and also stores it permanently as m_i. Upon an exchange between nodes P_i and P_j, each change their value to:

$$v_i, v_j \leftarrow \max\{v_i, v_j\}$$

Each node P_i for which $m_i < v_i$ will lose the competition for being the initiator in starting the computation of the average. In the end, there will be a single winner. Of course, although it is easy to conclude that a node has lost, it is much more difficult to decide that it has won, as it remains uncertain whether all results have come in. The solution to this problem is to be optimistic: a node always assumes it is the winner until proven otherwise. At that point, it simply resets the variable it is using for computing the average to zero. Note that by now, several computations (in our example, computing a maximum and computing an average) may be executing simultaneously.

5.5.2 A peer-sampling service

An important aspect in epidemic protocols is the ability of a node P to choose another node Q *at random* from all available nodes in the network. When giving the matter some thought, we may actually have a serious problem: if the network consists of thousands of nodes, how can P ever pick one of these nodes at random without having a complete overview of the network? For smaller networks, one could often resort to a central service that had registered every participating node. Obviously, this approach can never scale to large networks.

A solution is to construct a fully decentralized **peer-sampling service**, or **PSS** for short. As it turns out, and somewhat counter-intuitive, a PSS can be built using an epidemic protocol. As explored by Jelasity et al. [2007], each node maintains a list of c neighbors, where, ideally, each of these neighbors represents a randomly chosen *live* node from the current set of nodes. This list of neighbors is also referred to as a **partial view**. There are many ways to construct such a partial view. In the solution of Jelasity et al., it is assumed that nodes regularly exchange entries from their partial view. Each entry identifies another node in the network, and has an associated age that indicates how old the reference to that node is. Two threads are used, as shown in Figure 5.25.

The different selection operations are specified as follows:

- `selectPeer`: Randomly select a neighbor from the local partial view
- `selectToSend`: Select some other entries from the partial view, and add to the list intended for the selected neighbor.
- `selectToKeep`: Add received entries to the partial view, remove repeated items, and shrink the view to c items.

```
1 selectPeer(&Q);
2 selectToSend(&bufs);
3 sendTo(Q, bufs);                  ⟶   3 receiveFromAny(&P, &bufr);
4                                       4 selectToSend(&bufs);
5 receiveFrom(Q, &bufr);            ⟵   5 sendTo(P, bufs);
6 selectToKeep(p_view, bufr);           6 selectToKeep(p_view, bufr);
```

(a) (b)

Figure 5.25: Communication between the (a) active and (b) passive thread in a peer-sampling service.

The active thread takes the initiative to communicate with another node. It selects that node from its current partial view. It continues by constructing a list containing $c/2 + 1$ entries, including an entry identifying itself. The other entries are taken from the current partial view. After sending the list to the selected neighbor, it waits for a response.

That neighbor, meanwhile, will also have constructed a list through the passive thread shown in Figure 5.25(b) whose activities strongly resemble that of the active thread.

The crucial point is the construction of a new partial view. This view, for contacting as well as for the contacted peer, will contain exactly c entries, part of which will come from the received list. In essence, there are two ways to construct the new view. First, the two nodes may decide to discard the entries that they had sent to each other. Effectively, this means that they will *swap* part of their original views. The second approach is to discard as many *old* entries as possible (meaning, in practice, that after every gossiping round, the age of each entry in every partial view is incremented by one).

As it turns out, as long as peers regularly run the exchange algorithm just described, selecting a random peer from a thus dynamically changing partial view, is statistically indistinguishable from randomly selecting a peer from the entire network. Of course, selecting a peer from a partial view should occur at approximately the same frequency as the refreshing of partial views. We have thus constructed a fully decentralized gossip-based peer-sampling service. A simple and often-used implementation of a peer-sampling service is Cyclon [Voulgaris et al., 2005].

5.5.3 Gossip-based overlay construction

Although it would seem that structured and unstructured peer-to-peer systems form strict independent classes, this need actually not be the case (see also Castro et al. [2005]). One key observation is that by carefully exchanging and selecting entries from partial views, it is possible to construct and maintain specific topologies of overlay networks. This topology management is achieved by adopting a two-layered approach, as shown in Figure 5.26.

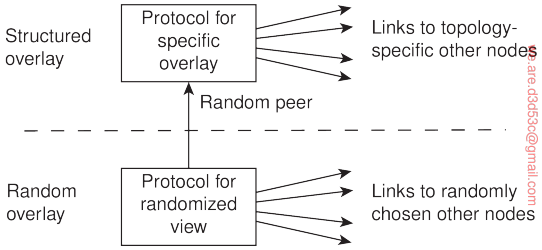


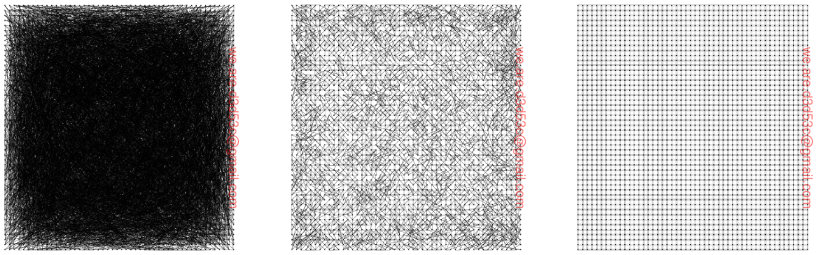
Figure 5.26: A two-layered approach for constructing and maintaining specific overlay topologies using techniques from unstructured peer-to-peer systems.

The lowest layer constitutes an unstructured peer-to-peer system in which nodes periodically exchange entries of their partial views with the aim to provide a peer-sampling service. Accuracy in this case refers to the fact that the partial view should be filled with entries referring to randomly selected *live* nodes.

The lowest layer passes its partial view to the higher layer, where an additional selection of entries takes place. This then leads to a second list of neighbors corresponding to the desired topology. Jelasity and Kermarrec [2006] propose to use a *ranking function* by which nodes are ordered according to some criterion relative to a given node. A simple ranking function is to order a set of nodes by increasing distance from a given node P. In that case, node P will gradually build up a list of its nearest neighbors, provided the lowest layer continues to pass randomly selected nodes.

As an illustration, consider a logical grid of size $N \times N$ with a node placed on each point of the grid. Every node is required to maintain a list of c nearest neighbors, where the distance between a node at (a_1, a_2) and (b_1, b_2) is defined as $d_1 + d_2$, with $d_i = \min(N - |a_i - b_i|, |a_i - b_i|)$. If the lowest layer periodically executes the protocol as outlined in Figure 5.25, the topology that will evolve is a two-dimensional torus, shown in Figure 5.27. In this example, $N = 50$ and we show the results after first initializing each list with random entries. Figure 5.27(b) shows the results after five rounds of exchanges, while the final result, shown in Figure 5.27(c) is achieved after 20 rounds.

Note 5.8 (Advanced:Gossip-based overlay construction in Python)
To better appreciate and understand the construction of overlay networks through gossiping, let us see what a concrete implementation in Python would look like. As we have mentioned, the principle is simple: we have two separate layers and each node is responsible for exchanging links with selected neighbors. A straightforward implementation would seem to be to use a separate thread per layer, as suggested in Figure 5.26. However, such a choice will instantly lead to potential deadlock problems. Suppose node A decides to exchange links with



(a) (b) (c)

Figure 5.27: Generating a torus overlay network using a two-layered unstructured peer-to-peer system.

node B, but B has chosen another node, say C for exchanging links. If C chooses A for exchanging links, we may have a deadlock if these three nodes first collectively choose their respective neighbor for exchanging links, and then collectively wait for that neighbor to respond.

The general solution to this problem is to essentially avoid **synchronous request-response behavior**, but instead to use **asynchronous, persistent communication**. In other words, we let the middleware store a message before it is delivered, after which the sender can continue doing other things. This type of communication is precisely supported by the `channel` package used for many of our Python examples. The outline of the code is shown in Figure 5.28. (The complete code is readily available as accompanying online material for the book.)

Note that for each layer, the approach is essentially always the same: select a peer, decide on the links to exchange, and send that information asynchronously to the selected peer. Thereafter, a node that initiated an exchange will have to wait for a response. When that comes in, it merely needs to update its own view. (There are many subtleties that need to be considered, which we will discuss shortly.) In principle, there will be a regular call to `maintainViews()`, if only to ensure that a node will also react to incoming exchange requests. Note that in our solution, we simply let a node ignore an incoming exchange request when it has an outstanding initiated exchange itself. Ignoring a request is done by telling the initiating node to give up its own attempt. Although this approach will prevent deadlocks, it bears the risk that no exchange will take place at all.

As mentioned, the actual code for handling gossip-based overlay construction involves more than what we have shown in Figure 5.28. Crucial are the criteria for selecting a neighboring peer, as well as the selection of links to exchange. As explained by Voulgaris and van Steen [2013], there are several things that can be done to speed up convergence of the overlay in comparison to random choices. At the same time, including random choices is essential to achieve convergence to the requested overlay.

In the first place, at the top layer for constructing the overlay, links should be exchanged that make most sense for the receiving peer. In our example, this

means that when a node A sends links to a node B, it should select the ones to neighbors that are closest to B. Furthermore, B should not return links that A had just sent: it is much better that A learns about new nodes first.

```
def maintainViews():
  for viewType in [viewOverlay, viewPSS]: # For each view, do the same
    peer[viewType] = None
    if time to maintain viewType: # This viewType needs to be updated
      peer[viewType] = selectPeer(viewType)            # Select a peer
      links = selectLinks(viewType, peer[viewType])    # Select links
      sendTo(peer[viewType], Request[viewType], links) # Send links asynchronously

  while True:
    block = (peer[viewOverlay] != None) or (peer[viewPSS] != None)
    sender, msgType, msgData = recvFromAny(block) # Block if expecting something

    if msg == None: # All work has been done, simply return from the call
      return

    for viewType in [viewOverlay, viewPSS]: # For each view, do the same
      if msgType == Response[viewType]:  # Response to previously sent links
        updateOwnView(viewType, msgData) # Just update the own view

      elif msgType == Request[viewType]: # Request for exchanging links
        if peer[viewType] == None:       # No outstanding exchange request
          links = selectLinks(viewType, sender)     # Select links
          sendTo(sender, Response[viewType], links) # Send them asynchronously
          updateOwnView(viewType,msgData)           # Update own view
        else: # This node already has a pending exchange request, ignore this one
          sendTo(sender, IgnoreRequest[viewType])

      elif msgType == IgnoreRequest[viewType]: # Request has been denied, give up
        peer[viewType] = None
```

Figure 5.28: Pseudocode in Python for implementing a gossip-based approach toward overlay construction.

Moreover, when A selects a peer, we should try to prevent selecting one with which a recent exchange has taken place. In other words, A should try to maximize diversity in selecting a peer. This diversity can be obtained by selecting peers in a round-robin fashion. This approach also has the benefit that in a dynamic network, nodes that have left are guaranteed to be discovered quickly and can thus be removed from a view.

If we were to use only the top layer, the greediness of exchanging only best links may lead to a situation that a collection of nodes will jointly form a closed set within the network: jointly, they are referring to each other and as such, may be missing out on better nodes. This is the reason introducing randomness is essential, and why we need to incorporate information from the lower random overlay in the selection process of the higher structured overlay.

This aspect is completely analogous to getting stuck in a local optimum when optimizing through hill-climbing procedures. What is needed is to introduce some randomness to ensure that indeed a global optimum will eventually be reached.

5.5.4 Secure gossiping

Gossiping is an attractive means for coordination. However, the speed by which a large collection of nodes manages to synchronize, is also an inherent vulnerability. The time it takes to exchange and disseminate proper information is also the time it takes to spread false information. This is perhaps best illustrated by means of an example. Consider a peer-sampling service for some large network. Assume that each node maintains a partial view of size c, and that, on average, $c/2$ references to neighboring nodes are exchanged. Let us also assume that there are c colluding attackers. Their behavior is simple: each attacker also maintains a partial view, but when returning $c/2$ references, only references to other colluding attackers are returned. The effect is that, gradually, the partial view of each benign node in the network is polluted by references to the attackers, to the extent that each partial view contains *only* references to attackers.

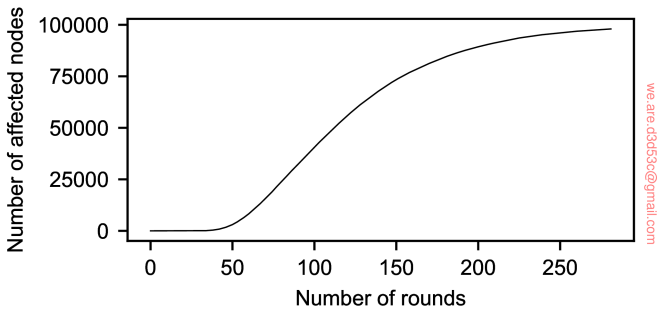


Figure 5.29: The speed by which a hub attack can take effect.

The effect can be dramatic. Figure 5.29 shows the speed by which partial views are fully polluted, that is, each contains references only to attackers. In this example, we consider a network of 100,000 nodes, a partial view size of 30, and only 30 attackers. After less than 300 rounds, all benign nodes have fully polluted partial views. Also note that, completely consistent with the push-pull behavior of epidemic protocols, it may take a while before any serious effect is seen. However, as soon as a few hundred nodes are completely

polluted, they increasingly contribute to contaminating the rest of the nodes, and so on.

It can be argued that there is no standard security measure that can combat these kinds of attacks. For example, even with using secure channels by which both parties have genuinely authenticated each other, there is no reason to assume that both of them will also behave as they should. In other words, authentication provides no guarantees for trust. In the case of gossiping systems, security often comes in the form of trying to detect and prevent malicious behavior. For our example, the question is how can a benign node can detect that it is under attack, and which countermeasures can it take to mitigate the effects of an attack?

A solution in the case of peer sampling, and developed by Jesi et al. [2010], works as follows. The essence is to discover if there are nodes that behave badly. The attack we just described is characterized by the fact that, eventually, many nodes will be referencing a very small of set of nodes. In terms of graphs, this means that the *indegree* of some nodes is extraordinarily high, while for most nodes, their indegree will be low, and actually 0. An indegree of 0 means that a node is unknown to any other node in the network.

When using Cyclon [Voulgaris et al., 2005] as the peer-sampling service, we know that the indegrees follow a normal distribution, as shown in Figure 5.30(a). In this example, the y-axis shows the fraction of nodes with a specific indegree value. Even with a mere 30 colluding attackers, we already see that after 10 rounds, the indegree distribution has started to becoming wider and shifting to the left, as shown in Figure 5.30(b). This continues, and after 40 rounds (Figure 5.30(e)), we already have that about 15% of all nodes have an indegree of 0. (Note that for Figure 5.29, we were counting the number of nodes that were referencing *only* attacking nodes, which is different from measuring indegrees. An attacked node can still be referenced by others.) Finally, after less than 300 rounds, virtually all nodes have an indegree of 0; the attackers have now full control over the entire network.

Figure 5.30 also gives us a hint toward a solution. As explained by Jesi et al. [2010], each node can gather statistics on which nodes are being referenced. If certain nodes are gradually being referenced unusually more often than others, then something fishy may be going on. There are a few issues we need to deal with. First, spreading of "bad" links can be so efficient, that we need to consider that once a benign node has come to the conclusion that it has identified an attacker, it may already be too late. Second, we also want to prevent that every benign node has to build up complete knowledge of the entire network, partly also because attackers have to be identified quickly. Third, attackers do not want to be caught, i.e., be recognized as acting maliciously, as they collectively do need some time to be successful.

Notably the last aspect, not wanting to be caught, provides a solution to mitigating an attack. In essence, a benign node can force an attacker to behave

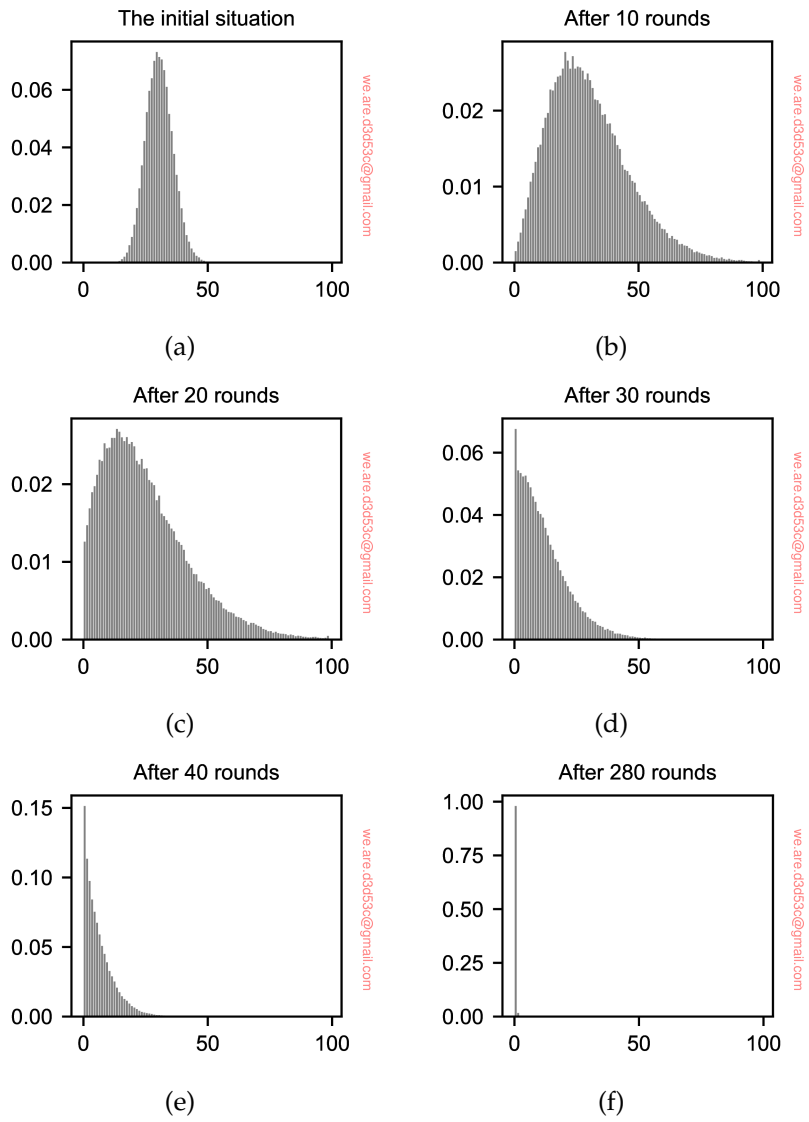


Figure 5.30: The development of the indegree distribution at several stages of a peer-sampling service under attack. Each x-axis shows the indegree, and the y-axis the fraction of nodes with that degree.

according to the rules. What happens is that when a benign node initiates an exchange of links with a selected node, it will process the returned links either for updating its own partial view, or for gathering statistics (it may do both, but this may not be on par with efficient resource usage). Moreover, to accelerate the process of discovering attackers, a node will initiate several link exchanges simultaneously (and, in the end, process only one of the replies). A malicious node is now faced with a dilemma: always returning references to its colluders will quickly reveal the attackers. The only thing that it can do, is occasionally play by the rules and return links to other, benign nodes. As it turns out, this dilemma is enough to force the attackers to behave decently enough, effectively rendering an attack useless. Note that this procedure will also be applied when an *attacker* initiates an exchange: the recipient can decide to use the links passed to it for updating its statistics on indegrees, as well as use it for running the exchange protocol. The attacker will, however, never know whether the recipient is going to collect statistics. To fly under the radar, it will have to behave according to the rules of the game enough in order not to be caught. The details can be found in [Jesi et al., 2010].

There are many other types of attacks possible. For example, attackers may decide to initiate information exchange at a much higher frequency than specified by the protocol. With a push-based approach, this may result in attackers rapidly polluting values of benign nodes, which are then further disseminated. Mousazadeh and Ladani [2015] come to the conclusion that for such attacks, in principle, only pulling should be allowed (as the benign nodes will then collectively dictate the pace at which information is disseminated).

Likewise, in the case of aggregation, attackers may deliberately return incorrect values. To illustrate, suppose that when collectively computing the average value as we described in Section 5.5.1, an attacker A always returns its own value v_A. Every node v_i communicating with A will therefore compute

$$v_i \leftarrow (v_i + v_A)/2$$

which gradually converges to the value v_A. In all these cases, attacks can be mitigated only by trying to detect if a node is not behaving according to the protocol. That detection mechanism, of course, should be known to the attacker, while at the same time will be effective if it forces an attacker to try to go ahead unnoticed. In turn, if well designed, the mechanism will force the attacker to actually behave well, or at least to the extent that any possible damage remains limited. More on how these **data-injection attacks** work and can be mitigated, are described by Gentz et al. [2016].

5.6 Distributed event matching

Let us now draw our attention to distributed event matching. Event matching, or more precisely, **notification filtering**, is at the heart of **publish-subscribe systems**. The problem boils down to the following:

- A process specifies through a subscription S in which events it is interested.
- When a process publishes a notification N on the occurrence of an event, the system needs to see if S *matches* N.
- In the case of a match, the system should send the notification N, possibly including the data associated with the event that took place, to the subscriber.

As a consequence, we need to facilitate at least two things: (1) matching subscriptions against events, and (2) notifying a subscriber in the case of a match. The two can be separated, but this need not always be the case. In the following, we assume the existence of a function *match*(S, N) which returns true when subscription S matches the notification N, and false otherwise. As we argue below, publish-subscribe systems turn out to face difficult scalability problems when the matching function cannot be easily distributed across multiple servers. This is often the case when subscriptions can be expressive, as in content-based publish-subscribe systems. We discuss scalability of distributed event matching separately below.

Another, rather nasty problem, is combining security and privacy while keeping publishers and subscribers mutually unaware of each other, but also ensuring that servers have only enough information available to perform their matching function. In this sense, it may seem that the goals of publish-subscribe systems, and those of secure distributed systems conflict. We return to this issue below as well.

5.6.1 Centralized implementations

A simple, naive implementation of event matching is to have a fully centralized server that handles all subscriptions and notifications. In such a scheme, a subscriber simply submits a subscription, which is subsequently stored. When a publisher submits a notification, that notification is checked against each and every subscription, and when a match is found, the notification is copied and forwarded to the associated subscriber.

Obviously, this is not a very scalable solution. Nevertheless, provided the matching can be done efficiently and the server itself has enough processing power, the solution is feasible for many cases. For example, using a centralized server is the canonical solution for implementing **Linda tuple spaces**. Likewise, many publish-subscribe systems that run within a single department or organization can be implemented through a central server. Important, in these cases, is that the matching function can be implemented efficiently. In practice, this is often the case when dealing with topic-based filtering: matching then resorts to checking for equality of attribute values.

Note that a simple way to scale up a centralized implementation, is to deterministically divide the work across multiple servers. A standard approach is to make use of two functions, as explained by Baldoni et al. [2009]:

- a function *sub2node*(S), which takes a subscription S and maps it to a nonempty subset of servers
- a function *not2node*(N), which takes a notification N and maps it to a nonempty subset of servers.

The servers to which *sub2node*(S) is mapped are called the **rendezvous nodes** for S. Likewise, *sub2node*(N) are the rendezvous nodes for N. The only constraint that needs to be satisfied, is that for any subscription S and matching notification N, $sub2node(S) \cap not2node(N) \neq \emptyset$. In other words, there must be at least one server that can handle the subscription when there is a matching notification. In practice, this constraint is satisfied by topic-based publish-subscribe systems by using a hashing function on the names of the topics.

The idea of having a central server can be extended by distributing the matching across multiple servers and dividing the work. The servers, generally referred to as **brokers**, are organized into an overlay network. The issue then becomes how to route notifications to the appropriate set of subscribers. A straightforward way to make sure that notifications reach their subscribers, is to deploy flooding. There are essentially two approaches. First, we store each subscription at every broker, while publishing notifications only at a single broker. The latter will handle identifying the matching subscriptions and subsequently copy and forward the notification. The alternative is to store a subscription only at one broker, while broadcasting notifications to all brokers. In that case, matching is distributed across the brokers, which may lead to a more balanced workload among the brokers.

Note 5.9 (Example: TIB/Rendezvous)
Flooding notifications is used in **TIB/Rendezvous**, of which the basic architecture is shown in Figure 5.31 [TIBCO]. In this approach, a notification is a message tagged with a compound keyword describing its content, such as news.comp.os.books. A subscriber provides (parts of) a keyword, or indicating the messages it wants to receive, such as news.comp.$*$.books. These keywords are said to indicate the **subject** of a message.

Fundamental to its implementation is the use of broadcasting common in local-area networks, although it also uses more efficient communication facilities when possible. For example, if it is known exactly where a subscriber resides, point-to-point messages will generally be used. Each host on such a network will run a **rendezvous daemon**, which takes care that messages are sent and delivered according to their subject. Whenever a message is published, it is multicasted to each host on the network running a rendezvous daemon. Typically, multicasting

is implemented using the facilities offered by the underlying network, such as IP-multicasting or hardware broadcasting.

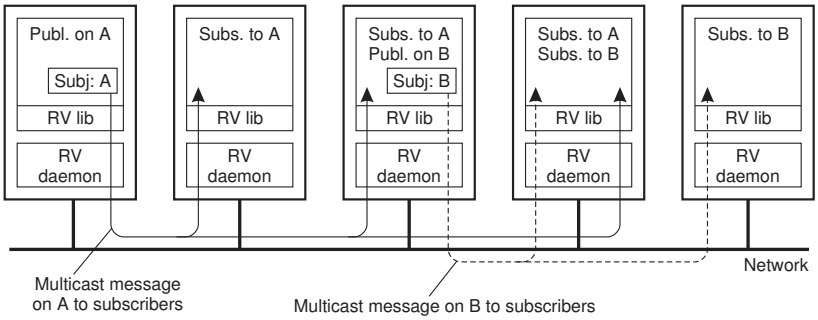


Figure 5.31: The principle of a publish/subscribe system as implemented in TIB/Rendezvous.

Processes that subscribe to a subject pass their subscription to their local daemon. The daemon constructs a table of (*process, subject*), entries and whenever a message on subject S arrives, the daemon simply checks in its table for local subscribers, and forwards the message to each one. If there are no subscribers for S, the message is discarded immediately.

When using multicasting as is done in TIB/Rendezvous, there is no reason why subscriptions cannot be elaborate and be more than string comparison, as is currently the case. The crucial observation here is that because messages are forwarded to every node anyway, the potentially complex matching of published data against subscriptions can be done entirely locally without further network communication needed.

When subscriptions are allowed to be more expressive, scalability may easily become a problem. In practice, expressive subscriptions take the form of (*attribute*,*value*) pairs in which a *value* returns as an expression on the possible values for the specified attribute. Examples include range values ("$1 \leq x < 10$"), containment ("$x \in \{\mathsf{red}, \mathsf{blue}\}$"), prefix and suffix expressions ("`url.startswith("https")`"), etc. More complicated expressions including combinations of conjunctions and disjunctions are, in theory, also possible, thus gradually resembling what one would find in SQL queries for databases. It is not hard to imagine that expressive subscriptions can be handled by centralized brokers, but when scalability is at stake, we may need to limit expressiveness altogether to the point that we find ourselves looking at topic-based event matching.

One way out that has been explored is combining peer-to-peer networks and publish-subscribe systems, as described in Kermarrec and Triantafillou [2013]. As the authors show, there is no simple general solution, and most solutions often have very specific limitations.

Selective routing

If we consider that publish-subscribe is all about routing messages between publishers and subscribers, one thought is to install specific filters in an overlay network that effectively ignore paths toward nodes that are not interested in what is being published. In other words, instead of flooding a network with either subscriptions or publications, we apply **selective routing**, as proposed by Carzaniga et al. [2004]. Consider a publish-subscribe system consisting of N brokers to which clients (i.e., applications) can send subscriptions and retrieve notifications. Carzaniga et al. propose a two-layered routing scheme, in which the lowest layer consists of a shared broadcast tree connecting the N brokers. There are various ways for setting up such a tree, ranging from network-level multicast support to application-level multicast trees, as we discussed in Chapter 4. Here, we also assume that such a tree has been set up with the N brokers as end nodes, along with a collection of intermediate nodes forming routers. Note that the distinction between a server and a router is only a logical one: a single machine may host both kinds of processes.

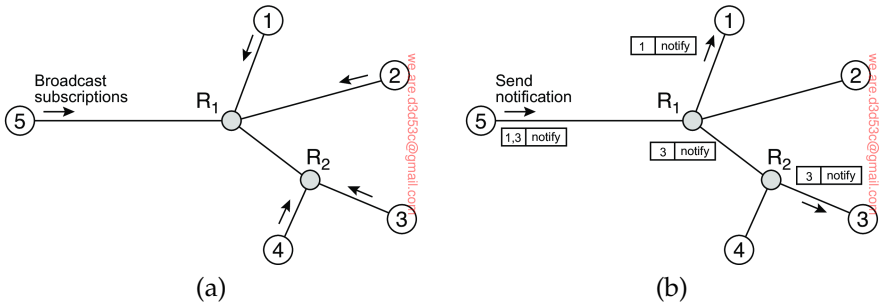


Figure 5.32: A first approach toward content-based routing in which brokers (a) first broadcast subscriptions to later (b) forward notifications only to relevant brokers.

Every broker broadcasts its subscriptions to all other brokers, as shown in Figure 5.32(a). As a result, every broker will be able to compile a list of (*subject*, *destination*) pairs. Then, whenever a process publishes a notification N, its associated broker prepends the destination brokers to that message and forwards the notification to the others, shown in Figure 5.32(b). When the message reaches a router, the latter can use the list to decide on the paths that the message should follow, as shown in Figure 5.32.

We can now refine the capabilities of routers for deciding where to forward notifications. To that end, each broker broadcasts its subscription across the network so that routers can compose **routing filters**. For example, assume that node 3 in Figure 5.32 subscribes to notifications for which an attribute a lies in the range $[0, 3]$, but that node 4 wants messages with $a \in [2, 5]$. In this

case, router R_2 will create a routing filter as a table with an entry for each of its outgoing links (in this case three: one to node 3, one to node 4, and one toward router R_1), as shown in Figure 5.33.

Interface	Filter
To node 3	$a \in [0,3]$
To node 4	$a \in [2,5]$
Toward router R_1	(unspecified)

Figure 5.33: A partially filled routing table.

More interesting is what happens at router R_1. In this example, the subscriptions from nodes 3 and 4 dictate that any notification with a lying in the interval $[0,3] \cup [2,5] = [0,5]$ should be forwarded along the path to router R_2, and this is precisely the information that R_1 will store in its table. It is not difficult to imagine that more intricate subscription compositions can be supported.

This simple example also illustrates that whenever a node leaves the system, or when it is no longer interested in specific notifications, it should cancel its subscription and essentially broadcast this information to all routers. This cancellation, in turn, may lead to adjusting various routing filters. Late adjustments will at worst lead to unnecessary traffic, as notifications may be forwarded along paths for which there are no longer subscribers. Nevertheless, timely adjustments are needed to keep performance at an acceptable level.

Gossiping

Selective routing may help to avoid broadcasting notifications to all routers, but ultimately, it may still not be enough to reach acceptable scalable solutions. Another approach is based on gossiping. The basic idea is that subscribers interested in the same notifications form their own overlay network (which is constructed through gossiping as we describe further below), so that once a notification is published, it merely needs to be routed to the appropriate overlay. For the latter, a **random walk** can be deployed. This approach is following in TERA [Baldoni et al., 2007].

A more sophisticated approach toward combining gossiping and event matching is followed in Sub-2-Sub [Voulgaris et al., 2006]. It is still one of the very few content-based approaches that combines scalability with expressiveness of subscriptions. Consider a publish-subscribe system in which data items can be described by N attributes $a_1, \ldots, a_N$ whose value can be directly mapped to a floating-point number. Such values include, for example, floats, integers, enumerations, Booleans, and strings. A subscription S takes the form of a tuple of (*attribute, value/range*) pairs, such as

$$S = \langle a_1 \rightarrow 3.0, a_4 \rightarrow [0.0,\ 0.5) \rangle$$

In this example, S specifies that a_1 should be equal to 3.0, and a_4 should lie in the interval $[0.0, 0.5)$. Other attributes are allowed to take on any value. For clarity, assume that every node i enters only one subscription S_i.

Note that each subscription S_i actually specifies a subset $\mathbf{S_i}$ in the N-dimensional space of floating-point numbers. Such a subset is also called a **hyperspace**. For the system as a whole, notifications that fall in the union $\overline{\mathbf{S}} = \cup \mathbf{S_i}$ of these hyperspaces are the only ones of interest. The whole idea is to automatically partition $\overline{\mathbf{S}}$ into M disjoint hyperspaces $\overline{\mathbf{S}}_\mathbf{1}, \ldots, \overline{\mathbf{S}}_\mathbf{M}$ such that each falls completely in one of the subscription hyperspaces $\mathbf{S_i}$, and together they cover all subscriptions. More formally, we have that:

$$(\overline{\mathbf{S}}_\mathbf{m} \cap \mathbf{S_i} \neq \emptyset) \Rightarrow (\overline{\mathbf{S}}_\mathbf{m} \subseteq \mathbf{S_i})$$

Sub-2-Sub keeps M minimal in the sense that there is no partitioning with fewer parts $\overline{\mathbf{S}}_\mathbf{m}$. To this end, for each hyperspace $\overline{\mathbf{S}}_\mathbf{m}$, it registers exactly those nodes i for which $\mathbf{S_m} \subseteq \mathbf{S_i}$. In that case, when a notification is published, the system need merely find the $\overline{\mathbf{S}}_\mathbf{m}$ to which the associated event belongs, from which point it can forward the notification to the appropriate nodes.

To this end, nodes regularly exchange subscriptions through gossiping. If two nodes i and j notice that their respective subscriptions intersect, that is, $\mathbf{S_{ij}} \equiv \mathbf{S_i} \cap \mathbf{S_j} \neq \emptyset$ they will record this fact and keep references to each other. If they discover a third node k with $\mathbf{S_{ijk}} \equiv \mathbf{S_{ij}} \cap \mathbf{S_k} \neq \emptyset$, the three of them will connect to each other so that a notification N from $\mathbf{S_{ijk}}$ can be efficiently disseminated. Note that if $\mathbf{S_{ij}} - \mathbf{S_{ijk}} \neq \emptyset$, nodes i and j will maintain their mutual references, but now associate it strictly with $\mathbf{S_{ij}} - \mathbf{S_{ijk}}$.

In essence, what we are seeking is a means to cluster nodes into M different groups, such that nodes i and j belong to the same group if and only if their subscriptions S_i and S_j intersect. Moreover, nodes in the same group should be organized into an overlay network that allows efficient dissemination of a data item in the hyperspace associated with that group. This situation for a single attribute is sketched in Figure 5.34.

Here, we see a total of seven nodes, in which the horizontal line for node i indicates its range of interest for the value of the single attribute. Also shown is the grouping of nodes into disjoint ranges of interests in values of the attribute. For example, nodes 3, 4, 7, and 10 will be grouped together representing the interval [16.5, 21.0]. Any data item with a value in this range should be disseminated to only these four nodes.

To construct these groups, the nodes are organized into a gossip-based unstructured network. Each node maintains a list of references to other neighbors (i.e., a **partial view**), which it periodically exchanges with one of its neighbors. Such an exchange allows a node to learn about random other nodes in the system. Every node keeps track of the nodes it discovers with overlapping interests (i.e., with an intersecting subscription).

At a certain moment, every node i will generally have references to other nodes with overlapping interests. As part of exchanging information with a

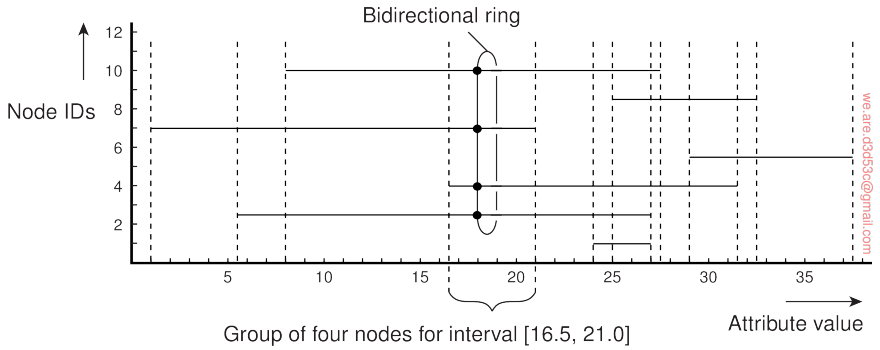


Figure 5.34: Grouping nodes for supporting range queries in a gossip-based publish-subscribe system.

node j, node i orders these nodes by their identifiers and selects the one with the lowest identifier $i_1 > j$, such that its subscription overlaps with that of node j, that is, $\mathbf{S}_{j,i_1} \equiv \mathbf{S}_{i_1} \cap \mathbf{S}_j \neq \emptyset$.

The next one to be selected is $i_2 > i_1$ such that its subscription also overlaps with that of j, but only if it contains elements not yet covered by node i_1. In other words, we should have that $\mathbf{S}_{j,i_1,i_2} \equiv (\mathbf{S}_{i_2} - \mathbf{S}_{j,i_1}) \cap \mathbf{S}_j \neq \emptyset$. This process is repeated until all nodes that have an overlapping interest with node i have been inspected, leading to an ordered list $i_1 < i_2 < \cdots < i_n$. Note that a node i_k is in this list because it covers a region $\mathbf{R}$ of common interest to node i and j not yet jointly covered by nodes with a lower identifier than i_k. In effect, node i_k is the *first* node that node j should forward a notification to that falls in this unique region $\mathbf{R}$. This procedure can be expanded to let node i construct a bidirectional ring. Such a ring is also shown in Figure 5.34.

Whenever a notification N is published, it is disseminated as quickly as possible to *any* node that is interested in it. As it turns out, with the information available at every node finding a node i interested in N is simple. From there on, node i need simply forward N along the ring of subscribers for the particular range that N falls into. To speed up dissemination, shortcuts are maintained for each ring as well.

5.6.2 Secure publish-subscribe solutions

A characteristic feature of publish-subscribe systems is that publishers and subscribers are **referentially decoupled**, and possibly also **temporally decoupled**. Notably, referential decoupling means that in a secure system messages should be able to flow from a publisher to subscribers while guaranteeing mutual anonymity. To guarantee confidentiality and integrity of messages, communicating parties usually set up a secure channel, but such an approach would break the party's anonymity.

A simple solution is to let a (possibly distributed) broker handle messages. The broker would handle all the matching and storage, if necessary. Obviously, such a broker would need to be trusted. Yet, it is not obvious at all that such trust is justified. For example, many publish-subscribe services are offered by third parties operating in a cloud setting. It is often unclear to which extent they can actually provide guarantees when it comes to protecting data. In addition, messages may often contain information that should definitely not, often even by law, be revealed to third parties, such as medical records.

We now seem to have several clashing requirements. First, for any publish-subscribe system, we need to ensure that the communicating parties remain referentially decoupled, that is, we need to guarantee mutual anonymity. Not knowing where messages come from imposes integrity issues: how can we be sure the messages have not been tampered with? Second, we cannot simply assume that brokers can be trusted. Facing confidentiality issues, we then cannot allow them to see (all of) the content of messages, yet this may impose serious routing problems. But there is more: if we cannot trust brokers, how can we ensure availability in the sense that subscribers, entitled to specific messages, actually receive those messages? Equally important is to ensure that messages are not delivered to unauthorized subscribers. Cui et al. [2021] also address an additional requirement: assuming there are colluders in place, brokers should not learn about the interests of innocent subscribers.

These problems are not unique to publish-subscribe systems, and have become more prevalent with the advent of cloud-based services. The key question is how to let a third party search through data to make decisions, yet without revealing that data. This is also known as **searchable encryption** [Bosch et al., 2014]. For publish-subscribe systems, an important technique is to conduct secure keyword search. Boneh et al. [2004] introduced **Public Key Encryption with Keyword Search**, generally known as **PEKS**. We will explain only the principle of PEKS without going into the cryptographic details.

In PEKS, a message is accompanied by a collection of keywords. To that end, using a public key PK, a message m and its n keywords $\mathsf{KW}_1, \ldots, \mathsf{KW}_n$ are stored at a server as the message m^*:

$$\mathsf{m}^* = [PK(\mathsf{m})|PEKS(\mathsf{PK}, \mathsf{KW}_1)|PEKS(\mathsf{PK}, \mathsf{KW}_2)|\cdots|PEKS(\mathsf{PK}, \mathsf{KW}_n)]$$

For each keyword KW_i a trapdoor $\mathsf{T}_{\mathsf{KW}_i}$ is generated: $\mathsf{T}_{\mathsf{W}}(\mathsf{m}^*)$ will return true if $\mathsf{W} \in \{\mathsf{KW}_1, \ldots, \mathsf{KW}_n\}$ and false otherwise.

Using PEKS, we can now set up a cloud-based publish-subscribe system with (conceptually) a single broker that resides in the cloud. Such a scheme is described in [Yang et al., 2017] and sketched in Figure 5.35. We note we have deliberately simplified matters significantly to focus on the essentials.

An important role is played by the key authority: a trusted third party that generates keys for publishers and subscribers. In our example, it provides the

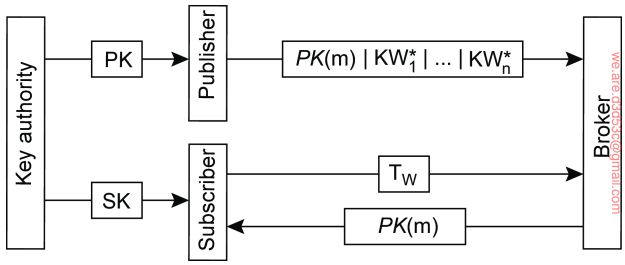


Figure 5.35: The principle of privacy-preserving publish-subscribe.

publisher with a key PK that can be used for encrypting the publication m. In addition, the key is used for constructing the encrypted tags $\mathsf{KW}_i^* = PEKS(\mathsf{PK}, \mathsf{KW}_i)$. Likewise, each subscriber is given a secret key SK that it can use to generate a trapdoor T_W to be used by the broker to see whether the tag W is associated with the publication m. If there is a match, the broker returns $PK(\mathsf{m})$ which can then be decrypted using SK.

Our simplified scheme misses many important details, for which we refer the interested reader to Yang et al. [2017]. The authors assume that the broker is **honest-but-curious** and does not collude with publishers or subscribers. Honest-but-curious means that the server will behave according to the specifications, but may keep track of everything it does and use that information. Cui et al. [2021] describe a system that drops the assumption on collusion, and is one of the few known publish-subscribe systems that can handle colluding brokers. An overview of the security issues in publish-subscribe systems is given by Uzunov [2016]. For a good overview on preserving confidentiality, we refer to Onica et al. [2016]. The authors also come to the conclusion that security becomes easier if brokers can be trusted. Otherwise, good support for operating on encrypted data is needed.

5.7 Location systems

When looking at large distributed systems that are dispersed across a wide-area network, it is often necessary to take proximity into account. Just imagine a distributed system organized as an overlay network in which two processes are neighbors in the overlay network, but are actually placed far apart in the underlying network. If these two processes communicate a lot, it may have been better to ensure that they are also physically placed in each other's proximity. In this section, we take a look at location-based techniques to coordinate the placement of processes and their communication.

5.7.1 GPS: Global Positioning System

Let us start by considering how to determine your geographical position anywhere on Earth. This positioning problem is by itself solved through a highly specific, dedicated distributed system, namely **GPS**, which is an

acronym for **Global Positioning System**. GPS is a satellite-based distributed system that was launched in 1978. Although it initially was used mainly for military applications, it by now has found its way to many civilian applications, notably for traffic navigation. However, many more application domains exist. For example, modern smartphones now allow owners to track each other's position. This principle can easily be applied to tracking other things as well, including pets, children, cars, boats, and so on.

GPS uses up to 72 satellites, each circulating in an orbit at a height of approximately 20,000 km. Each satellite has up to four atomic clocks, which are regularly calibrated from special stations on Earth. A satellite continuously broadcasts its position, and time stamps each message with its local time. This broadcasting allows every receiver on Earth to accurately compute its own position using, in principle, only four satellites. To explain, let us first assume that all clocks, including the receiver's, are synchronized.

To compute a position, consider first the two-dimensional case, as shown in Figure 5.36, in which three satellites are drawn, along with the circles representing points at the same distance from each respective satellite. We see that the intersection of the three circles is a unique point.

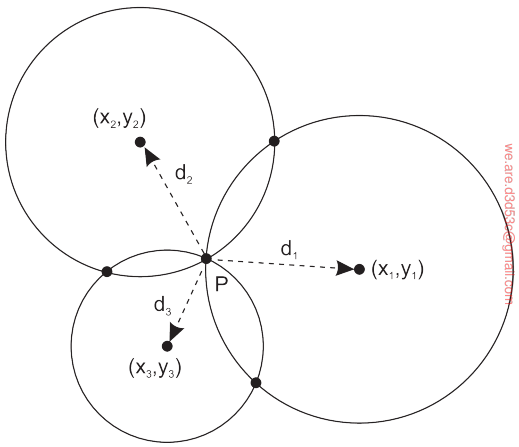


Figure 5.36: Computing a node's position in a two-dimensional space.

This principle of intersecting circles can be expanded to three dimensions, meaning that we need to know the distance to four satellites to determine the longitude, latitude, and altitude of a receiver on Earth. This positioning is all fairly straightforward, but determining the distance to a satellite becomes complicated when we move from theory to practice. There are at least two important real-world facts that we need to take into account:

1. It takes a while before data on a satellite's position reaches the receiver.
2. The receiver's clock is generally not in sync with that of a satellite.

Assume that the timestamp from a satellite is completely accurate. Let Δ_r denote the deviation of the receiver's clock from the actual time. When a message is received from satellite S_i with timestamp T_i, then the measured delay Δ_i by the receiver consists of two components: the actual delay, along with its own deviation:

$$\Delta_i = (T_{now} - T_i) + \Delta_r$$

where T_{now} is the actual current time. As signals travel with the speed of light, c, the *measured* distance $\tilde{d}_i$ to satellite S_i is equal to $c \cdot \Delta_i$. With

$$d_i = c \cdot (T_{now} - T_i)$$

being the real distance between the receiver and satellite S_i, the measured distance can be rewritten to $\tilde{d}_i = d_i + c \cdot \Delta_r$. The real distance is now computed as:

$$\tilde{d}_i - c \cdot \Delta_r = \sqrt{(x_i - x_r)^2 + (y_i - y_r)^2 + (z_i - z_r)^2}$$

where x_i, y_i, and z_i denote the coordinates of satellite S_i. What we see now is a system of quadratic equations with four unknowns (x_r, y_r, z_r, and also Δ_r). We thus need four reference points (i.e., satellites) to find a unique solution that will also give us Δ_r. A GPS measurement will thus also give an account of the actual time.

So far, we have assumed that measurements are perfectly accurate. Of course, they are not. There are many sources of errors, starting with the fact that the atomic clocks in the satellites are not always in perfect sync, the position of a satellite is not known precisely, the receiver's clock has a finite accuracy, the signal propagation speed is not constant (as signals appear to slow down when entering, e.g., the ionosphere), and so on. On average, this leads to an error of some 5–10 meters. Special modulation techniques, as well as special receivers, are needed to improve accuracy. Using so-called **differential GPS**, by which corrective information is sent through wide-area links, accuracy can be further improved. More information can be found in [LaMarca and de Lara, 2008], as well as an excellent overview by Zogg [2002].

5.7.2 When GPS is not an option

A major drawback of GPS is that it generally cannot be used indoors. For that purpose, other techniques are necessary. An increasingly popular technique is to make use of the numerous WiFi access points available. The basic idea is simple: if we have a database of known access points along with their coordinates, and we can estimate our distance to an access point, then with only three detected access points, we should be able to compute our position. Of course, it really is not that simple at all.

A major problem is determining the coordinates of an access point. A popular approach is to do this through **war driving**: using a WiFi-enabled

device along with a GPS receiver, someone drives or walks through an area and records observed access points. An access point can be identified through its SSID or its MAC-level network address. An access point AP should be detected at several locations before its coordinates can be estimated. A simple method is to compute the centroid: assume we have detected AP at N different locations $\{\vec{x_1}, \vec{x_2}, \ldots, \vec{x_N}\}$, where each location $\vec{x_i}$ consists of a (*latitude, longitude*)-pair as provided by the GPS receiver. We then simply estimate AP's location $\vec{x}_{AP}$ as

$$\vec{x}_{AP} = \frac{\sum_{i=1}^{N} \vec{x_i}}{N}.$$

Accuracy can be improved by taking the observed signal strength into account, and giving more weight to a location with relatively high observed signal strength than to a location where only a weak signal was detected. In the end, we obtain an estimation of the coordinates of the access point. The accuracy of this estimation is strongly influenced by:

- the accuracy of each GPS detection point $\vec{x_i}$
- the fact that an access point has a nonuniform transmission range
- the number of sampled detection points N.

Studies show that estimates of the coordinates of an access point may be tens of meters off from the actual location (see, e.g., Kim et al. [2006] or Tsui et al. [2010]). Moreover, access points come and go at a relatively high rate. Nevertheless, locating and positioning access points is widely popular, exemplified by the open-access Wigle database which is populated through crowdsourcing.[1]

5.7.3 Logical positioning of nodes

Instead of trying to find the absolute location of a node in a distributed system, an alternative is to use a logical, proximity-based location. In **geometric overlay networks** each node is given a position in an m-dimensional geometric space, such that the distance between two nodes in that space reflects a real-world performance metric. Computing such a position is the core business of a **Network Coordinates System**, or simply **NCS**, which are surveyed by Donnet et al. [2010]. The simplest, and most applied example, is where distance corresponds to internode latency. In other words, given two nodes P and Q, then the distance $\hat{d}(\mathsf{P}, \mathsf{Q})$ reflects how long it would take for a message to travel from P to Q and *vice versa*. We use the notation $\hat{d}$ to denote distance in a system where nodes have been assigned coordinates.

There are many applications of geometric overlay networks. Consider the situation where a Website at server O has been replicated to multiple servers

[1] See `wigle.net`.

$S_1, \ldots, S_N$ on the Internet, as typically happens with a **Content Delivery Network** (CDN). When a client C requests a page from O, the CDN may decide to redirect that request to the server closest to C, that is, the one that will give the best response time. If the geometric location of C is known, as well as those of each replica server, the CDN can then simply pick that server S_i for which $\hat{d}(C, S_i)$ is minimal.

Another example is optimal replica placement. Consider again a CDN that has gathered the positions of clients for one of its major customers. If the CDN were to replicate content to N servers, it can compute the N best positions where to place replicas such that the average client-to-replica response time is minimal. Performing such computations is almost trivially feasible if clients and servers have geometric positions that reflect internode latencies.

As a last example, consider **position-based routing** (see, e.g., [Popescu et al., 2012] or [Bilal et al., 2013]). In such schemes, a message is forwarded to its destination using only positioning information. For example, a naive routing algorithm to let each node forward a message to the neighbor closest to the destination. Although it can be easily shown that this specific algorithm need not converge, it illustrates that only local information is used to decide. There is no need to propagate link information or such to all nodes in the network, as is the case with conventional routing algorithms.

Centralized positioning

Positioning a node in an m-dimensional geometric space requires $m + 1$ distance measures to nodes with known positions. Assuming that node P wants to compute its own position, it contacts three other nodes with known positions and measures its distance to each of them. Contacting only one node would tell P about the circle it is located on; contacting only two nodes would tell it about the position of the intersection of two circles (which generally consists of two points); a third node would subsequently allow P to compute its actual location.

Node P can compute its own coordinates (x_P, y_P) by solving the three *quadratic* equations with the two unknowns x_P and y_P:

$$\tilde{d}_i = \sqrt{(x_i - x_P)^2 + (y_i - y_P)^2} \qquad (i = 1, 2, 3)$$

Here, we use $\tilde{d}$ to denote measured, or estimated distance. As said, $\tilde{d}_i$ generally corresponds to measuring the latency between P and the node at (x_i, y_i). This latency can be estimated as being half the round-trip delay, but it should be clear that its value will be different over time. The effect is a different positioning whenever P would want to recompute its position. Moreover, if other nodes would use P's current position to compute their own coordinates, then it should be clear that the error in positioning P will affect the accuracy of the positioning of other nodes.

It should also be clear that measured distances between a set of nodes will generally not even be consistent. For example, assume we are computing distances in a one-dimensional space, as shown in Figure 5.37. In this example, we see that although R measures its distance to Q as 2.0, and $\tilde{d}(P,Q)$ has been measured to be 1.0, when R measures $\tilde{d}(P,R)$ it finds 2.8, which is clearly inconsistent with the other two measurements.

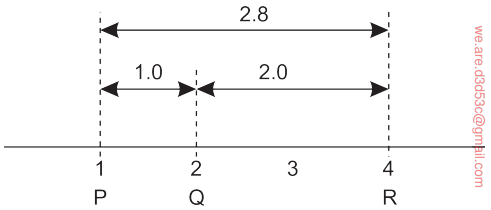


Figure 5.37: Inconsistent distance measurements in a one-dimensional space.

Figure 5.37 also suggests how this situation can be improved. In our simple example, we could solve the inconsistencies by merely computing positions in a two-dimensional space. This by itself, however, is not a general solution when dealing with many measurements. In fact, considering that Internet latency measurements may violate the **triangle inequality**, it is generally impossible to resolve inconsistencies completely. The triangle inequality states that in a geometric space, for any arbitrary three nodes P, Q, and R it must always be true that

$$d(P,R) \leq d(P,Q) + d(Q,R).$$

There are various ways to approach these issues. One common approach, proposed by Ng and Zhang [2002], is to use N special nodes $L_1, \ldots, L_N$, known as **landmarks**. Landmarks measure their pairwise latencies $\tilde{d}(L_i, L_j)$ and subsequently let a central node compute the coordinates for each landmark. To this end, the central node seeks to minimize the following aggregated error function:

$$\sum_{i=1}^{N} \sum_{j=i+1}^{N} \left(\frac{\tilde{d}(L_i,L_j) - \hat{d}(L_i,L_j)}{\tilde{d}(L_i,L_j)} \right)^2$$

where, again, $\hat{d}(L_i, L_j)$ corresponds to the distance after nodes L_i and L_j have been positioned.

The hidden parameter in minimizing the aggregated error function is the dimension m. Obviously, we have that $N > m$, but nothing prevents us from choosing a value for m that is much smaller than N. In that case, a node P measures its distance to each of the N landmarks and computes its coordinates by minimizing

$$\sum_{i=1}^{N} \left(\frac{\tilde{d}(L_i,P) - \hat{d}(L_i,P)}{\tilde{d}(L_i,P)} \right)^2$$

As it turns out, with well-chosen landmarks, m can be as small as 6 or 7, with $\hat{d}(\mathsf{P},\mathsf{Q})$ being no more than a factor 2 different from the actual latency $d(\mathsf{P},\mathsf{Q})$ for arbitrary nodes P and Q [Szymaniak et al., 2004; 2008].

Decentralized positioning

Another way to tackle this problem is to view the collection of nodes as a huge system in which nodes are attached to each other through springs. In this case, $|\tilde{d}(\mathsf{P},\mathsf{Q}) - \hat{d}(\mathsf{P},\mathsf{Q})|$ indicates to what extent nodes P and Q are displaced relative to the situation in which the system of springs would be at rest. By letting each node (slightly) change its position, it can be shown that the system will eventually converge to an optimal organization in which the aggregated error is minimal. This approach is followed in Vivaldi, described in [Dabek et al., 2004a].

In a system with N nodes $\mathsf{P}_1,\ldots,\mathsf{P}_N$, Vivaldi aims at minimizing the following aggregated error:

$$\sum_{i=1}^{N}\sum_{j=1}^{N}|\tilde{d}(\mathsf{P}_i,\mathsf{P}_j) - \hat{d}(\mathsf{P}_i,\mathsf{P}_j)|^2$$

where $\tilde{d}(\mathsf{P}_i,\mathsf{P}_j)$ is the *measured* distance (i.e., latency) between nodes P_i and P_j, and $\hat{d}(\mathsf{P}_i,\mathsf{P}_j)$ the distance computed from the network coordinates of each node. Let $\vec{x}_i$ denote the coordinates of node P_i. In a situation that each node is placed in a geometric space, the *force* that node P_i exerts on node P_j is computed as:

$$\vec{F}_{ij} = \big(\tilde{d}(\mathsf{P}_i,\mathsf{P}_j) - \hat{d}(\mathsf{P}_i,\mathsf{P}_j)\big) \times u(\vec{x}_i - \vec{x}_j)$$

with $u(\vec{x}_i - \vec{x}_j)$ denoting the unit vector in the direction of $\vec{x}_i - \vec{x}_j$. In other words, if $F_{ij} > 0$, node P_i will push P_j away from itself, and will otherwise pull it toward itself. Node P_i now repeatedly executes the following steps:

1. Measure the latency $\tilde{d}_{ij}$ to node P_j, and also receive P_j's coordinates $\vec{x}_j$.
2. Compute the error $e = \tilde{d}(\mathsf{P}_i,\mathsf{P}_j) - \hat{d}(\mathsf{P}_i,\mathsf{P}_j)$
3. Compute the direction $\vec{u} = u(\vec{x}_i - \vec{x}_j)$.
4. Compute the force vector $F_{ij} = e \cdot \vec{u}$
5. Adjust own position by moving along the force vector: $\vec{x}_i \leftarrow \vec{x}_i + \delta \cdot \vec{u}$.

A crucial element is the choice of δ: too large and the system will oscillate; too small and convergence to a stable situation will take a long time. The trick is to have an adaptive value, which is large when the error is large as well, but small when only small adjustments are needed. Details can be found in [Dabek et al., 2004a].

5.8 Summary

Strongly related to communication between processes is the issue of how processes in distributed systems synchronize. Synchronization is all about doing the right thing at the right time. A problem in distributed systems, and computer networks in general, is that there is no notion of a globally shared clock. In other words, processes on different machines have their own idea of what time it is.

There are various ways to synchronize clocks in a distributed system, but all methods are essentially based on exchanging clock values, while considering the time it takes to send and receive messages. Variations in communication delays, and the way those variations are dealt with, largely determine the accuracy of clock synchronization algorithms.

Knowing the absolute time is often not necessary. What counts is that related events at different processes happen in the correct order. Lamport showed that by introducing a notion of logical clocks, it is possible for a collection of processes to reach global agreement on the correct ordering of events. In essence, each event e, such as sending or receiving a message, is assigned a globally unique logical timestamp $C(e)$ such that when event a happened before b, $C(a) < C(b)$. Lamport timestamps can be extended to vector timestamps: if $C(a) < C(b)$, we even know that event a causally preceded b.

An important class of synchronization algorithms is that of distributed mutual exclusion. These algorithms ensure that in a distributed collection of processes, at most one process at a time has access to a shared resource. Distributed mutual exclusion can easily be achieved if we make use of a coordinator that keeps track of whose turn it is. Fully distributed algorithms also exist, but have the drawback that they are generally more susceptible to communication and process failures.

Synchronization between processes often requires that one process acts as a coordinator. In those cases where the coordinator is not fixed, it is necessary that processes in a distributed computation decide on who is going to be that coordinator. Such a decision is taken by election algorithms. Election algorithms are primarily used in cases where the coordinator can crash. However, they can also be applied for the selection of superpeers in peer-to-peer systems.

The most important aspect in gossip-based coordination is being able to select another peer randomly from an entire overlay. As it turns out, we can implement such a peer-sampling service using gossiping, by ensuring that the partial view is refreshed regularly and randomly. Combining peer sampling with a selective replacement of entries in a partial view allows us to efficiently construct structured overlay networks.

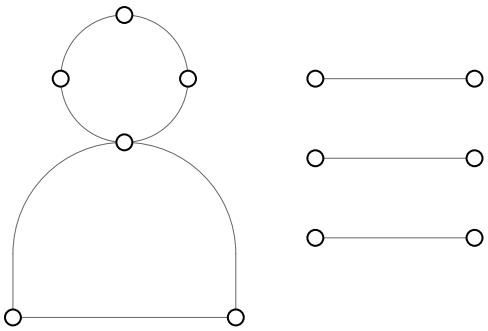
Particularly challenging when it comes to coordination is distributed event matching, which sits at the core of publish-subscribe systems. Relatively

simple is the case when we have a central implementation where matching subscriptions against notifications can be done by essentially doing one-to-one comparisons. However, as soon as we aim at distributing the load, we are faced with the problem of deciding beforehand which node is responsible for which part of the subscriptions, without knowing what kind of notifications to expect. This is particularly problematic for content-based matching, which, in the end, requires advanced filtering techniques to route notifications to the proper subscribers.

Related to these synchronization problems is positioning nodes in a geometric overlay. The basic idea is to assign each node coordinates from an m-dimensional space such that the geometric distance can be used as an accurate measure for the latency between two nodes. The method of assigning coordinates strongly resembles the one applied in determining the location and time in GPS.

06

NAMING



Names play an important role in all computer systems. They are used to sharing resources, to uniquely identify entities, to refer to locations, and more. An important issue with naming is that a name can be resolved to the entity it refers to. Name resolution thus allows a process to access the named entity. To resolve names, it is necessary to implement a naming system. The difference between naming in distributed systems and nondistributed systems lies in the way naming systems are implemented.

In a distributed system, the implementation of a naming system is itself often distributed across multiple machines. How this distribution is done plays a key role in the efficiency and scalability of the naming system. In this chapter, we concentrate on three different, important ways that names are used in distributed systems.

First, we consider so-called flat-naming systems. In such systems, entities are referred to by an identifier that, in principle, has no meaning at all. In addition, flat names bear no structure, implying that we need special mechanisms to trace the location of such entities. We discuss various approaches, ranging from distributed hash tables to hierarchical location services.

In practice, humans prefer to use readable names. Such names are often structured, as is well known from the way Web pages are referred to. Structured names allow for a highly systematic way of finding the server responsible for the named entity, as exemplified by the Domain Name System. We discuss the general principles, as well as scalability issues.

Finally, humans often prefer to describe entities through various characteristics, leading to a situation in which we need to resolve a description using the attributes assigned to an entity. As we shall see, this type of name resolution is notoriously difficult, especially with searching.

Name resolution and routing are closely related to each other. Normally, we look up an address given a name, to subsequently access the named entity through its address. As an alternative to looking up an address, researchers have been exploring how a name can be directly used to route toward the entity, to subsequently return a copy of its associated data. This so-called named-based routing is gradually maturing as an alternative for a future Internet. We discuss it briefly in this chapter.

6.1 Names, identifiers, and addresses

A name in a distributed system is a string of bits or characters that is used to refer to an entity. An entity in a distributed system, can be practically anything. Typical examples include resources such as hosts, printers, disks, and files. Other well-known examples of entities that are often explicitly named are processes, users, mailboxes, Web pages, graphical windows, messages, network connections, and so on.

Entities can be operated on. For example, a resource such as a printer offers an interface containing operations for printing a document, requesting the status of a print job, and the like. Furthermore, an entity such as a network connection may provide operations for sending and receiving data, setting quality-of-service parameters, requesting the status, and so forth.

To operate on an entity, it is necessary to access it, for which we need an **access point**. An access point is yet another, but special, kind of entity in a distributed system. The name of an access point is called an **address**. The address of an access point of an entity is also simply called an address of that entity.

An entity can offer more than one access point. As a comparison, a telephone can be viewed as an access point of a person, whereas the phone number corresponds to an address. Indeed, many people have several phone numbers, each number corresponding to a point where they can be reached. In a distributed system, a typical example of an access point is a host running a specific server, with its address formed by the combination of, for example, an IP address and port number (i.e., the server's transport-level address).

An entity may change its access points over time. For example, when a mobile computer moves to another location, it is often assigned a different IP address than the one it had before. Likewise, when a person changes jobs, there is likely a change in phone numbers as well. Similarly, changing your Internet Service Provider, often means changing an e-mail address.

An address is thus just a special kind of name: it refers to an access point of an entity. Because an access point is tightly associated with an entity, it would seem convenient to use the address of an access point as a regular name for the associated entity. Nevertheless, this is hardly ever done, as such naming is generally very inflexible and often human unfriendly.

For example, it is not uncommon to regularly reorganize a distributed system so that a specific server is now running on a different host than it did previously. The old machine on which the server used to be running may be reassigned to a different server. In other words, an entity may easily change an access point, or an access point may be reassigned to a different entity. If an address is used to refer to an entity, we will have an invalid reference the instant the access point changes or is reassigned to another entity. Therefore, it is much better to let a service be known by a separate name independent of the address of the associated server.

Likewise, if an entity offers more than one access point, it is not clear which address to use as a reference. For instance, many organizations distribute their Web service across several servers. If we would use the addresses of those servers as a reference for the Web service, it is not obvious which address should be chosen as the best one. Again, a much better solution is to have a single name for the Web service, independent of the addresses of the different Web servers.

These examples illustrate that a name for an entity that is independent of its addresses is often much easier and more flexible to use. Such a name is called **location independent**.

In addition to addresses, there are other types of names that deserve special treatment, such as names that are used to uniquely identify an entity. A true **identifier** is a name that has the following properties [Wieringa and de Jonge, 1995]:

1. An identifier refers to at most one entity.
2. Each entity is referred to by at most one identifier.
3. An identifier always refers to the same entity (i.e., it is never reused).

By using identifiers, it becomes much easier to unambiguously refer to an entity. For example, assume two processes each refer to an entity by an identifier. To check if the processes are referring to the same entity, it is sufficient to test if the two identifiers are equal. Such a test would not be sufficient if the two processes were using regular, nonunique, nonidentifying names. For example, the name "John Smith" cannot be taken as a unique reference to just a single person.

Likewise, if an address can be reassigned to a different entity, we cannot use an address as an identifier. Consider again the use of phone numbers, which are reasonably stable in the sense that a phone number will often for some time refer to the same person or organization. However, using a phone number as an identifier will not work, as it can be, and often is, reassigned over time. Consequently, Bob's new bakery may be receiving phone calls for Alice's old antique store for a long time. In this case, it would have been better to use a true identifier for Alice instead of her phone number.

Addresses and identifiers are two important types of names that are each used for very different purposes. In many computer systems, addresses and identifiers are represented in machine-readable form only, that is, in the form of bit strings. For example, an Ethernet address is essentially a random string of 48 bits. Likewise, memory addresses are typically represented as 32-bit or 64-bit strings.

Another important type of name is that which is tailored to be used by humans, also referred to as **human-friendly names**. In contrast to addresses and identifiers, a human-friendly name is generally represented as a character string. These names appear in many different forms. For example, files in Unix systems have character-string names that can generally be as long as 255 characters, and which are defined entirely by the user. Similarly, DNS names are represented as relatively simple case-insensitive character strings.

Having names, identifiers, and addresses brings us to the central theme of this chapter: how do we resolve names and identifiers to addresses? As we shall see, there are essentially two approaches. In the first one, we maintain

a (generally distributed) table of *(name, address)* pairs. This is the approach followed by naming systems such as DNS, which we discuss extensively in this chapter. In the second approach, a name is resolved by routing the request gradually to the name's associated address, or even directly to an access point. Typically, this is the approach followed in structured peer-to-peer systems, but also in what is known as **information-centric networking** (which we also discuss later in this chapter). It is interesting to note that in such cases the boundaries between name resolution and message routing are starting to blur, as was noted by Shoch [1978] already almost 50 years ago. In fact, we will challenge the need for name-to-address resolution when discussing **named-data networking**. To stay in historical perspective, it is then also interesting to note how certain discussions become topical again: in a sense, naming and resolving a name to an address is precisely what is referred to as the **identifier-location split** (see also Ramirez et al. [2014] and Feng et al. [2017]).

In the following sections, we will consider four different classes of naming systems. First, we will take a look at how identifiers can be resolved to addresses. In this case, we will also see an example where name resolution is actually indistinguishable from message routing. After that, we consider human-friendly names and then descriptive names (i.e., entities that are described by a collection of names). Finally, we pay attention to named-based networking.

6.2 Flat naming

Above, we explained that identifiers are convenient to uniquely represent entities. Often, identifiers are simply random bit strings, which we conveniently refer to as unstructured, or **flat names**. An important property of such a name is that it does not contain any information whatsoever on how to locate the access point of its associated entity. In the following, we will take a look at how flat names can be resolved, or, equivalently, how we can locate an entity when given only its identifier.

6.2.1 Simple solutions

We first consider two simple solutions for locating an entity: broadcasting and forwarding pointers. Both solutions are mainly applicable only to local-area networks. Nevertheless, in that environment, they often do the job well, making their simplicity particularly attractive.

Broadcasting

Consider a distributed system built on a computer network that offers efficient broadcasting facilities. Typically, such facilities are offered by local-area

networks, in which all machines are connected to a single cable or the logical equivalent thereof. Also, local-area wireless networks fall into this category.

Locating an entity in such an environment is simple: a message containing the identifier of the entity is broadcasted to each machine, and each machine is requested to check whether it has that entity. Only the machines that can offer an access point for the entity send a reply message containing the address of that access point.

This principle is used in the Internet **Address Resolution Protocol** (**ARP**) to find the data-link address of a machine when given only an IP address [Plummer, 1982]. In essence, a machine broadcasts a packet on the local-network asking who is the owner of a given IP address. When the message arrives at a machine, the receiver checks whether it should listen to the requested IP address. If so, it sends a reply packet containing, for example, its Ethernet address.

Broadcasting becomes inefficient when the network grows. Not only is network bandwidth wasted by request messages, but, more seriously, too many hosts may be interrupted by requests they cannot answer. One possible solution is to switch to multicasting, by which only a restricted group of hosts receives the request. Ethernet networks support data-link level multicasting directly in hardware.

Multicasting can also be used to locate entities in point-to-point networks. For example, the Internet supports network-level multicasting by allowing hosts to join a specific multicast group. Such groups are identified by a multicast address. When a host sends a message to a multicast address, the network layer provides a best-effort service to deliver that message to all group members. Efficient implementations for multicasting on the Internet are discussed in Deering and Cheriton [1990] and Deering et al. [1996].

A multicast address can be used as a general location service for multiple entities. Consider an organization where each employee has his or her own mobile computer. When such a computer connects to the locally available network, it is dynamically assigned an IP address. In addition, it joins a specific multicast group. When a process wants to locate a computer A, it sends a "where is A?" request to the multicast group. If A is connected, it responds with its current IP address.

Another way to use a multicast address is to associate it with a replicated entity, and to use multicasting to locate the *nearest* replica. When sending a request to the multicast address, each replica responds with its current (normal) IP address. A crude way to select the nearest replica is to choose the one whose reply comes in first, but as it turns out, selecting a nearest replica is generally not that easy.

Forwarding pointers

Another approach to locating mobile entities is to make use of forwarding pointers [Fowler, 1985]. The principle is simple: when an entity moves from A to B, it leaves behind in A a reference to its new location at B. The main advantage of this approach is its simplicity: as soon as an entity has been located, for example by using a traditional naming service, a client can look up the current address by following the chain of forwarding pointers.

There are also drawbacks. First, if no special measures are taken, a chain for a highly mobile entity can become so long that locating that entity is prohibitively expensive. Second, all intermediate locations in a chain will have to maintain their part of the chain of forwarding pointers as long as needed. A third (and related) drawback is the vulnerability to broken links. As soon as any forwarding pointer is lost, the entity can no longer be reached. An important issue is, therefore, to keep chains relatively short, and to ensure that forwarding pointers are robust.

6.2.2 Home-based approaches

A popular approach to supporting mobile entities in large-scale networks is to introduce a **home location**, which keeps track of the current location of an entity. Special techniques may be applied to safeguard against network or process failures. In practice, the home location is often chosen to be the place where an entity was created.

The home-based approach is used as a fall-back mechanism for location services based on forwarding pointers. Another example where the home-based approach is followed is in Mobile IP [Perkins et al., 2011]. Each mobile host uses a fixed IP address. All communication to that IP address is initially directed to the mobile host's **home agent**. This home agent is located on the local-area network corresponding to the network address contained in the mobile host's IP address. In the case of IPv6, it is realized as a network-layer component. Whenever the mobile host moves to another network, it requests a temporary address that it can use for communication. This **care-of address** is registered at the home agent.

When the home agent receives a packet for the mobile host, it looks up the host's current location. If the host is on the current local network, the packet is simply forwarded. Otherwise, it is *tunneled* to the host's current location, that is, wrapped as data in an IP packet and sent to the care-of address. At the same time, the sender of the packet is informed of the host's current location. This principle is shown in Figure 6.1 Note that the IP address is effectively used as an identifier for the mobile host.

An important aspect is that this whole mechanism is largely hidden for applications. In other words, the original IP address associated with the mobile host can be used by an application without further ado. Client-side

software that is part of the application-independent communication layer will handle the redirection to the target's current location. Likewise, at the target's location, a message that has been tunneled will be unpacked and handed to the application on the mobile host as if it were using its original address. Indeed, Mobile IP establishes a high degree of location transparency.

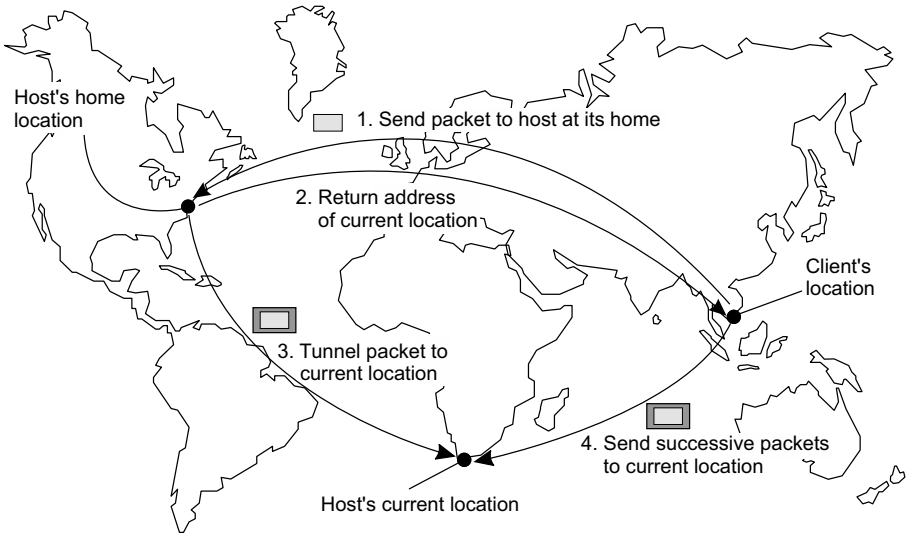


Figure 6.1: The principle of Mobile IP.

Figure 6.1 also illustrates a drawback of home-based approaches in large-scale networks. To communicate with a mobile entity, a client first has to contact the home, which may be at an entirely different location than the entity itself. The result is an increase in communication latency.

Another drawback of the home-based approach is the use of a fixed home location. For one thing, it must be ensured that the home location always exists. Otherwise, contacting the entity will become impossible. Problems are aggravated when a long-lived entity decides to move permanently to an entirely different part of the network than where its home is located. In that case, it would have been better if the home could have moved along with the host.

A solution to this problem is to register the home at a traditional naming service and to let a client first look up the location of the home. Because the home location can be assumed to be relatively stable, that location can be effectively cached after it has been looked up.

6.2.3 Distributed hash tables

Let us now take a closer look at how to resolve an identifier to the address of the associated entity. We have already mentioned distributed hash tables a number of times, but have deferred discussion on how they actually work. In this section, we correct this situation by first considering the Chord system as an easy-to-explain DHT-based system.

General mechanism

Many DHT-based systems have been developed in the past decade, with the Chord system [Stoica et al., 2003] being a typical representative. Chord uses an m-bit identifier space to assign randomly chosen identifiers to nodes as well as keys to specific entities. The latter can be virtually anything: files, processes, etc. The number m of bits is usually 128 or 160, depending on which hash function is used. An entity with key k falls under the jurisdiction of the node with the smallest identifier $id \geq k$. This node is referred to as the **successor** of k and denoted as $succ(k)$. To keep our notation simple and consistent, in the following, we refer to a node with identifier p as node p.

The main issue in DHT-based systems is to efficiently resolve a key k to the address of $succ(k)$. A naive approach is to let each node p keep track of the successor $succ(p+1)$. In that case, whenever a node p receives a request to resolve key k, it will simply forward the request to its successor, unless $pred(p) < k \leq p$, in which case node p should return its own address to the process that initiated the resolution of key k. A lookup request would, on average, need to travel half of the ring. Obviously, this approach is not scalable. Keeping track of the predecessor and forwarding a request in that direction will cut the expected number of hops by only 50%, which is not by far good enough.

Instead of this linear approach toward key lookup, each Chord node maintains a **finger table** containing $s \leq m$ entries. If FT_p denotes the finger table of node p, then

$$\mathsf{FT}_\mathsf{p}[i] = succ(p + 2^{i-1})$$

Put in other words, the ith entry points to the first node succeeding p by at least 2^{i-1} units. Note that these references are actually shortcuts to existing nodes in the identifier space, where the short-cutted distance from node p increases exponentially as the index in the finger table increases. To look up a key k, node p will then immediately forward the request to node q with index j in p's finger table where:

$$q = \mathsf{FT}_\mathsf{p}[j] \leq k < \mathsf{FT}_\mathsf{p}[j+1]$$

or $q = \mathsf{FT}_\mathsf{p}[1]$ when $p < k < \mathsf{FT}_\mathsf{p}[1]$. (For clarity, we ignore modulo arithmetic.) Note that when the finger-table size s is equal to 1, a Chord lookup corresponds to naively traversing the ring linearly, as we just discussed.

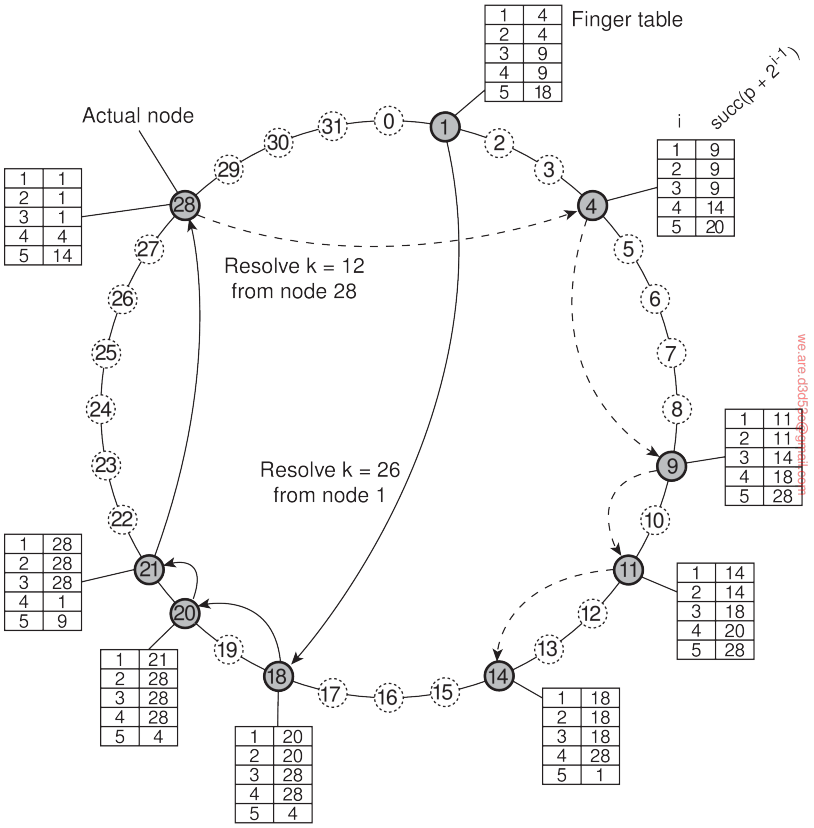
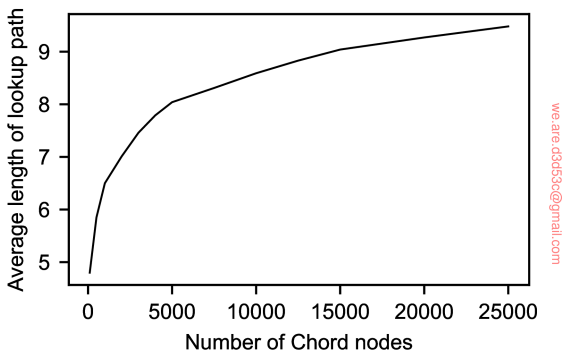


Figure 6.2: Resolving key 26 from node 1 and key 12 from node 28 in a Chord system.

To illustrate this lookup, consider resolving $k = 26$ from node 1 as shown in Figure 6.2. First, node 1 will look up $k = 26$ in its finger table to discover that this value is larger than $\mathsf{FT}_1[5]$, meaning that the request will be forwarded to node $18 = \mathsf{FT}_1[5]$. Node 18, in turn, will select node 20, as $\mathsf{FT}_{18}[2] \leq k < \mathsf{FT}_{18}[3]$. Finally, the request is forwarded from node 20 to node 21 and from there to node 28, which is responsible for $k = 26$. At that point, the address of node 28 is returned to node 1 and the key has been resolved. For similar reasons, when node 28 is requested to resolve the key $k = 12$, a request will be routed as shown by the dashed line in Figure 6.2.

It should come as no surprise that a lookup will generally require $\mathcal{O}(\log(N))$ steps, with N being the number of nodes in the system. The result of a simple experiment by which we look up $k = p - 1$ starting at node p, yet initially ignore p's predecessor, is shown in Figure 6.3. Note that, by ignoring p's predecessor at the first lookup step, we need to effectively go to a node at distance roughly 2^{i-1} for decreasing values of i.



we.are.d3d53c@gmail.com

Figure 6.3: The length of a lookup path as function of the size of a Chord ring.

In large distributed systems, the collection of participating nodes can be expected to change all the time. Not only will nodes join and leave voluntarily, we also need to consider the case of nodes failing (and thus effectively leaving the system), to later recover again (at which point they rejoin).

Joining a DHT-based system such as Chord is relatively simple. Suppose node p wants to join. It simply contacts an arbitrary node in the existing system and requests a lookup for $succ(p+1)$. Once this node has been identified, p can insert itself into the ring. Likewise, leaving can be just as simple. Note that nodes also keep track of their predecessor.

Obviously, the complexity comes from keeping the finger tables up-to-date. Most important is that for every node q, $\mathsf{FT}_q[1]$ is correct as this entry refers to the next node in the ring, that is, the successor of $q+1$. To achieve this goal, each node q regularly runs a simple procedure that contacts $succ(q+1)$ and requests to return $pred(succ(q+1))$. If $q = pred(succ(q+1))$ then q knows its information is consistent with that of its successor. Otherwise, if q's successor has updated its predecessor, then apparently a new node p had entered the system, with $q < p \leq succ(q+1)$, so that q will adjust $\mathsf{FT}_q[1]$ to p. At that point, it will also check whether p has recorded q as its predecessor. If not, another adjustment of $\mathsf{FT}_q[1]$ is needed.

Similarly, to update a finger table, node q simply needs to find the successor for $k = q + 2^{i-1}$ for each entry i. Again, this can be done by issuing a request to resolve $succ(k)$. In Chord, such requests are issued regularly using a background process.

Likewise, each node q will regularly check whether its predecessor is alive. If the predecessor has failed, the only thing that q can do is record the fact by setting $pred(q)$ to "unknown." On the other hand, when node q is updating its link to the next known node in the ring, and finds that the predecessor of $succ(q+1)$ has been set to "unknown," it will simply notify $succ(q+1)$ that

it suspects it to be the predecessor. By and large, these simple procedures ensure that a Chord system is generally consistent, only perhaps except for a few nodes. The details can be found in [Stoica et al., 2003].

Note 6.1 (Advanced: Chord in Python)
Coding Chord in Python is relatively simple. Again, omitting many of the nonessential coding details, the core of the behavior of a Chord node can be described as shown in Figure 6.4. The function `finger(i)` with `succNode(i)` computes $succ(i)$ for the given node. All nodes known to a specific Chord node are collected in a local set `nodeSet`, which is sorted by node identifier. The node first looks up its own position in this set, and that of its right-hand neighbor. The operation `inbetween(k,l,u)` computes if $k \in [l, u)$, taking modulo arithmetic into account. Computing `inbetween(k,l+1,u+1)` is therefore the same as testing whether $k \in (l, u]$. We thus see that `finger(i)` returns the largest existing node identifier, less or equal to i.

```
class ChordNode:

  def __succNode(self, key):
    if (key <= self.nodeSet[0] or
        key > self.nodeSet[len(self.nodeSet)-1]): # key is in segment for which
      return self.nodeSet[0]                      # this node is responsible
    for i in range(1,len(self.nodeSet)):
      if (key <= self.nodeSet[i]):                # key is in segment for which
        return self.nodeSet[i]                    # node (i+1) may be responsible

  def __finger(self, i):
    return self.__succNode((self.nodeID + pow(2,i-1)) % self.MAXPROC) # succ(p+2^(i-1))

  def __recomputeFingerTable(self):
    self.FT[0]  = self.nodeSet[(self.nodeInd - 1)%len(self.nodeSet)] # Predecessor
    self.FT[1:] = [self.__finger(i) for i in range(1,self.nBits+1)]  # Successors
    self.FT.append(self.nodeID)                                      # This node

  def __localSuccNode(self, key):
    if self.__inbetween(key, self.FT[0]+1, self.nodeID+1):  # key in (pred,self]
      return self.nodeID                                    # this node is responsible
    elif self.__inbetween(key, self.nodeID+1, self.FT[1]):  # key in (self,FT[1]]
      return self.FT[1]                                     # successor responsible
    for i in range(1, self.nBits+2):                        # go through rest of FT
      if self.__inbetween(key, self.FT[i], self.FT[(i+1)]): # key in [FT[i],FT[i+1])
        return self.FT[i]                                   # FT[i] is responsible
```

Figure 6.4: The essence of a Chord node expressed in Python.

Every time a node learns about a new node in the system (or discovers that one has left), it simply adjusts the local `nodeSet` and recomputes its finger table by calling `recomputeFingerTable`. The finger table itself is implemented as a local table `FT`, with `FT[0]` pointing to the node's predecessor and `FT[nBits+1]` to the

node itself, where `nBits` is the number of bits used for node identifiers and keys.

The core of what a node does during a lookup is encoded in `localSuccNode(k)`. When handed a key k, it will either return itself, its immediate successor `FT[1]`, or go through the finger table to search the entry satisfying `FT[i]` $\leq k <$ `FT[i+1]`. The code does not show what is done with the returned value (which is a node identifier), but typically in an *iterative scheme*, the referenced node will be contacted to continue looking up k, unless the node had returned itself as the one being responsible for k. When deploying a *recursive scheme*, the node itself contacts the referenced node.

Exploiting network proximity

One of the potential problems with systems such as Chord is that requests may be routed erratically across the Internet. For example, assume that node 1 in Figure 6.2 is placed in Amsterdam, The Netherlands; node 18 in San Diego, California; node 20 in Amsterdam again; and node 21 in San Diego. The result of resolving key 26 will then incur three wide-area message transfers, which arguably could have been reduced to at most one. To minimize these pathological cases, designing a DHT-based system requires taking the underlying network into account.

Castro et al. [2002a] distinguish three different ways for making a DHT-based system aware of the underlying network. In the case of **topology-based assignment of node identifiers** the idea is to assign identifiers such that two nearby nodes will have identifiers that are also close to each other. It is not difficult to imagine that this approach may impose severe problems in the case of relatively simple systems such as Chord. In the case where node identifiers are sampled from a one-dimensional space, mapping a logical ring to the Internet is far from trivial. Moreover, such a mapping can easily expose correlated failures: nodes on the same enterprise network will have identifiers from a relatively small interval. When that network becomes unreachable, we suddenly have a gap in the otherwise uniform distribution of identifiers.

With **proximity routing**, nodes maintain a list of alternatives to forward a request to. For example, instead of having only a single successor, each node in Chord could equally well keep track of r successors. In fact, this redundancy can be applied for every entry in a finger table. For node p, $\mathsf{FT}_\mathsf{p}[i]$ normally points to the first node in the range $[p + 2^{i-1}, p + 2^i - 1]$. Whenever it needs to look up a key k, it tries to prevent "overshooting" by passing the request to a node q with $k < q$ without knowing for sure if there is a node q' with $k \leq q' < q$. For this reason, p passes k to the node known to p with the *largest* identifier smaller or equal to k.

However, there is no reason why p cannot keep track of r nodes in range $[p + 2^{i-1}, p + 2^i - 1]$: each node q in this range can be used to route a lookup request for a key k as long as $q \leq k$. In that case, when choosing to forward

a lookup request, a node can pick one of the r successors that is closest to itself while making sure not to "overshoot." An additional advantage of having multiple successors for every table entry is that node failures need not immediately lead to failures of lookups, as multiple routes can be explored.

Finally, in **proximity neighbor selection** the idea is to optimize routing tables such that the nearest node is selected as neighbor. This selection works only when there are more nodes to choose from. In Chord, this is normally not the case. However, in other protocols such as Pastry [Rowstron and Druschel, 2001], when a node joins, it receives information about the current overlay from multiple other nodes. This information is used by the new node to construct a routing table. Obviously, when there are alternative nodes to choose from, proximity neighbor selection will allow the joining node to choose the best one.

Note that it may not be that easy to draw a line between proximity routing and proximity neighbor selection. In fact, when Chord is modified to include r successors for each finger table entry, proximity neighbor selection resorts to identifying the closest r neighbors, which comes very close to proximity routing, as we just explained [Dabek et al., 2004b].

6.2.4 Hierarchical approaches

We now discuss a general approach to a hierarchical location scheme, including a number of optimizations. The approach we present is based on the Globe location service [van Steen et al., 1998]. A detailed description can be found in [Ballintijn, 2003]. This is a general-purpose location service that is representative of many hierarchical location services proposed for what are called Personal Communication Systems, of which a general overview can be found in Pitoura and Samaras [2001].

In a hierarchical scheme, a network is divided into a collection of **domains**. There is a single top-level domain that spans the entire network. Each domain can be subdivided into multiple, smaller subdomains. A lowest-level domain, called a **leaf domain**, typically corresponds to a local-area network in a computer network or a cell in a mobile telephone network. The general assumption is that within a smaller domain, the average time it takes to transfer a message from one node to another is less than in a large domain.

Each domain D has an associated directory node $dir(\mathsf{D})$ that keeps track of the entities in that domain. This leads to a tree of directory nodes. The directory node of the top-level domain, called the **root (directory) node**, knows about all entities. This general organization of a network into domains and directory nodes is illustrated in Figure 6.5

To keep track of the whereabouts of an entity, each entity currently located in a domain D is represented by a **location record** in the directory node $dir(\mathsf{D})$. A location record for entity E in the directory node N for a leaf domain D contains the entity's current address in that domain. In contrast,

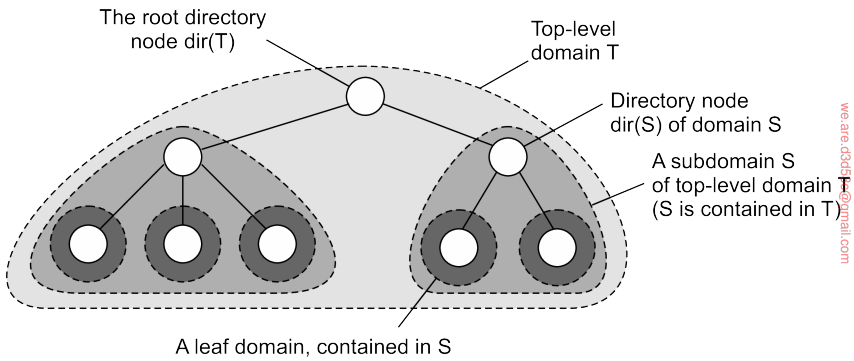


Figure 6.5: Hierarchical organization of a location service into domains, each having an associated directory node.

the directory node N′ for the next higher-level domain D′ that contains D, will have a location record for E containing only a pointer to N. Likewise, the parent node of N′ will store a location record for E containing only a pointer to N′. Consequently, the root node will have a location record for each entity, where each location record stores a pointer to the directory node of the next lower-level subdomain where that record's associated entity is currently located.

An entity may have multiple addresses, for example if it is replicated. If an entity has an address in leaf domain D_1 and D_2 respectively, then the directory node of the smallest domain containing both D_1 and D_2, will have two pointers, one for each subdomain containing an address. This leads to the general organization of the tree as shown in Figure 6.6.

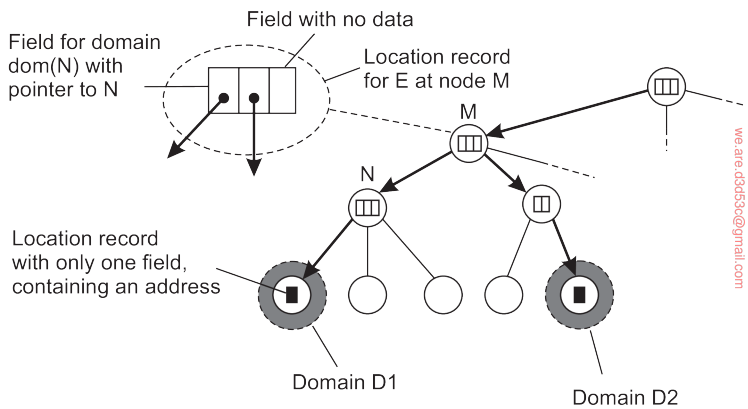


Figure 6.6: An example of storing information of an entity having two addresses in different leaf domains.

Let us now consider how a lookup operation proceeds in such a hierarchical location service. As is shown in Figure 6.7, client,nt wishing to locate an entity E, issues a lookup request to the directory node of the leaf domain D in which the client resides. If the directory node does not store a location record for the entity, then the entity is currently not located in D. Consequently, the node forwards the request to its parent. Note that the parent node represents a larger domain than its child. If the parent also has no location record for E, the lookup request is forwarded to a next level higher, and so on.

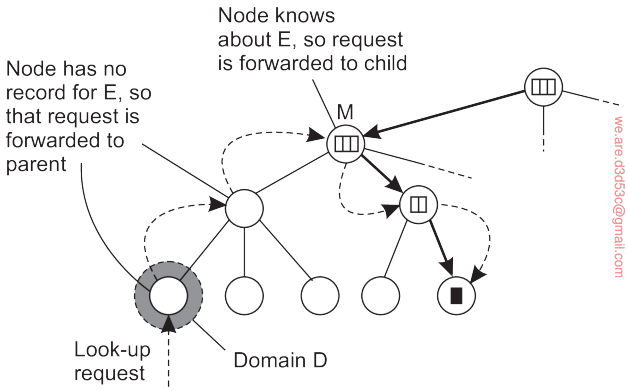
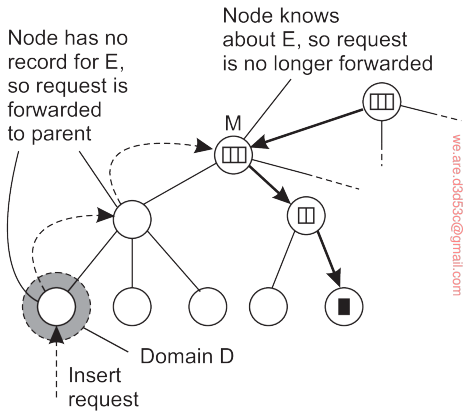


Figure 6.7: Looking up a location in a hierarchically organized location service.

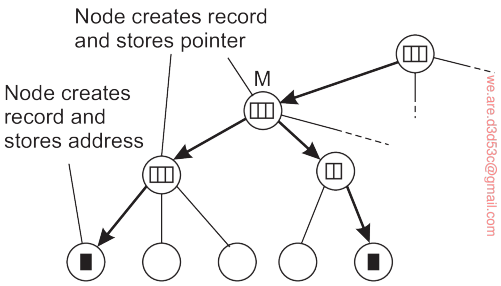
As soon as the request reaches a directory node M that stores a location record for entity E, we know that E is somewhere in the domain $dom(M)$ represented by node M. In Figure 6.7, M is shown to store a location record containing a pointer to one of its subdomains. The lookup request is then forwarded to the directory node of that subdomain, which in turn forwards it further down the tree, until the request finally reaches a leaf node. The location record stored in the leaf node will contain the address of E in that leaf domain. This address can then be returned to the client that initially requested the lookup to take place.

An important observation regarding hierarchical location services is that the lookup operation exploits locality. In principle, the entity is searched in a gradually increasing ring centered around the requesting client. The search area is expanded each time the lookup request is forwarded to a next higher-level directory node. In the worst case, the search continues until the request reaches the root node. Because the root node has a location record for each entity, the request can then simply be forwarded along a downward path of pointers to one of the leaf nodes.

Update operations exploit locality similarly, as shown in Figure 6.8. Consider an entity E that has created a replica in a leaf domain D for which it needs to insert its address. The insertion is initiated at the leaf node $dir(D)$



(a)



(b)

Figure 6.8: (a) An insert request is forwarded to the first node that knows about entity E. (b) A chain of forwarding pointers to the leaf node is created.

of D which immediately forwards the insert request to its parent. The parent will forward the insert request as well, until it reaches a directory node M that already stores a location record for E.

Node M will then store a pointer in the location record for E, referring to the child node from where the insert request was forwarded. At that point, the child node creates a location record for E, containing a pointer to the next lower-level node from where the request came. This process continues until we reach the leaf node from which the insert was initiated. The leaf node, finally, creates a record with the entity's address in the associated leaf domain.

Inserting an address as just described leads to installing the chain of pointers in a top-down fashion, starting at the lowest-level directory node that has a location record for entity E. An alternative is to create a location

record before passing the insert request to the parent node. In other words, the chain of pointers is constructed from the bottom up. The advantage of the latter is that an address becomes available for lookups as soon as possible. Consequently, if a parent node is temporarily unreachable, the address can still be looked up within the domain represented by the current node.

A delete operation is analogous to an insert operation. When an address for an entity E in a leaf domain D needs to be removed, the directory node $dir(\mathsf{D})$ is requested to remove that address from its location record for E. If that location record becomes empty, that is, it contains no other addresses for E in D, the record can be removed. In that case, the parent node of $dir(\mathsf{D})$ wants to remove its pointer to $dir(\mathsf{D})$. If the location record for E at the parent now also becomes empty, that record should be removed as well and the next higher-level directory node should be informed. Again, this process continues until a pointer is removed from a location record that remains nonempty afterward or until the root is reached.

Note 6.2 (Advanced: Scalability issues)
One question that immediately comes to mind is whether the hierarchical approach just described can actually scale. A seemingly obvious design flaw, is that the root node needs to keep track of *all* identifiers. However, it is important to make a distinction between a logical design and its physical implementation. Let us make this distinction here and see how we can actually come to a highly scalable implementation of a hierarchical location service.

To this end, we assume that each entity is assigned a unique identifier uniform at random from a large space of m-bit identifiers, just as in Chord. Furthermore, let us assume that there are a total of N physical hosts $\{\mathsf{H}_1, \mathsf{H}_2, \ldots, \mathsf{H}_\mathsf{N}\}$ that can accommodate the lookup service, spread across the Internet. Each host is capable of running one or more location servers. Typically, two servers running on the same host will represent two nodes at different levels of the logical tree. Let $D_k(\mathsf{A})$ denote the domain at level k that contains address A, with $k = 0$ denoting the root domain. Likewise, let $LS_k(\mathsf{E}, \mathsf{A})$ denote the unique location server in $D_k(\mathsf{A})$ responsible for keeping track of the whereabouts of the entity E.

We can now make a distinction between a logical root and its implementation. Let $\mathbf{D}_k = \{\mathsf{D}_{k,1}, \mathsf{D}_{k,2}, \ldots, \mathsf{D}_{k,\mathsf{N}_k}\}$ denote the N_k domains at level k, with, obviously, $N_0 = |\mathbf{D}_0| = 1$. For each level k, the set of hosts is partitioned into N_k subsets, with each host running a location server representing exactly one of the domains $\mathsf{D}_{k,i}$ from $\mathbf{D}_k$. This principle is shown in Figure 6.9.

In this example, we consider a simple tree with four levels and nine hosts. There are two level-1 domains, four level-2 domains, and eight leaf domains. We also show a tree for one specific entity E: any contact address associated with E will be stored in one of the eight level-3 location servers, depending, of course, on the domain to which that address belongs. The root location server for E is running on host H_3. Note that this host also runs a leaf-level location server for E.

As explained by van Steen and Ballintijn [2002], by judiciously choosing which host should run a location server for E, we can combine the principle of local

operations (which is good for geographical scalability) and full distribution of higher level servers (which is good for size scalability).

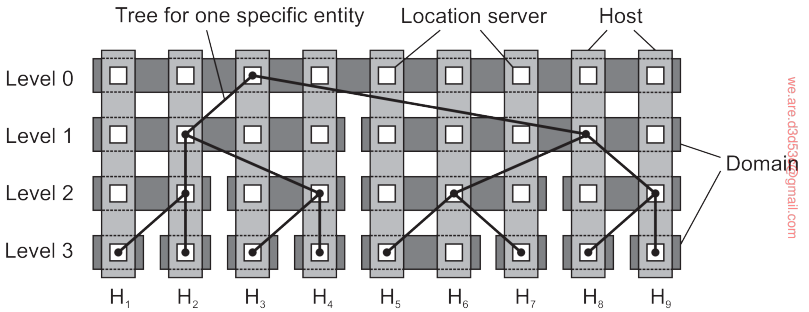


Figure 6.9: The principle of distributing logical location servers over physical hosts.

6.2.5 Secure flat naming

Flat names do not contain any information on how to resolve a name to the entity it is referring to. As a consequence, one needs to, in principle, entirely rely on the name-resolution process to eventually access the associated entity. If the name-resolution process cannot be trusted, there is no reason to believe that any response can be trusted as well. There are then, essentially, two approaches to follow: (1) secure the name-resolution process, or (2) secure the identifier-to-entity association. Let us first take a look at this last option.

Securing the identifier-to-entity association is what happens in a so-called **self-certifying name**. As a general principle, we can compute an identifier from an entity by simply using a hash function:

$$id(entity) = hash(data\ associated\ with\ the\ entity).$$

The crux is, of course, which associated data is used. In the case of nonmodifiable entities, such as read-only files, a client can simply check that it received the correct file by separately computing the hash over the file and comparing it to the identifier it used for looking up that file. Of course, to make this work, the client would need to know which hash function to use along with perhaps other information.

In practice, entities are modified, such as when dealing with mobile objects. In that case, the entity is the current address of the object. What we need to know, for sure, is that the returned address is indeed the one that can be used for accessing the object. Likewise, when dealing with modifiable files, the returned information should allow the client to check that it is

indeed accessing the correct file. Omitting various details, a scheme proposed in [Dannewitz et al., 2010] is to have

$$id(entity) = public\ key(entity).$$

A client will be returned the associated entity, along with additional data that will allow it to verify that it is indeed dealing with the correct entity. For example, the additional data could contain a signed digest of the entity, effectively forming a signature of the entity's owner. If necessary, the returned information could also contain a certificate stating the validity of the public key that has been used as an identifier.

By securing the identifier-to-entity binding, it becomes less important to trust the name-resolution process. The worse that can now happen is that the process returns false or no entities. However, whatever it returns, the client will be able to verify its correctness. Trust is now degraded to providing a decent service.

Securing the name-resolution process is more involved, yet equally important, if only to guarantee proper reliability (we will return to the relationship between security and reliability in Chapter 8). For lookup services such as those for mobile IP or hierarchical location services, we need to ensure that returned (intermediate) answers make sense. We will discuss this in the next section when zooming into Secure DNS. As mentioned before, securing DHT-based systems has proved to be rather problematic. There are several problems to deal with, yet the most important ones deal with addressing **Sybil** and **eclipse attacks**, as also discussed in Section 9.4.2. The bottom line is that robustness for DHT-based systems comes largely from ensuring that a node identifier is genuine: it belongs to exactly one owner, and this fact can be verified. In practice, this means that a centralized authority should be used for handing out identifiers [Urdaneta et al., 2011].

6.3 Structured naming

Flat names are good for machines, but are generally not very convenient for humans to use. As an alternative, naming systems generally support structured names that are composed from simple, human-readable names. Not only file naming, but also host naming on the Internet, follows this approach. In this section, we concentrate on structured names and the way that these names are resolved to addresses.

6.3.1 Name spaces

Names are commonly organized into what is called a **name space**. Name spaces for structured names can be represented as a labeled, directed graph with two types of nodes. A **leaf node** represents a named entity and has

the property that it has no outgoing edges. A leaf node generally stores information on the entity it is representing–for example, its address–so that a client can access it. Alternatively, it can store the state of that entity, such as in the case of file systems in which a leaf node actually contains the complete file it is representing. We return to the contents of nodes below.

In contrast to a leaf node, a **directory node** has a number of outgoing edges, each labeled with a name, as shown in Figure 6.10 Each node in a naming graph is considered as yet another entity in a distributed system, and, in particular, has an associated identifier. A directory node stores a table in which an outgoing edge is represented as a pair *(node identifier, edge label)*. Such a table is called a **directory table**.

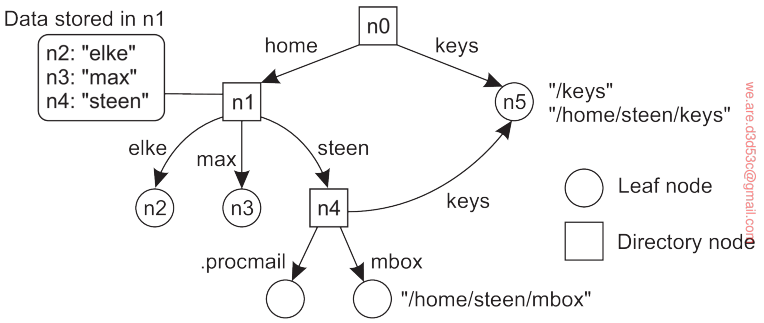


Figure 6.10: A general naming graph with a single root node.

The naming graph shown in Figure 6.10 has one node, namely n0, which has only outgoing and no incoming edges. Such a node is called the **root (node)** of the naming graph. Although it is possible for a naming graph to have several root nodes, for simplicity, many naming systems have only one. Each path in a naming graph can be referred to by the sequence of labels corresponding to the edges in that path, such as N:[$\text{label}_1, \text{label}_2, ..., \text{label}_n$], where N refers to the first node in the path. Such a sequence is called a **path name**. If the first node in a path name is the root of the naming graph, it is called an **absolute path name**. Otherwise, it is called a **relative path name**.

It is important to realize that names are always organized in a name space. As a consequence, a name is always defined relative only to a directory node. In this sense, the term "absolute name" is somewhat misleading. Likewise, the difference between global and local names can often be confusing. A **global name** is a name that denotes the same entity, no matter where that name is used in a system. In other words, a global name is always interpreted regarding the same directory node. In contrast, a **local name** is a name whose interpretation depends on where that name is being used. Put differently, a local name is essentially a relative name whose directory in which it is contained is (implicitly) known.

This description of a naming graph comes close to what is implemented in many file systems. However, instead of writing the sequence of edge labels to represent a path name, path names in file systems are generally represented as a single string in which the labels are separated by a special separator character, such as a slash ("/"). This character is also used to indicate whether a path name is absolute. For example, in Figure 6.10 instead of using n0:[home, steen, mbox], that is, the actual path name, it is common practice to use its string representation "/home/steen/mbox." Note also that when there are several paths that lead to the same node, that node can be represented by different path names. For example, node n5 in Figure 6.10 can be referred to by "/home/steen/keys" as well as "/keys." The string representation of path names can be equally well applied to naming graphs other than those used for only file systems. In Plan 9 [Pike et al., 1995], all resources, such as processes, hosts, I/O devices, and network interfaces, are named in the same fashion as traditional files. This approach is analogous to implementing a single naming graph for all resources in a distributed system.

There are many ways to organize a name space. As we mentioned, most name spaces have only a single root node. Often, a name space is also strictly hierarchical, in the sense that the naming graph is organized as a tree. This means that each node, except the root, has exactly one incoming edge; the root has no incoming edges. As a consequence, each node also has exactly one associated (absolute) path name.

The naming graph shown in Figure 6.10 is an example of **directed acyclic graph**. In such an organization, a node can have more than one incoming edge, but the graph is not permitted to have a cycle. There are also name spaces that do not have this restriction.

Note 6.3 (More information: Implementing the Unix naming graph)
To make matters more concrete, consider the way that files in a traditional Unix file system are named. In a naming graph for Unix a directory node represents a file directory, whereas a leaf node represents a file. There is a single root directory, represented in the naming graph by the root node. The implementation of the naming graph is an integral part of the complete implementation of the file system. That implementation consists of a contiguous series of blocks from a logical disk, generally divided into a boot block, a superblock, a series of index nodes (called inodes), and file data blocks. See also [Silberschatz et al., 2019] or [Tanenbaum and Bos, 2022]. This organization is shown in Figure 6.11.

The boot block is a special block of data and instructions that are automatically loaded into main memory when the system is booted. The boot block is used to load the operating system into main memory.

The superblock contains information on the entire file system, such as its size, which blocks on disk are not yet allocated, which inodes are not yet used, and so on. Inodes are referred to by an index number, starting at number zero, which is reserved for the inode representing the root directory.

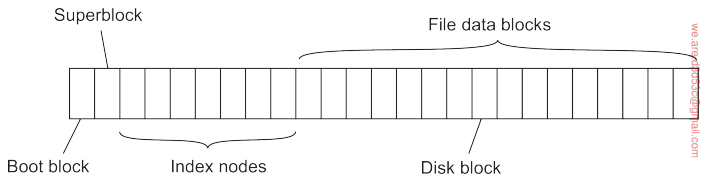


Figure 6.11: The general organization of the Unix file system implementation on a logical disk of contiguous disk blocks.

Each inode contains information on where the data of its associated file can be found on disk. In addition, an inode contains information on its owner, time of creation and last modification, protection, and the like. Consequently, when given the index number of an inode, it is possible to access its associated file. Each directory is implemented as a file as well. This is also the case for the root directory, which contains a mapping between file names and index numbers of inodes. It is thus seen that the index number of an inode corresponds to a node identifier in the naming graph.

6.3.2 Name resolution

Name spaces offer a convenient mechanism for storing and retrieving information about entities through names. More generally, given a path name, it should be possible to look up any information stored in the node referred to by that name. The process of looking up a name is called **name resolution**.

To explain how name resolution works, let us consider a path name such as N:[$\mathsf{label}_1, \mathsf{label}_2, ..., \mathsf{label}_n$]. Resolution of this name starts at node N of the naming graph, where the name label_1 is looked up in the directory table, and which returns the identifier of the node to which label_1 refers. Resolution then continues at the identified node by looking up the name label_2 in its directory table, and so on. Assuming that the named path actually exists, resolution stops at the last node referred to by label_n, by returning that node's content.

Note 6.4 (More information: The Unix naming graph again)
A name lookup returns the identifier of a node, from where the name resolution process continues. In particular, it is necessary to access the directory table of the identified node. Consider again a naming graph for a Unix file system. As mentioned, a node identifier is implemented as the index number of an inode. Accessing a directory table means that first the inode has to be read to find out where the actual data are stored on disk, and then subsequently to read the data blocks containing the directory table.

Closure mechanism

Name resolution can take place only if we know how and where to start. In our example, the starting node was given, and we assumed we had access to its directory table. Knowing how and where to start name resolution is generally referred to as a **closure mechanism**. Essentially, a closure mechanism deals with selecting the initial node in a name space, from which name resolution is to start [Radia, 1989]. What makes closure mechanisms sometimes hard to understand is that they are necessarily partly implicit and may be very different when comparing them to each other.

Consider, for example, the string "00312059837784". Many people will not know what to do with these numbers, unless they are told that the sequence is a telephone number. That information is enough to start the resolution process, in particular, by entering the number at a device for making phone calls. The telephone system subsequently does the rest.

As another example, consider the use of global and local names in distributed systems. A typical example of a local name is an environment variable. For example, in Unix systems, the variable named HOME is used to refer to the home directory of a user. Each user has their own copy of this variable, which is initialized to the global, systemwide name corresponding to the user's home directory. The closure mechanism associated with environment variables ensure that the name of the variable is properly resolved by looking it up in a user-specific table.

Note 6.5 (More information: The Unix naming graph and its closure mechanism) Name resolution in the naming graph for a Unix file system makes use of the fact that the inode of the root directory is the first inode in the logical disk representing the file system. Its actual byte offset is calculated from the values in other fields of the superblock, together with hard-coded information in the operating system itself on the internal organization of the superblock.

To make this point clear, consider the string representation of a file name, such as `/home/steen/mbox`. To resolve this name, it is necessary to already have access to the directory table of the root node of the appropriate naming graph. Being a root node, the node itself cannot have been looked up unless it is implemented as a different node in another naming graph, say *G*. But in that case, it would have been necessary to already have access to the root node of *G*. Consequently, resolving a file name requires that some mechanism has already been implemented by which the resolution process can start.

In this respect, observe how the closure mechanism works in the case of containers: the `chroot` command is used to make applications within a specific container see a different root than those in another container. In other words, each container offers its own naming graph to the contained applications, whereas the operating system hosting the containers provides a closure to properly start name resolution independent of each other.

Linking and mounting

Strongly related to name resolution is the use of **aliases**. An alias is another name for the same entity. An environment variable is an example of an alias. In terms of naming graphs, there are basically two different ways to implement an alias. The first approach is to simply allow multiple absolute paths names to refer to the same node in a naming graph. This approach is illustrated in Figure 6.10, in which the node n5 can be referred to by two different path names. In Unix terminology, both path names /keys and /home/steen/keys in Figure 6.12 are called **hard links** to node n5.

The second approach is to represent an entity by a leaf node, say N, but instead of storing the address or state of that entity, the node stores an absolute path name. When first resolving an absolute path name that leads to N, name resolution will return the path name stored in N, at which point it can continue with resolving that new path name. This principle corresponds to the use of **symbolic links** in Unix file systems, and is illustrated in Figure 6.12 In this example, the path name /home/steen/keys, which refers to a node containing the absolute path name /keys, is a symbolic link to node n5.

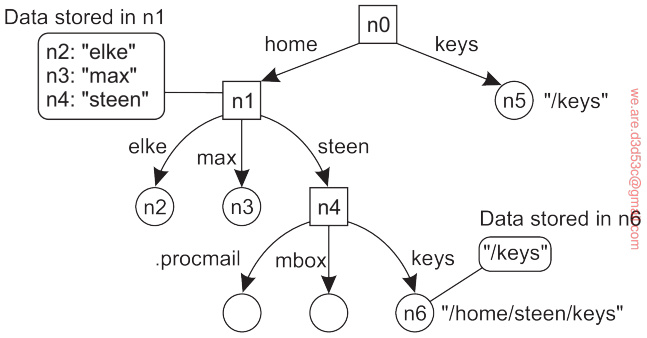


Figure 6.12: The concept of a symbolic link explained in a naming graph.

Name resolution as described so far takes place completely within a single name space. However, name resolution can also be used to merge different name spaces transparently. Let us first consider a **mounted file system**. In terms of our naming model, a mounted file system corresponds to letting a directory node store the identifier of a directory node from a *different* name space, which we refer to as a **foreign name space**. The directory node storing the node identifier is called a **mount point**. Accordingly, the directory node in the foreign name space is called a **mounting point**. Normally, the mounting point is the root of a name space. During name resolution, the mounting point is looked up and resolution proceeds by accessing its directory table.

The principle of mounting can be generalized to other name spaces as well. In particular, what is needed is a directory node that acts as a mount

point and stores all the necessary information for identifying and accessing the mounting point in the foreign name space. This approach is followed in many distributed file systems.

Consider a collection of name spaces that is distributed across different machines. In particular, each name space is implemented by a different server, each possibly running on a separate machine. Consequently, if we want to mount a foreign name space NS_2 into a name space NS_1, it may be necessary to communicate over a network with the server of NS_2, as that server may be running on a different machine than the server for NS_1. To mount a foreign name space in a distributed system requires at least the following information:

1. The name of an access protocol.
2. The name of the server.
3. The name of the mounting point in the foreign name space.

Note that each of these names needs to be resolved. The name of an access protocol needs to be resolved to the implementation of a protocol by which communication with the server of the foreign name space can take place. The name of the server needs to be resolved to an address where that server can be reached. As the last part in name resolution, the name of the mounting point needs to be resolved to a node identifier in the foreign name space.

In nondistributed systems, none of the three points may actually be needed. For example, in Unix there is no access protocol and no server. Also, the name of the mounting point is not necessary, as it is simply the root directory of the foreign name space.

The name of the mounting point is to be resolved by the server of the foreign name space. However, we also need name spaces and implementations for the access protocol and the server name. One possibility is to represent the three names listed above as a URL.

To make matters concrete, consider a situation in which a user with a laptop computer wants to access files that are stored on a remote file server. The client machine and the file server are both configured with the **Network File System** (**NFS**). In particular, to allow NFS to work across the Internet, a client can specify exactly which file it wants to access by means of an NFS URL, for example, `nfs://flits.cs.vu.nl/home/steen`. This URL names a file (which happens to be a directory) called `/home/steen` on an NFS file server `flits.cs.vu.nl`, which can be accessed by a client through the NFS protocol [Haynes, 2015; Noveck and Lever, 2020].

The name `nfs` is a well-known name, in the sense that worldwide agreement exists on how to interpret that name. Given that we are dealing with a URL, the name `nfs` will be resolved to an implementation of the NFS protocol. The server name is resolved to its address using DNS, which is discussed in a later section. As we said, `/home/steen` is resolved by the server of the foreign name space.

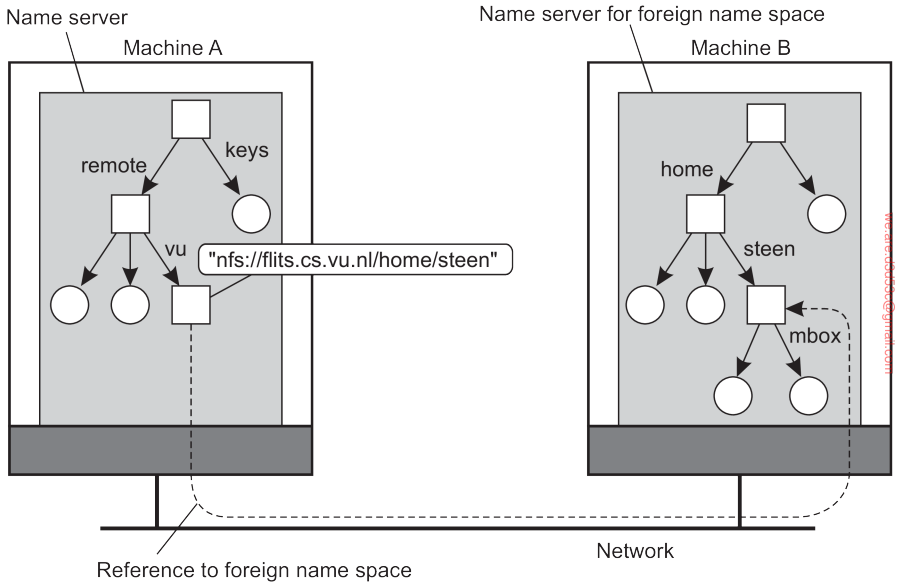


Figure 6.13: Mounting remote name spaces through a specific protocol.

The organization of a file system on the client machine is partly shown in Figure 6.13 The root directory has a number of user-defined entries, including a subdirectory called `/remote`. This subdirectory is intended to include mount points for foreign name spaces, such as the user's home directory at VU University. To this end, a directory node named `/remote/vu` is used to store the URL `nfs://flits.cs.vu.nl/home/steen`.

Now consider the name `/remote/vu/mbox`. This name is resolved by starting in the root directory on the client's machine and continues until the node `/remote/vu` is reached. The process of name resolution then continues by returning the URL `nfs://flits.cs.vu.nl/home/steen`, in turn leading the client machine to contact the file server `flits.cs.vu.nl` through the NFS protocol, and to subsequently access directory `/home/steen`. Name resolution can then be continued by reading the file named `mbox` in that directory, after which the resolution process stops.

Distributed systems that allow mounting a remote file system as just described allow a client machine to, for example, execute the following commands (assume the client machine is named `horton`):

```
horton$ cd /remote/vu
horton$ ls -l
```

which subsequently lists the files in the directory `/home/steen` on the remote file server. The beauty of all this is that the user is spared the details of the

actual access to the remote server. Ideally, only some loss in performance is noticed compared to accessing locally available files. In effect, to the client it appears that the name space rooted on the local machine, and the one rooted at /home/steen on the remote machine, form a single name space.

Note 6.6 (More information: Mounting across a network in Unix)
There are many ways in which mounting across a network can take place. One practical solution and adopted by many small-scale distributed systems, is to simply assign fixed IP addresses to machines and subsequently offer mounting points to clients. Consider the following example. Suppose we have a Unix machine named coltrane using the private address 192.168.2.3, storing a collection of music files under the local directory /audio. This directory can be *exported* as a mounting point, and as a consequence can be *imported* by another machine.

Let quandar be such a machine, and suppose it wants to mount the collection of audio files at the local mount point /home/maarten/Music. The following command will do the job (assuming the correct privileges have been set):

```
quandar$ mount -t nfs 192.168.2.3:/audio /home/maarten/Music
```

From that moment on, all files available on coltrane in its directory /audio can be accessed by quandar in the directory /home/maarten/Music. The beauty of this scheme is that once mounted, there is no need to think of remote access anymore (until something fails, of course).

6.3.3 The implementation of a name space

A name space forms the heart of a naming service, that is, a service that allows users and processes to add, remove, and look up names. A naming service is implemented by name servers. If a distributed system is restricted to a local-area network, it is often feasible to implement a naming service by only a single name server. However, in large-scale distributed systems with many entities, possibly spread across a large geographical area, it is necessary to distribute the implementation of a name space over multiple name servers.

Name space distribution

Name spaces for a large-scale, possibly worldwide distributed system, are usually organized hierarchically. As before, assume such a name space has only a single root node. To effectively implement such a name space, it is convenient to partition it into logical layers. Cheriton and Mann [1989] distinguish the following three layers.

The **global layer** is formed by highest-level nodes, that is, the root node and other directory nodes logically close to the root, namely its children. Nodes in the global layer are often characterized by their stability, in the sense that directory tables are rarely changed. Such nodes may represent

organizations, or groups of organizations, for which names are stored in the name space.

The **administrational layer** is formed by directory nodes that, together, are managed within a single organization. A characteristic feature of the directory nodes in the administrational layer is that they represent groups of entities that belong to the same organization or administrational unit. For example, there may be a directory node for each department in an organization, or a directory node from which all hosts can be found. Another directory node may be used as the starting point for naming all users, and so forth. The nodes in the administrational layer are relatively stable, although changes generally occur more frequently than to nodes in the global layer.

Finally, the **managerial layer** consists of nodes that may typically change regularly. For example, nodes representing hosts in the local network belong to this layer. For the same reason, the layer includes nodes representing shared files, such as those for libraries or binaries. Another important class of nodes includes those that represent user-defined directories and files. In contrast to the global and administrational layer, the nodes in the managerial layer are maintained not only by system administrators, but also by individual end users of a distributed system.

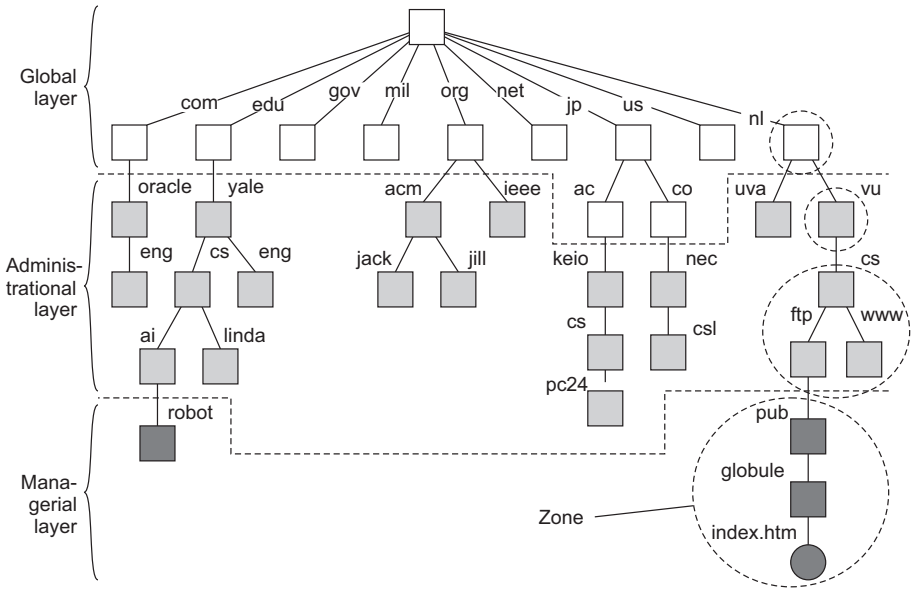


Figure 6.14: An example partitioning of the DNS name space, including Internet-accessible files, into three layers.

To make matters more concrete, Figure 6.14 shows an example of the partitioning of part of the DNS name space, including the names of files within

an organization that can be accessed through the Internet, for example, Web pages and transferable files. The name space is divided into nonoverlapping parts, called **zones** in DNS [Mockapetris, 1987a;b]. A zone is a part of the name space that is implemented by a separate name server. Some of these zones are illustrated in Figure 6.14.

If we take a look at availability and performance, name servers in each layer have to meet different requirements. High availability is especially critical for name servers in the global layer. If a name server fails, a large part of the name space will be unreachable because name resolution cannot proceed beyond the failing server.

Performance is somewhat subtle. Due to the low rate of change of nodes in the global layer, the results of lookup operations generally remain valid for a long time. Consequently, those results can be effectively cached (i.e., stored locally) by the clients. The next time the same lookup operation is performed, the results can be retrieved from the client's cache instead of letting the name server return the results. As a result, name servers in the global layer do not have to respond quickly to a single lookup request. On the other hand, throughput may be important, especially in large-scale systems with millions of users.

The availability and performance requirements for name servers in the global layer can be met by replicating servers, with client-side caching. Updates in this layer generally do not have to come into effect immediately, making it much easier to keep replicas consistent.

Availability for a name server in the administrational layer is primarily important for clients in the same organization as the name server. If the name server fails, many resources within the organization become unreachable because they cannot be looked up. On the other hand, it may be less important that resources in an organization are temporarily unreachable for users outside that organization.

Regarding performance, name servers in the administrational layer have similar characteristics as those in the global layer. Because changes to nodes do not occur all that often, caching lookup results can be highly effective, making performance less critical. However, in contrast to the global layer, the administrational layer should take care that lookup results are returned within a few milliseconds, either directly from the server or from the client's local cache. Likewise, updates should generally be processed quicker than those of the global layer. For example, it is unacceptable that an account for a new user takes hours to become effective.

These requirements can often be met by using relatively powerful machines to run name servers. In addition, client-side caching should be applied, combined with replication, for increased overall availability.

Availability requirements for name servers at the managerial level are generally less demanding. In particular, it often suffices to use a single

machine to run name servers at the risk of temporary unavailability. However, performance is crucial: operations must take place immediately. Because updates occur regularly, client-side caching is often less effective.

Issue	Global	Administrational	Managerial
Geographical scale	Worldwide	Organization	Department
Number of nodes	Few	Many	Vast numbers
Responsiveness to lookups	Seconds	Milliseconds	Immediate
Update propagation	Lazy	Immediate	Immediate
Number of replicas	Many	None or few	None
Client-side caching	Yes	Yes	Sometimes

Figure 6.15: A comparison between name servers for implementing nodes from a large-scale name space partitioned into a global layer, an administrational layer, and a managerial layer.

A comparison between name servers at different layers is shown in Figure 6.15. In distributed systems, name servers in the global and administrational layer are the most difficult to implement. Difficulties are caused by replication and caching, which are needed for availability and performance, but which also introduce consistency problems. Some of the problems are aggravated by the fact that caches and replicas are spread across a wide-area network, which may introduce long communication delays during lookups.

Implementation of name resolution

The distribution of a name space across multiple name servers affects the implementation of name resolution. To explain the implementation of name resolution in large-scale name services, we assume for the moment that name servers are not replicated and that no client-side caches are used. Each client has access to a local **name resolver**, which is responsible for ensuring that the name resolution process is carried out. Referring to Figure 6.14, assume the (absolute) path name root:[nl, vu, cs, ftp, pub, globe, index.html] is to be resolved. Using a URL notation, this path name would correspond to ftp://ftp.cs.vu.nl/pub/globe/index.html. There are now two ways to implement name resolution.

In **iterative name resolution**, a name resolver hands over the complete name to the root name server. It is assumed that the address where the root server can be contacted is well known. The root server will resolve the path name as far as it can, and return the result to the client. In our example, the root server can resolve only the label nl, for which it will return the address of the associated name server.

At that point, the client passes the remaining path name (i.e., nl:[vu, cs, ftp, pub, globe, index.html]) to that name server. This server can resolve only the

label vu, and returns the address of the associated name server, along with the remaining path name vu:[cs, ftp, pub, globe, index.html].

The client's name resolver will then contact this next name server, which responds by resolving the label cs, and subsequently also ftp, returning the address of the FTP server along with the path name ftp:[pub, globe, index.html]. The client then contacts the FTP server, requesting it to resolve the last part of the original path name. The FTP server will subsequently resolve the labels pub, globe, and index.html, and transfer the requested file (in this case using FTP). This process of iterative name resolution is shown in Figure 6.16. (The notation #[cs] is used to indicate the address of the server responsible for handling the node referred to by [cs].)

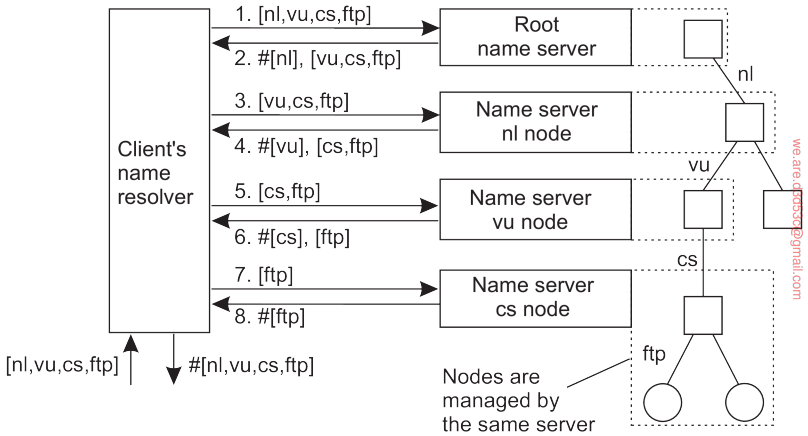


Figure 6.16: The principle of iterative name resolution.

In practice, the last step, namely contacting the FTP server and requesting it to transfer the file with path name ftp:[pub, globe, index.html], is carried out separately by the client process. In other words, the client would normally hand only the path name root:[nl, vu, cs, ftp] to the name resolver, from which it would expect the address where it can contact the FTP server, as is also shown in Figure 6.16

An alternative to iterative name resolution is to use recursion during name resolution. Instead of returning each intermediate result to the client's name resolver, with **recursive name resolution**, a name server passes the result to the next name server it finds. So, for example, when the root name server finds the address of the name server implementing the node named nl, it requests that name server to resolve the path name nl:[vu, cs, ftp, pub, globe, index.html]. Using recursive name resolution as well, this next server will resolve the complete path and eventually return the file index.html to the root server, which, in turn, will pass that file to the client's name resolver.

Recursive name resolution is shown in Figure 6.17. As in iterative name resolution, the last step (contacting the FTP server and asking it to transfer the indicated file) is generally carried out as a separate process by the client.

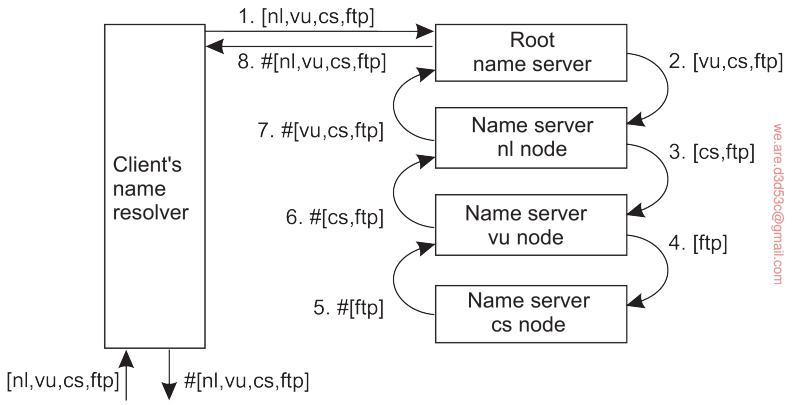


Figure 6.17: The principle of recursive name resolution.

The main drawback of recursive name resolution is that it puts a higher performance demand on each name server. Basically, a name server is required to handle the complete resolution of a path name, although it may do so in cooperation with other name servers. This additional burden is generally so high that name servers in the global layer of a name space support only iterative name resolution.

There are two important advantages to recursive name resolution. The first advantage is that caching results is more effective compared to iterative name resolution. The second advantage is that communication costs may be reduced. To explain these advantages, assume that a client's name resolver will accept path names referring only to nodes in the global or administrational layer of the name space. To resolve that part of a path name that corresponds to nodes in the managerial layer, a client will separately contact the name server returned by its name resolver, as we discussed above.

Recursive name resolution allows each name server to gradually learn the address of each name server responsible for implementing lower-level nodes. As a result, caching can be effectively used to enhance performance. For example, when the root server is requested to resolve the path name root:[nl, vu, cs, ftp], it will eventually get the address of the name server implementing the node referred to by that path name. To come to that point, the name server for the nl node has to look up the address of the name server for the vu node, whereas the latter has to look up the address of the name server handling the cs node.

Because changes to nodes in the global and administrational layer do not occur often, the root name server can effectively cache the returned address.

Moreover, because the address is also returned, by recursion, to the name server responsible for implementing the vu node and to the one implementing the nl node, it might as well be cached at those servers too.

Likewise, the results of intermediate name lookups can also be returned and cached. For example, the server for the nl node will have to look up the address of the vu node server. That address can be returned to the root server when the nl server returns the result of the original name lookup. A complete overview of the resolution process, and the results that can be cached by each name server, is shown in Figure 6.18.

Server for node	Should resolve	Looks up	Passes to child	Receives and caches	Returns to requester
cs	[ftp]	#[ftp]	—	—	#[ftp]
vu	[cs, ftp]	#[cs]	[ftp]	#[ftp]	#[cs] #[cs, ftp]
nl	[vu, cs, ftp]	#[vu]	[cs, ftp]	#[cs] #[cs, ftp]	#[vu] #[vu, cs] #[vu, cs, ftp]
root	[nl, vu, cs, ftp]	#[nl]	[vu, cs, ftp]	#[vu] #[vu, cs] #[vu, cs, ftp]	#[nl] #[nl, vu] #[nl, vu, cs] #[nl, vu, cs, ftp]

Figure 6.18: Recursive name resolution of [nl, vu, cs, ftp]. Name servers cache intermediate results for subsequent lookups.

The main benefit of this approach is that, eventually, lookup operations can be handled quite efficiently. For example, suppose that another client later requests resolution of the path name root:[nl, vu, cs, flits]. This name is passed to the root, which can immediately forward it to the name server for the cs node, and request it to resolve the remaining path name cs:[flits].

With iterative name resolution, caching is necessarily restricted to the client's name resolver. Consequently, if a client A requests the resolution of a name, and another client B later requests that same name to be resolved, name resolution will have to pass through the same name servers as was done for client A. As a compromise, many organizations use a local, intermediate name server that is shared by all clients. This local name server handles all naming requests and caches results. Such an intermediate server is also convenient from a management point of view. For example, only that server needs to know where the root name server is located; other machines do not require this information.

The second advantage of recursive name resolution is that it is often cheaper regarding communication. Again, consider the resolution of the path name root:[nl, vu, cs, ftp] and assume the client is located in San Francisco. Assuming that the client knows the address of the server for the nl node,

with recursive name resolution, communication follows the route from the client's host in San Francisco to the nl server in The Netherlands, shown as R1 in Figure 6.19 From there on, communication is subsequently needed between the nl server and the name server of VU University on the campus in Amsterdam, The Netherlands. This communication is shown as R2. Finally, communication is needed between the vu server and the name server in the Computer Science Department, shown as R3. The route for the reply is the same, but in the opposite direction. Clearly, communication costs are dictated by the message exchange between the client's host and the nl server.

In contrast, with iterative name resolution, the client's host has to communicate separately with the nl server, the vu server, and the cs server, of which the total costs may be roughly three times that of recursive name resolution. The arrows in Figure 6.19 labeled I1, I2, and I3 show the communication path for iterative name resolution.

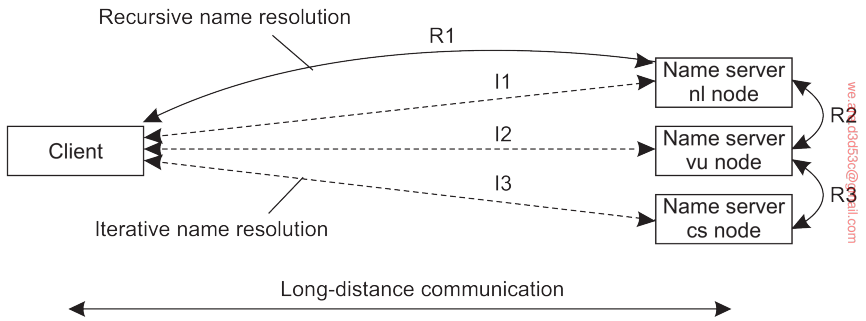


Figure 6.19: The comparison between recursive and iterative name resolution regarding communication costs.

6.3.4 Example: The Domain Name System

One of the largest distributed naming services in use today is the Internet **Domain Name System** (**DNS**). DNS is primarily used for looking up IP addresses of hosts and mail servers. In the following pages, we concentrate on the organization of the DNS name space and the information stored in its nodes. Also, we take a closer look at the actual implementation of DNS. More information can be found in [Mockapetris, 1987a;b] and [Liu and Albitz, 2006]. An excellent tutorial on DNS is provided by van der Toorn et al. [2022].

The DNS name space

The DNS name space is hierarchically organized as a rooted tree. A label is a case-insensitive string made up of alphanumeric characters. A label has a maximum length of 63 characters; the length of a complete path name is

restricted to 255 characters. The string representation of a path name consists of listing its labels, starting with the rightmost one, and separating the labels by a dot (“.”). The root is represented by a dot. So, for example, the path name root:[nl, vu, cs, flits], is represented by the string “flits.cs.vu.nl.”, which includes the rightmost dot to indicate the root node. We generally omit this dot for readability.

Because each node in the DNS name space has exactly one incoming edge (except for the root node, which has no incoming edges), the label attached to a node’s incoming edge is also used as the name for that node. A subtree is called a **domain**; a path name to its root node is called a **domain name**. Note that, just like a path name, a domain name can be either absolute or relative.

The contents of a node is formed by a collection of **resource records**. There are different types of resource records. The major ones are shown in Figure 6.20.

Type	Refers to	Description
SOA	Zone	Holds info on the represented zone
A	Host	IP addr. of host this node represents
MX	Domain	Mail server to handle mail for this node
SRV	Domain	Server handling a specific service
NS	Zone	Name server for the represented zone
CNAME	Node	Symbolic link
PTR	Host	Canonical name of a host
HINFO	Host	Info on this host
TXT	Any kind	Any info considered useful

Figure 6.20: The most important types of resource records forming the contents of nodes in the DNS name space.

A node in the DNS name space will often represent several entities at the same time. For example, a domain name such as vu.nl is used to represent a domain and a zone. In this case, the domain is implemented by means of several (nonoverlapping) zones.

An SOA (start of authority) resource record contains information such as an e-mail address of the system administrator responsible for the represented zone, the name of the host where data on the zone can be fetched, and so on.

An A (address) record, represents a particular host on the Internet. The A record contains an IP address for that host to allow communication. If a host has several IP addresses, as is the case with multi-homed machines, the node will contain an A record for each address.

Another type of record is the MX (mail exchange) record, which is like a symbolic link to a node representing a mail server. For example, the node representing the domain cs.vu.nl has an MX record that used to contain the

name zephyr.cs.vu.nl which refers to a mail server. That server would handle all incoming mail addressed to users in the cs.vu.nl domain. There may be several MX records stored in a node.

Related to MX records are SRV records, which contain the name of a server for a specific service. The service itself is identified through a name along with the name of a protocol. For example, the Web server in the cs.vu.nl domain could be named using an SRV record, such as website.cs.vu.nl. This record would then refer to the actual name of the server (which is soling.cs.vu.nl). An important advantage of SRV records is that clients need no longer know the DNS name of the host providing a specific service. Instead, only service names need to be standardized, after which the providing host can be looked up.

Nodes that represent a zone, contain one or more NS (name server) records. Like MX records, an NS record contains the name of a name server that implements the zone represented by the node. In principle, each node in the name space can store an NS record referring to the name server that implements it. However, as we discuss below, the implementation of the DNS name space is such that only nodes representing zones need to store NS records.

DNS distinguishes aliases from what are called **canonical names**. Each host is assumed to have a canonical, or primary name. An alias is implemented using a node storing a CNAME record containing the canonical name of a host. The name of the node storing such a record is thus the same as a symbolic link, as was shown in Figure 6.12.

DNS maintains an inverse mapping of IP addresses to host names using PTR (pointer) records. To accommodate the lookups of host names when given only an IP address, DNS maintains a domain named in-addr.arpa, which contains nodes that represent Internet hosts and which are named by the IP address of the represented host. For example, host www.cs.vu.nl once had IP address 130.37.20.20. DNS creates a node named 20.20.37.130.in-addr.arpa, which is used to store the canonical name of that host (which happens to be soling.cs.vu.nl in a PTR record).

Finally, an HINFO (host info) record is used to store additional information on a host, such as its machine type and operating system. Similarly, TXT records are used for any other kind of data that a user finds useful to store about the entity represented by the node.

DNS implementation

In essence, the DNS name space can be divided into a global layer and an administrational layer, as shown in Figure 6.14. The managerial layer, which is generally formed by local file systems, is formally not part of DNS, and is therefore also not managed by it.

Each zone is implemented by a name server, which is virtually always replicated for availability. Updates for a zone are normally handled by the primary name server. Updates take place by modifying the DNS database local to the primary server. Secondary name servers do not access the database directly, but, instead, request the primary server to transfer its content. The latter is called a **zone transfer** in DNS terminology.

A DNS database is implemented as a (small) collection of files, of which the most important one contains all the resource records for *all* the nodes in a particular zone. This approach allows nodes to be simply identified through their domain name, by which the notion of a node identifier reduces to an (implicit) index into a file.

Note 6.7 (More information: An example DNS database)
To better understand these implementation issues, Figure 6.21 shows a small part of the file that contains most of the information for a previous organization of the cs.vu.nl domain. Note that we have deliberately chosen an outdated version for security reasons. The file has been edited for readability. It shows the content of several nodes that used to be part of the cs.vu.nl domain, where each node is identified through its domain name.

In this example, the node cs.vu.nl represents the domain as well as the zone. Its SOA resource record contains specific information on the validity of this file, which will not concern us further. There are four name servers for this zone, referred to by their canonical host names in the NS records. The TXT record is used to give some additional information on this zone, but cannot be automatically processed by any name server. Furthermore, there is a single mail server that can handle incoming mail addressed to users in this domain. The number preceding the name of a mail server specifies a selection priority. A sending mail server should always first attempt to contact the mail server with the lowest number.

The host star.cs.vu.nl operates as a name server for this zone. Name servers are critical to any naming service. What can be seen about this name server is that additional robustness has been created by giving two separate network interfaces, each represented by a separate A resource record. In this way, the effects of a broken network link can be somewhat alleviated, as the server will remain accessible.

The next four lines (for zephyr.cs.vu.nl) give the necessary information about one of the department's mail servers. Note that this mail server is also backed up by another mail server, whose path is tornado.cs.vu.nl.

The next six lines show a typical configuration in which the department's Web server, as well as the department's FTP server, are implemented by a single machine, called soling.cs.vu.nl. By executing both servers on the same machine (and essentially using that machine only for Internet services and not anything else), system management becomes easier. For example, both servers will have the same view of the file system, and for efficiency, part of the file system may be implemented on soling.cs.vu.nl. This approach is often applied in the case of Web and FTP services.

The following two lines show information on one of the department's server clusters at that time. In this case, it tells us that the address 130.37.198.0 is associated with the host name vucs-das1.cs.vu.nl.

Name	Record type	Record value
cs.vu.nl.	SOA	star.cs.vu.nl. hostmaster.cs.vu.nl. 2005092900 7200 3600 2419200 3600
cs.vu.nl.	TXT	"VU University - Computer Science"
cs.vu.nl.	MX	1 mail.few.vu.nl.
cs.vu.nl.	NS	ns.vu.nl.
cs.vu.nl.	NS	top.cs.vu.nl.
cs.vu.nl.	NS	solo.cs.vu.nl.
cs.vu.nl.	NS	star.cs.vu.nl.
star.cs.vu.nl.	A	130.37.24.6
star.cs.vu.nl.	A	192.31.231.42
star.cs.vu.nl.	MX	1 star.cs.vu.nl.
star.cs.vu.nl.	MX	666 zephyr.cs.vu.nl.
star.cs.vu.nl.	HINFO	"Sun" "Unix"
zephyr.cs.vu.nl.	A	130.37.20.10
zephyr.cs.vu.nl.	MX	1 zephyr.cs.vu.nl.
zephyr.cs.vu.nl.	MX	2 tornado.cs.vu.nl.
zephyr.cs.vu.nl.	HINFO	"Sun" "Unix"
ftp.cs.vu.nl.	CNAME	soling.cs.vu.nl.
www.cs.vu.nl.	CNAME	soling.cs.vu.nl.
soling.cs.vu.nl.	A	130.37.20.20
soling.cs.vu.nl.	MX	1 soling.cs.vu.nl.
soling.cs.vu.nl.	MX	666 zephyr.cs.vu.nl.
soling.cs.vu.nl.	HINFO	"Sun" "Unix"
vucs-das1.cs.vu.nl.	PTR	0.198.37.130.in-addr.arpa.
vucs-das1.cs.vu.nl.	A	130.37.198.0
inkt.cs.vu.nl.	HINFO	"OCE" "Proprietary"
inkt.cs.vu.nl.	A	192.168.4.3
pen.cs.vu.nl.	HINFO	"OCE" "Proprietary"
pen.cs.vu.nl.	A	192.168.4.2
localhost.cs.vu.nl.	A	127.0.0.1

Figure 6.21: An excerpt from an (old) DNS database for the zone cs.vu.nl.

The next four lines show information on two major printers connected to the local network. Note that addresses in the range 192.168.0.0 to 192.168.255.255 are private: they can be accessed only from inside the local network and are inaccessible from an arbitrary Internet host.

Because the cs.vu.nl domain was implemented as a single zone, Figure 6.21 does not include references to other zones. The way to refer to nodes in a subdomain that are implemented in a different zone is shown in Figure 6.22. What needs to be done is to specify a name server for the subdomain by simply

giving its domain name and IP address. When resolving a name for a node that lies in the cs.vu.nl domain, name resolution will continue at a certain point by reading the DNS database stored by the name server for the cs.vu.nl domain.

Name	Record type	Record value
cs.vu.nl.	NS	solo.cs.vu.nl.
cs.vu.nl.	NS	star.cs.vu.nl.
cs.vu.nl.	NS	ns.vu.nl.
cs.vu.nl.	NS	top.cs.vu.nl.
ns.vu.nl.	A	130.37.129.4
top.cs.vu.nl.	A	130.37.20.4
solo.cs.vu.nl.	A	130.37.20.5
star.cs.vu.nl.	A	130.37.24.6
star.cs.vu.nl.	A	192.31.231.42

Figure 6.22: Part of the description for the vu.nl domain which contains the cs.vu.nl domain.

Modern DNS

Following the terminology from van der Toorn et al. [2022], we speak of the *modern DNS* to refer to the current-day implementation of DNS. There are a number of changes to the DNS implementation that are important for understanding its behavior. The presentation so far, roughly assumes that that applications contact a local DNS resolver, and that this resolver subsequently contacts various name servers, as shown in Figure 6.23(a). The bounding boxes with dashed lines represent the borders of the organization where the clients, resolver, and name servers belong to, respectively. In the traditional setup, the clients contact the local DNS resolver, which can be thought of as connected to the same local network, or the one from the local ISP. The name servers each belong to a different organization.

In the modern DNS, we see three phenomena. First, many organizations will make use of an external DNS resolver. We explained in Section 3.4.4 that services like CDNs use the client's address to select a nearby server when resolving a URL. However, the address that the CDN service gets is not that of the client, but of the DNS resolver that the client uses. Clearly, if the DNS resolver is not close to the client, then a CDN may make a poor decision on selecting a nearby server. This problem can be mitigated if the local DNS resolver allows the client (or the requesting host) to also specify its own IP address. In that case, a CDN DNS name server can use that address to select the best server to eventually redirect the client to [Contavalli et al., 2016].

Second, is that many clients, and notably browsers, may actually bypass any configuration of their local organization and directly contact the DNS

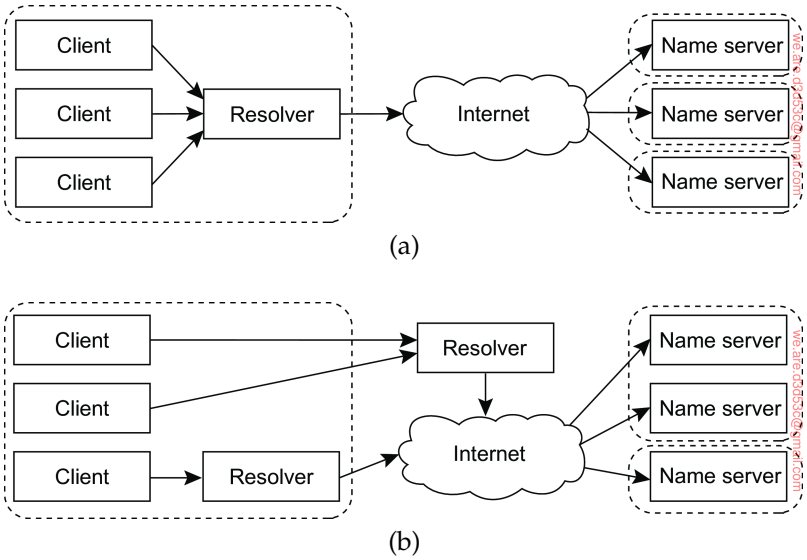


Figure 6.23: (a) The traditional organization of the implementation of DNS and (b) the modern organization that has already been partly realized (adapted from [van der Toorn et al., 2022]).

resolver of their own choice. Of course, there is nothing wrong with this, yet it makes it much harder to measure what is happening with DNS requests from an organization's perspective, especially if the client-to-resolver communication is also encrypted. In addition, the drawback of using an external resolver for redirecting requests to a nearby server also holds in this case.

The third phenomenon is that increasingly fewer organizations are running their own DNS servers. Instead, the name-resolution process is outsourced to third parties. In a sense, this means that the DNS is becoming increasingly less decentralized [Moura et al., 2020]. It is yet unclear what the consequences are of this centralization, but considering the many discussions on the role of the Big Tech companies, striving for spreading the DNS across multiple organizations may not be a bad idea.

As with so many (public) naming and lookup services, we need to provide the means for checking the validity of an answer. DNS is in the sense no exception. Moreover, being one of the most used and even necessary services on the Internet, securing the DNS is crucial. At the same time, when we realize that the DNS is becoming more centralized *and* end users are actually submitting a lot of information on how they make use of the Internet using their DNS queries, we also need to consider protecting the privacy of end users. Let us look briefly into these two subjects. A systematic overview of security and privacy issues in DNS is provided by Khormali et al. [2021].

Securing DNS responses Discussions on facilitating the validation of DNS responses started already in the 1990s. At present, we see that in a vast majority of countries worldwide, all top-level domains support what is known as **DNSSEC**, i.e., **DNS Security extensions**. DNSSEC is, in principle, a straightforward system by which resource records are signed by the organization owning a zone. To this end, DNS needed to be extended in two ways. The first, obvious one, is that new fields needed to be added to the set of records for containing signatures and keys. Second, and quite important, is that larger queries and responses had to be supported than what traditional UDP messages would allow. The original DNS completely relied on fitting everything into 512-byte packets. Using so-called extension mechanisms for DNS allows for the acceptance of DNS queries and responses beyond 512 bytes [Damas et al., 2013].

The basic idea is simple. First, resource records of the same type are grouped into a separate set, which is then subsequently signed. So, for example, a zone will contain a separate set with all its IPv4 addresses, a separate set for all its IPv6 addresses, its name servers, and so on. The public key associated with the secret key used for signing a set of resource records is also added to a zone. At that point, any client receiving a set of resource records can check that set against the provided signature. To recall, this means that the set of resource records is hashed, and the hashed value is encrypted with the secret key of the zone owner. When receiving the set, the client hashes it as well, and decrypts the version that was also in the response. When the two match, the client will believe the returned records are valid.

Of course, the client will need to trust that the provided public key, known as a **zone-signing key** is valid. To this end, all the provided zone-signing keys are grouped again into a separate set, which is subsequently signed using yet another secret key. The associated public key is known as the **key-signing key**. What happens is that a hash of the key-signing key is stored, and signed by the parent domain. The parent domain, of course, signs this hash with the secret key associated with its zone-signing key, which, in turn, is signed with the secret key associated with the parent's key-signing key. In this way, we establish a **trust chain** all the way to the root. This scheme of signing in DNSSEC is shown in Figure 6.24.

We show part of the zone information available at three different levels, including the root level (level 0). For the lowest level, we show only one specific set of resource records RR. The zone at each level k includes information concerning which hash function HZ_k is used for signing records, or HK_k for signing a zone-signing key. We use the notation SKZ_k to denote the secret key associated with the (public) key ZSK_k and, likewise, SKK_k for the secret key associated with the key-signing key KSK_k. Knowing HZ_2 and ZSK_2 will allow a client to verify the signature $SKZ_2(HZ_2(\mathsf{RR}))$ by checking

$$ZSK_2(SKZ_2(HZ_2(\mathsf{RR}))) \stackrel{?}{=} HZ_2(\mathsf{RR})$$

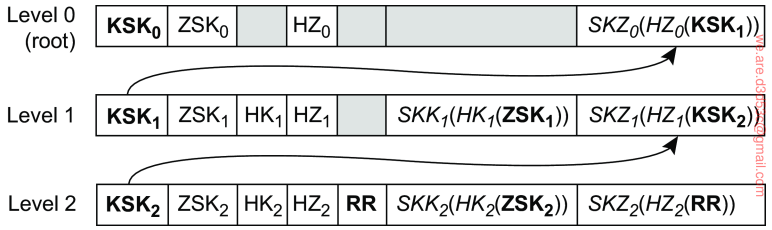


Figure 6.24: The relation between various resource records and keys in DNSSEC across three levels.

If the client wants to be sure that the public zone-signing key ZSK_2 is in order, it needs to get information from the parent zone, i.e., the zone at level 1. At that level, the zone owner has stored a signed hash value $SKZ_1(HZ_1(\mathsf{KSK}_2))$, effectively allowing anyone to validate the key-signing key KSK_2 and the use of its associated secret key SKK_2 that was used for signing ZSK_2. Obviously, the parent zone would never store such information without having going through a process of ensuring that its child zone is indeed properly owned and operating as it should.

Protecting DNS users In general, one can state that information stored in DNS is public: it can be viewed as a huge open database that allows us to resolve human-friendly names to addresses and other data. However, *what* a client is asking from the DNS is, in principle, no one else's business. That means that queries should be kept confidential. There are roughly two increasingly often supported protection mechanisms deployed in the modern DNS.

First, systems today allow applications to set up a secure channel to a remote DNS resolver through TLS, mostly by that application's local resolver, which runs on the local operating system. We discuss TLS in Section 9.3.2. In effect, having DNS over TLS prevents a third party from discovering to which Websites an application is actually referring. Obviously, both the local DNS resolver and the remote resolver need to be trusted not to leak information, and are sufficiently protected against attacks. Note that facilitating DNS over TLS requires that a resolver uses a specific port (in this case, port 853).

Second, many modern browsers support issuing DNS queries over **HTTPS**. In this case, the browser is configured to directly access a remote DNS resolver that supports DNS over HTTPS. As HTTPS runs over TLS, this mechanism essentially offers the same protection as DNS over TLS. A major difference, of course, is that DNS queries are now completely handled out of the control of local administrators. This also means that local policies concerning allowing or denying access to certain sites are bypassed, for better or for worse.

Using DNS over either TLS or HTTPS is already an improvement over

having no confidentiality concerning DNS queries, yet it may not be sufficient. Name resolution as explained so far, and illustrated in Figure 6.16 and Figure 6.17, shows that the entire path is sent to a name server. Of course, the queries between name servers can also be protected, yet a more efficient and generally sufficient protection mechanism is to let a name server ask for resolving *only* the relevant part of a path. So, for example, instead of asking to return an answer for ftp.cs.vu.nl (and get the address for the name server handling the nl domain), a resolver asks the root server to resolve .nl. We omit a few important details (such as how does the resolver know whether multiple components in a path are handled by the same name server), yet it can be seen that simply limiting a query can indeed help in attaining confidentiality.

Note 6.8 (Advanced: Decentralized versus hierarchical DNS implementations)
The implementation of DNS we described so far is the standard one. It follows a hierarchy of servers with 13 well-known root nodes and ending in millions of servers at the leaves (but read on). An important observation is that higher-level nodes receive many more requests than lower-level nodes. Only by caching the name-to-address bindings of these higher levels, it becomes possible to avoid sending requests to them and thus swamping them.

These scalability problems can, in principle, be avoided altogether with fully decentralized solutions. In particular, we can compute the hash of a DNS name, and subsequently take that hash as a key value to be looked up in a distributed hash table or a hierarchical location service with a fully partitioned root node. The obvious drawback of this approach is that we lose the structure of the original name. This loss may prevent efficient implementations of, for example, finding all children in a specific domain.

As argued by Walfish et al. [2004], when there is a need for many names, using identifiers as a semantic-free way of accessing data will allow different systems to make use of a single naming system. The reason is simple: by now, it is well understood how a huge collection of (flat) names can be efficiently supported. What needs to be done is to maintain the mapping of identifier-to-name information, where in this case a name may come from the DNS space, be a URL, and so on. Using identifiers can be made easier by letting users or organizations use a strict local name space. The latter is completely analogous to maintaining a private setting of environment variables on a computer.

Nevertheless, stating that a decentralized implementation of DNS will circumvent many of its scalability problems is too simple. In a comparative study, Pappas et al. [2006] showed that there are many trade-offs to consider and that the current, hierarchical design of DNS is not so bad for at least two reasons:

- In a hierarchical design, not all nodes are equal and in the case of DNS, notably the higher-level nodes are *engineered* differently. For example, despite that there are officially 13 root nodes, each of these nodes is highly distributed and replicated for performance and availability. To illustrate, the root node provided by RIPE NCC is implemented at some 25 different

sites (all using the same IP address), each implemented as a highly robust and replicated server cluster.

Again, we see the important difference between a logical and physical design. Exploiting this difference is crucial for the operation of a distributed system such as DNS. However, in virtually all DHT-based systems, making this distinction can be much more difficult when dealing with a logical naming hierarchy, as all names are necessarily treated to be equal. In such cases, it becomes much more difficult to engineer the system so that, for example, top-level domains are separated out by special (physical) nodes.

Of course, the obvious drawback of not having all nodes being equal, is that special measures need to be taken to protect the more important parts of a system against abuse. We have already mentioned that top-level nodes in DNS are implemented as distributed and replicated servers (clusters), but also that an associated server will not provide recursive name resolution. Such implementation decisions are necessary also from a perspective of robustness.

- DNS caches are highly effective and driven almost entirely by the local distribution of queries: if a domain D is queried often at a server S, then the references for name servers of D will be cached at S. The behavior at another server S′ is determined by what is queried at S′. This important feature has been confirmed in a more recent study that also shows how difficult it can be to understand the effectiveness of caching and the locality principles of DNS resolvers. In particular, an ISP's DNS resolver may be very effective in redirecting traffic to content that is localized in that ISP [Ager et al., 2010].

  In contrast, caching and replication in DHT-based systems generally does not show such principles of locality: results are simply cached at nodes on the return path of a lookup and have very little to do with the fact that a lookup was locally initiated at a specific node in the DHT, or a resolver for which the local ISP can assist in looking up content.

The fact remains that replacing DNS by a decentralized implementation is not necessarily a good idea. DNS as it stands today, is a well-engineered system that is difficult to beat when it comes to performance and robustness (see Vixie [2009], Vixie [2014], but also Allman [2020]).

6.3.5 Example: The Network File System

As another, and very different example, consider naming in NFS. The fundamental idea underlying the NFS naming model is to provide clients complete transparent access to a remote file system as maintained by a server. This transparency is achieved by letting a client be able to mount a remote file system into its own local file system, as shown in Figure 6.25.

Instead of mounting an entire file system, NFS allows clients to mount only part of a file system, as also shown in Figure 6.25. A server is said to

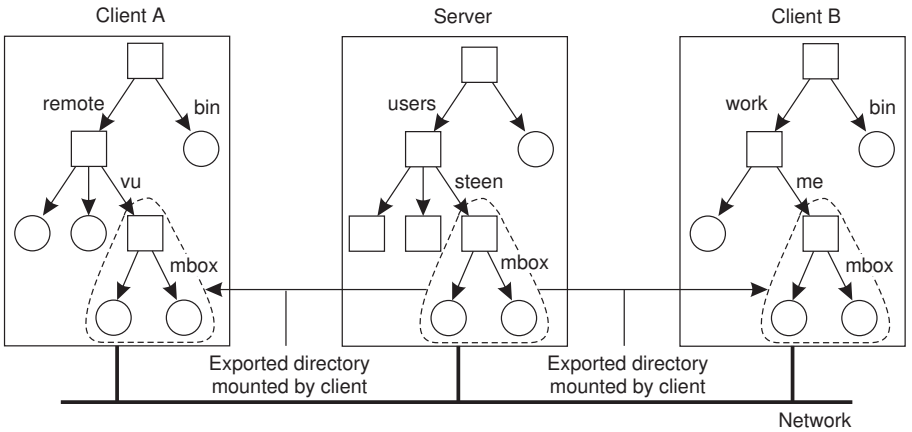


Figure 6.25: Mounting (part of) a remote file system in NFS.

export a directory when it makes that directory and its entries available to clients. An exported directory can be mounted into a client's local name space.

This design approach has a serious implication: in principle, users do not share name spaces. As shown in Figure 6.25 the file named /remote/vu/mbox at client A is named /work/me/mbox at client B. A file's name therefore depends on how clients organize their own local name space, and where exported directories are mounted. The drawback of this approach in a distributed file system is that sharing files becomes much harder. For example, Alice cannot tell Bob about a file using the name she assigned to that file, for that name may have an entirely different meaning in Bob's name space of files.

There are several ways to solve this problem, but the most common one is to provide each client with a name space that is partly standardized. For example, each client may be using the local directory /usr/bin to mount a file system containing a standard collection of programs that are available to everyone. Likewise, the directory /local may be used as a standard to mount a local file system that is located on the client's host.

An NFS server can itself mount directories that are exported by other servers. However, it is not allowed to export those directories to its own clients. Instead, a client will have to explicitly mount such a directory from the server that maintains it, as shown in Figure 6.26. This restriction comes partly from simplicity. If a server could export a directory that it mounted from another server, it would have to return special file handles that include an identifier for a server. NFS does not support such file handles.

To explain this point in more detail, assume that the server A hosts a file system FS_A from which it exports the directory /packages. This directory contains a subdirectory /draw that acts as a mount point for a file system

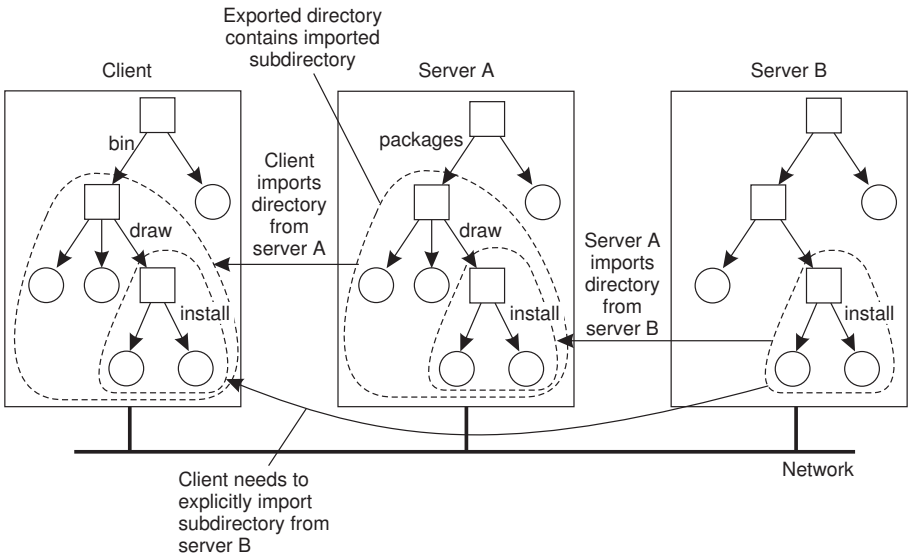


Figure 6.26: Mounting nested directories from multiple servers in NFS.

FS_B that is exported by the server B and mounted by A. Let A also export /packages/draw to its own clients, and assume that a client has mounted /packages into its local directory /bin as shown in Figure 6.26.

If name resolution is iterative, then to resolve the name /bin/draw/install, the client contacts server A when it has locally resolved /bin and requests A to return a **file handle** for directory /draw. In that case, server A should return a file handle that includes an identifier for server B, as only B can resolve the rest of the path name, in this case /install. As we have said, this kind of name resolution is not supported by NFS.

Name resolution in earlier versions of NFS is strictly iterative, in the sense that only a single file name at a time can be looked up. In other words, resolving a name such as /bin/draw/install requires three separate calls to the NFS server. Moreover, the client is fully responsible for implementing the resolution of a path name. NFSv4 also supports recursive name lookups. In this case, a client can pass a complete path name to a server and request that server to resolve it.

There is another peculiarity with NFS name lookups that has been solved with the most recent version (NFSv4). Consider a file server hosting several file systems. With the strict iterative name resolution, whenever a lookup is done for a directory on which another file system was mounted, the lookup would return the file handle of the directory. Subsequently, reading that directory would return its *original* content, not that of the root directory of the mounted file system.

To explain, assume that in our previous example that both file systems FS_A and FS_B are hosted by a single server. If the client has mounted /packages into its local directory /bin, then looking up the file name draw at the server would return the file handle for draw. A subsequent call to the server for listing the directory entries of draw by readdir would then return the list of directory entries that were *originally* stored in FS_A in subdirectory /packages/draw. Only if the client had also mounted file system FS_B, would it be possible to properly resolve the path name draw/install relative to /bin.

NFSv4 solves this problem by allowing lookups to cross mount points at a server. In particular, lookup returns the file handle of the *mounted* directory instead of that of the original directory. The client can detect that the lookup has crossed a mount point by inspecting the file system identifier of the looked up file. If required, the client can locally mount that file system as well.

A **file handle** is a reference to a file within a file system. It is independent of the name of the file it refers to. A file handle is created by the server that is hosting the file system and is unique regarding all file systems exported by the server. It is created when the file is created. The client is kept ignorant of the actual content of a file handle; it is completely opaque. File handles were 32 bytes in NFS version 2, but were variable up to 64 bytes in version 3 and 128 bytes in version 4. Of course, the length of a file handle is not opaque.

Ideally, a file handle is implemented as a true identifier for a file relative to a file system. For one thing, this means that as long as the file exists, it should have one and the same file handle. This persistence requirement allows a client to store a file handle locally once the associated file has been looked up through its name. One benefit is performance: as most file operations require a file handle instead of a name, the client can avoid having to look up a name repeatedly before every file operation. Another benefit of this approach is that the client can now access the file regardless which (current) name it has.

Because a file handle can be locally stored by a client, it is also important that a server does not reuse a file handle after deleting a file. Otherwise, a client may mistakenly access the wrong file when it uses its locally stored file handle.

Note that the combination of iterative name lookups and not letting a lookup operation allow crossing a mount point introduces a problem with getting an initial file handle. To access files in a remote file system, a client will need to provide the server with a file handle of the directory where the lookup should take place, along with the name of the file or directory that is to be resolved. NFSv3 solves this problem through a separate mount protocol, by which a client actually mounts a remote file system. After mounting, the client is passed back the **root file handle** of the mounted file system, which it can subsequently use as a starting point for looking up names.

In NFSv4, this problem is solved by providing a separate operation putrootfh that tells the server to solve all file names relative to the root file

handle of the file system it manages. The root file handle can be used to look up any other file handle in the server's file system. This approach has the additional benefit that there is no need for a separate mount protocol. Instead, mounting can be integrated into the regular protocol for looking up files. A client can simply mount a remote file system by requesting the server to resolve names relative to the file system's root file handle using putrootfh.

Note 6.9 (Advanced: Automounting)
As we mentioned, the NFS naming model essentially provides users with their own name space. Sharing in this model may become difficult if users name the same file differently. One solution to this problem is to provide each user with a local name space that is partly standardized, and subsequently mounting remote file systems the same for each user.

Another problem with the NFS naming model has to do with deciding *when* a remote file system should be mounted. Consider a large system with thousands of users. Assume that each user has a local directory /home that is used to mount the home directories of other users. For example, Alice's home directory may be locally available to her as /home/alice, although the actual files are stored on a remote server. This directory can be automatically mounted when Alice logs into her workstation. In addition, she may have access to Bob's public files by accessing Bob's directory through /home/bob.

The question, however, is whether Bob's home directory should also be mounted automatically when Alice logs in. The benefit of this approach would be that the whole business of mounting file systems would be transparent to Alice. However, if this policy were followed for every user, logging in could incur a lot of communication and administrative overhead. In addition, it would require that all users are known in advance. A much better approach is to transparently mount another user's home directory on demand, that is, when it is first needed.

On-demand mounting of a remote file system (or actually an exported directory) is handled in NFS by an **automounter**, which runs as a separate process on the client's machine. The principle underlying an automounter is relatively simple. Consider a simple automounter implemented as a user-level NFS server on a Unix operating system. (For other implementations, see [Callaghan, 2000]).

Assume that for each user, the home directories of all users are available through the local directory /home, as described above. When a client machine boots, the automounter starts with mounting this directory. The effect of this local mount is that whenever a program attempts to access /home, the Unix kernel will forward a lookup operation to the NFS client, which, in this case, will forward the request to the automounter in its role as NFS server, as shown in Figure 6.27.

For example, suppose that Alice logs in. The login program will attempt to read the directory /home/alice to find information such as login scripts. The automounter will thus receive the request to look up subdirectory /home/alice, for which reason it first creates a subdirectory /alice in /home. It then looks up the NFS server that exports Alice's home directory to subsequently mount that directory in /home/alice. At that point, the login program can proceed.

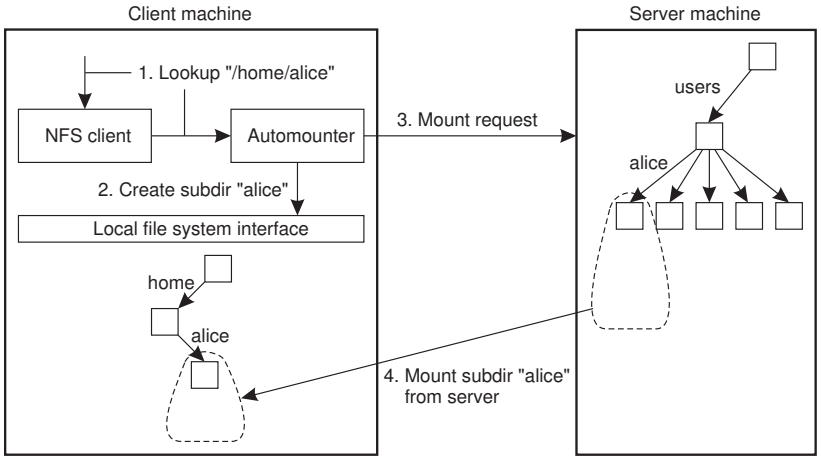


Figure 6.27: A simple automounter for NFS.

The problem with this approach is that the automounter will have to be involved in all file operations to guarantee transparency. If a referenced file is not locally available because the corresponding file system has not yet been mounted, the automounter will have to know. In particular, it will need to handle all read and write requests, even for file systems that have already been mounted. This approach may incur a large performance problem. It would be better to have the automounter only mount and unmount directories, and otherwise, stay out of the loop. A simple solution is to let the automounter mount directories in a special subdirectory, and install a symbolic link to each mounted directory. This approach is shown in Figure 6.28.

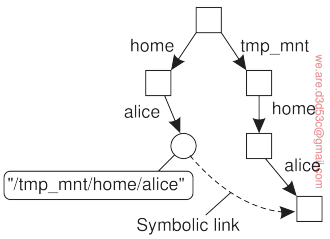


Figure 6.28: Using symbolic links with automounting.

In our example, the user home directories are mounted as subdirectories of `/tmp_mnt`. When Alice logs in, the automounter mounts her home directory in `/tmp_mnt/home/alice` and creates a symbolic link `/home/alice` that refers to that subdirectory. In this case, whenever Alice executes a command such as `ls -l /home/alice` the NFS server that exports Alice's home directory is contacted directly without further involvement of the automounter.

6.4 Attribute-based naming

Flat and structured names generally provide a unique and location-independent way of referring to entities. Moreover, structured names have been partly designed to provide a human-friendly way to name entities so that they can be conveniently accessed. In most cases, it is assumed that the name refers to only a single entity. However, location independence and human friendliness are not the only criterion for naming entities. In particular, as more information is being made available, it becomes important to effectively search for entities. This approach requires that a user can provide merely a description of what she is looking for.

There are many ways in which descriptions can be provided, but a popular one in distributed systems is to describe an entity in terms of (*attribute, value*) pairs, generally referred to as **attribute-based naming**. In this approach, an entity is assumed to have an associated collection of attributes. Each attribute says something about that entity. By specifying which values a specific attribute should have, a user essentially constrains the set of entities that she is interested in. It is up to the naming system to return one or more entities that meet the user's description. In this section, we take a closer look at attribute-based naming systems.

6.4.1 Directory services

Attribute-based naming systems are also known as **directory services**, whereas systems that support structured naming are generally called **naming systems**. With directory services, entities have a set of associated attributes that can be used for searching. In some cases, the choice of attributes can be relatively simple. For example, in an e-mail system, messages can be tagged with attributes for the sender, recipient, subject, and so on. However, even in the case of e-mail, matters become difficult when other types of descriptors are needed, as is illustrated by the difficulty of developing filters that will allow only certain messages (based on their descriptors) to be passed through.

What it all boils down to is that designing an appropriate set of attributes is not trivial. In most cases, attribute design has to be done manually. Even if there is consensus on the set of attributes to use, practice shows that setting the values consistently by a diverse group of people is a problem by itself, as many will have experienced when accessing music and video databases on the Internet.

To alleviate some of these problems, research has been conducted on unifying the ways that resources can be described. In the context of distributed systems, one particularly relevant development is the **Resource Description Framework** (**RDF**). Fundamental to the RDF model is that resources are described as triplets consisting of a subject, a predicate, and an object. For example, (Person, name, Alice) describes a resource referred to as Person whose

name is Alice. In RDF, each subject, predicate, or object can be a resource itself. This means that Alice may be implemented as a reference to a file that can be subsequently retrieved. In the case of a predicate, such a resource could contain a textual description of that predicate. Resources associated with subjects and objects can be anything. References in RDF are essentially URLs.

If resource descriptions are stored, it becomes possible to query that storage in a way that is common for many attribute-based naming systems. For example, an application could ask for the information associated with a person named Alice. Such a query would return a reference to the person resource associated with Alice. This resource can then subsequently be fetched by the application.

In this example, the resource descriptions are stored at a central location. There is no reason why the resources should reside at the same location as well. However, not having the descriptions in the same place may incur a serious performance problem. Unlike structured naming systems, looking up values in an attribute-based naming system essentially requires an exhaustive search through all descriptors. (Various techniques can be applied to avoid such exhaustive searches, an obvious one being indexing.) When considering performance, an exhaustive search may be less of a problem within a single, nondistributed data store, but simply sending a search query to hundreds of servers that jointly implement a distributed data store is generally not such a good idea. In the following, we will take a look at different approaches to solving this problem in distributed systems.

6.4.2 Hierarchical implementations: LDAP

A common approach to tackling distributed directory services is to combine structured naming with attribute-based naming. This approach has been widely adopted, for example, in Microsoft's Active Directory service and other systems. Many of these systems use, or rely on the **Lightweight Directory Access Protocol** commonly referred simply as **LDAP**. The LDAP directory service has been derived from OSI's X.500 directory service. As with many OSI services, the quality of their associated implementations hindered widespread use, and simplifications were needed to make it useful. Detailed information on LDAP can be found in [Arkills, 2003].

Conceptually, an LDAP directory service consists of a number of records, usually referred to as directory entries. A directory entry is comparable to a resource record in DNS. Each record is made up of a collection of (*attribute, value*) pairs, where each attribute has an associated type. A distinction is made between single-valued attributes and multiple-valued attributes. The latter typically represent arrays and lists. As an example, a simple directory entry identifying the network addresses of some general servers from Figure 6.22 is shown in Figure 6.28.

Attribute	Abbr.	Value
Country	C	NL
Locality	L	Amsterdam
Organization	O	VU University
OrganizationalUnit	OU	Computer Science
CommonName	CN	Main server
Mail_Servers	–	137.37.20.3, 130.37.24.6, 137.37.20.10
FTP_Server	–	130.37.20.20
WWW_Server	–	130.37.20.20

Figure 6.29: A simple example of an LDAP directory entry using LDAP naming conventions.

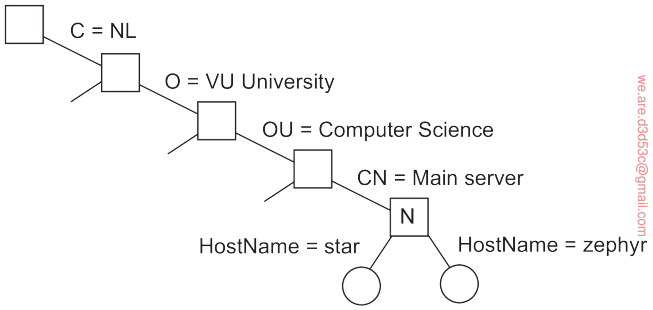
In our example, we have used a naming convention described in the LDAP standards, which applies to the first five attributes. The attributes Organization and OrganizationUnit describe, respectively, the organization and the department associated with the data that are stored in the record. Likewise, the attributes Locality and Country provide additional information on where the entry is stored. The CommonName attribute is often used as an (ambiguous) name to identify an entry within a limited part of the directory. For example, the name "Main server" may be enough to find our example entry given the specific values for the other four attributes Country, Locality, Organization, and OrganizationalUnit. In our example, only attribute Mail_Servers has multiple values associated with it. All other attributes have only a single value.

The collection of all directory entries in an LDAP directory service is called a **directory information base** (**DIB**). An important aspect of a DIB is that each record is uniquely named so that it can be looked up. Such a globally unique name appears as a sequence of naming attributes in each record. Each naming attribute is called a **relative distinguished name**, or **RDN** for short. In our example in Figure 6.29 the first five attributes are all naming attributes. Using the conventional abbreviations for representing naming attributes in LDAP, as shown in Figure 6.29, the attributes Country, Organization, and OrganizationalUnit could be used to form the name

$$/C = NL/O = VU\ University/OU = Computer\ Science$$

which is globally unique, analogous to the DNS name nl.vu.cs.

As in DNS, the use of globally unique names by listing RDNs in sequence, leads to a hierarchy of the collection of directory entries, which is referred to as a **directory information tree** (**DIT**). A DIT essentially forms the naming graph of an LDAP directory service, in which each node represents a directory entry. In addition, a node may also act as a directory in the traditional sense, in that there may be several children for which the node acts as parent. To explain, consider the naming graph, as partly shown in Figure 6.30. (Recall that labels are associated with edges.)



we.are.d3d53c@gmail.com

(a)

Attribute	Value	Attribute	Value
Locality	Amsterdam	Locality	Amsterdam
Organization	VUUniversity	Organization	VUUniversity
OrganizationalUnit	ComputerScience	OrganizationalUnit	ComputerScience
CommonName	Mainserver	CommonName	Mainserver
HostName	star	HostName	zephyr
HostAddress	192.31.231.42	HostAddress	137.37.20.10

(b)

Figure 6.30: (a) Part of a directory information tree. (b) Two directory entries having HostName as RDN.

The node N corresponds to the directory entry shown in Figure 6.29. At the same time, this node acts as a parent to a number of other directory entries that have an additional naming attribute HostName that is used as an RDN. For example, such entries may be used to represent hosts as shown in Figure 6.30.

A node in an LDAP naming graph can thus simultaneously represent a directory in the traditional sense, as we discussed previously, as well as an LDAP record. This distinction is supported by two different lookup operations. The read operation is used to read a single record given its path name in the DIT. In contrast, the list operation is used to list the names of all outgoing edges of a given node in the DIT. Each name corresponds to a child node of the given node. Note that the list operation does not return any records; it merely returns names. In other words, calling read with as input the name

$$/C = NL/O = VU\ University/OU = Computer\ Science/CN = Main\ server$$

will return the record shown in Figure 6.30, whereas calling list will return the names star and zephyr from the entries shown in Figure 6.30 as well as the names of other hosts that have been registered in a similar way.

Implementing an LDAP directory service proceeds in much the same way as implementing a naming service, such as DNS, except that LDAP supports more lookup operations, as we will discuss shortly. When dealing with a large-scale directory, the DIT is usually partitioned and distributed across several servers, known as **directory service agents** (**DSA**). Each part of a partitioned DIT thus corresponds to a zone in DNS. Likewise, each DSA behaves very much the same as a normal name server, except that it implements a number of typical directory services, such as advanced search operations.

Clients are represented by what are called **directory user agents**, or simply **DUA**. A DUA is similar to a name resolver in structured-naming services. A DUA exchanges information with a DSA according to a standardized access protocol.

What makes an LDAP implementation different from a DNS implementation are the facilities for searching through a DIB. In particular, facilities are provided to search for a directory entry given a set of criteria that attributes of the searched entries should meet. For example, suppose that we want a list of all main servers at VU University. Using the notation defined in Smith and Howes [2006], such a list can be returned using a search operation like

```
search("(C=NL)(O=VU University)(OU=*)(CN=Main server)")
```

In this example, we have specified that the place to look for main servers is the organization named VU_University in country NL, but that we are not interested in a particular organizational unit. However, each returned result should have the CN attribute equal to Main_server.

As we already mentioned, searching in a directory service is generally an expensive operation. For example, to find all main servers at VU University requires searching all entries at each department and combining the results in a single answer. In other words, we will generally need to access several leaf nodes of a DIT in order to get an answer. In practice, this also means that several DSAs need to be accessed. In contrast, naming services can often be implemented in such a way that a lookup operation requires accessing only a single leaf node.

This whole setup of LDAP can be taken one step further by allowing several trees to co-exist, while also being linked to each other. This approach is followed in Microsoft's Active Directory leading to a *forest* of LDAP domains [Allen and Lowe-Norris, 2003]. Obviously, searching in such an organization can be overwhelmingly complex. To circumvent some of the scalability problems, Active Directory usually assumes there is a global index server (called a global catalog) that can be searched first. The index will indicate which LDAP domains need to be searched further.

Although LDAP by itself already exploits hierarchy for scalability, it is common to combine LDAP with DNS. For example, every tree in LDAP needs to be accessible at the root (known in Active Directory as a domain controller).

The root is often known under a DNS name, which, in turn, can be found through an appropriate SRV record, as we explained above.

6.4.3 Decentralized implementations

Notably with the advent of peer-to-peer systems, researchers have also been looking for solutions for decentralized attribute-based naming systems. In particular, peer-to-peer systems are often used to store files. Initially, files could not be searched—they could only be looked up by their key. However, having the possibility to search for a file based on descriptors can be extremely convenient, where each descriptor is nothing but an *(attribute, value)* pair. Obviously, querying every node in a peer-to-peer system to see if it contains a file matching one or more of such pairs, is infeasible. What we need is a mapping of *(attribute, value)* pairs to **index servers**, which, in turn, point to files matching those pairs.

Using a distributed index

Let us first look at the situation of building a (distributed) index. The basic idea is that a search query is formulated as a list of *(attribute, value)* pairs, just as in the case of our LDAP examples. The result should be a list of (references to) entities that match *all* pairs. In the case of a peer-to-peer system storing files, a list of keys to relevant files may be returned, after which the client can look up each of those files using the returned keys.

A straightforward approach toward a distributed index is the following. Assume there are d different attributes. In that case, we can use a server for each of the d attributes, where a server for attribute A maintains a set of (E,val) pairs for each entity E that has the value val for attribute A. A search query such as

```
search("(Country=NL)(Organization=VU University)
       (OrganizationalUnit=*)(CommonName=Main server)")
```

would be sent to the servers for Country, Organization, and CommonName, respectively, after which the client would need to see which entities occur in *all* three sets as returned by the servers. To prevent that a server needs to maintain a large set of entities, the set for each server can be further partitioned and distributed across several subservers, each subserver associated with the same attribute.

More precisely, if we have a set of attributes $\{a^1, \ldots, a^N\}$, then for each attribute a^k we associate a set $\mathbf{S^k} = \{S_1^k, \ldots, S_{n_k}^k\}$ of n_k servers. Assuming that an attribute a^k takes values from a set R^k, we construct a global mapping F such that

$$F(a^k, v) = S_j^k \text{ with } S_j^k \in \mathbf{S^k} \text{ and } v \in R^k$$

In this example, the server S_j^k would keep track of each key associated with a file having $a^k = v$. The beauty of this scheme is its simplicity. If $L(a^k, v)$ is the set of keys returned by the server $F(a^k, v)$, then a query can be formulated as a logical expression such as

$$\big(F(a^1, v^1) \wedge F(a^2, v^2)\big) \vee F(a^3, v^3)$$

which can then be processed on the client side by constructing the set

$$\big(L(a^1, v^1) \cap L(a^2, v^2)\big) \cup L(a^3, v^3)$$

Unfortunately, there are important drawbacks to this scheme. First, any query involving k attributes requires contacting k index servers, which may incur significant communication costs. Furthermore, and related, is that the client is required to process the sets returned by the index servers. Just imagine that each file has two attributes firstName and lastName, respectively, and that a client is looking for the file owned by Pheriby Smith. Now, although Pheriby may be quite unique for a first name, Smith definitely is not. However, our poor client will have to receive perhaps millions of keys of files for which lastName $=$ Smith, while there may actually be only a handful of files for which firstName $=$ Pheriby. Thirdly, although this scheme does allow leaving certain attributes unspecified (by simply not mentioning them in the query), it does not easily support range queries, such as, price $= [1000 - -2500]$.

Space-filling curves

A common approach to implementing decentralized attribute-based naming systems is to use what are known as **space-filling curves**. The basic idea is to map the N-dimensional space covered by the N attributes $\{a^1, \ldots, a^N\}$ into a single dimension, and then use, for example, a simple hashing technique to distribute the resultant space among index servers. One of the key issues is to have *(attribute, value)* pairs that are "close" to each other be handled by the same index server.

Let us make matters concrete and look into one popular case, namely Hilbert space-filling curves (see, for example, [Lawder and King, 2000]). These are easiest to explain by looking at only two dimensions, that is, considering only two distinct attributes. The possible values that each attribute can have corresponds to one axis in a two-dimensional space. Without loss of generality, we assume that each attribute takes on values in the interval $[0, 1)$. As a first approximation of the square, we divide it into four quadrants, as shown in Figure 6.31(a). All data values (x, y) with $0 \leq x, y < 0.5$ are associated with index 0. Values (x, y) with $0.5 \leq x, y < 1.0$ are associated with index 2.

We can repeat this procedure recursively for each subsquare: divide it into four smaller squares and connect the smaller squares through a single line. Using rotation and reflection, we make sure that this line can be nicely

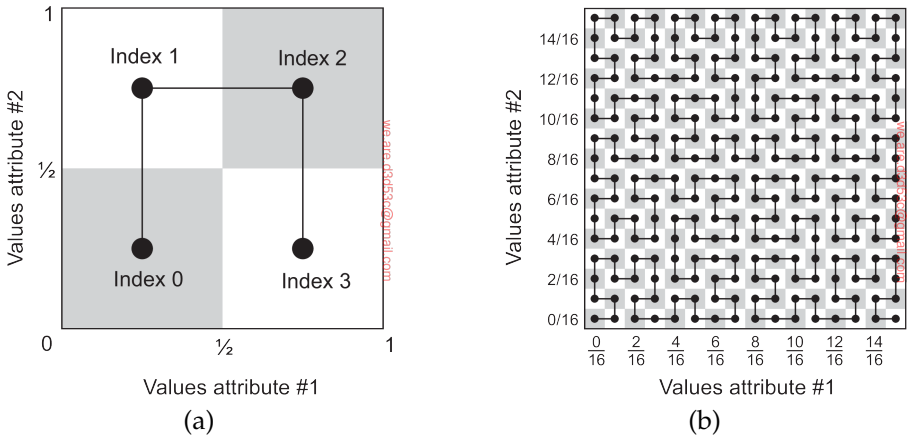


Figure 6.31: Reducing a two-dimensional space to a single dimension through a Hilbert space-filling curve of (a) order 1, and (b) order 4.

connected to the one in the previously neighboring larger subsquare (which has also been divided into smaller squares). To illustrate, where Figure 6.31(a) shows a Hilbert curve of order 1, Figure 6.31(b) shows a curve of order 4 with 256 indices. In general, a Hilbert curve of order k connects 2^{2k} subsquares, and thus has also 2^{2k} indices. There are various ways in which we can systematically draw a curve through a two-dimensional space that has been partitioned into equally sized squares. Furthermore, the process can be easily expanded to higher dimensions, as explained by Sagan [1994] and Bader [2013].

An important property of space-filling curves is that they preserve locality: two indices that are close to each other on the curve correspond to two points that are also close to each other in the multidimensional space. (Note that the reverse is not always true: two points close to each other in the multidimensional space need not necessarily lie close to each other on the curve.)

To complete the story, several things need to be done. First, attribute values need to be indexed. Assume that we are dealing with a total of N possible attributes $\{a^1, \ldots, a^N\}$, and that each entity assigns a value to each of these N attributes (possibly including the equivalent of a "don't care" value). To keep matters simple, we assume that each attribute value is normalized to a value in the interval $[0, 1)$. Then clearly, an entity E having the tuple of values $(v^1, \ldots, v^N)$ is associated with a real-valued coordinate in an N-dimensional space, in turn uniquely associated with an N-dimensional subsquare as we discussed for the two-dimensional case. The center of such a subsquare corresponds to an index on the associated Hilbert space-filling curve, and is now the index associated with the entity E. Of course, multiple entities whose associated

coordinates fall in the same subsquare will all have the same index. If we want to avoid such collisions as much as possible, we need to use high-ordered space-filling curves. Orders of 32 or 64 are not uncommon.

Second, we also need to be able to search for entities. The principle of searching for entities based on their attribute values should now be clear. Suppose we were looking for files whose two attribute values a^1 and a^2 lie in intervals $[v_l^1, v_u^1)$ and $[v_l^2, v_u^2)$, respectively (with $v_l^i < v_u^i$). Clearly, this delineates a rectangular region through which the curve passes, and all files indexed by those segments of the curve that intersect with that region match the search criterion. We therefore need an operation that returns a series of curve-related indices given a region (expressed in terms of subsquares) in the associated N-dimensional space. Such an operation is clearly dependent on which space-filling curve has been used, but interestingly, need not be dependent on actual entities.

Finally, we need to maintain (references to) the entities associated with indices. One approach, used in the Squid system [Schmidt and Parashar, 2008], is to use a Chord ring. In Squid, the index space is chosen to be the same as that of the Chord ring, that is, both use m-bit identifiers. Then clearly, the Chord node responsible for index i will store (references to) the entities indexed by i.

Note 6.10 (Example: The SWORD system)
Decentralized implementations of attribute-based naming systems have received a lot of attention. The ones based on space-filling curves are relatively popular, but several alternatives have been proposed as well. As an example, we discuss a solution adopted in the SWORD resource discovery system [Albrecht et al., 2008].

In SWORD, (*attribute, value*) pairs are first transformed into a key for a DHT. These pairs always contain a single value; only queries may contain value ranges for attributes. When computing the key (by means of a hash) the name of the attribute and its value are kept separate. Specific bits in the key will identify the attribute name, while others identify its value. In addition, the key will contain a number of random bits to guarantee uniqueness among all keys that need to be generated. In this way, the space of attributes is conveniently partitioned: if n bits are reserved to code attribute names, 2^n different server groups can be used, one group for each attribute name. Likewise, by using m bits to encode values, a further partitioning per server group can be applied to store specific (*attribute, value*) pairs. DHTs are used only for distributing attribute names.

For each attribute name, the possible range of its value is partitioned into subranges and a single server is assigned to each subrange. To explain, consider a resource description with two attributes: a^1 taking values in the range [1..10] and a^2 taking values in the range [101...200]. Assume there are two servers for a^1: S^{11} takes care of recording values of a^1 in [1..5], and S^{12} for values in [6..10]. Likewise, server S^{21} records values for a^2 in range [101..150] and server S^{22} for values in [151..200]. Then, when an entity E has associated attribute values

($\mathsf{a}^1 = 7, \mathsf{a}^2 = 175$), server S^{12} *and* server S^{22} will maintain a copy of, or a reference to E.

The advantage of this scheme is that range queries can be easily supported. When a query is issued to return resources that have a^2 lying between 165 and 189, the query can be forwarded to the server S^{22} who can then return the resources that match the query range. The drawback, however, is that updates need to be sent to multiple servers. Moreover, it is not immediately clear how well the load is balanced between the various servers. In particular, if certain range queries turn out to be very popular, specific servers will receive a high fraction of all queries.

Summarizing remarks There are indeed many ways of supporting attribute-based naming systems in a decentralized fashion. The essence in all cases is to assign attributes to servers so that clients know where to direct their queries, yet at the same time make sure that there is a balance in the load for the set of servers. In this light, supporting range queries requires special attention, if only to decide which server will be responsible for which subrange.

In practice, we see that when dealing with N attributes, many systems model the collection of *(attribute, value)* pairs as an N-dimensional space in which each entity is represented by a unique point in that space. Conceptually, a search addresses a subspace and leads to identifying the servers responsible for that subspace. In the simplest case, we assign each attribute to one server, leading to $\mathcal{O}(N)$ servers. In this scheme, a query addressing k attributes needs to be sent to k servers, while the querying client needs to combine the results. We have discussed this case previously. The problem is to divide the ranges per attribute among subservers such that we have a reasonable balance of the workload. A solution to this problem is discussed in [Bharambe et al., 2004].

Instead of letting a client combine results, we can let servers collaborate. To this end, the N-dimensional space is divided into subspaces by splitting each dimension d into n_d intervals. This splitting leads to a total of $n_1 \times \cdots \times n_N$ subspaces, where each subspace is assigned to a separate server. Even with $n_d = 2$ for each dimension, we will face a total of $\mathcal{O}(2^N)$ servers. Using space-filling curves, we can reduce the number of dimensions to one, and use a separate technique for deciding which N-dimensional subspace is served by which server. Practice indicates that load balancing may become an issue. An alternative solution in which the number of dimensions is still reduced, but larger than one, while also maintaining load balancing, has been built into HyperDex [Escriva et al., 2012]. The authors also address the problem of replication and consistency in case the naming system at the same time stores the entities which it indexes. In that case, whenever a server indexes an entity E, it will have to be copied to the respective server.

Attribute-based naming is particularly relevant for distributed systems when it comes to resource discovery and selection. An interesting case is

described by Stratan et al. [2012]. Again, the attribute space is modeled by an N-dimensional space in which each resource is associated with a coordinate. In this situation, each resource maintains a link to another resource, but one that is responsible for a subspace of exponentially increasing size. The net effect is that each resource needs to have only a fixed number of neighbors, while routing a query to the relevant subspace, takes only a linear number of steps. The organization is akin to the use of finger tables in Chord. An extensive overview of resource discovery in distributed systems is discussed by Zarrin et al. [2018].

6.5 Named-data networking

In our discussion so far, we have made a distinction between names, identifiers, and addresses. In particular, we have argued that a name needs to be resolved to an address, to access the named entity (recall that we referred to an address as the name of an entity's access point). At least on one occasion, we dropped this name-resolution process, namely when discussing how an identifier as used in Chord, or other DHTs, could be directly used to route a lookup request to a specific node. Let us now challenge the need for name-to-address resolution by taking a closer look into **Information-centric networking**, or simply **ICN** [Ahlgren et al., 2012]. In particular, we concentrate on its perhaps most popular form, namely **named-data networking** (**NDN**).

6.5.1 Basics

Named-data networking revolves around the principle that applications are not really interested to know *where* an entity is stored, but rather that they can get a copy to access it locally when needed. To this end, much research has been spent since approximately 2007 on designing an alternative to the host-based addressing schemes that are common in today's Internet. In particular, the main idea is that an application can retrieve an entity from the network by using that entity's name. The network takes that name as input, and subsequently routes a request to an appropriate location where the entity is stored, to return a copy to the requester. In effect, NDN takes over the role of IP in a future architecture of the Internet, as illustrated in Figure 6.32.

An important consequence of this organization is that instead of first resolving a name to an address, and then routing a request toward that address, the name of an entity is directly used to fetch the associated data. We already came across this scheme when discussing how a lookup request in DHTs, based on a unique key associated with an entity, is routed toward a node responsible for that key.

In NDN, names are assumed to be structured. For example, this chapter may be referred to as

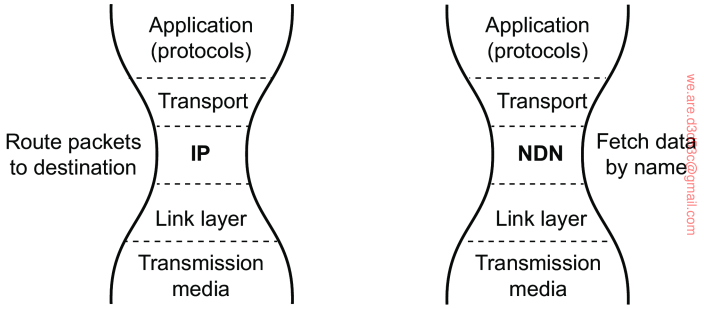


Figure 6.32: The role of NDN in comparison to IP when viewing the Internet protocol stack as an hourglass (slightly adapted from [Afanasyev et al., 2018]).

/distributed-systems.net/books/Distributed Systems/4/01/Naming

indicating the book at distributed-systems.net entitled Distributed Systems, its 4th edition, version 01, and, in particular, the chapter called Naming. This immediately raises the question how users and applications can and should name entities. One way or the other, the name, or naming scheme, of an entity should be globally known, and otherwise it becomes impossible to fetch the data associated with a name. Likewise, globally available entities should have globally unique names. How this naming is realized is outside the scope of NDN. A general overview and introduction to NDN can be found in [Afanasyev et al., 2018] and [Saxena et al., 2016].

Note that named-data networking assumes that data cannot be modified without generating a new name. Without this assumption, efficiency may be at stake, as caching by routers becomes by far less effective (as we describe shortly). This may seem to be a drawback, yet as long as a user knows that updates may be available, she can always ask for the latest version. There are various ways to achieve this. For example, when sending out a request for named data, a source may respond with the required data, but also with additional information that an update is available (and may even send that update along with the originally requested data). An example of how such schemes can be deployed for keeping up-to-date in a chat conversation is described in [Zhu and Afanasyev, 2013].

To better facilitate updates, named-data networking provides separate synchronization protocols, which are surveyed in [Moll et al., 2021]. In essence, all these protocols allow the owner of named data to announce updates to interested parties, thus essentially sending a multicast message within an NDN network. Depending on what is actually announced, a recipient may subsequently decide to fetch the update through an explicit request (which, again, is a name for the updated data).

6.5.2 Routing

For our discussion, perhaps the most interesting part of NDN is how to realize routing and forwarding of requests for named data. Initially, one might think that with seemingly arbitrary structured names routing may be inherently subject to serious scalability problems, for how to find content on the Internet by using just its name? When giving the matter some thought, the problem is fundamentally not very different when deploying IP addresses, for how would one find the access point associated with the IPv4 address 145.100.190.243, or the IPv6 address 2001:610:508:108:192:87:108:15 (both belong to surf.nl)? As argued by Zhang et al. [2019], there is really no difference and a decision needs to be made on which part of a name or address (i.e., a prefix) should be announced within a global routing substrate, just as is currently done with IPv4 addresses with BGP routers. Once a named packet has found its way into an organization's network, specific techniques can be used to forward it to places where the associated content can be found.

Returning to our example name, for

distributed-systems.net/books/Distributed Systems/4/01/Naming

we could decide to use /distributed-systems.net as a global prefix that should be used at the level of BGP routers for globally routing requests to an organization's network responsible for that prefix. Within that network, we can use the rest of the name to search for the associated content. Special NDN routers will be needed for this purpose, which we explain next.

An NDN router, shown in Figure 6.33, consists of three elements:

1. A **content store** is essentially a cache for keeping data associated with a (previously looked up) name. If a named request enters a router, and the named data is in this cache, the router immediately returns that data to the requester, as shown in Figure 6.33(a).
2. A **pending interest table** is nothing but a table that keeps track of a *(name,interface)* pair: if a request for data named N arrived through interface I of the router, then (N,I) is stored. This will allow a router to return a response whenever the associated data arrives at the router. As shown in Figure 6.33(b), if data enters the router but there is no (longer any) interest, the router can decide to drop the data.
3. A **forwarding information base** tells the router what to do when it cannot serve a request, as shown in Figure 6.33(a). For example, the request may be flooded to all neighboring routers, a random walk may be initiated, or, perhaps for whatever reason, the request may be dropped. Again, note that these decisions are fundamentally not different from routing using IPv4 or IPv6 address.

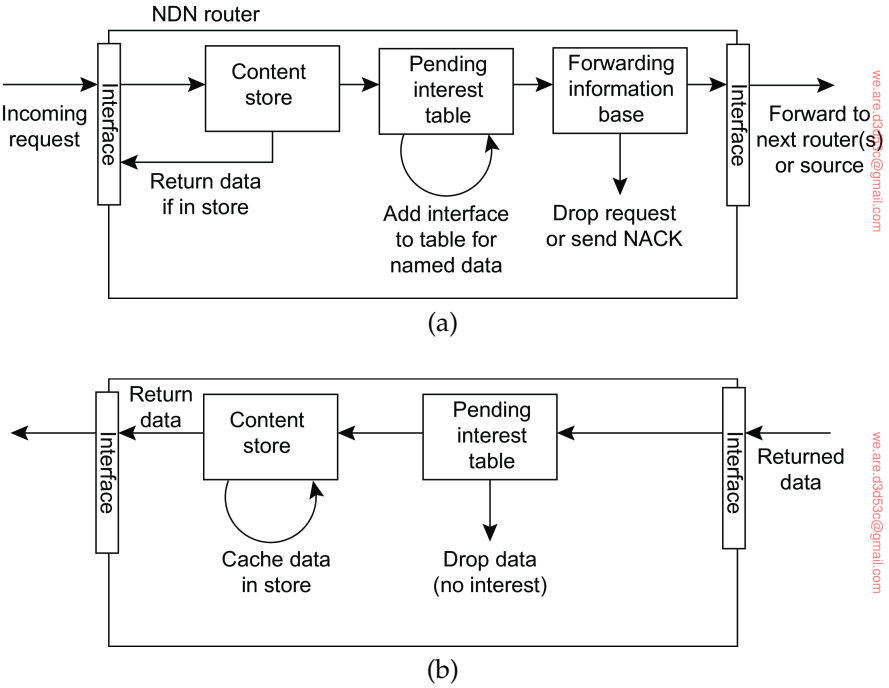


Figure 6.33: The proposed general organization of an NDN router when (a) a request flows in to the router and (b) the data is returned from another router or the source.

When a request forwarded by a router eventually returns a response, the router will look up whether there was a pending request, send it through the associated interface, and clear the entry in the pending interest table. Also, the router may decide to store the returned content in its content store for future requests, as also shown in Figure 6.33(b).

This design opens many opportunities for routing optimizations, including decisions on what to cache, when to evict data from the content store, how and to whom to forward requests, and so on. Practice will have to show whether named-data networking can indeed form a (partial) replacement of the current IP-based Internet.

6.5.3 Security in named-data networking

There are various aspects related to security in named-data networking. Besides secure naming, a weak aspect of any naming system is that the name servers and routers need to be protected as well. There are numerous threats to the NDN routers, which in the end all boil down to effectively attempting to disrupt the service. A complete overview of the various threats and possible

solutions is provided by Tourani et al. [2018] to which we refer the interested reader. Here, analogous to our discussion on secure DNS, we briefly discuss secure naming.

Secure naming is all about ensuring that the provided name is unforgeably linked to the content it refers to. The simplest way of doing this, is by including a signed digest of that content into the name, similar to self-certifying names, as we discussed in Section 6.2.5. Assuming additional information is available to the recipient of the named data, such as information on which hash function has been used, as well as the public key (perhaps including its certificate), the recipient can verify that the returned content is indeed associated with the name.

6.6 Summary

Names are used to refer to entities. Essentially, there are three types of names. An address is the name of an access point associated with an entity, also simply called the address of an entity. An identifier is another type of name. It has three properties: each entity is referred to by exactly one identifier, an identifier refers to only one entity, and is never assigned to another entity. Finally, human-friendly names are targeted to be used by humans, and as such are represented as character strings. Given these types, we make a distinction between flat naming, structured naming, and attribute-based naming.

Systems for flat naming essentially need to resolve an identifier to the address of its associated entity. This locating of an entity can be done in different ways. The first approach is to use broadcasting or multicasting. The identifier of the entity is broadcast to every process in the distributed system. The process offering an access point for the entity responds by providing an address for that access point. Obviously, this approach has limited scalability.

A second approach is to use forwarding pointers. Each time an entity moves to a next location, it leaves behind a pointer telling where it will be next. Locating the entity requires traversing the path of forwarding pointers. To avoid large chains of pointers, it is important to reduce chains periodically

A third approach is to allocate a home to an entity. Each time an entity moves to another location, it informs its home where it is. Locating an entity proceeds by first asking its home for the current location.

A fourth approach is to organize all nodes into a structured peer-to-peer system, and systematically assign nodes to entities, taking their respective identifiers into account. By subsequently devising a routing algorithm by which lookup requests are moved toward the node responsible for a given entity, efficient and robust name resolution is possible.

A fifth approach is to build a hierarchical search tree. The network is divided into nonoverlapping domains. Domains can be grouped into higher-level (nonoverlapping) domains, and so on. There is a single top-level domain

that covers the entire network. Each domain at every level has an associated directory node. If an entity is located in a domain D, the directory node of the next higher-level domain will have a pointer to D. A lowest-level directory node stores the address of the entity. The top-level directory node knows about all entities.

Structured names are easily organized in a name space. A name space can be represented by a naming graph in which a node represents a named entity and the label on an edge represents the name under which that entity is known. A node having multiple outgoing edges represents a collection of entities and is also known as a context node or directory. Large-scale naming graphs are often organized as rooted acyclic directed graphs.

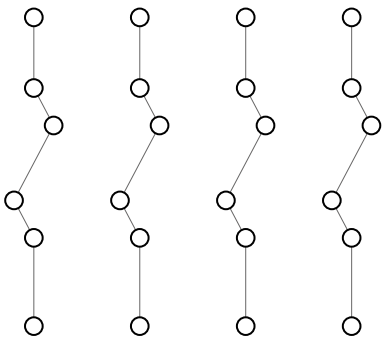
Naming graphs are convenient to organize human-friendly names in a structured way. An entity can be referred to by a path name. Name resolution is the process of traversing the naming graph by looking up the components of a path name, one at a time. A large-scale naming graph is implemented by distributing its nodes across multiple name servers. When resolving a path name by traversing the naming graph, name resolution continues at the next name server as soon as a node is reached, implemented by that server.

More problematic are attribute-based naming schemes in which entities are described by a collection of (*attribute, value*) pairs. Queries are also formulated as such pairs, essentially requiring an exhaustive search through all descriptors. Such a search is feasible only when the descriptors are stored in a single database. However, alternative solutions have been devised by which the pairs are mapped onto DHT-based systems, essentially leading to a distribution of the collection of entity descriptors. Using space-filling curves, we can then make different nodes responsible for different values of an attribute, which helps in effectively distributing the load among the nodes in the case of search operations.

Instead of making the distinction between the name and an address of an entity, researchers have been exploring the possibility of routing requests directly based on an entity's name, similar to using an identifier when looking up information in a peer-to-peer system. In so-called named-data networks, an application systematically fetches (read only) data by providing a name. In the end, the network returns the associated data instead of an address of the access point.

07

CONSISTENCY AND REPLICATION



An important issue in distributed systems is the replication of data. Data are generally replicated to enhance reliability or improve performance. One of the major problems is keeping replicas consistent. Informally, this means that when one copy is updated, we need to ensure that the other copies are updated as well; otherwise the replicas will no longer be the same. In this chapter, we take a detailed look at what consistency of replicated data actually means, and the various ways that consistency can be achieved.

We start with a general introduction discussing why replication is useful and how it relates to scalability. We then continue by focusing on what consistency actually means. An important class of what are known as consistency models assumes that multiple processes simultaneously access shared data. Consistency for these situations can be formulated regarding what processes can expect when reading and updating the shared data, knowing that others are accessing that data as well.

Consistency models for shared data are often hard to implement efficiently in large-scale distributed systems. Moreover, often simpler models can be used, which are also often easier to implement. One specific class is formed by client-centric consistency models, which concentrate on consistency from the perspective of a single (possibly mobile) client. Client-centric consistency models are discussed in a separate section.

Consistency is only half of the story. We also need to consider how consistency is actually implemented. There are essentially two, more or less independent, issues we need to consider. First, we start with concentrating on managing replicas, which considers not only the placement of replica servers, but also how content is distributed to these servers.

The second issue is how replicas are kept consistent. In most cases, applications require a strong form of consistency. Informally, this means that updates are to be propagated more or less immediately between replicas. There are various alternatives for implementing strong consistency, which are discussed in a separate section. Also, attention is paid to caching protocols, which form a special case of consistency protocols.

Being arguably the largest distributed system, we pay separate attention to caching and replication in Web-based systems, notably looking at content delivery networks as well as edge-server caching techniques.

7.1 Introduction

In this section, we start with discussing the important reasons for wanting to replicate data in the first place. We concentrate on replication as a technique for achieving scalability, and motivate why reasoning about consistency is so important.

7.1.1 Reasons for replication

There are two primary reasons for replicating data. First, data are replicated to increase the reliability of a system. If a file system has been replicated, it may be possible to continue working after one replica crashes by simply switching to one of the other replicas. Also, by maintaining multiple copies, it becomes possible to provide better protection against corrupted data. For example, imagine there are three copies of a file, and every read and write operation is performed on each copy. We can safeguard ourselves against a single, failing write operation, by considering the value that is returned by at least two copies as being the correct one.

The other reason for replicating data is performance. Replication for performance is important when a distributed system needs to scale in terms of size or in terms of the geographical area it covers. Scaling regarding size occurs, for example, when an increasing number of processes needs to access data that are managed by a single server. In that case, performance can be improved by replicating the server and subsequently dividing the workload among the processes accessing the data.

Scaling regarding a geographical area may also require replication. The basic idea is that by placing a copy of data in proximity of the process using them, the time to access the data decreases. As a consequence, the performance as perceived by that process increases. This example also illustrates that the benefits of replication for performance may be hard to evaluate. Although a client process may perceive better performance, it may also be the case that more network bandwidth is now consumed keeping all replicas up to date.

If replication helps to improve reliability and performance, who could be against it? Unfortunately, there is a price to be paid when data are replicated. The problem with replication is that having multiple copies may lead to consistency problems. Whenever a copy is modified, that copy becomes different from the rest. Consequently, modifications have to be carried out on all copies to ensure consistency. Exactly when and how those modifications need to be carried out determines the price of replication.

To understand the problem, consider improving access times to Web pages. If no special measures are taken, fetching a page from a remote Web server may sometimes even take seconds to complete. To improve performance, Web browsers often locally store a copy of a previously fetched Web page (i.e., they **cache** a Web page). If a user requires that page again, the browser automatically returns the local copy. The access time as perceived by the user, is excellent. However, if the user always wants to have the latest version of a page, she may be in for bad luck. The problem is that if the page has been modified in the meantime, modifications will not have been propagated to cached copies, making those copies out-of-date.

One solution to the problem of returning a stale copy to the user is to forbid the browser to keep local copies in the first place, effectively letting the

server be fully in charge of replication. However, this solution may still lead to poor access times if no replica is placed near the user. Another solution is to let the Web server invalidate or update each cached copy, but this requires that the server keeps track of all caches and sending them messages. This, in turn, may degrade the overall performance of the server. We return to performance versus scalability issues below.

In the following, we will mainly concentrate on replication for performance. Replication for reliability is discussed in Chapter 8.

7.1.2 Replication as scaling technique

Replication and caching for performance are widely applied as scaling techniques. Scalability issues generally appear in the form of performance problems. Placing copies of data close to the processes using them can improve performance through reduction of access time, and thus solve scalability problems.

A possible trade-off that needs to be made is that keeping copies up to date may require more network bandwidth. Consider a process P that accesses a local replica N times per second, whereas the replica itself is updated M times per second. Assume that an update completely refreshes the previous version of the local replica. If $N \ll M$, that is, the access-to-update ratio is very low, we have the situation where many updated versions of the local replica will never be accessed by P, rendering the network communication for those versions useless. In this case, it may have been better not to install a local replica close to P, or to apply a different strategy for updating the replica.

A more serious problem, however, is that keeping multiple copies consistent may itself be subject to serious scalability problems. Intuitively, a collection of copies is consistent when the copies are always the same. This means that a read operation performed at any copy will always return the same result. Consequently, when an update operation is performed on one copy, the update should be propagated to all copies before a subsequent operation takes place, no matter at which copy that operation is initiated or performed.

This type of consistency is sometimes informally (and imprecisely) referred to as tight consistency, as provided by what is also called synchronous replication. (In Section 7.2, we will provide precise definitions of consistency and introduce a range of consistency models.) The key idea is that an update is performed at all copies as a single atomic operation, or transaction. Unfortunately, implementing atomicity involving many replicas that may be widely dispersed across a large-scale network is inherently difficult when operations are also required to complete quickly.

Difficulties come from the fact that we need to synchronize all replicas. In essence, this means that all replicas first need to reach agreement on when

exactly an update is to be performed locally. For example, replicas may need to decide on a global ordering of operations using Lamport timestamps, or let a coordinator assign such an order. Global synchronization simply takes a lot of communication time, especially when replicas are spread across a wide-area network.

We are now faced with a dilemma. On the one hand, scalability problems can be alleviated by applying replication and caching, leading to improved performance. On the other hand, to keep all copies consistent generally requires global synchronization, which is inherently costly in terms of performance. The cure may be worse than the disease.

Often, the only real solution is to relax the consistency constraints. In other words, if we can relax the requirement that updates need to be executed as atomic operations, we may be able to avoid (instantaneous) global synchronizations, and may thus gain performance. The price paid is that copies may not always be the same everywhere. As it turns out, to what extent consistency can be relaxed depends highly on the access and update patterns of the replicated data, as well as on the purpose for which those data are used.

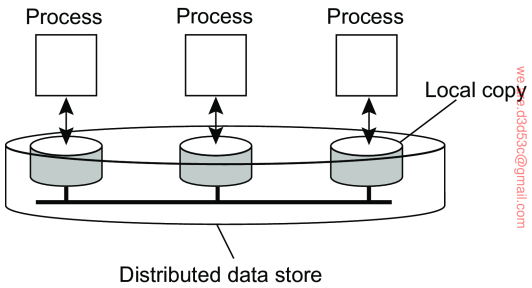
There is a range of consistency models and many ways to implement models through what are called distribution and consistency protocols. Approaches to classifying consistency and replication can be found in [Gray et al., 1996; Wiesmann et al., 2000; Aguilera and Terry, 2016], and [Viotti and Vukolic, 2016].

7.2 Data-centric consistency models

Traditionally, consistency has been discussed in the context of read and write operations on shared data, available through (distributed) shared memory, a (distributed) shared database, or a (distributed) file system. Here, we use the broader term **data store**. A data store may be physically distributed across multiple machines. In particular, each process that can access data from the store is assumed to have a local (or nearby) copy available of the entire store. Write operations are propagated to the other copies, as shown in Figure 7.1. A data operation is classified as a write operation when it changes the data, and is otherwise classified as a read operation.

A **(date centric) consistency model** is essentially a contract between processes and the data store. It says that if processes agree to obey certain rules, the store promises to work correctly. Normally, a process that performs a read operation on a data item, expects the operation to return a value that shows the results of the last write operation on that data.

Lacking a global clock, it is difficult to define precisely which write operation is the last one. As an alternative, we need to provide other definitions, leading to a range of consistency models. Each model effectively restricts the values that a read operation on a data item can return. As is to be expected,



www.cdc653c@gmail.com

Figure 7.1: The general organization of a logical data store, physically distributed and replicated across multiple processes.

the ones with major restrictions are easy to use, for example when developing applications, whereas those with minor restrictions are generally considered to be difficult to use in practice. The trade-off is, of course, that the easy-to-use models do not perform nearly as well as the difficult ones. Such is life.

7.2.1 Consistent ordering of operations

There is a huge body of work on data-centric consistency models from the past decades. An important class of models comes from the field of parallel programming. Confronted with the fact that in parallel and distributed computing multiple processes will need to share resources and access these resources simultaneously, researchers have sought to express the semantics of concurrent accesses when shared resources are replicated. The models that we discuss here all deal with consistently ordering operations on shared, replicated data.

Sequential consistency

In the following, we will use a special notation in which we draw the operations of a process along a time axis. The time axis is always drawn horizontally, with time increasing from left to right. We use the notation $W_i(x)a$ to denote that process P_i writes value a to data item x. Similarly, $R_i(x)b$ represents the fact that process P_i reads x and is returned the value b. We assume that each data item has an initial value NIL. When there is no confusion about which process is accessing data, we omit the index from the symbols W and R.

As an example, in Figure 7.2 P_1 does a write to a data item x, modifying its value to a. Note that, according to our system model the operation $W_1(x)a$ is first performed on a copy of the data store that is local to P_1, and only then is it propagated to the other local copies. In our example, P_2 later reads the value NIL, and some time after that a (from its local copy of the store). What

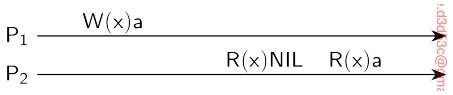


Figure 7.2: Behavior of two processes operating on the same data item. The horizontal axis is time.

we are seeing here is that it took some time to propagate the update of x to P_2, which is perfectly acceptable.

Sequential consistency is an important data-centric consistency model, which was first defined by Lamport [1979] in the context of shared memory for multiprocessor systems. A data store is said to be sequentially consistent when it satisfies the following condition:

> *The result of any execution is the same as if the (read and write) operations by all processes on the data store were executed in some sequential order and the operations of each individual process appear in this sequence in the order specified by its program.*

What this definition means is that when processes run concurrently on (possibly) different machines, any valid interleaving of read and write operations is acceptable behavior, but *all processes see the same interleaving of operations*. Note that nothing is said about time; that is, there is no reference to the "most recent" write operation on a data item. Also, a process "sees" the writes from all process, but only through its own reads.

That time does not play a role can be seen from Figure 7.3. Consider four processes operating on the same data item x. In Figure 7.3(a) process P_1 first performs $W_1(x)a$ on x. Later (in absolute time), process P_2 also performs a write operation $W_2(x)b$, by setting the value of x to b. However, both processes P_3 and P_4 *first* read value b, and *later* value a. In other words, the write operation $W_2(x)b$ of process P_2 appears to have taken place before $W_1(x)a$ of P_1.

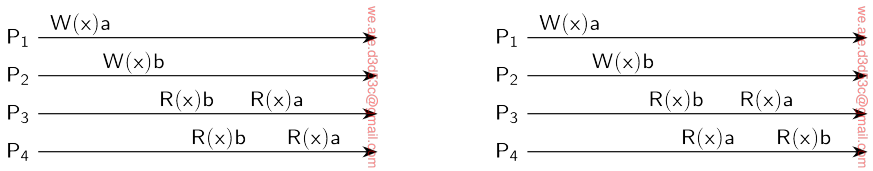


Figure 7.3: (a) A sequentially consistent data store. (b) A data store that is not sequentially consistent.

In contrast, Figure 7.3(b) violates sequential consistency because not all processes see the same interleaving of write operations. In particular, to

process P_3, it appears as if the data item has first been changed to b, and later to a. On the other hand, P_4 will conclude that the final value is b.

Process P_1	Process P_2	Process P_3
x ← 1;	y ← 1;	z ← 1;
print(y,z);	print(x,z);	print(x,y);

Figure 7.4: Three concurrently executing processes.

To make the notion of sequential consistency more concrete, consider three concurrently executing processes P_1, P_2, and P_3, shown in Figure 7.4 (taken from [Dubois et al., 1988]). The data items in this example are formed by the three integer variables x, y, and z, which are stored in a (possibly distributed) shared sequentially consistent data store. We assume that each variable is initialized to 0. In this example, an assignment corresponds to a write operation, whereas a print statement corresponds to a simultaneous read operation of its two arguments. All statements are assumed to be indivisible.

Various interleaved execution sequences are possible. With six independent statements, there are potentially 720 (6!) possible execution sequences, although some of these violate program order. Consider the 120 (5!) sequences that begin with x ← 1. Half of these have print(x,z) before y ← 1 and thus violate program order. Half also have print(x,y) before z ← 1 and also violate program order. Only 1/4 of the 120 sequences, or 30, are valid. Another 30 valid sequences are possible starting with y ← 1 and another 30 can begin with z ← 1, for a total of 90 valid execution sequences. Four of these are shown in Figure 7.5.

Execution 1	Execution 2	Execution 3	Execution 4
P_1: x ← 1;	P_1: x ← 1;	P_2: y ← 1;	P_2: y ← 1;
P_1: print(y,z);	P_2: y ← 1;	P_3: z ← 1;	P_1: x ← 1;
P_2: y ← 1;	P_2: print(x,z);	P_3: print(x,y);	P_3: z ← 1;
P_2: print(x,z);	P_1: print(y,z);	P_2: print(x,z);	P_2: print(x,z);
P_3: z ← 1;	P_3: z ← 1;	P_1: x ← 1;	P_1: print(y,z);
P_3: print(x,y);	P_3: print(x,y);	P_1: print(y,z);	P_3: print(x,y);
Prints: 001011	*Prints:* 101011	*Prints:* 010111	*Prints:* 111111
Signature: 0 0 1 0 1 1	*Signature:* 1 0 1 0 1 1	*Signature:* 1 1 0 1 0 1	*Signature:* 1 1 1 1 1 1
(a)	(b)	(c)	(d)

Figure 7.5: Four valid execution sequences for the processes of Figure 7.4. The vertical axis is time.

In Figure 7.5(a) the three processes are run in order, first P_1, then P_2, then P_3. The other three examples demonstrate different, but equally valid, interleaving of the statements in time. Each of the three processes prints

two variables. Since the only values each variable can take on are the initial value (0), or the assigned value (1), each process produces a 2-bit string. The numbers after *Prints* are the actual outputs that appear on the output device.

If we concatenate the output of P_1, P_2, and P_3 in that order, we get a 6-bit string that characterizes a particular interleaving of statements. This is the string listed as the *Signature* in Figure 7.5. Below, we will characterize each ordering by its time-independent signature rather than by its printout.

Not all 64 signature patterns are allowed. As a trivial example, 00 00 00 is not permitted because that would imply that the print statements ran before the assignment statements, violating the requirement that statements are executed in program order. A more subtle example is 00 10 01. The first two bits, 00, mean that y and z were both 0 when P_1 did its printing. This situation occurs only when P_1 executes both statements before P_2 or P_3 starts. The next two bits, 10, mean that P_2 must run after P_1 has started but before P_3 has started. The last two bits, 01, mean that P_3 must complete before P_1 starts, but we have already seen that P_1 must go first. Therefore, 00 10 01 is not allowed.

In short, the 90 different valid statement orderings produce a variety of different program results (less than 64, though) that are allowed under the assumption of sequential consistency. The contract between the processes and the distributed shared data store is that the processes must accept all of these as valid results. In other words, the processes must accept the four results shown in Figure 7.5 and all the other valid results as proper answers, and must work correctly if any of them occurs. A program that works for some of these results and not for others violates the contract with the data store and is incorrect.

This example also shows that sequential consistency is not easy to understand at first sight. In the following note, we zoom more into the intricacies.

Note 7.1 (Advanced: The importance and intricacies of sequential consistency) There is no doubt that sequential consistency is an important model. In essence, of all consistency models that exist and have been developed, it is the easiest one to understand when developing concurrent and parallel applications. This is because the model matches our expectations best when we let several programs operate on shared data simultaneously. At the same time, implementing sequential consistency is far from trivial [Adve and Boehm, 2010]. To illustrate, consider the example involving two variables x and y, shown in Figure 7.6.

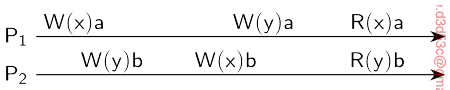


Figure 7.6: Both x and y are each handled in a sequentially consistent manner, but taken together, sequential consistency is violated.

If we just consider the write and read operations on x, the fact that P_1 reads

the value a is perfectly consistent. The same holds for the operation $R_2(y)b$ by process P_2. However, when taken together, there is no way that we can order the write operations on x and y such that we can have $R_1(x)a$ and $R_2(y)b$ (note that we need to keep the ordering as executed by each process):

Ordering of operations	Result	
$W_1(x)a$; $W_1(y)a$; $W_2(y)b$; $W_2(x)b$	$R_1(x)b$	$R_2(y)b$
$W_1(x)a$; $W_2(y)b$; $W_1(y)a$; $W_2(x)b$	$R_1(x)b$	$R_2(y)a$
$W_1(x)a$; $W_2(y)b$; $W_2(x)b$; $W_1(y)a$	$R_1(x)b$	$R_2(y)a$
$W_2(y)b$; $W_1(x)a$; $W_1(y)a$; $W_2(x)b$	$R_1(x)b$	$R_2(y)a$
$W_2(y)b$; $W_1(x)a$; $W_2(x)b$; $W_1(y)a$	$R_1(x)b$	$R_2(y)a$
$W_2(y)b$; $W_2(x)b$; $W_1(x)a$; $W_1(y)a$	$R_1(x)a$	$R_2(y)a$

In terms of transactions, the operations carried out by P_1 and P_2 are not **serializable**. Our example shows that sequential consistency is not **compositional**: when having data items that are each kept sequentially consistent, their composition as a set need not be so [Herlihy et al., 2021]. The problem of noncompositional consistency can be solved by assuming **linearizability**. This is best explained by making a distinction between the start and completion of an operation, and assuming that it may take some time. Linearizability [Herlihy and Wing, 1991] states that:

> *Each operation should appear to take effect instantaneously at some moment between its start and completion.*

Returning to our example, Figure 7.7 shows the same set of write operations, but we have now also indicated when they take place: the shaded area designates the time the operation is being executed. Linearizability states that the effect of an operation should take place somewhere during the interval indicated by the shaded area. In principle, this means that at the time of completion of a write operation, the results should be propagated to the other data stores.

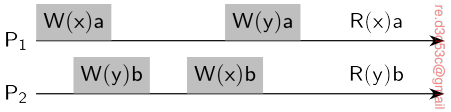


Figure 7.7: An example of taking linearizable sequential consistency into account, with only one possible outcome for x and y.

With that in mind, the possibilities for properly ordering become limited:

Ordering of operations	Result	
$W_1(x)a$; $W_2(y)b$; $W_1(y)a$; $W_2(x)b$	$R_1(x)b$	$R_2(y)a$
$W_1(x)a$; $W_2(y)b$; $W_2(x)b$; $W_1(y)a$	$R_1(x)b$	$R_2(y)a$
$W_2(y)b$; $W_1(x)a$; $W_1(y)a$; $W_2(x)b$	$R_1(x)b$	$R_2(y)a$
$W_2(y)b$; $W_1(x)a$; $W_2(x)b$; $W_1(y)a$	$R_1(x)b$	$R_2(y)a$

In particular, $W_2(y)b$ is completed before $W_1(y)a$ starts, so that y will have the value a. Likewise, $W_1(x)a$ completes before $W_2(x)b$ starts, so that x will have

value b. It should not come as a surprise that implementing linearizability on a many-core architecture may impose serious performance problems. Yet at the same time, it eases programmability considerably, so a trade-off needs to be made.

Causal consistency

The **causal consistency** model [Hutto and Ahamad, 1990] represents a weakening of sequential consistency in that it makes a distinction between events that are potentially causally related and those that are not. We already came across causality when discussing vector timestamps in the previous chapter. If event b is caused or influenced by an earlier event a, causality requires that everyone else first see a, then see b.

Consider a simple interaction using a distributed shared database. Suppose that process P_1 writes a data item x. Then P_2 reads x and writes y. Here, the reading of x and the writing of y are potentially causally related because the computation of y may have depended on the value of x as read by P_2 (i.e., the value written by P_1).

On the other hand, if two processes spontaneously and simultaneously write two different data items, these are not causally related. Operations that are not causally related are said to be **concurrent**.

For a data store to be considered causally consistent, it is necessary that the store obeys the following condition:

> *Writes that are potentially causally related must be seen by all processes in the same order. Concurrent writes may be seen in a different order on different machines.*

As an example of causal consistency, consider Figure 7.8. Here we have an event sequence that is allowed with a causally consistent store, but which is forbidden with a sequentially consistent store or a strictly consistent store. The thing to note is that the writes $W_2(x)b$ and $W_1(x)c$ are concurrent, so it is not required that all processes see them in the same order.

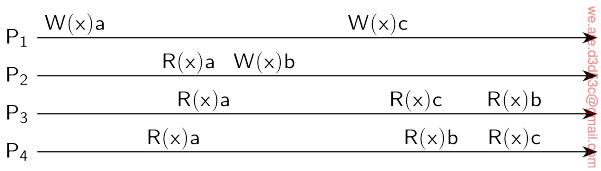


Figure 7.8: This sequence is allowed with a causally consistent store, but not with a sequentially consistent store.

Now consider a second example. In Figure 7.9(a) we have $W_2(x)b$ potentially depending on $W_1(x)a$ because writing the value b into x may be a result

of a computation involving the previously read value by $R_2(x)a$. The two writes are causally related, so all processes must see them in the same order. Therefore, Figure 7.9(a) is incorrect. On the other hand, in Figure 7.9(b) the read has been removed, so $W_1(x)a$ and $W_2(x)b$ are now concurrent writes. A causally consistent store does not require concurrent writes to be globally ordered, so Figure 7.9(b) is correct. Note that Figure 7.9(b) reflects a situation that would not be acceptable for a sequentially consistent store.

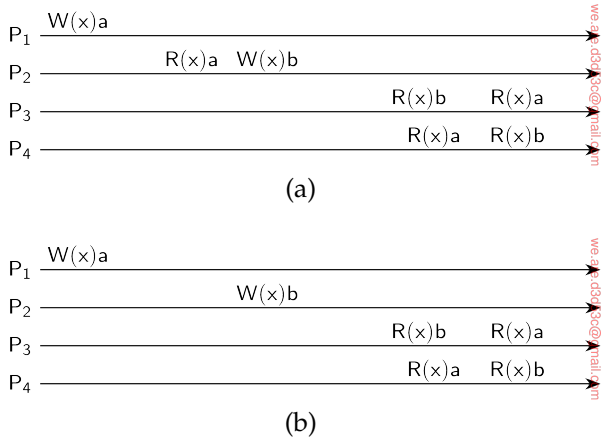


Figure 7.9: (a) A violation of a causally consistent store. (b) A correct sequence of events in a causally consistent store.

Implementing causal consistency requires keeping track of which processes have seen which writes. There are many subtle issues to consider. To illustrate, assume we replace $W_2(x)b$ in Figure 7.9(a) with $W_2(y)b$, and likewise $R_3(x)b$ with $R_3(y)b$, respectively. This situation is shown in Figure 7.10.

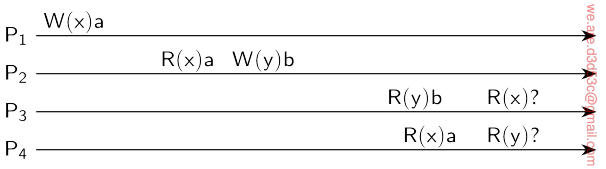


Figure 7.10: A slight modification of Figure 7.9(a). What should $R_3(x)$ or $R_4(y)$ return?

Let us first look at operation $R_3(x)$. Process P_3 executes this operation after $R_3(y)b$. We know at this point for sure that $W(x)a$ *happened before* $W(y)b$. In particular, $W(x)a \rightarrow R(x)a \rightarrow W(y)b$, meaning that if we are to preserve causality, reading x after reading b from y can return only a. If the system returned NIL to P_3 it would violate the preservation of causal relationships.

What about $R_4(y)$? Could it return the initial value of y, namely NIL? The answer is affirmative: although we have the formal *happened-before* relationship $W(x)a \rightarrow W(y)b$, without having read b from y, process P_4 can still justifiably observe that $W(x)a$ took place independently of the initialization of y.

Implementation wise, preserving causality introduces some interesting questions. Consider, for example, the middleware underlying process P_3 from Figure 7.10. At the point that this middleware returns the value b from reading y, it must know about the relationship $W(x)a \rightarrow W(y)b$. In other words, when the most recent value of y was propagated to P_3's middleware, at the very least, metadata on y's dependency should have been propagated as well. Alternatively, the propagation may have also been done together with updating x at P_3's node. By-and-large, the bottom line is that we need a dependency graph of which operation is dependent on which other operations. Such a graph may be pruned at the moment that dependent data is also locally stored.

Grouping operations

Many consistency models are defined at the level of elementary read and write operations. This level of granularity is for historical reasons: these models have initially been developed for shared-memory multiprocessor systems and were actually implemented at the hardware level.

The fine granularity of these consistency models often does not match the granularity as provided by applications. What we see there is that concurrency between programs sharing data is generally kept under control through synchronization mechanisms for mutual exclusion and transactions. Effectively, what happens is that at the program level, read and write operations are bracketed by the pair of operations ENTER_CS and LEAVE_CS. A process that has successfully executed ENTER_CS will be ensured that all the data in its local store is up-to-date. At that point, it can safely execute a series of read and write operations on that store, and subsequently wrap things up by calling LEAVE_CS. Data and instructions between ENTER_CS and LEAVE_CS is denoted as a **critical region**.

In essence, what happens is that within a program, the data that are operated on by a series of read and write operations are protected against concurrent accesses that would lead to seeing something else than the result of executing the series as a whole. Put differently, the bracketing turns the series of read and write operations into an atomically executed unit, thus raising the level of granularity.

To reach this point, we do need to have precise semantics concerning the operations ENTER_CS and LEAVE_CS. These semantics can be formulated in terms of shared **synchronization variables**, or simply **locks**. A lock has shared data items associated with it, and each shared data item is associated with at most one lock. In the case of coarse-grained synchronization, all

shared data items would be associated to just a single lock. Fine-grained synchronization is achieved when each shared data item has its own unique lock. Of course, these are just two extremes of associating shared data to a lock. When a process enters a critical region, it should *acquire* the relevant locks, and likewise when it leaves the critical region, it *releases* these locks.

Each lock has a current owner, namely, the process that last acquired it. A process not currently owning a lock but wanting to acquire it has to send a message to the current owner asking for ownership and the current values of the data associated with that lock. While having *exclusive access* to a lock, a process is allowed to perform read and write operations. It is also possible for several processes to simultaneously have *nonexclusive access* to a lock, meaning that they can read, but not write, the associated data. Of course, nonexclusive access can be granted if and only if there is no other process having exclusive access.

We now demand that the following criteria are met [Bershad et al., 1993]:

- Acquiring a lock can succeed only when all updates to its associated shared data have completed.
- Exclusive access to a lock can succeed only if no other process has exclusive or nonexclusive access to that lock.
- Nonexclusive access to a lock is allowed only if any previous exclusive access has been completed, including updates to the lock's associated data.

Note that we are effectively demanding that the usage of locks is linearized, adhering to sequential consistency. Figure 7.11 shows an example of what is known as **entry consistency**. We associate a lock with each data item separately. We use the notation $L(x)$ as an abbreviation for acquiring the lock for x, that is, *locking* x. Likewise, $U(x)$ stands for releasing the lock on x, or *unlocking* it. In this case, P_1 locks x, changes x once, after which it locks y. Process P_2 also acquires the lock for x but not for y so that it will read value a for x, but may read NIL for y. However, because process P_3 first acquires the lock for y, it will read the value b when y was unlocked by P_1. It is important to note here that each process has a *copy* of a variable, but that this copy need not be instantly or automatically updated. When locking or unlocking a variable, a process is explicitly telling the underlying distributed system that the copies of that variable need to be synchronized. A simple read operation without locking may thus result in reading a local value that is effectively stale.

One of the programming problems with entry consistency is properly associating data with locks. One straightforward approach is to explicitly tell the middleware which data will be accessed, as is generally done by declaring which database tables will be affected by a transaction. In an object-

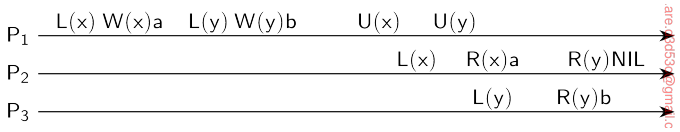


Figure 7.11: A valid event sequence for entry consistency.

based approach, we could associate a unique lock with each declared object, effectively serializing all invocations to such objects.

Note 7.2 (More information: Consistency models, serializability, transactions)
Coming to this point, the consistency models may at first sight appear to be overwhelming. Yet, despite their seemingly complexity, there are strong analogies with models that many of us are used to. In particular, let us consider transactions. As discussed in Section 1.3.2, transactions effectively group a series of read and write operations such that it is guaranteed that these operations are executed in the order of their appearance in the transaction. Moreover, when considering a collection of transactions that simultaneously operate on a data set, then the final values of that data set can be explained by one specific ordering of those transactions.

The keyword here is **serializability**: can we order the execution of operations that comprise the transactions in such a way that the final result matches a serial execution of the transactions? Consider the example shown in Figure 7.12. It is important to note that it is not necessary to know what is exactly being computed, but instead, only that we are dealing with read and write operations. In this sense, each transaction T_i can be represented as the series $\langle W_i(x); R_i(x); W_i(x)\rangle$.

```
BEGIN_TRANSACTION   BEGIN_TRANSACTION   BEGIN_TRANSACTION
  x = 0               x = 0               x = 0
  x = x + 1           x = x + 2           x = x + 3
END_TRANSACTION     END_TRANSACTION     END_TRANSACTION
```
Transaction T_1 | Transaction T_2 | Transaction T_3

Figure 7.12: Three transactions.

To ensure consistency, the underlying transaction system will have to devise legal **schedules**: orderings of the read and write operations from the three transactions such that the final result corresponds to a serial execution of those transactions. Consider the four schedules shown in Figure 7.13.

Both schedules S1 and S2 produce the result `x == 3`. This result can be explained by assuming that the scheduler executed the transactions in the order $T_1 \rightarrow T_2 \rightarrow T_3$. This ordering happens to match S1, but not S2, yet that is not relevant. Likewise, S3 is an illegal schedule, having the final result `x == 5`. Schedule S4 is interesting: the final result is `x == 3`, but that is only because of a coincidence. As the scheduler sees only (read and) write operations, and not the

effects of write operations, scheduling the operations $x = x + 1$ and then $x = x + 2$ is illegal.

	Time ⟶						
S1	x = 0	x = x + 1	x = 0	x = x + 2	x = 0	x = x + 3	Legal
S2	x = 0	x = 0	x = x + 1	x = x + 2	x = 0	x = x + 3	Legal
S3	x = 0	x = 0	x = x + 1	x = 0	x = x + 2	x = x + 3	Illegal
S4	x = 0	x = 0	x = x + 3	x = 0	x = x + 1	x = x + 2	Illegal

Figure 7.13: Various schedules from the transactions shown in Figure 7.12.

Note that what we have just described corresponds to entry consistency. Transactions consisting of only a single operation are, from a consistency point of view, analogous to sequential consistency.

Consistency versus coherence

At this point, it is useful to clarify the difference between two closely related concepts. The models we have discussed so far, all deal with the fact that a number of processes execute read and write operations on a set of data items. A **consistency model** describes what can be expected regarding that set when multiple processes concurrently operate on that data. The set is then said to be consistent if it adheres to the rules described by the model.

Where data consistency is concerned with a set of data items, **coherence models** describe what can be expected to hold for only a single data item [Cantin et al., 2005]. In this case, we assume that a data item is replicated; it is said to be coherent when the various copies abide to the rules as defined by its associated consistency model. A popular model is that of sequential consistency, but now applied to only a single data item. In effect, it means that in the case of concurrent writes, all processes will eventually see the same order of updates taking place.

7.2.2 Eventual consistency

To what extent processes actually operate in a concurrent fashion, and to what extent consistency needs to be guaranteed, may vary. There are many examples in which concurrency appears only in a restricted form. For example, in many database systems, most processes hardly ever perform update operations; they mostly read data from the database. Only one, or very few, processes perform update operations. The question then is how fast updates should be made available to only-reading processes. In the advent of globally operating content delivery networks, developers often choose to propagate updates slowly, implicitly assuming that most clients are always redirected to the same replica and will therefore never experience inconsistencies.

Another example is the Web. In virtually all cases, Web pages are updated by a single authority, such as a webmaster or the actual owner of the page. There are normally no write-write conflicts to resolve. On the other hand, to improve efficiency, browsers and Web proxies are often configured to keep a fetched page in a local cache and to return that page upon the next request. An important aspect of both types of Web caches is that they may return out-of-date Web pages. In other words, the cached page that is returned to the requesting client is an older version compared to the one available at the actual Web server. As it turns out, many users find this inconsistency acceptable (to a certain degree), as long as they have access only to the same cache. In effect, they remain unaware of the fact that an update had taken place, just as in the previous case of content delivery networks.

Yet another example, is a worldwide naming system such as DNS. The DNS name space is partitioned into domains, where each domain is assigned to a naming authority, which acts as owner of that domain. Only that authority is allowed to update its part of the name space. Consequently, conflicts resulting from two operations that both want to perform an update on the same data (i.e., **write-write conflicts**), never occur. The only situation that needs to be handled are **read-write conflicts**, in which one process intends to update a data item while another is concurrently attempting to read that item. As it turns out, also in this case it is often acceptable to propagate an update in a lazy fashion, meaning that a reading process will see an update only after some time has passed since the update took place.

These examples can be viewed as cases of (large scale) distributed and replicated databases that tolerate a relatively high degree of inconsistency. They have in common that if no updates take place for a long time, all replicas will gradually become consistent, that is, have the same data stored. This form of consistency is called **eventual consistency** [Vogels, 2009].

Data stores that are eventually consistent thus have the property that lacking write-write conflicts, all replicas will converge toward identical copies of each other. Eventual consistency essentially requires only that updates are guaranteed to propagate to all replicas. Write-write conflicts are often relatively easy to solve when assuming that only a small group of processes can perform updates. In practice, we often also see that in the case of conflicts, one specific write operation is (globally) declared as "winner," overwriting the effects of any other conflicting write operation. Eventual consistency is therefore often cheap to implement.

Note 7.3 (Advanced: Making eventual consistency stronger)
Eventual consistency is a relatively easy model to understand, but equally important is the fact that it is also relatively easy to implement. Nevertheless, it is a weak-consistency model with its own peculiarities. Consider a calendar shared between Alice, Bob, and Chuck. A meeting M has two attributes: a pro-

posed starting time and a set of people who have confirmed their attendance. When Alice proposes to start meeting M at time T, and assuming no one else has confirmed attendance, she executes the operation $W_A(M)[T, \{A\}]$. When Bob confirms his attendance, he will have read the tuple $[T, \{A\}]$ and update M accordingly: $W_B(M)[T, \{A, B\}]$. In our example two meetings M_1 and M_2 need to be planned.

Assume the sequence of events

$$W_A(M_1)[T_1, \{A\}] \rightarrow R_B(M_1)[T_1, \{A\}] \rightarrow$$

$$W_B(M_1)[T_1, \{A, B\}] \rightarrow W_B(M_2)[T_2, \{B\}].$$

In other words, Bob confirms his attendance at M_1 and then immediately proposes to schedule M_2 at T_2. Unfortunately, Chuck *concurrently* proposes to schedule M_1 at T_3 when Bob confirms he can attend M_1 at T_1. Formally, using the symbol "$\|$" to denote concurrent operations, we have,

$$W_B(M_1)[T_1, \{A, B\}] \parallel W_C(M_1)[T_3, \{C\}].$$

Using our usual notation, these operations can be illustrated as shown in Figure 7.14.

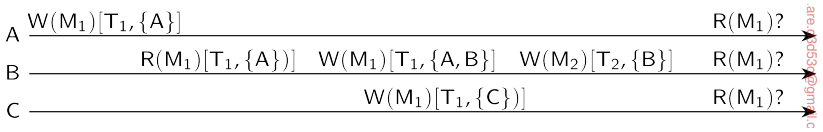


Figure 7.14: The situation of updating two meetings M_1 and M_2.

Eventual consistency may lead to very different scenarios. There are a number of write-write conflicts, but in any case, eventually $[T_2, \{B\}]$ will be stored for meeting M_2, as the result of the associated write operation by Bob. For the value of meeting M_1 there are different options. In principle, we have *three* possible outcomes: $[T_1, \{A\}]$, $[T_1, \{A, B\}]$, and $[T_3, \{C\}]$. Assuming we can maintain some notion of a global clock, it is not very likely that $W_A(M_1)[T_1, \{A\}]$ will prevail. However, the two write operations $W_B(M_1)[T_1, \{A, B\}]$ and $W_C(M_1)[T_3, \{C\}]$ are truly in conflict. In practice, one of them will win, presumably through a decision by a central coordinator.

Researchers have been seeking to combine eventual consistency with stricter guarantees on ordering. Bailis et al. [2013] propose to use a separate layer that operates on top of an eventually consistent, distributed store. This layer implements **causal consistency**, of which it has been formerly proven that it is the best attainable consistency in the presence of network partitioning [Mahajan et al., 2011]. In our example, we have only one chain of dependencies:

$$W_A(M_1)[T_1, \{A\}] \rightarrow R_B(M_1)[T_1, \{A\}] \rightarrow$$

$$W_B(M_1)[T_1, \{A, B\}] \rightarrow W_B(M_2)[T_2, \{B\}].$$

An important observation is that with causal consistency in place, once a process reads $[T_2, \{B\}]$ for meeting M_2, obtaining the value for M_1 returns either $[T_1, \{A, B\}]$ or $[T_3, \{C\}]$, but certainly not $[T_1, \{A\}]$. The reason is that $W_B(M_1)[T_1, \{A, B\}]$ immediately precedes $W_B(M_2)[T_2, \{B\}]$, and at worse may have been overwritten by $W_C(M_1)[T_3, \{C\}]$. Causal consistency rules out that the system could return $[T_1, \{A\}]$.

However, eventual consistency may overwrite previously stored data items. In doing so, dependencies may be lost. To make this point clear, it is important to realize that in practice, an operation at best keeps track of the immediate preceding operation it depends on. As soon as $W_c(M_1)[T_3, \{C\}]$ *overwrites* $W_B(M_1)[T_1, \{A, B\}]$ (and propagates to all replicas), we also break the chain of dependencies

$$W_A(M_1)[T_1, \{A\}] \rightarrow R_B(M_1)[T_1, \{A\}] \rightarrow \cdots \rightarrow W_B(M_2)[T_2, \{B\}]$$

which would normally prevent $W_A(M_1)[T_1, \{A\}]$ ever overtaking $W_B(M_1)[T_1, \{A, B\}]$ and any operation depending on it. As a consequence, maintaining causal consistency requires that we do maintain a history of dependencies, instead of just keeping track of immediately preceding operations.

Assuming that write-write conflicts hardly occur is generally not realistic. As mentioned, improvements can be made by grouping operations and using locks. What this entails, is that processes need to *coordinate* their actions by making use of mutual-exclusion mechanisms. For many large-scale systems, coordination among processes has turned out to form a real performance bottleneck. Note that we are dealing here with coordination for consistency. Alleviating potential bottleneck problems then boils down to bringing down coordination, or weakening consistency requirements. At least two approaches have been, by now, reasonably well explored, and are gradually finding their way into practical solutions.

The first one, referred to as **strong eventual consistency**, ensures that if there are conflicting updates, that nevertheless the replicas where those updates have taken place, are in the same state. Note that this is indeed stronger than (weak) eventual consistency, as in that case, nothing is specified what to do when the same replicated data item is updated by two different processes, except that the conflict may be reported. The seminal paper by Shapiro et al. [2011] introduced the **Conflict-Free Replicated Data Type**, or simply **CRDT**. A CRDT is a data type that can be replicated at many different sites, yet most importantly, can be updated in a concurrent fashion without further coordination. Of course, the meaning of concurrent updates may be different per type. For example, in the case of a variable for which the last update wins, its implementation will need to keep track of dependencies. A crude, but perhaps acceptable semantics, is that a good approximation of the actual time when an update is initiated is considered (e.g., using NTP), along

with some random number to come to a deterministic choice when several updates happen within the same small time span. An alternative is to keep all updates so that a final decision can be made at application level. Many of such examples are discussed by Preguica [2018]. Kleppmann and Beresford [2017] describe how CRDTs can be implemented to support data structures in which no update is lost. In effect, it boils down to keeping all updates while at the same time avoiding clear duplicates, and maintaining clear dependencies.

The second approach is to move away from data-centric consistency and to look at **program consistency**. Again motivated by the observation that coordination for consistency may be extremely costly, the question arises if and where coordination is truly necessary. Program consistency is all about the question whether a program produces the expected outcome, despite various kinds of anomalies that may occur (like race conditions). A key observation is that of **problem monotonicity**:

A problem P is monotonic if for any input sets S and T, $P(S) \subseteq P(T)$.

The basic idea behind a monotonic problem is that it can come to a (partial) solution even if certain input information is missing. In other words, a program solving a monotonic problem can start with incomplete information, but is guaranteed not to have to roll back when missing information becomes available. A typical example of such a problem is filling a (digital) shopping cart: operations can be performed in any order by any server. Indeed, there is no write-write conflict. However, if we also need to support the removal of items, we bump into a problem. In this case, monotonicity can help if we split the add and delete operations into two different sets, and let each of them simply grow to a final state. In this case, we need to coordinate *only* when each set has converged to a final state. The difference between what has been added and what has been deleted is what needs to be coordinated to reach consistency. Further details on program consistency can be found in [Hellerstein and Alvaro, 2020].

7.2.3 Continuous consistency

There is no such thing as the best solution to replicating data. Replicating data poses consistency problems that cannot be solved efficiently in a general way. Only if we loosen consistency can there be hope for attaining efficient solutions. Unfortunately, there are also no general rules for loosening consistency: exactly what can be tolerated is highly dependent on applications.

There are different ways for applications to specify what inconsistencies they can tolerate. Yu and Vahdat [2002] take a general approach by distinguishing three independent axes for defining inconsistencies: deviation in numerical values between replicas, deviation in staleness between replicas, and deviation with respect to the ordering of update operations. They refer to these deviations as forming **continuous consistency** ranges.

Measuring inconsistency in terms of numerical deviations can be used by applications for which the data have numerical semantics. One obvious example is the replication of records containing stock market prices. In this case, an application may specify that two copies should not deviate more than $0.02, which would be an *absolute numerical deviation*. Alternatively, a *relative numerical deviation* could be specified, stating that two copies should differ by no more than, for example, 0.5%. In both cases, we would see that if a stock goes up (and one of the replicas is immediately updated) without violating the specified numerical deviations, the replicas would still be considered to be mutually consistent.

Numerical deviation can also be understood in terms of the number of updates that have been applied to a given replica, but have not yet been seen by others. For example, a Web cache may not have seen a batch of operations carried out by a Web server. In this case, the associated deviation in the *value* is also referred to as its *weight*.

Staleness deviations relate to the last time a replica was updated. For some applications, it can be tolerated that a replica provides old data as long as it is not *too* old. For example, weather reports typically stay reasonably accurate over some time, say a few hours. In such cases, a main server may receive timely updates, but may decide to propagate updates to the replicas only once in a while.

Finally, there are classes of applications in which the ordering of updates is allowed to be different at the various replicas, as long as the differences remain bounded. One way of looking at these updates is that they are applied tentatively to a local copy, awaiting global agreement from all replicas. As a consequence, some updates may need to be rolled back and applied in a different order before becoming permanent. Intuitively, ordering deviations are much harder to grasp than the other two consistency metrics.

The notion of a conit

To define inconsistencies, Yu and Vahdat introduce a **consistency unit**, abbreviated to **conit**. A conit specifies the unit over which consistency is to be measured. For example, in our stock-exchange example, a conit could be defined as a record representing a single stock. Another example is an individual weather report.

To give an example of a conit, and at the same time illustrate numerical and ordering deviations, consider the situation of keeping track of a fleet of cars. In particular, the fleet owner is interested in knowing how much she pays on average for gas. To this end, whenever a driver tanks gasoline, she reports the amount of gasoline that has been tanked (recorded as `g`), the price paid (recorded as `p`), and the total distance since the last time she tanked (recorded by the variable `d`). Technically, the three variables `g`, `p`, and `d` form a conit. This conit is replicated across two servers, as shown in Figure 7.15, and

a driver regularly reports her gas usage to one of the servers by separately updating each variable (without further considering the car in question).

The task of the servers is to keep the conit "consistently" replicated. To this end, each replica server maintains a two-dimensional vector clock. We use the notation $\langle T, \mathsf{R} \rangle$ to express an operation that was carried out by replica R at (its) logical time T.

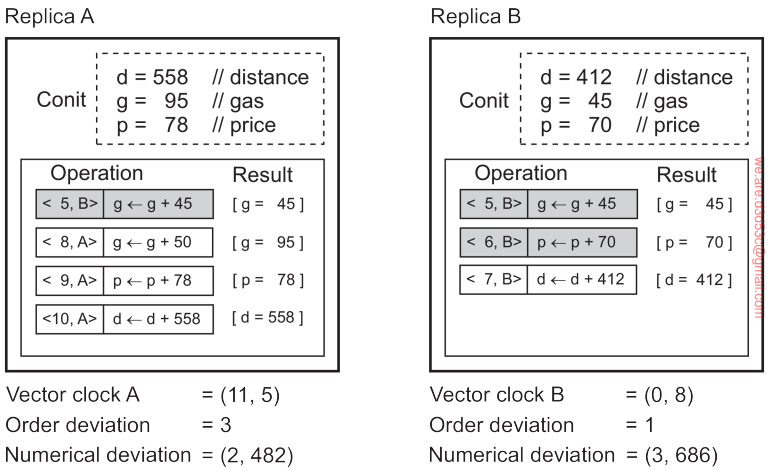


Figure 7.15: An example of keeping track of consistency deviations.

In this example, we see two replicas that operate on a conit containing the data items g, p, and d from our example. All variables are assumed to have been initialized to 0. Replica A received the operation

$$\langle 5, \mathsf{B} \rangle : \mathsf{g} \leftarrow \mathsf{g} + 45$$

from replica B. We have shaded this operation gray to indicate that A has *committed* this operation to its local store. In other words, it has been made permanent and cannot be rolled back. Replica A also has three *tentative* update operations listed: $\langle 8, \mathsf{A} \rangle$, $\langle 9, \mathsf{A} \rangle$, and $\langle 10, \mathsf{A} \rangle$, respectively. In terms of continuous consistency, the fact that A has three tentative operations pending to be committed is referred to as an **order deviation** of, in this case, value 3. Analogously, with in total three operations of which two have been committed, B has an order deviation of 1.

From this example, we see that A's logical clock value is now 11. Because the last operation from B that A had received had timestamp 5, the vector clock at A will be $(11, 5)$, where we assume the first component of the vector is used for A and the second for B. Along the same lines, the logical clock at B is $(0, 8)$.

The *numerical deviation* at a replica R consists of two components: the number of operations at all *other* replicas that have not yet been seen by

R, along with the sum of corresponding missed values (more sophisticated schemes are, of course, also possible). In our example, A has not yet seen operations $\langle 6, B\rangle$ and $\langle 7, B\rangle$ with a total value of 70 + 412 units, leading to a numerical deviation of $(2, 482)$. Likewise, B is still missing the three tentative operations at A, with a total summed value of 686, bringing B's numerical deviation to $(3, 686)$.

Using these notions, it becomes possible to specify specific consistency schemes. For example, we may restrict order deviation by specifying an acceptable maximal value. Likewise, we may want two replicas to never numerically deviate by more than 1000 units. Having such consistency schemes does require that a replica knows how much it is deviating from other replicas, implying that we need separate communication to keep replicas informed. The underlying assumption is that such communication is much less expensive than communication to keep replicas synchronized. Admittedly, it is questionable if this assumption also holds for our example.

Note 7.4 (Advanced: On the granularity of conits)
There is a trade-off between maintaining fine-grained and coarse-grained conits. If a conit represents a lot of data, such as a complete database, then updates are aggregated for all the data in the conit. As a consequence, this may bring replicas sooner in an inconsistent state. For example, assume that in Figure 7.16 two replicas may differ in no more than one outstanding update. In that case, when the data items in Figure 7.16 have each been updated once at the first replica, the second one will need to be updated as well. This is not the case when choosing a smaller conit, as shown in Figure 7.16 There, the replicas are still considered to be up-to-date. This problem is particularly important when the data items contained in a conit are used completely independently, in which case they are said to **falsely share** the conit.

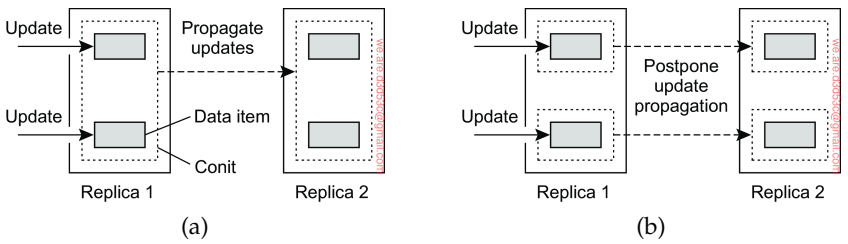


Figure 7.16: Choosing the appropriate granularity for a conit. (a) Two updates lead to update propagation. (b) No update propagation is needed.

Unfortunately, making conits small is not a good idea, for the simple reason that the total number of conits that need to be managed grows as well. In other words, there is an overhead related to managing the conits that needs to be considered. This overhead, in turn, may adversely affect overall performance, which also has to be considered.

Although, from a conceptual point of view, conits form an attractive means for capturing consistency requirements, there are two important issues that need to be dealt with before they can be put to practical use. First, to enforce consistency, we need to have protocols. Protocols for continuous consistency are discussed later in this chapter.

A second issue is that program developers must specify the consistency requirements for their applications. Practice indicates that obtaining such requirements may be extremely difficult. Programmers are generally not used to handling replication, let alone understanding what it means to provide detailed information on consistency. Therefore, it is mandatory that there are simple and easy-to-understand programming interfaces.

Note 7.5 (Advanced: Programming conits)
Continuous consistency can be implemented as a toolkit which appears to programmers as just another library that they link with their applications. A conit is simply declared alongside an update of a data item. For example, the fragment of pseudocode

```
AffectsConit(ConitQ, 1, 1);
append message m to queue Q;
```

states that appending a message to queue `Q` belongs to a conit named `ConitQ`. Likewise, operations may now also be declared as being dependent on conits:

```
DependsOnConit(ConitQ, 4, 0, 60);
read message m from head of queue Q;
```

In this case, the call to `DependsOnConit()` specifies that the numerical deviation, ordering deviation, and staleness should be limited to the values 4, 0, and 60 (seconds), respectively. This can be interpreted as that there should be at most 4 unseen update operations at other replicas, there should be no tentative local updates, and the local copy of `Q` should have been checked for staleness no more than 60 seconds ago. If these requirements are not fulfilled, the underlying middleware will attempt to bring the local copy of `Q` to a state such that the read operation can be carried out.

The question, of course, is how does the system know that `Q` is associated with `ConitQ`? For practical reasons, we can avoid explicit declarations of conits and concentrate only on the grouping of operations. The data to be replicated is collectively considered belonging together. By subsequently associating a write operation with a named conit, and likewise for a read operation, we tell the middleware layer when to start synchronizing the *entire* replica. Indeed, there may be a considerable amount of false sharing in such a case. If false sharing needs to be avoided, we would have to introduce a separate programming construct to explicitly declare conits.

7.3 Client-centric consistency models

Data-centric consistency models aim at providing a systemwide consistent view on a data store. An important assumption is that concurrent processes may be simultaneously updating the data store, and that it is necessary to provide consistency in the face of such concurrency. For example, in the case of object-based entry consistency, the data store guarantees that when an object is called, the calling process is provided with a copy of the object that reflects all changes to the object that have been made so far, possibly by other processes. During the call, it is also guaranteed that no other process can interfere, that is, mutual exclusive access is provided to the calling process.

Being able to handle concurrent operations on shared data while maintaining strong consistency is fundamental to distributed systems. For performance reasons, strong consistency may be guaranteed only when processes use mechanisms such as transactions or synchronization variables. Along the same lines, it may be impossible to guarantee strong consistency, and weaker forms need to be accepted, such as causal consistency with eventual consistency.

In this section, we take a look at a special class of distributed data stores. The data stores we consider are characterized by the lack of simultaneous updates, or when such updates happen, it is assumed that they can be relatively easily resolved. Most operations involve reading data. These data stores offer a weak consistency model, such as eventual consistency. By introducing special **client-centric consistency models**, it turns out that many inconsistencies can be hidden in a relatively cheap way.

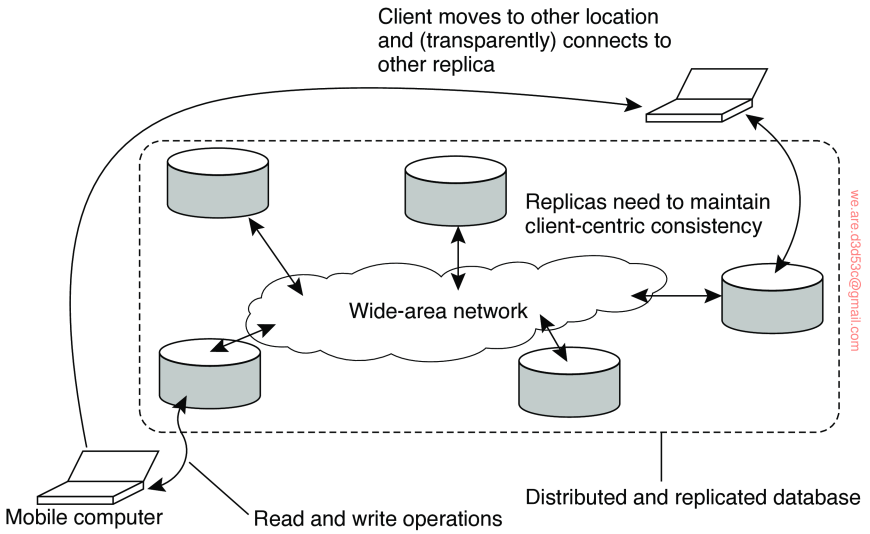


Figure 7.17: The principle of a mobile user accessing different replicas of a distributed database.

Eventually, consistent data stores generally work fine as long as clients always access the same replica. However, problems arise when different replicas are accessed over a short period of time. This is best illustrated by considering a mobile user accessing a distributed database, as shown in Figure 7.17.

The mobile user, say, Alice, accesses the database by connecting to one of the replicas in a transparent way. In other words, the application running on Alice's mobile device is unaware on which replica it is actually operating. Assume Alice performs several update operations and then disconnects again. Later, she accesses the database again, possibly after moving to a different location or by using a different access device. At that point, she may be connected to a different replica than before, as shown in Figure 7.17. However, if the updates performed previously have not yet been propagated, Alice will notice inconsistent behavior. In particular, she would expect to see all previously made changes, but instead, it appears as if nothing at all has happened.

This example is typical for eventually consistent data stores and is caused by the fact that users may sometimes operate on different replicas, while updates have not been fully propagated. The problem can be alleviated by introducing **client-centric consistency**. In essence, client-centric consistency provides guarantees *for a single client* concerning the consistency of accesses to a data store by that client. No guarantees are given concerning concurrent accesses by different clients. If Bob modifies data that is shared with Alice but which is stored at a different location, we may easily create write-write conflicts. Moreover, if neither Alice nor Bob access the same location for some time, such conflicts may take a long time before they are discovered.

Client-centric consistency models originate from the work on Bayou and, more general, from mobile-data systems (see, for example, Terry et al. [1994], Terry et al. [1998], or Terry [2008]). Bayou is a database system developed for mobile computing, where it is assumed that network connectivity is unreliable and subject to various performance problems. Wireless networks and networks that span large areas, such as the Internet, fall into this category.

Bayou essentially distinguishes four different consistency models. To explain these models, we again consider a data store that is physically distributed across multiple machines. When a process accesses the data store, it generally connects to the locally (or nearest) available copy, although, in principle, any copy will do just fine. All read and write operations are performed on that local copy. Updates are eventually propagated to the other copies.

Client-centric consistency models are described using the following notations. Let x_i denote the *version* of data item x. The version x_i is the result of a series of write operations that took place since initialization, its **write set** $WS(x_i)$. By appending write operations to thatseries, series we obtain another version x_j and say that x_j *follows from* x_i. We use the notation $W(x_i;x_j)$

to indicate that x_j follows from x_i. If we do not know if x_j follows from x_i, we use the notation $W(x_i|x_j)$.

7.3.1 Monotonic reads

The first client-centric consistency model is that of monotonic reads. A (distributed) data store is said to provide **monotonic-read consistency** if the following condition holds:

> *If a process reads the value of a data item x, any successive read operation on x by that process will always return that same value or a more recent value.*

In other words, monotonic-read consistency guarantees that once a process has seen a value of x, it will never see an older version of x.

As an example where monotonic reads are useful, consider a distributed e-mail database. In such a database, each user's mailbox may be distributed and replicated across multiple machines. Mail can be inserted in a mailbox at any location. However, updates are propagated in a lazy (i.e., on demand) fashion. Only when a copy needs certain data for consistency are those data propagated to that copy. Suppose a user reads her mail in San Francisco. Assume that only reading mail does not affect the mailbox, that is, messages are not removed, stored in subdirectories, or even tagged as having already been read, and so on. When the user later flies to New York and opens her mailbox again, monotonic-read consistency guarantees that the messages that were in the mailbox in San Francisco will also be in the mailbox when it is opened in New York.

Using a notation similar to that for data-centric consistency models, monotonic-read consistency can be graphically represented as shown in Figure 7.18. Rather than showing *processes* along the vertical axis, we now show *local data stores*, in our example L_1 and L_2. A write or read operation is indexed by the process that executed the operation, that is, $W_1(x)a$ denotes that process P_1 wrote value a to x. As we are not interested in specific values of shared data items, but rather their versions, we use the notation $W_1(x_2)$ to indicate that process P_1 produces version x_2 without knowing anything about other versions. $W_2(x_1;x_2)$ indicates that process P_2 is responsible for producing version x_2 that follows from x_1. Likewise, $W_2(x_1|x_2)$ denotes that process P_2 producing version x_2 *concurrently* to version x_1 (and thus potentially introducing a write-write conflict). $R_1(x_2)$ simply means that P_1 reads version x_2.

In Figure 7.18(a) process P_1 first performs a write operation on x at L_1, producing version x_1 and later reads this version. At L_2 process P_2 first produces version x_2, following from x_1. When process P_1 moves to L_2 and reads x again, it finds a more recent value, but one that at least took its previous write into account.



Figure 7.18: The read operations performed by a single process P at two different local copies of the same data store. (a) A monotonic-read consistent data store. (b) A data store that does not provide monotonic reads.

Figure 7.18(b) shows a situation in which monotonic-read consistency is violated. After process P_1 has read x_1 at L_1, it later performs the operation $R_1(x_2)$ at L_2. However, the preceding write operation $W_2(x_1|x_2)$ by process P_2 at L_2 is known to produce a version that does not follow from x_1. As a consequence, P_1's read operation at L_2 is known not to include the effect of the write operations when it performed $R_1(x_1)$ at location L_1.

7.3.2 Monotonic writes

In many situations, it is important that write operations are propagated in the correct order to all copies of the data store. This property is expressed in monotonic-write consistency. In a **monotonic-write consistent** store, the following condition holds:

> *A write operation by a process on a data item* x *is completed before any successive write operation on* x *by the same process.*

More formally, if we have two successive operations $W_k(x_i)$ and $W_k(x_j)$ by process P_k, then, regardless where $W_k(x_j)$ takes place, we also have $W_k(x_i;x_j)$. Thus, completing a write operation means that the copy on which a successive operation is performed reflects the effect of a previous write operation by the same process, no matter where that operation was initiated. In other words, a write operation on a copy of item x is performed only if that copy has been brought up to date through any preceding write operation by that same process, which may have taken place on other copies of x. If need be, the new write must wait for old ones to finish.

Note that monotonic-write consistency resembles data-centric FIFO consistency. The essence of FIFO consistency is that write operations by the same process are performed in the correct order everywhere. This ordering constraint also applies to monotonic writes, except that we are now considering consistency only for a single process, instead of for a collection of processes.

Bringing a copy of x up to date need not be necessary when each write operation completely overwrites the present value of x. However, write operations are often performed on only part of the state of a data item. Consider, for

example, a software library. Often, updating such a library is done by replacing one or more functions, leading to a next version. With monotonic-write consistency, guarantees are given that if an update is performed on a copy of the library, all preceding updates will be performed first. The resulting library will then indeed become the most recent version and will include all updates that have led to previous versions of the library.

Monotonic-write consistency is shown in Figure 7.19. In Figure 7.19(a) process P_1 performs a write operation on x at L_1, presented as the operation $W_1(x_1)$. Later, P_1 performs another write operation on x, but this time at L_2, shown as $W_1(x_2;x_3)$. The version produced by P_1 at L_2 follows from an update by process P_2, in turn based on version x_1. The latter is expressed by the operation $W_2(x_1;x_2)$. To ensure monotonic-write consistency, it is necessary that the previous write operation at L_1 has already been propagated to L_2, and possibly updated.

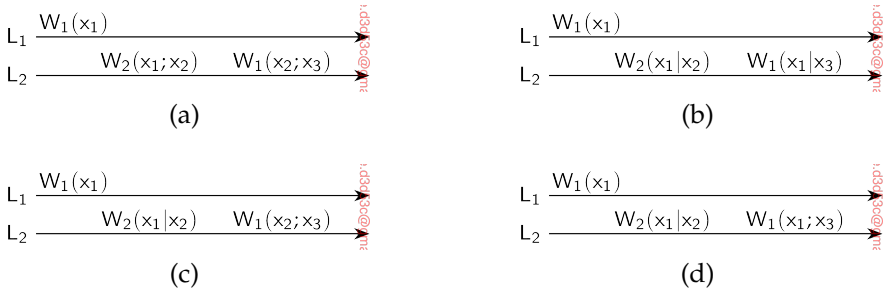


Figure 7.19: The write operations performed at two different local copies of the same data store. (a) A monotonic-write consistent data store. (b) A data store that does not provide monotonic-write consistency. (c) Again, no consistency as $W_2(x_1|x2)$ and thus also $W_1(x_1|x_3)$. (d) Consistent as $W_1(x_1;x_3)$ although x_1 has apparently overwritten x_2.

In contrast, Figure 7.19(b) shows a situation in which monotonic-write consistency is not guaranteed. Compared to Figure 7.19(a), what is missing is the propagation of x_1 to L_2 before another version of x is produced, expressed by the operation $W_2(x_1|x_2)$. In this case, process P_2 produced a concurrent version to x_1, after which process P_1 simply produces version x_3, but again concurrently to x_1. Only slightly more subtle, but still violating monotonic-write consistency, is the situation sketched in Figure 7.19(c). Process P_1 now produces version x_3 which follows from x_2. However, because x_2 does not incorporate the write operations that led to x_1, that is, $W_2(x_1|x_2)$, we also have $W_1(x_1|x_3)$.

An interesting case is shown in Figure 7.19(d). The operation $W_2(x_1|x_2)$ produces version x_2 concurrently to x_1. However, later, process P_1 produces version x_3, but apparently based on the fact that version x_1 had become

available at L_2. How and when x_1 was transferred to L_2 is left unspecified, but in any case a write-write conflict was created with version x_2 and resolved in favor of x_1. A consequence is that the situation shown in Figure 7.19(d) follows the rules for monotonic-write consistency. Note, however, that any subsequent write by process P_2 at L_2 (without having read version x_1) will immediately violate consistency again. How such a violation can be prevented is left as an exercise to the reader.

Note that, by the definition of monotonic-write consistency, write operations by the same process are performed in the same order as they are initiated. A somewhat weaker form of monotonic writes is one in which the effects of a write operation are seen only if all preceding writes have been carried out as well, but perhaps not in the order in which they have been originally initiated. This consistency is applicable in those cases in which write operations are commutative, so that ordering is really not necessary. Details are found in [Terry et al., 1994].

7.3.3 Read your writes

A data store is said to provide **read-your-writes consistency**, if the following condition holds:

> *The effect of a write operation by a process on data item* x *will always be seen by a successive read operation on* x *by the same process.*

In other words, a write operation is always completed before a successive read operation by the same process, no matter the location where that read operation takes place.

The absence of read-your-writes consistency is sometimes experienced when updating Web documents and subsequently viewing the effects. Update operations frequently take place through a standard editor or word processor, perhaps embedded as part of a content management system, which then saves the new version on a file system that is shared by the Web server. The user's Web browser accesses that same file, possibly after requesting it from the local Web server. However, once the file has been fetched, either the server or the browser often caches a local copy for subsequent accesses. Consequently, when the Web page is updated, the user will not see the effects if the browser or the server returns the cached copy instead of the original file. Read-your-writes consistency can guarantee that if the editor and browser are integrated into a single program, the cache is invalidated when the page is updated, so that the updated file is fetched and displayed.

Similar effects occur when updating passwords. For example, to enter a digital library on the Web, it is often necessary to have an account with an accompanying password. However, changing a password may take some time to come into effect, with the result that the library may be inaccessible to the user for a few minutes. The delay can be caused because a separate server

is used to manage passwords, and it may take some time to subsequently propagate (encrypted) passwords to the various servers that constitute the library.

Figure 7.20(a) shows a data store that provides read-your-writes consistency. Note that Figure 7.20(a) is very similar to Figure 7.18(a), except that consistency is now determined by the last write operation by process P_1, instead of its last read.



Figure 7.20: (a) A data store that provides read-your-writes consistency. (b) A data store that does not.

In Figure 7.20(a) process P_1 performed a write operation $W_1(x_1)$ and later a read operation at a different local copy. Read-your-writes consistency guarantees that the effects of the write operation can be seen by the succeeding read operation. This is expressed by $W_2(x_1;x_2)$, which states that a process P_2 produced a new version of x, yet one based on x_1. In contrast, in Figure 7.20(b) process P_2 produces a version concurrently to x_1, expressed as $W_2(x_1|x_2)$. This means that the effects of the previous write operation by process P_1 have not been propagated to L_2 at the time x_2 was produced. When P_1 reads x_2, it will not see the effects of its own write operation at L_1.

7.3.4 Writes follow reads

The last client-centric consistency model is one in which updates are propagated as the result of previous read operations. A data store is said to provide **writes-follow-reads** consistency, if the following holds.

> *A write operation by a process on a data item x following a previous read operation on x by the same process is guaranteed to take place on the same or a more recent value of x that was read.*

In other words, any successive write operation by a process on a data item x will be performed on a copy of x that is up to date with the value most recently read by that process.

Writes-follow-reads consistency can be used to guarantee that users of a network newsgroup see a posting of a reaction to an article only after they have seen the original article [Terry et al., 1994]. To understand the problem, assume that a user first reads an article A. Then, she reacts by posting a response B. By requiring writes-follow-reads consistency, B will be written to any copy of the newsgroup only after A has been written as well. Note

that users who only read articles need not require any specific client-centric consistency model. The writes-follows-reads consistency assures that reactions to articles are stored at a local copy only if the original is stored there as well.



Figure 7.21: (a) A writes-follow-reads consistent data store. (b) A data store that does not provide writes-follow-reads consistency.

This consistency model is shown in Figure 7.21. In Figure 7.21(a), process P_2 reads version x_1 at local copy L_1. This version of x was previously produced at L_1 by process P_1 through the operation $W_1(x_1)$. That version was subsequently propagated to L_2, and used by another process P_3 to produce a new version x_2, expressed as $W_3(x_1;x_2)$. When process P_2 later updates its version of x after moving to L_2, it is known that it will operate on a version that follows from x_1, expressed as $W_2(x_2;x_3)$. Because we also have $W_3(x_1;x_2)$, we known that $W_1(x_1;x_3)$.

The situation shown in Figure 7.21(b) is different. Process P_3 produces a version x_2 concurrently to that of x_1. As a consequence, when P_2 updates x after reading x_1, it will be updating a version it had not read before. Writes-follow-reads consistency is then violated.

7.3.5 Example: client-centric consistency in ZooKeeper

An interesting example in which we see elements of data-centric and client-centric consistency combined, is the model provided by ZooKeeper [Hunt et al., 2010]. First, ZooKeeper guarantees that update operations are **serializable** and keep precedence. What this means, is that the state of ZooKeeper can always be explained by some linear ordering in the execution of *all* submitted update operations, while preserving monotonic writes. In other words, while multiple clients can concurrently update ZooKeeper's state, the final result can be understood by some interleaving of those updates while the ordering of operations as submitted per client is preserved. In addition, ZooKeeper guarantees monotonic reads. It does not, however, guarantee read-your-writes or writes-follow-read consistency.

A simple perspective is to consider ZooKeeper as a collection of servers, while each client is connected to its own specific server. There is one, fixed primary server to which all write operations are forwarded, and processed in the order as submitted by their respective clients. Read operations are executed by the server to which the client is connected (and in the order as submitted by that client). Nothing, however, is said about when the primary

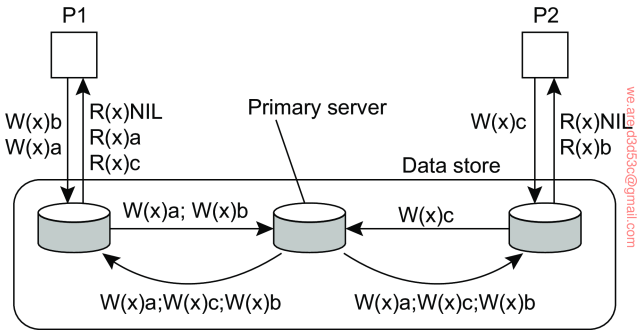


Figure 7.22: The consistency model of ZooKeeper explained through a naive implementation.

server brings the other servers up to date. In other words, a client is left in the dark concerning when it will see its own updates, or concurrent updates from other clients. This model is sketched in Figure 7.22.

Assume that P_1 first submits the operation $W(x)a$ and then $W(x)b$. Process P_2 submits the operation $W(x)c$. What is important is that monotonic writes are supported. In our example, the primary server receives the three write operations and decides to forward the operations $W(x)a$, $W(x)c$, and $W(x)b$ in that order to the other stores. We have not specified *when* these operations are submitted or actually performed. Important is that this order needs to be obeyed.

If P_1 submits read requests $R(x)$, it may first read the initial value NIL, and then subsequently $R(x)a$ and later $R(x)c$ (and eventually, it will be able to read only $R(x)b$). Likewise, P_2 may initially read NIL and later the final value $R(x)b$. Note that both P_1 and P_2 may read NIL even after having submitted their respective write operations. This demonstrates that ZooKeeper does not provide read-your-writes consistency. Also, even if P_1 had first read NIL and then submitted $W(x)a;W(x)b$, it may still read $R(x)c$ if its latest write operation ($R(x)b$) had not yet been processed. This demonstrates that ZooKeeper also does not provide writes-follow-reads consistency.

7.4 Replica management

A key issue for any distributed system that supports replication is to decide where and when replicas should be placed, and subsequently which mechanisms to use for keeping the replicas consistent. The placement problem itself should be split into two subproblems: that of placing *replica servers*, and that of placing *content*. The difference is subtle and the two issues are often not clearly separated. Replica-server placement is concerned with finding the best locations to place a server that can host (part of) a data store. Content

placement deals with finding the best servers for placing content. Note that this often means that we are looking for the optimal placement of only a single data item. Obviously, before content placement can take place, replica servers will have to be placed first.

7.4.1 Finding the best server location

Where perhaps over a decade ago one could be concerned about where to place an individual server, matters have changed considerably with the advent of the many large-scale data centers located across the Internet. Likewise, connectivity continues to improve, making *precisely* locating servers less critical.

There are various ways to compute the best placement of replica servers, but virtually all boil down to an optimization problem in which the best K out of N locations need to be selected ($K < N$). These problems are known to be computationally complex and can be solved only through heuristics. Important in this respect is to decide on the criteria to even judge on a "best" solution. In an extensive overview, Sahoo et al. [2017] distinguish cost-related criteria and network-related criteria. Typical network-related criteria are latencies between a server and its clients, the available bandwidth, the (logical) distance as discussed in Section 5.7.3, and hop count. Of course, combinations are possible as well.

By now, there are many (often theoretical) models for deciding on the placement of replica servers. Sahoo et al. [2017] classify these models as shown in Figure 7.23.

Main class	Subclass
QoS Aware	Optimized QoS
	Bounded QoS
Consistency Aware	Periodic update
	Aperiodic update
	Expiration-based update
	Cache-based update
Energy	
Others	

Figure 7.23: Taxonomy of replica-placement algorithms (adapted from [Sahoo et al., 2017]).

Let us take a closer look at these different classes. When considering **Quality of Service**, that is **QoS**, server placement is decided by optimizing for some or more QoS parameters. QoS is generally expressed in terms of one or more of the network-related parameters. In the case of bounded QoS, solutions are sought that guarantee a value of a certain QoS parameter, for example, guaranteed bandwidth.

As mentioned, QoS awareness requires solving a computationally difficult problem, meaning in practice that we need to resort to heuristics, and thus approximations of optimal values. This is generally not a problem, considering that the QoS parameters itself may be rather imprecise. Consider, for example, accurately measuring latency between a client (end user) and a replica server. As discussed in Section 5.7.3, this is by itself not an easy problem to solve when we cannot simply access that client. The best one can hope for, is that an *existing* server is often enough connected to a client to get some idea of latencies. The same problem occurs for other parameters, in particular, bandwidth. As a consequence, we can try to accurately optimize on QoS, but when the basic input values are already very difficult to measure accurately, optimizations can quickly become quite theoretical. Quick-and-dirty heuristics may be enough, yet we will need to take their results with a grain of salt.

Consistency-aware algorithms take an entirely different approach. The basic assumption is that there is an existing network for which we can decide where to place replica servers. Placement of servers will then involve costs for keeping replicated content up to date. Those costs can be expressed in terms of when updates take place, as well as how. We return to these matters later in Section 7.4.3 where we discuss replica management. The four different subclasses deal with deciding on *when* updates are propagated to the replica server (periodically, instantly, or only after some time), or from *where* (is a replica server going to check the main server, or is it going to see whether a nearby server has an update). Again, we see that many theoretical models have been developed to optimize consistency-aware algorithms, yet in this case, such optimizations rely heavily on knowledge concerning read and update patterns. In practice, such knowledge is not readily available and may even be hard to get, let alone when it comes to predicting future patterns.

Energy-aware algorithms also assume an existing network to which clients connect. Placement of a replica server is then decided based on energy consumption. To keep it simple, placing a server at a specific node and keeping it up and running when there are only few clients, may be costly in terms of energy. The server in that case does not have enough work to do, while its idle time does consume energy. Another criterion is to decide on placement at nodes that support energy efficiency, for example, by switching between different power modes. Finally, placement may also be influenced by optimizing the distribution of work (measured in terms of energy consumption) among more or less replica servers, given that we know the various access patterns of a set of clients. As before, we see that much needs to be known about those patterns, as well as the capacities and facilities of the network, to make practically sensible decisions.

Finally, there are other classes for deciding on replica-server placement. Sahoo et al. have concentrated on server placement for CDNs. Such distributed systems are spread across many organizations (formally known as

Autonomous Systems, that is, an organization in charge of a separate network within the Internet). Replica-server placement then also involves monetary costs that need to be negotiated with those organizations. Likewise, placement decisions can be based on the connectivity of clients to a specific autonomous system, along with the costs in terms of QoS parameters or monetary costs. Further details can be found in [Sahoo et al., 2017].

7.4.2 Content replication and placement

When it comes to content replication and placement, three different types of replicas can be distinguished, logically organized as shown in Figure 7.24.

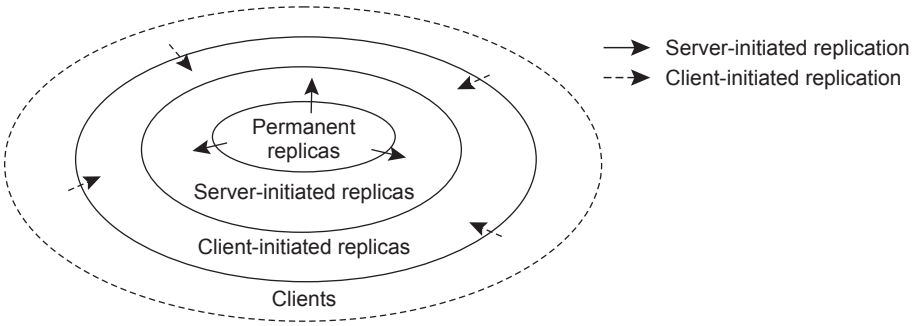


Figure 7.24: The logical organization of different kinds of copies of a data store into three concentric rings.

Permanent replicas

Permanent replicas can be considered as the initial set of replicas that constitute a distributed data store. Often, the number of permanent replicas is small. Consider, for example, a Website. Distribution of a Website generally comes in one of two forms. The first kind of distribution is one in which the files that constitute a site are replicated across a limited number of servers at a single location. Whenever a request comes in, it is forwarded to one of the servers, for instance, using a round-robin strategy.

The second form of distributed Websites is what is called **mirroring**. In this case, a Website is copied to a limited number of servers, called **mirror sites**, which are geographically spread across the Internet. Often, clients simply choose one of the various mirror sites from a list offered to them, or are transparently forwarded to one of the mirrors. Mirrored Websites have in common with cluster-based Websites that there are only a few replicas, which are more or less statically configured.

Similar static organizations also appear with distributed databases [Kemme et al., 2010; Özsu and Valduriez, 2020]. Again, the database can be distributed

and replicated across a number of servers that together form a cluster of servers, often referred to as a **shared-nothing architecture**, emphasizing that neither disks nor main memory are shared by processors. Alternatively, a database is distributed and possibly replicated across a number of geographically dispersed sites. This architecture is generally deployed in federated databases [Sheth and Larson, 1990; Azevedo et al., 2020].

Server-initiated replicas

In contrast to permanent replicas, **server-initiated replicas** are copies of a data store that exist to enhance performance, and created at the initiative of the (owner of the) data store. Consider, for example, a Web server placed in New York. Normally, this server can handle incoming requests effortlessly, but it may happen that over a couple of days a sudden burst of requests come in from an unexpected location far from the server. In that case, it may be worthwhile to install a number of temporary replicas in regions where requests are coming from.

Web hosting has often been taken over by Content Delivery Networks, and for good reasons. Most important is that Web pages are no longer just a simple collection of static pages, but combine databases with static and dynamic content. As a result, replication needs to be much more sophisticated than just pushing pages to specific locations. We outlined the principal operation of a CDN in Section 3.4.4. Important is that, in general, replication is on demand. The CDN notices that requests are coming from a client, the client is directed to a nearest replica server, and that server checks to see whether it has the requested content cached locally. Obviously, there are many options concerning decisions on where to redirect the client to, and also when and which content to actually store. For our discussion here, despite that content is replicated to a server because of a client request, because the CDN is in control when it comes to the aforementioned decisions, it is still appropriate to speak of server-initiated replication. That the initiative has been delegated to a CDN is less important.

Note 7.6 (More information: An example of dynamic Web-content placement)
The problem of dynamically placing replicas has since long been addressed in Web hosting services (and by now in content delivery networks). Historically, these services offered an often relatively static collection of servers spread across the Internet that maintained and provided access to Web files belonging to third parties. To provide optimal facilities, such hosting services dynamically replicated files to servers where those files are needed to enhance performance, that is, close to demanding (groups of) clients.

Given that the replica servers are already in place, deciding where to place content is not that difficult. An early case toward dynamic replication of files in the case of a Web hosting service is described by Rabinovich et al. [1999]. The

algorithm is designed to support Web pages, for which reason it assumes that updates are relatively rare compared to read requests. Using files as the unit of data, the algorithm works as follows.

The algorithm for dynamic replication takes two issues into account. First, replication can take place to reduce the load on a server. Second, specific files on a server can be migrated or replicated to servers placed in the proximity of clients that issue many requests for those files. In the following, we concentrate only on this second issue. We also omit a number of details, which can be found in [Rabinovich et al., 1999].

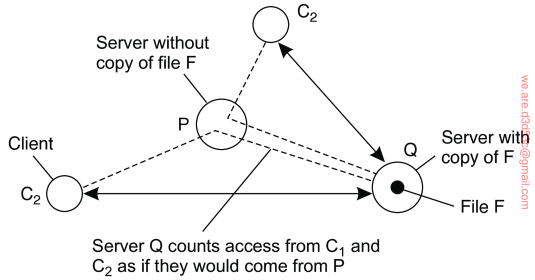


Figure 7.25: Counting access requests from different clients.

Each server keeps track of access counts per file, and where access requests come from. In particular, when a client C enters the service, it does so through a server close to it. If client C_1 and client C_2 share the same closest server P, all access requests for file F at server Q from C_1 and C_2 are jointly registered at Q as a single access count $\mathsf{cnt}_Q(P, F)$. This situation is shown in Figure 7.25.

When the number of requests for a specific file F at server S drops below a deletion threshold $\mathsf{del}(S, F)$, that file can be removed from S. As a consequence, the number of replicas of that file is reduced, possibly leading to higher work loads at other servers. Special measures are taken to ensure that at least one copy of each file continues to exist.

A replication threshold $\mathsf{rep}(S, F)$, which is always chosen higher than the deletion threshold, indicates that the number of requests for a specific file is so high that it may be worthwhile replicating it on another server. If the number of requests lies somewhere between the deletion and replication threshold, the file is allowed to be only migrated. In other words, in that case, it is important to at least keep the number of replicas for that file the same.

When a server Q decides to reevaluate the placement of the files it stores, it checks the access count for each file. If the total number of access requests for F at Q drops below the deletion threshold $\mathsf{del}(Q, F)$, it will delete F unless it is the last copy. Furthermore, if for some server P, $\mathsf{cnt}_Q(P, F)$ exceeds more than half of the total requests for F at Q, server P is requested to take over the copy of F. In other words, server Q will attempt to migrate F to P.

Migration of file F to server P may not always succeed, for example because P is already heavily loaded or is out of disk space. In that case, Q will attempt to

replicate F on other servers. Of course, replication can take place only if the total number of access requests for F at Q exceeds the replication threshold $\mathsf{rep}(Q, F)$. Server Q checks all other servers in the Web hosting service, starting with the one farthest away. If, for some server R, $\mathsf{cnt}_Q(R, F)$ exceeds a certain fraction of all requests for F at Q, an attempt is made to replicate F to R.

What we have just described is an alternative to simply caching content at a replica server when a client request comes in, as we described for content delivery networks. There is no fundamental reason why the algorithm for copying and migration of content cannot be applied in CDNs as well.

Client-initiated replicas

An important kind of replica is the one initiated by a client. **Client-initiated replicas** are more commonly known as **client caches**. In essence, a cache is a local storage facility that is used by a client to temporarily store a copy of the data it has just requested. In principle, managing the cache is left entirely to the client. The data store from where the data had been fetched has nothing to do with keeping cached data consistent. However, there are many occasions in which the client can rely on participation from the data store to inform it when cached data has become stale.

Client caches are used only to improve access times to data. Normally, when a client wants access to some data, it connects to the nearest copy of the data store from where it fetches the data it wants to read, or to where it stores the data it had just modified. When most operations involve only reading data, performance can be improved by letting the client store requested data in a nearby cache. Such a cache could be located on the client's machine, or on a separate machine in the same local-area network as the client. The next time that same data needs to be read, the client can simply fetch it from this local cache. This scheme works fine as long as the fetched data have not been meanwhile modified.

Data are generally kept in a cache for a limited amount of time, for example, to prevent extremely stale data from being used, or simply to make room for other data. Whenever requested data can be fetched from the local cache, a **cache hit** is said to have occurred. To improve the number of cache hits, caches can be shared between clients. The underlying assumption is that a data request from client C_1 may also be useful for a request from another nearby client C_2.

Whether this assumption is correct depends very much on the type of data store. For example, in traditional file systems, data files are rarely shared at all (see, e.g., Muntz and Honeyman [1992] and Blaze [1993]) rendering a shared cache useless. Likewise, it turns out that using Web caches to share data has been losing ground, partly also because of the improvement in network

and server performance. Instead, server-initiated replication schemes are becoming more effective.

Placement of client caches is relatively simple: a cache is normally placed on the same machine as its client, or otherwise on a machine shared by clients on the same local-area network. However, in some cases, extra levels of caching are introduced by system administrators by placing a shared cache between a number of departments or organizations, or even placing a shared cache for an entire region such as a province or country.

Yet another approach is to place (cache) servers at specific points in a wide-area network and let a client locate the nearest server. When the server is located, it can be requested to hold copies of the data the client was previously fetching from somewhere else [Noble et al., 1999].

7.4.3 Content distribution

Replica management also deals with propagation of (updated) content to the relevant replica servers. There are various trade-offs to make.

State versus operations

An important design issue concerns what is actually to be propagated. Basically, there are three possibilities:

- Propagate only a notification of an update.
- Transfer data from one copy to another.
- Propagate the update operation to other copies.

Propagating a notification is what **invalidation protocols** do. In an invalidation protocol, other copies are informed that an update has taken place and that the data they contain are no longer valid. The invalidation may specify which part of the data store has been updated, so that only part of a copy is actually invalidated. The important issue is that no more than a notification is propagated. Whenever an operation on an invalidated copy is requested, that copy generally needs to be updated first, depending on the specific consistency model that is to be supported.

The main advantage of invalidation protocols is that they use little network bandwidth. The only information that needs to be transferred is a specification of which data are no longer valid. Such protocols generally work best when there are many update operations compared to read operations, that is, the read-to-write ratio is relatively small.

Consider, for example, a data store in which updates are propagated by sending the modified data to all replicas. If the size of the modified data is large, and updates occur frequently compared to read operations, we may have the situation that two updates occur after one another without any read

operation being performed between them. Consequently, propagation of the first update to all replicas is effectively useless, as it will be overwritten by the second update. Instead, sending a notification that the data have been modified would have been more efficient.

Transferring the modified data among replicas is the second alternative, and is useful when the read-to-write ratio is relatively high. In that case, the probability that an update will be effective in the sense that the modified data will be read before the next update takes place is high. Instead of propagating modified data, it is also possible to log the changes and transfer only those logs to save bandwidth. In addition, transfers are often aggregated in the sense that multiple modifications are packed into a single message, thus saving communication overhead.

The third approach is not to transfer any data modifications at all, but to tell each replica which update operation it should perform (and sending only the parameter values that those operations need). This approach, also referred to as **active replication**, assumes that each replica is represented by a process capable of "actively" keeping its associated data up to date by performing operations [Schneider, 1990]. The main benefit of active replication is that updates can often be propagated at minimal bandwidth costs, provided the size of the parameters associated with an operation are relatively small. Moreover, the operations can be of arbitrary complexity, which may allow further improvements in keeping replicas consistent. On the other hand, more processing power may be required by each replica, especially in those cases when operations are relatively complex.

Pull versus push protocols

Another design issue is whether updates are pulled or pushed. In a **push-based approach**, also referred to as **server-based protocols**, updates are propagated to other replicas without those replicas even asking for the updates. Push-based approaches are often used between permanent and server-initiated replicas, but can also be used to push updates to client caches. Server-based protocols are generally applied when strong consistency is required.

This need for strong consistency is related to the fact that permanent and server-initiated replicas, as well as large shared caches, are often shared by many clients, which, in turn, mainly perform read operations. Consequently, the read-to-update ratio at each replica is relatively high. In these cases, push-based protocols are efficient in the sense that every pushed update can be expected to be of use for at least one, but perhaps more readers. In addition, push-based protocols make consistent data immediately available when asked for.

In contrast, in a **pull-based approach**, a server or client requests another server to send it any updates it has at that moment. Pull-based protocols, also called **client-based protocols**, are often used by client caches. For example, a

common strategy applied to Web caches is first to check whether cached data items are still up-to-date. When a cache receives a request for items that are still locally available, the cache checks with the original Web server, whether those data items have been modified since they were cached. In the case of a modification, the modified data are first transferred to the cache, and then returned to the requesting client. If no modifications took place, the cached data are returned. In other words, the client polls the server to see whether an update is needed. A similar approach is followed in content delivery networks where a replica server caches content, yet checks with the origin server whether updates are available before delivering its (cached) content to the actual client (i.e., end user). Note that in this respect, the replica server acts as a client to the origin server.

A pull-based approach is efficient when the read-to-update ratio is relatively low. This is often the case with (nonshared) client caches, which have only one client. However, even when a cache is shared by many clients, a pull-based approach may also be efficient when the cached data items are rarely shared. The main drawback of a pull-based strategy in comparison to a push-based approach is that the response time increases in the case of a cache miss.

When comparing push-based and pull-based solutions, there are a number of trade-offs to be made, as shown in Figure 7.26. For simplicity, consider a client-server system consisting of a single, nondistributed server, and a number of client processes, each having their own cache. We make no distinction between clients at end users (such as Web browsers), or clients within, for example, a content delivery network (such as replica servers).

Issue	Push-based	Pull-based
State at server	List of client replicas and caches	None
Messages sent	Update (and possibly fetch update later)	Poll and update
Response time at client	Immediate (or fetch-update time)	Fetch-update time

Figure 7.26: A comparison between push-based and pull-based protocols in the case of multiple-client, single-server systems.

An important issue is that in push-based protocols, the server needs to keep track of all client caches, be they at end users or replica servers. Apart from the fact that stateful servers are often less fault tolerant, keeping track of all such caches may introduce a considerable overhead at the server. For example, in a push-based approach, a Web server may easily need to keep track of tens of thousands of client caches. Each time a Web page is updated, the server will have to go through its list of client caches holding a copy of that page and subsequently propagate the update. Worse yet, if a client purges a

page due to lack of space, it has to inform the server, leading to even more communication.

The messages that need to be sent between a client and the server also differ. In a push-based approach, the only communication is that the server sends updates to each client. When updates are only invalidations, additional communication is needed by a client to fetch the modified data. In a pull-based approach, a client will have to poll the server, and, if necessary, fetch the modified data.

Finally, the response time at the client is also different. When a server pushes modified data to the client caches, it is clear that the response time at the client side is zero. When invalidations are pushed, the response time is the same as in the pull-based approach, and is determined by the time it takes to fetch the modified data from the server.

These trade-offs have lead to a hybrid form of update propagation based on leases. In the case of replica management, a **lease** is a promise by the server that it will push updates to the client for a specified time. When a lease expires, the client is forced to poll the server for updates and pull in the modified data if necessary. An alternative is that a client requests a new lease for pushing updates when the previous lease expires.

Leases, originally introduced by Gray and Cheriton [1989], provide a convenient mechanism for dynamically switching between a push-based and pull-based strategy. Consider the following lease system that allows the expiration time to be dynamically adapted depending on different lease criteria, described in [Duvvuri et al., 2003]. We distinguish the following three types of leases. (Note that in all cases, updates are pushed by the server as long as the lease has not expired.)

First, **age-based leases** are given out on data items depending on the last time the item was modified. The underlying assumption is that data that have not been modified for a long time can be expected to remain unmodified for some time yet to come. This assumption has shown to be reasonable in the case of, for example, Web-based data and regular files. By granting long-lasting leases to data items that are expected to remain unmodified, the number of update messages can be strongly reduced compared to the case where all leases have the same expiration time.

Another lease criterion is how often a specific client requests its cached copy to be updated. With **renewal-frequency-based leases**, a server will hand out a long-lasting lease to a client whose cache often needs to be refreshed. On the other hand, a client that asks only occasionally for a specific data item will be handed a short-term lease for that item. The effect of this strategy is that the server essentially keeps track only of those clients where its data are popular; moreover, those clients are offered a high degree of consistency.

The last criterion is that of state-space overhead at the server. When the server realizes that it is gradually becoming overloaded, it lowers the

expiration time of new leases it hands out to clients. The effect of this **state-based lease** strategy is that the server needs to keep track of fewer clients as leases expire more quickly. In other words, the server dynamically switches to a more stateless mode of operation, thereby expecting to offload itself so that it can handle requests more efficiently. The obvious drawback is that it may need to do more work when the read-to-update ratio is high.

Unicasting versus multicasting

Related to pushing or pulling updates is deciding whether unicasting or multicasting should be used. In unicast communication, when a server that is part of the data store sends its update to N other servers, it does so by sending N separate messages, one to each server. With multicasting, the underlying network takes care of sending a message efficiently to multiple receivers.

Often, it is cheaper to use available multicasting facilities. An extreme situation is when all replicas are located in the same local-area network and that hardware broadcasting is available. In that case, broadcasting or multicasting a message is no more expensive than a single point-to-point message. Unicasting updates would then be less efficient.

Multicasting can often be efficiently combined with a push-based approach to propagating updates. When the two are carefully integrated, a server that decides to push its updates to a number of other servers simply uses a single multicast group to send its updates. In contrast, with a pull-based approach, it is generally only a single client or server that requests its copy to be updated. In that case, unicasting may be the most efficient solution.

7.4.4 Managing replicated objects

As we mentioned, data-centric consistency for distributed objects comes naturally in the form of entry consistency. Recall that in this case, the goal is to group operations on shared data using synchronization variables (e.g., in the form of locks). As objects naturally combine data and the operations on that data, locking objects during an invocation serializes access and keeps them consistent.

Although conceptually associating a lock with an object is simple, it does not necessarily provide a proper solution when an object is replicated. There are two issues that need to be solved for implementing entry consistency. The first one is that we need a means to prevent concurrent execution of multiple invocations on the same object. In other words, when any method of an object is being executed, no other methods may be executed. This requirement ensures that access to the internal data of an object is indeed serialized. Simply using local locking mechanisms will ensure this serialization.

The second issue is that in the case of a replicated object, we need to ensure that all changes to the replicated state of the object are the same. In other

words, we need to make sure that no two independent method invocations take place on different replicas at the same time. This requirement implies that we need to order invocations such that each replica sees all invocations in the same order. We describe a few general solutions in Section 7.5.

Often, designing replicated objects is done by first designing a single object, possibly protecting it against concurrent access through local locking, and subsequently replicating it. The role of middleware is to ensure that if a client invokes a replicated object, the invocation is passed to the replicas and handed to the their respective object servers in the same order everywhere. However, we also need to ensure that all threads in those servers process those requests in the correct order as well. The problem is sketched in Figure 7.27.

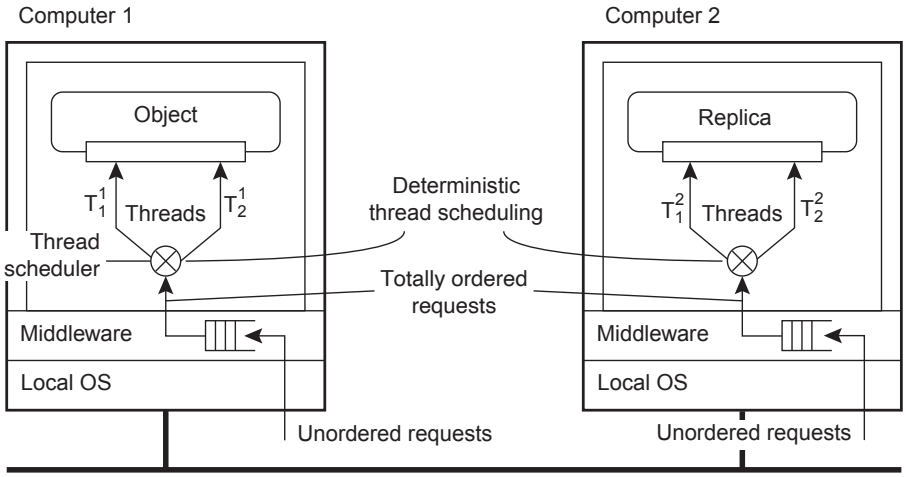


Figure 7.27: Deterministic thread scheduling for replicated object servers.

Multithreaded (object) servers simply pick up an incoming request, pass it on to an available thread, and wait for the next request to come in. The server's thread scheduler subsequently allocates the CPU to runnable threads. Of course, if the middleware has done its best to provide a total ordering for request delivery, the thread schedulers should operate in a deterministic fashion in order not to mix the ordering of method invocations on the same object. In other words, If threads T_1^1 and T_1^2 from Figure 7.27 handle the same incoming (replicated) invocation request, they should both be scheduled before T_2^1 and T_2^2, respectively.

Simply scheduling *all* threads deterministically is not necessary. In principle, if we already have totally ordered request delivery, we need only to ensure that all requests for the same replicated object are handled in the order they were delivered. Such an approach would allow invocations for different objects to be processed concurrently, and without further restrictions from the thread scheduler. Only few systems exist that support such concurrency.

One approach, described by Basile et al. [2002], ensures that threads sharing the same (local) lock are scheduled in the same order on every replica. At the basics lies a primary-based scheme in which one of the replica servers takes the lead in determining, for a specific lock, which thread goes first. An improvement that avoids frequent communication between servers is described in [Basile et al., 2003]. Note that threads that do not share a lock can thus operate concurrently on each server.

One drawback of this scheme is that it operates at the level of the underlying operating system, meaning that every lock needs to be managed. By providing application-level information, a huge improvement in performance can be made by identifying only those locks that are required for serializing access to replicated objects (see Taiani et al. [2005]).

Note 7.7 (Advanced: Replicated invocations)
Another problem that needs to be solved is that of **replicated invocations**. Consider an object A calling another object B as shown in Figure 7.28. Object B is assumed to call yet another object C. If B is replicated, each replica of B will, in principle, call C independently. The problem is that C is now called multiple times instead of only once. If the called method on C results in the transfer of $100,000, then clearly, someone is, sooner or later, going to complain.

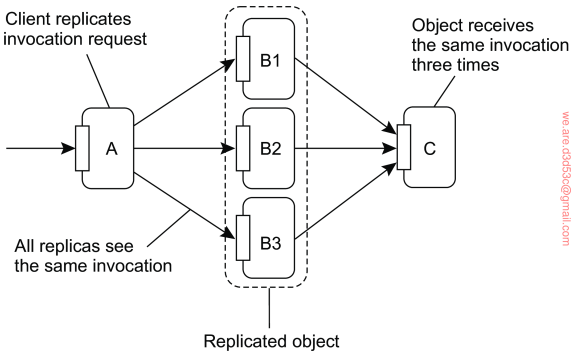


Figure 7.28: The problem of replicated object invocations.

There are not many general-purpose solutions to solve the problem of replicated invocations. One solution is to simply forbid it [Maassen et al., 2001], which makes sense when performance is at stake. However, when replicating for fault tolerance, the following solution proposed by Mazouni et al. [1995] may be deployed. Their solution is independent of the replication policy, that is, the exact details of how replicas are kept consistent. The essence is to provide a replication-aware communication layer on top of which (replicated) objects execute. When a replicated object B invokes another replicated object C, the invocation request is first assigned the same, unique identifier by each replica of B. At that point, a coordinator of the replicas of B forwards its request to all the replicas of object C,

while the other replicas of B hold back their copy of the invocation request, as shown in Figure 7.29. The result is that only a single request is forwarded to each replica of C.

The same mechanism is used to ensure that only a single reply message is returned to the replicas of B. This situation is shown in Figure 7.29. A coordinator of the replicas of C notices it is dealing with a replicated reply message that has been generated by each replica of C. However, only the coordinator forwards that reply to the replicas of object B, while the other replicas of C hold back their copy of the reply message.

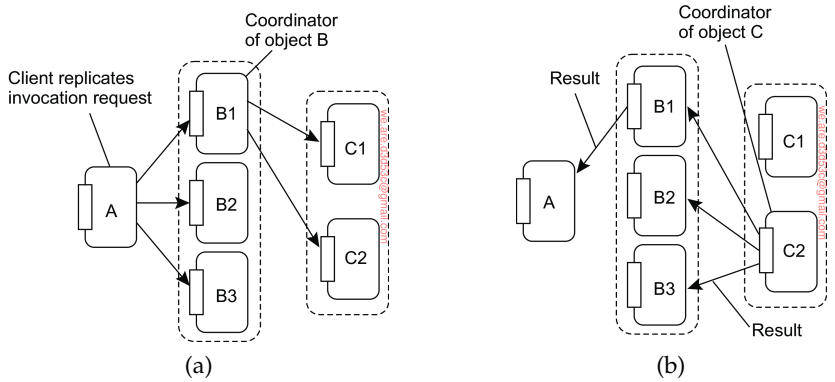


Figure 7.29: (a) Forwarding an invocation request from a replicated object to another replicated object. (b) Returning a reply from one replicated object to another.

When a replica of B receives a reply message for an invocation request it had either forwarded to C or held back because it was not the coordinator, the reply is then handed to the actual object.

In essence, the scheme just described is based on using multicast communication, but preventing that the same message is multicast by different replicas. As such, it is essentially a sender-based scheme. An alternative solution is to let a receiving replica detect multiple copies of incoming messages belonging to the same invocation, and to pass only one copy to its associated object. Details of this scheme are left as an exercise.

7.5 Consistency protocols

We now concentrate on the actual implementation of consistency models by taking a look at several consistency protocols. A **consistency protocol** describes an implementation of a specific consistency model. We follow the organization of our discussion on consistency models by first taking a look at data-centric models, followed by protocols for client-centric models.

7.5.1 Sequential consistency: Primary-based protocols

In practice, we see that distributed applications generally follow consistency models that are relatively easy to understand. These models include those for bounding staleness deviations, and to a lesser extent also those for bounding numerical deviations. When it comes to models that handle consistent ordering of operations, sequential consistency, notably those in which operations can be grouped through locking or transactions, are popular.

As soon as consistency models become slightly difficult to understand for application developers, we see that they are ignored even if performance could be improved. The bottom line is that if the semantics of a consistency model are not intuitively clear, application developers will have a hard time building correct applications. Simplicity is appreciated (and perhaps justifiably so).

In the case of sequential consistency, it turns out that **primary-based protocols** prevail. In these protocols, each data item x in the data store has an associated primary, which is responsible for coordinating write operations on x. A distinction can be made whether the primary is fixed at a remote server or if write operations can be carried out locally after moving the primary to the process where the write operation is initiated.

Remote-write protocols

The simplest primary-based protocol that supports replication is the one in which all write operations need to be forwarded to a fixed single server. Read operations can be carried out locally. Such schemes are also known as **primary-backup protocols** [Budhijara et al., 1993]. A primary-backup protocol works as shown in Figure 7.30. A process wanting to perform a write operation on data item x, forwards that operation to the primary server for x. The primary performs the update on its local copy of x, and subsequently forwards the update to the backup servers. Each backup server performs the update as well, and sends an acknowledgment to the primary. When all backups have updated their local copy, the primary sends an acknowledgment to the initial process, which, in turn, informs the client.

A potential performance problem with this scheme is that it may take a relatively long time before the process that initiated the update is allowed to continue. In effect, an update is implemented as a blocking operation. An alternative is to use a nonblocking approach. As soon as the primary has updated its local copy of x, it returns an acknowledgment. After that, it tells the backup servers to perform the update as well. Nonblocking primary-backup protocols are discussed in [Budhiraja and Marzullo, 1992].

The main problem with nonblocking primary-backup protocols has to do with fault tolerance. In a blocking scheme, the client process knows for sure that the update operation is backed up by several other servers. This is not

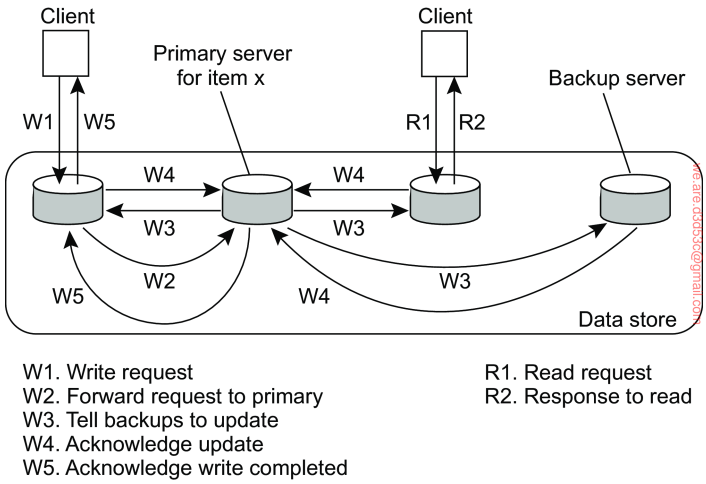


Figure 7.30: The principle of a primary-backup protocol.

the case with a nonblocking solution. The advantage, of course, is that write operations may speed up considerably.

Primary-backup protocols provide a straightforward implementation of sequential consistency, as the primary can order all incoming writes in a globally unique order. Evidently, all processes see all write operations in the same order, no matter which backup server they use to perform read operations. Also, with blocking protocols, processes will always see the effects of their most recent write operation (note that this cannot be guaranteed with a nonblocking protocol without taking special measures).

Local-write protocols

A variant of primary-backup protocols is one in which the primary copy migrates between processes that wish to perform a write operation. As before, whenever a process wants to update a data item x, it locates the primary copy of x, and subsequently moves it to its own location, as shown in Figure 7.31. The main advantage of this approach is that multiple, successive write operations can be carried out locally, while reading processes can still access their local copy. However, such an improvement can be achieved only if a nonblocking protocol is followed by which updates are propagated to the replicas after the primary has finished with locally performing the updates.

This primary-backup local-write protocol can also be applied to mobile computers that can operate in disconnected mode. Before disconnecting, the mobile computer becomes the primary server for each data item it expects to update. While being disconnected, all update operations are carried out locally, while other processes can still perform read operations (but no updates).

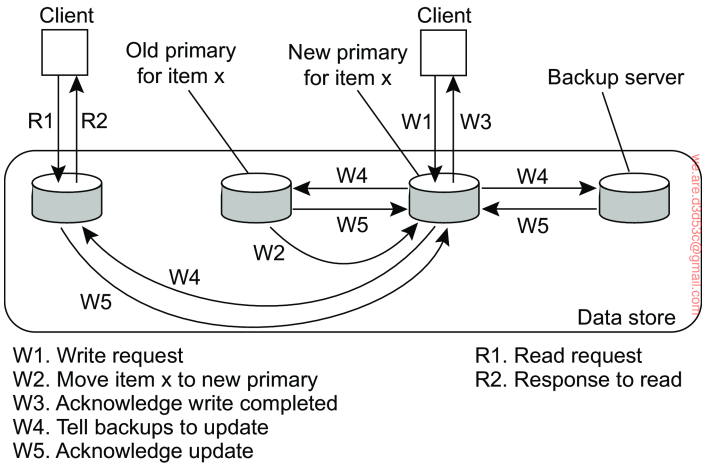


Figure 7.31: Primary-backup protocol in which the primary migrates to the process wanting to perform an update.

Later, when connecting again, updates are propagated from the primary to the backups, bringing the data store in a consistent state again.

As a last variant of this scheme, nonblocking local-write primary-based protocols are also used for distributed file systems in general. In this case, there may be a fixed central server through which normally all write operations take place, as in the case of remote-write primary backup. However, the server temporarily allows one of the replicas to perform a series of local updates, as this may considerably speed up performance. When the replica server is done, the updates are propagated to the central server, from where they are then distributed to the other replica servers.

7.5.2 Sequential consistency: Replicated-write protocols

In **replicated-write protocols**, write operations can be carried out at multiple replicas instead of only one, as in the case of primary-based replicas. A distinction can be made between active replication, in which an operation is forwarded to all replicas, and consistency protocols based on majority voting.

Active replication

In **active replication**, each replica has an associated process that carries out update operations. In contrast to other protocols, updates are generally propagated through the write operation that causes the update. In other words, the operation is sent to each replica. However, it is also possible to send the update.

One problem with active replication is that operations need to be carried out in the same order everywhere. Consequently, what is needed is a totally ordered multicast mechanism. A practical approach to accomplish total ordering is by a central coordinator, also called a **sequencer**. One approach is to first forward each operation to the sequencer, which assigns it a unique sequence number and subsequently forwards the operation to all replicas. Operations are carried out in the order of their sequence number.

Note 7.8 (Advanced: Achieving scalability)
Note that using a sequencer may easily introduce scalability problems. In fact, if totally ordered multicasting is needed, a combination of symmetric multicasting using Lamport timestamps [Lamport, 1978] and sequencers may be necessary. Such a solution is described by Rodrigues et al. [1996]. The essence of that solution is to have multiple sequencers multicast update operations to each other and order the updates using Lamport's total-ordering mechanism, as described in Chapter 5. Nonsequencing processes are grouped such that each group uses a single sequencer. Any nonsequencing process sends update requests to its sequencer and waits until it receives an acknowledgment that its request has been processed (i.e., multicast to the other sequencers in a totally ordered fashion). Obviously, there is a trade-off between the number of processes that act as sequencer and those that do not, as well as the choice of processes to act as sequencer. As it turns out, this trade-off depends very much on the application and, in particular, the relative update rate at each process.

Quorum-based protocols

A different approach to supporting replicated writes is to use voting, as originally proposed by Thomas [1979] and generalized by Gifford [1979]. The basic idea is to require clients to request and acquire the permission of multiple servers before either reading or writing a replicated data item.

As a simple example of how the algorithm works, consider a distributed file system and suppose that a file is replicated on N servers. We could make a rule stating that to update a file, a client must first contact at least half the servers plus one (a majority) and get them to agree to do the update. Once they have agreed, the file is changed and a new version number is associated with the new file. The version number is used to identify the version of the file and is the same for all the newly updated files.

To read a replicated file, a client must also contact at least half the servers plus one and ask them to send the version numbers associated with the file. If all the version numbers are the same, this must be the most recent version because an attempt to update only the remaining servers would fail because there are not enough of them.

For example, if there are five servers and a client determines that three of them have version 8, it is impossible that the other two have version 9. After

all, any successful update from version 8 to version 9 requires getting three servers to agree to it, not just two.

When quorum-based replication was originally introduced, a somewhat more general scheme was proposed. In it, to read a file of which N replicas exist, a client needs to assemble a **read quorum**, an arbitrary collection of any N_R servers, or more. Similarly, to modify a file, a **write quorum** of at least N_W servers is required. The values of N_R and N_W are subject to the following two constraints:

1. $N_R + N_W > N$
2. $N_W > N/2$

The first constraint is used to prevent read-write conflicts, whereas the second prevents write-write conflicts. Only after the appropriate number of servers has agreed to participate can a file be read or written.

To see how this algorithm works, consider Figure 7.32(a) which has $N_R = 3$ and $N_W = 10$. Imagine that the most recent write quorum consisted of the 10 servers C through L. All of these get the new version and the new version number. Any subsequent read quorum of three servers will have to contain at least one member of this set. When the client looks at the version numbers, it will know which is most recent and take that one.

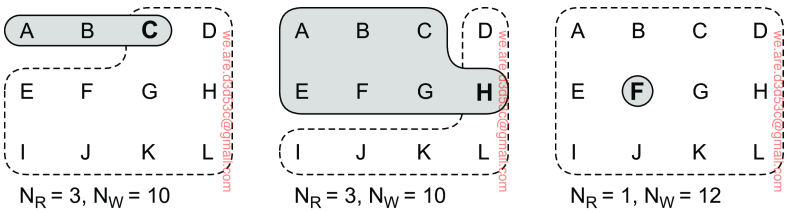


Figure 7.32: Three examples of the voting algorithm. The gray areas denote a read quorum; the white ones a write quorum. Servers in the intersection are denoted in boldface. (a) A correct choice of read and write set. (b) A choice that may lead to write-write conflicts. (c) A correct choice, known as ROWA (read one, write all).

In Figure 7.32 we see two more examples. In Figure 7.32(b) a write-write conflict may occur because $N_W \leq N/2$. In particular, if one client chooses {A, B, C, E, F, G} as its write set and another client chooses {D, H, I, J, K, L} as its write set, then clearly we will run into trouble, as the two updates will both be accepted without detecting that they actually conflict.

The situation shown in Figure 7.32(c) is especially interesting because it sets N_R to one, making it possible to read a replicated file by finding any copy and using it. The price paid for this good read performance, however, is that

write updates need to acquire all copies. This scheme is generally referred to as **Read-One, Write-All**, (**ROWA**).

Quorum-based replication has been an active field of research and an inspiration for various protocols. However, in practice, we often see primary-based protocols at work (to which we return in Chapter 8). Jalote [1994] provides a good overview of various quorum-based protocols. A more formal introduction is given by Merideth and Reiter [2010], whereas many formal details and analyses of different protocols are described in [Vukolić, 2012].

7.5.3 Cache-coherence protocols

Caches form a special case of replication, in the sense that they are generally controlled by clients instead of servers. However, **cache-coherence protocols**, which ensure that a cache is consistent with the server-initiated replicas are, in principle, not very different from the consistency protocols discussed so far.

There has been much research in the design and implementation of caches, especially in the context of shared-memory multiprocessor systems. Many solutions are based on support from the underlying hardware, for example, by assuming that snooping or efficient broadcasting can be done. In the context of middleware-based distributed systems that are built on top of general-purpose operating systems, software-based solutions to caches are more interesting. In this case, two separate criteria are often maintained to classify caching protocols (see also Min and Baer [1992], Lilja [1993], or Tartalja and Milutinovic [1997]).

First, caching solutions may differ in their **coherence detection strategy**, that is, *when* inconsistencies are actually detected. In static solutions, a compiler is assumed to perform the necessary analysis before execution, and to determine which data may actually lead to inconsistencies because they may be cached. The compiler simply inserts instructions that avoid inconsistencies. Dynamic solutions are typically applied in the distributed systems studied in this book. In these solutions, inconsistencies are detected at runtime. For example, a check is made with the server to see whether the cached data have been modified since they were cached.

In the case of distributed databases, dynamic detection-based protocols can be further classified by considering exactly when, during a transaction, the detection is done. Franklin et al. [1997] distinguish the following three cases. First, when during a transaction a cached data item is accessed, the client needs to verify whether that data item is still consistent with the version stored at the (possibly replicated) server. The transaction cannot proceed to use the cached version until its consistency has been definitively validated.

A second, optimistic, approach is to let the transaction proceed while verification is taking place. In this case, it is assumed that the cached data were up-to-date when the transaction started. If that assumption later proves to be false, the transaction will have to abort.

The third approach is to verify whether the cached data are up-to-date only when the transaction commits. In effect, the transaction just starts operating on the cached data and hopes for the best. After all the work has been done, accessed data are verified for consistency. When stale data were used, the transaction is aborted.

Another design issue for cache-coherence protocols is the **coherence enforcement strategy**, which determines *how* caches are kept consistent with the copies stored at servers. The simplest solution is to disallow shared data to be cached at all. Instead, shared data are kept only at the servers, which maintain consistency using one of the primary-based or replication-write protocols discussed above. Clients are allowed to cache only private data. Obviously, this solution can offer only limited performance improvements.

When shared data can be cached, there are two approaches to enforce cache coherence. The first is to let a server send an invalidation to all caches whenever a data item is modified. The second is to simply propagate the update. Most caching systems use one of these two schemes. Dynamically choosing between sending invalidations or updates is sometimes supported in client-server databases.

Finally, we also need to consider what happens when a process modifies cached data. When read-only caches are used, update operations can be performed only by servers, which subsequently follow some distribution protocol to ensure that updates are propagated to caches. Often, a pull-based approach is followed. In this case, a client detects that its cache is stale, and requests a server for an update.

An alternative approach is to allow clients to directly modify the cached data, and forward the update to the servers. This approach is followed in **write-through caches**, which are often used in distributed file systems. In effect, write-through caching is similar to a primary-based local-write protocol in which the client's cache has become a temporary primary. To guarantee (sequential) consistency, it is necessary that the client has been granted exclusive write permissions, or otherwise write-write conflicts may occur.

Write-through caches potentially offer improved performance over other schemes, as all operations can be carried out locally. Further enhancements can be made if we delay the propagation of updates by allowing multiple writes to take place before informing the servers. This leads to what is known as a **write-back cache**, which is, again, mainly applied in distributed file systems.

Note 7.9 (Example: Client-side caching in NFS)
As a practical example, consider the general caching model in NFS as shown in Figure 7.33. Each client can have a memory cache that contains data previously

read from the server. In addition, there may also be a disk cache that is added as an extension to the memory cache, using the same consistency parameters.

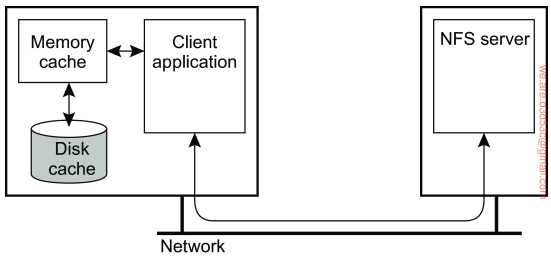


Figure 7.33: Client-side caching in NFS.

Typically, clients cache file data, attributes, file handles, and directories. Different strategies exist to handle consistency of the cached data, cached attributes, and so on. Let us first take a look at caching file data.

NFSv4 supports two different approaches for caching file data. The simplest approach is when a client opens a file and caches the data it obtains from the server as the result of various read operations. In addition, write operations can be carried out in the cache as well. When the client closes the file, NFS requires that if modifications have taken place, the cached data must be flushed back to the server. This approach corresponds to implementing session semantics as discussed earlier.

Once (part of) a file has been cached, a client can keep its data in the cache even after closing the file. Also, several clients on the same machine can share a single cache. NFS requires that whenever a client opens a previously closed file that has been (partly) cached, the client must immediately revalidate the cached data. Revalidation takes place by checking when the file was last modified and invalidating the cache in case it contains stale data.

In NFSv4 a server may delegate some of its rights to a client when a file is opened. **Open delegation** takes place when the client machine is allowed to locally handle open and close operations from other clients on the same machine. Normally, the server is in charge of checking whether opening a file should succeed, for example because share reservations need to be considered. With open delegation, the client machine is sometimes allowed to make such decisions, avoiding the need to contact the server.

For example, if a server has delegated the opening of a file to a client that requested write permissions, file locking requests from other clients on the same machine can also be handled locally. The server will still handle locking requests from clients on other machines, by simply denying those clients access to the file. Note that this scheme does not work in the case of delegating a file to a client that requested only read permissions. In that case, whenever another local client wants to have write permissions, it will have to contact the server; it is not possible to handle the request locally.

An important consequence of delegating a file to a client is that the server

needs to be able to recall the delegation, for example, when another client on a different machine needs to obtain access rights to the file. Recalling a delegation requires that the server can do a callback to the client, as illustrated in Figure 7.34.

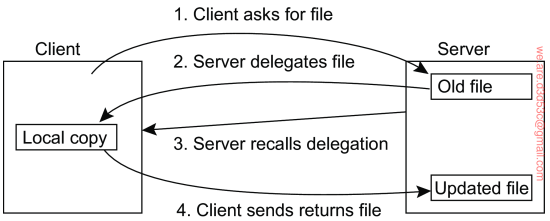


Figure 7.34: Using the NFSv4 callback mechanism to recall file delegation.

A callback is implemented in NFS using its underlying RPC mechanisms. Note, however, that callbacks require that the server keeps track of clients to which it has delegated a file. Here, we see another example where an NFS server cannot be implemented in a stateless manner. Note, however, that the combination of delegation and stateful servers may lead to various problems in the presence of client and server failures. For example, what should a server do when it had delegated a file to a now unresponsive client?

Clients can also cache attribute values, but are largely left on their own when it comes to keeping cached values consistent. In particular, attribute values of the same file cached by two different clients may be different unless the clients keep these attributes mutually consistent. Modifications to an attribute value should be immediately forwarded to the server, thus following a write-through cache coherence policy.

A similar approach is followed for caching file handles (or rather, the name-to-file handle mapping) and directories. To mitigate the effects of inconsistencies, NFS uses leases on cached attributes, file handles, and directories. After some time has elapsed, cache entries are thus automatically invalidated and revalidation is needed before they are used again.

7.5.4 Implementing continuous consistency

As part of their work on continuous consistency, Yu and Vahdat [2000] have developed a number of protocols to tackle the three forms of continuous consistency. In the following, we briefly consider a number of solutions, omitting details for clarity.

Bounding numerical deviation

We first concentrate on one solution for keeping the numerical deviation within bounds. Again, our purpose is not to go into all the details for each protocol, but rather to give the general idea. Details for bounding numerical deviation can be found in [Yu and Vahdat, 2000].

We concentrate on writes to a single data item x. Each write W(x) has an associated *value* that represents the numerical value by which x is updated, denoted as $val(W(x))$, or simply $val(W)$. For simplicity, we assume that $val(W) > 0$. Each write W is initially submitted to one out of the N available replica servers, in which case that server becomes the write's *origin*, denoted as $origin(W)$. If we consider the system at a specific moment we will see several submitted writes that still need to be propagated to all servers. To this end, each server S_i will keep track of a *log* L_i of writes that it has performed on its own local copy of x.

Let $TW[i,j]$ be the effect of performing the writes executed by server S_i that originated from server S_j:

$$TW[i,j] = \sum\{val(W) | origin(W) = S_j \text{ and } W \in L_i\}$$

Note that $TW[i,i]$ represents the aggregated writes submitted to S_i. Our goal is for any time t, to let the current value v_i of x at server S_i deviate within bounds from the actual value v of x. This actual value is completely determined by all submitted writes. That is, if v_0 is the initial value of x, then

$$v = v_0 + \sum_{k=1}^{N} TW[k,k]$$

and

$$v_i = v_0 + \sum_{k=1}^{N} TW[i,k]$$

Note that $v_i \leq v$. Let us concentrate only on absolute deviations. In particular, for every server S_i, we associate an upper bound δ_i such that we need to enforce:

$$v - v_i \leq \delta_i$$

Writes submitted to a server S_i will need to be propagated to all other servers. There are different ways in which this can be done, but typically an epidemic protocol will allow rapid dissemination of updates. In any case, when a server S_i propagates a write originating from S_j to S_k, the latter will be able to learn about the value $TW[i,j]$ at the time the write was sent. In other words, S_k can maintain a *view* $TW_k[i,j]$ of what it believes S_i will have as value for $TW[i,j]$. Obviously,

$$0 \leq TW_k[i,j] \leq TW[i,j] \leq TW[j,j]$$

The whole idea is that when a server S_k notices that S_i has not been staying in the right pace with the updates that have been submitted to S_k, it forwards writes from its log to S_i. This forwarding effectively advances the view $TW_k[i,k]$ that S_k has of $TW[i,k]$, making the deviation $TW[i,k] - TW_k[i,k]$ smaller. In particular, S_k advances its view on $TW[i,k]$ when an application submits a new write that would increase $TW[k,k] - TW_k[i,k]$ beyond $\delta_i/(N-1)$. We leave it as an exercise to the reader to show that advancement always ensures that $v - v_i \leq \delta_i$.

Bounding staleness deviations

There are many ways to keep the staleness of replicas within specified bounds. One simple approach is to let the server S_k keep a real-time vector clock RVC_k where $RVC_k[i] = t_i$ means that S_k has seen all writes that have been submitted to S_i up to time t_i. In this case, we assume that each submitted write is timestamped by its origin server, with t_i denoting the time *local to* S_i.

If the clocks between the replica servers are loosely synchronized, then an acceptable protocol for bounding staleness would be the following. Whenever the server S_k notes that $t_k - RVC_k[i]$ is about to exceed a specified limit, it simply starts pulling in writes that originated from S_i with a timestamp later than $RVC_k[i]$.

Note that in this case, a replica server is responsible for keeping its copy of x up to date regarding writes that have been issued elsewhere. In contrast, when maintaining numerical bounds, we followed a push approach by letting an origin server keep replicas up to date by forwarding writes. The problem with pushing writes in the case of staleness is that no guarantees can be given for consistency when it is unknown in advance what the maximal propagation time will be. This situation is somewhat improved by pulling in updates, as multiple servers can help to keep a server's copy of x up to date.

Bounding ordering deviations

Recall that ordering deviations in continuous consistency are caused by the fact that a replica server tentatively applies updates that have been submitted to it. As a result, each server will have a local queue of tentative writes for which the actual order in which they are to be applied to the local copy of x still needs to be determined. The deviation is bounded by specifying the maximal length of the queue of tentative writes.

As a consequence, detecting when ordering consistency needs to be enforced is simple: when the length of this local queue exceeds a specified maximal length. At that point, a server will no longer accept any newly submitted writes, but will instead attempt to commit tentative writes by negotiating with other servers in which order its writes should be executed. We thus have to enforce a globally consistent ordering of tentative writes.

7.5.5 Implementing client-centric consistency

For our last topic on consistency protocols, let us draw our attention to implementing client-centric consistency. Implementing client-centric consistency is relatively straightforward if performance issues are ignored.

In a naive implementation of client-centric consistency, each write operation W is assigned a globally unique identifier. Such an identifier is assigned by the server to which the write had been submitted. We refer to this server as the *origin* of W. Then, for each client, we keep track of two sets of writes.

The read set for a client consists of the writes relevant for the read operations performed by a client. Likewise, the write set consists of the (identifiers of the) writes performed by the client.

Monotonic-read consistency is implemented as follows. When a client wants to perform a read operation at a server, that server is handed the client's read set to check whether all the identified writes have taken place locally. If not, it contacts the other servers to ensure that it is brought up to date before carrying out the read operation. Alternatively, the read operation is forwarded to a server where the write operations have already taken place. After the read operation is performed, the write operations that have taken place at the selected server and which are relevant for the read operation are added to the client's read set.

Note that it should be possible to determine exactly where the write operations identified in the read set have taken place. For example, the write identifier could include the identifier of the server to which the operation was submitted. That server is required to, for example, log the write operation so that it can be replayed at another server. In addition, write operations should be performed in the order they were submitted. Ordering can be achieved by letting the client generate a globally unique sequence number that is included in the write identifier. If each data item can be modified only by its owner, the latter can supply the sequence number.

Monotonic-write consistency is implemented analogous to monotonic reads. Whenever a client initiates a new write operation at a server, the server is handed over the client's write set. (Again, the size of the set may be prohibitively large in the face of performance requirements. An alternative solution is discussed below.) It then ensures that the identified write operations are performed first and in the correct order. After performing the new operation, that operation's write identifier is added to the write set. Note that bringing the current server up to date with the client's write set may introduce a considerable increase in the client's response time, since the client then waits for the operation to fully complete.

Likewise, read-your-writes consistency requires that the server where the read operation is to be performed has seen all the write operations in the client's write set. The writes can simply be fetched from other servers before the read operation is actually executed, although this may lead to a poor response time. Alternatively, the client-side software can search for a server where the identified write operations in the client's write set have already been performed.

Finally, writes-follow-reads consistency can be implemented by first bringing the selected server up to date with the write operations in the client's read set, and then later adding the identifier of the write operation to the write set, along with the identifiers in the read set (which have now become relevant for the write operation just performed).

Note 7.10 (Advanced: Improving efficiency)
It is easy to see that the read set and write set associated with each client can become very large. To keep these sets manageable, a client's read and write operations are grouped into sessions. A *session* is typically associated with an application: it is opened when the application starts and is closed when it exits. However, sessions may also be associated with applications that are temporarily exited, such as user agents for e-mail. Whenever a client closes a session, the sets are simply cleared. Of course, if a client opens a session that it never closes, the associated read and write sets can still become very large.

The main problem with a naive implementation lies in the representation of the read and write sets. Each set consists of a number of identifiers for write operations. Whenever a client forwards a read or write request to a server, a set of identifiers is handed to the server as well to see whether all write operations relevant to the request have been carried out by that server.

This information can be more efficiently represented by vector timestamps as follows. First, whenever a server accepts a new write operation W, it assigns that operation a globally unique identifier along with a timestamp $ts(\mathsf{W})$. A subsequent write operation submitted to that server is assigned a higher-valued timestamp. Each server $\mathsf{S_i}$ maintains a vector timestamp $\mathsf{WVC_i}$, where $\mathsf{WVC_i}[j]$ is equal to the timestamp of the most recent write operation originating from $\mathsf{S_j}$ that has been processed by $\mathsf{S_i}$.

For clarity, assume that for each server, writes from $\mathsf{S_j}$ are processed in the order that they were submitted. Whenever a client issues a request to perform a read or write operation O at a specific server, that server returns its current timestamp along with the results of O. Read and write sets are subsequently represented by vector timestamps. More specifically, for each session A, we construct a vector timestamp $\mathsf{SVC_A}$ with $\mathsf{SVC_A}[i]$ set equal to the maximum timestamp of all write operations in A that originate from server $\mathsf{S_i}$:

$$\mathsf{SVC_A}[j] = \max\{ts(\mathsf{W}) | \mathsf{W} \in \mathsf{A} \text{ and } origin(\mathsf{W}) = \mathsf{S_j}\}$$

In other words, the timestamp of a session always represents the latest write operations that have been seen by the applications that are being executed as part of that session. The compactness is obtained by representing all observed write operations originating from the same server through a single timestamp.

As an example, suppose a client, as part of session A, logs in at server $\mathsf{S_i}$. To that end, it passes $\mathsf{SVC_A}$ to $\mathsf{S_i}$. Assume that $\mathsf{SVC_A}[j] > \mathsf{WVC_i}[j]$. What this means is that $\mathsf{S_i}$ has not yet seen all the writes originating from $\mathsf{S_j}$ that the client has seen. Depending on the required consistency, server $\mathsf{S_i}$ may now have to fetch these writes before being able to consistently report back to the client. Once the operation has been performed, server $\mathsf{S_i}$ will return its current timestamp $\mathsf{WVC_i}$. At that point, $\mathsf{SVC_A}$ is adjusted to:

$$\mathsf{SVC_A}[j] \leftarrow \max\{\mathsf{SVC_A}[j], \mathsf{WVC_i}[j]\}$$

Again, we see how vector timestamps can provide an elegant and compact way of representing history in a distributed system.

7.6 Example: Caching and replication in the Web

The Web is arguably the largest distributed system ever built. Originating from a relatively simple client-server architecture, it is now a sophisticated system consisting of many techniques to ensure stringent performance and availability requirements. These requirements have led to numerous proposals for caching and replicating Web content. Where the original schemes (which are still largely deployed) have been targeted toward supporting static content, much effort has also been put into supporting dynamic content, that is, supporting documents that are generated on-the-spot as the result of a request, as well as those containing scripts and such. An overview of traditional Web caching and replication is provided by Rabinovich and Spastscheck [2002].

Client-side caching in the Web generally occurs at two places. In the first place, most browsers are equipped with a relatively simple caching facility. Whenever a document is fetched, it is stored in the browser's cache, from where it is loaded the next time. In the second place, a client's site often runs a Web proxy. A Web proxy accepts requests from local clients and passes these to Web servers. When a response comes in, the result is passed to the client. The advantage of this approach is that the proxy can cache the result and return that result to another client, if necessary. In other words, a Web proxy can implement a shared cache. With so many documents being generated on the fly, the server generally provides the document in pieces, instructing the client to cache only those parts that are not likely to change when the document is requested a next time.

In addition to caching at browsers and proxies, ISPs generally also place caches in their networks. Such schemes are mainly used to reduce network traffic (which is good for the ISP) and to improve performance (which is good for end users). However, with multiple caches along the request path from client to server, there is a risk of increased latencies when caches do not contain the requested information.

Note 7.11 (Advanced: Cooperative caching)
As an alternative to building hierarchical caches, one can also organize caches for cooperative deployment, as shown in Figure 7.35. In **cooperative caching** or **distributed caching**, whenever a cache miss occurs at a Web proxy, the proxy first checks a number of neighboring proxies to see if one of them contains the requested document. If such a check fails, the proxy forwards the request to the Web server responsible for the document. In more traditional settings, this scheme is primarily deployed with Web caches belonging to the same organization or institution.

A study by Wolman et al. [1999] shows that cooperative caching may be effective for only relatively small groups of clients (in the order of tens of thousands of users). However, such groups can also be serviced by using a single proxy cache, which is much cheaper in terms of communication and resource usage.

However, in a study from a decade later, Wendell and Freedman [2011] show that in a highly decentralized system, cooperative caching actually turned out to be highly effective. These studies do not necessarily contradict each other: in both cases, the conclusion is that the effect of cooperative caching depends highly on the demands from clients.

A comparison between hierarchical and cooperative caching by Rodriguez et al. [2001] makes clear that there are various trade-offs to make. For example, because cooperative caches are generally connected through high-speed links, the transmission time needed to fetch a document is much lower than for a hierarchical cache. Also, as is to be expected, storage requirements are less strict for cooperative caches than hierarchical ones.

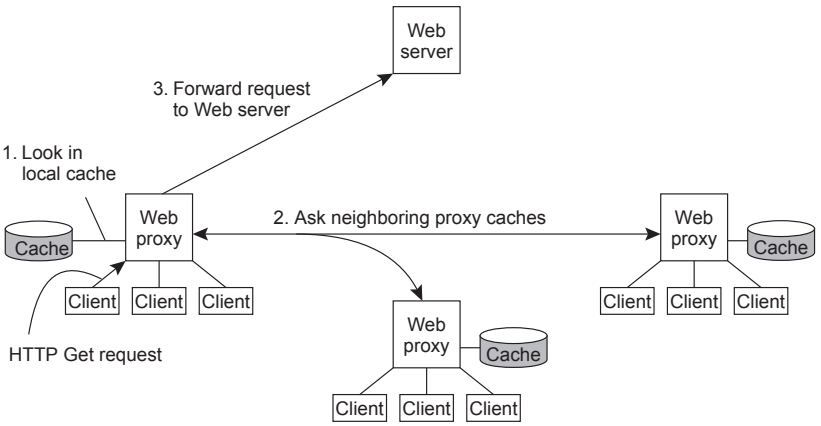


Figure 7.35: The principle of cooperative caching.

Different cache-consistency protocols have been deployed in the Web. To guarantee that a document returned from the cache is consistent, some Web proxies first send a conditional HTTP get request to the server with an additional If-Modified-Since request header, specifying the last modification time associated with the cached document. Only if the document has been changed since that time, will the server return the entire document. Otherwise, the Web proxy can simply return its cached version to the requesting local client, which corresponds to a pull-based protocol.

Unfortunately, this strategy requires that the proxy contacts a server for each request. To improve performance at the cost of weaker consistency, the widely used Squid Web proxy [Wessels, 2004] assigns an expiration time T_{expire} that depends on how long ago the document was last modified when it is cached. In particular, if $T_{last_modified}$ is the last modification time of a document (as recorded by its owner), and T_{cached} is the time it was cached, then

$$T_{expire} = \alpha(T_{cached} - T_{last_modified}) + T_{cached}$$

with $\alpha = 0.2$ (this value has been derived from practical experience). Until T_{expire}, the document is considered valid, and the proxy will not contact the server. After the expiration time, the proxy requests the server to send a fresh copy, unless it had not been modified. We note that Squid also allows the expiration time to be bounded by a minimum and a maximum time.

As an alternative to a pull-based protocol is that the server notifies proxies that a document has been modified by sending an invalidation. The problem with this approach for Web proxies is that the server may need to keep track of many proxies, inevitably leading to a scalability problem. However, by combining leases and invalidations, the state to be maintained at the server can be kept within acceptable bounds. Note that this state is largely dictated by the expiration times set for leases: the lower, the less caches a server needs to keep track of. Nevertheless, invalidation protocols for Web proxy caches are hardly ever applied. A comparison of Web caching consistency policies can be found in [Cao and Ozsu, 2002]. Their conclusion is that letting the server send invalidations can outperform any other method in terms of bandwidth and perceived client latency, while maintaining cached documents consistent with those at the origin server.

Finally, we should also mention that much research has been conducted to find out what the best cache replacement strategies are. Numerous proposals exist, but by-and-large, simple replacement strategies such as evicting the least recently used object work well enough. An in-depth survey of replacement strategies is presented by Podling and Boszormenyi [2003]; Ali et al. [2011] provide a more recent overview, which also includes Web prefetching techniques.

As the importance of the Web continues to increase as a vehicle for organizations to present themselves and to directly interact with end users, we see a shift between maintaining the content of a Web site and making sure that the site is easily and continuously accessible. This distinction has paved the way for **Content Delivery Networks** (**CDN**). The main idea underlying these CDNs is that they act as a Web hosting service, providing an infrastructure for distributing and replicating the Web documents of multiple sites across the Internet. The size of the infrastructure can be impressive. As mentioned before, as of 2022, Akamai is reported to have over 400,000 servers worldwide.

The sheer size of a CDN requires that hosted documents are automatically distributed and replicated. In most cases, a large-scale CDN is organized similar to a feedback-control loop, as shown in Figure 7.36 and which is described extensively in [Sivasubramanian et al., 2004b].

There are essentially three different kinds of aspects related to replication in Web hosting systems: metric estimation, adaptation triggering, and taking appropriate measures. The latter can be subdivided into replica placement decisions, consistency enforcement, and client-request routing. In the following, we briefly pay attention to each of these.

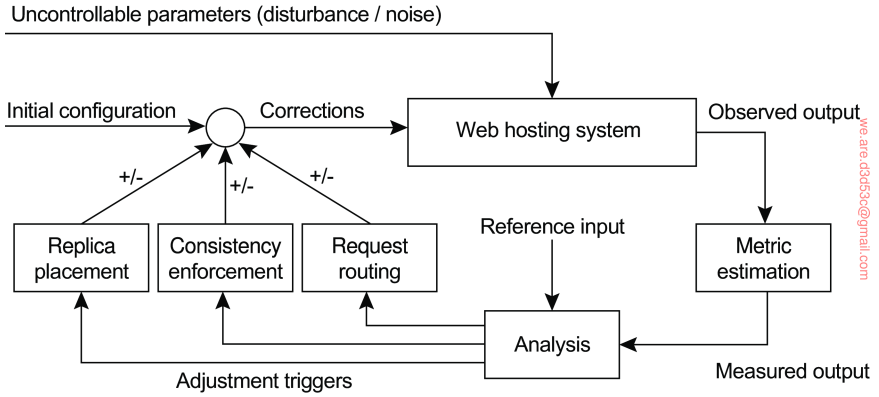


Figure 7.36: The general organization of a CDN as a feedback-control system.

An interesting aspect of CDNs is that they need to make a trade-off between many aspects when it comes to hosting replicated content. For example, access times for a document may be optimal if a document is massively replicated, but at the same time this incurs a financial cost, as well as a cost in terms of bandwidth usage for disseminating updates. By and large, there are many proposals for estimating how well a CDN is performing. These proposals can be grouped into several classes.

First, there are *latency metrics*, by which the time is measured for an action, for example, fetching a document, to take place. Trivial as this may seem, estimating latencies becomes difficult when, for example, a process deciding on the placement of replicas needs to know the delay between a client and some remote server. Typically, an algorithm for globally positioning nodes as discussed in Section 5.7.3 will need to be deployed.

Instead of estimating latency, it may be more important to measure the available bandwidth between two nodes. This information is particularly important when large documents have to be transferred, as in that case, the responsiveness of the system is largely dictated by the time that a document can be transferred. There are various tools for measuring available bandwidth, but in all cases it turns out that accurate measurements can be difficult to attain (see also Strauss et al. [2003], Shriram and Kaur [2007], Chaudhari and Biradar [2015], and Atxutegi et al. [2016]).

Another class consists of *spatial metrics*, which mainly consist of measuring the distance between nodes in terms of the number of network-level routing hops, or hops between autonomous systems. Again, determining the number of hops between two arbitrary nodes can be very difficult, and may also not even correlate with latency [Huffaker et al., 2002]. Moreover, simply looking at routing tables is not going to work when low-level techniques such as **Multi-Protocol Label Switching** (**MPLS**) are deployed. MPLS circumvents

network-level routing by using virtual-circuit techniques to immediately and efficiently forward packets to their destination (see also Guichard et al. [2005]). Packets may thus follow entirely different routes than advertised in the tables of network-level routers.

A third class is formed by *network usage metrics*, which most often entails consumed bandwidth. Computing consumed bandwidth in terms of the number of bytes to transfer is generally easy. However, to do this correctly, we need to consider how often the document is read, how often it is updated, and how often it is replicated.

Consistency metrics tell us to what extent a replica is deviating from its master copy. We already discussed extensively how consistency can be measured in the context of continuous consistency [Yu and Vahdat, 2002].

Finally, *financial metrics* form another class for measuring how well a CDN is doing. Although not technical at all, considering that most CDN operate on a commercial basis, it is clear that often financial metrics will be decisive. Moreover, the financial metrics are closely related to the actual infrastructure of the Internet. For example, most commercial CDNs place servers at the edge of the Internet, meaning that they hire capacity from ISPs directly servicing end users. At this point, business models become intertwined with technological issues, an area that is not at all well understood. There is only few material available on the relation between financial performance and technological issues [Janiga et al., 2001].

From these examples, it should become clear that simply measuring the performance of a CDN, or even estimating its performance, may by itself be an extremely complex task. In practice, for commercial CDNs the issue that really counts is whether they can meet the service-level agreements that have been made with customers. These agreements are often formulated simply in terms of how quickly customers are to be serviced. It is then up to the CDN to make sure that these agreements are met.

Another question that needs to be addressed is when and how adaptations are to be triggered. A simple model is to periodically estimate metrics and subsequently take measures as needed. This approach is often seen in practice. Special processes located at the servers collect information and periodically check for changes.

An interesting aspect of this scheme is the simplicity by which consistency of documents can be enforced. Clearly, whenever a main document is changed, a client will always be able to fetch it from the origin server. In the case of embedded documents, a different approach needs to be followed as these documents are, in principle, fetched from a nearby replica server. To this end, a URL for an embedded document not only refers to a special host name that eventually leads to a CDN DNS server, but also contains a unique identifier that is changed every time the embedded document changes. In effect, this identifier changes the name of the embedded document. As a consequence,

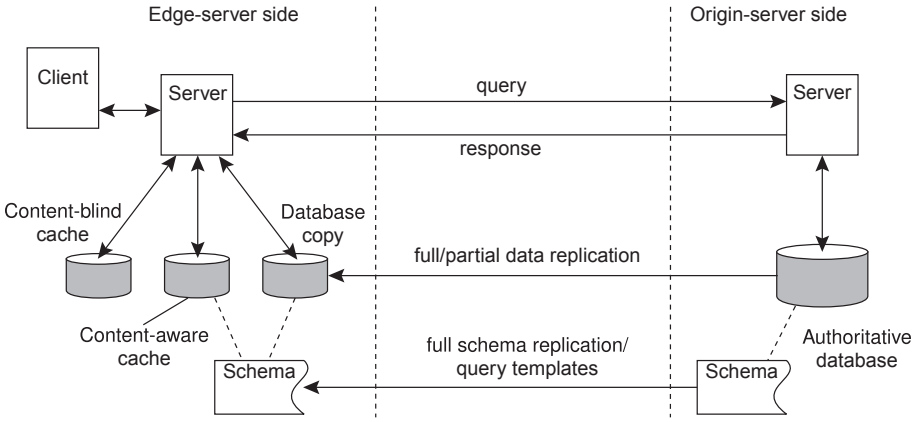


Figure 7.37: Alternatives for caching and replication with Web applications.

when the client is redirected to a specific CDN server, that server will not find the named document in its cache and will thus fetch it from the origin server. The old document will eventually be evicted from the server's cache as it is no longer referenced.

Up to this point, we have mainly concentrated on caching and replicating static Web content. In practice, we see that the Web is increasingly offering more dynamically generated content, but that it is also expanding toward offering services that can be called by remote applications. Also in these situations we see that caching and replication can help considerably in improving the overall performance, although the methods to reach such improvements are more subtle than what we discussed so far (see also Conti et al. [2005]).

When considering improving performance of Web applications through caching and replication, matters are complicated by the fact that several solutions can be deployed, with no single one standing out as the best. Let us consider the edge-server situation as sketched in Figure 7.37 (see also Sivasubramanian et al. [2007]). In this case, we assume a CDN in which each hosted site has an origin server that acts as the authoritative site for all read and update operations. An edge server is used to handle client requests, and has the ability to store (partial) information as also kept at an origin server.

Recall that in an edge-server architecture, Web clients request data through an edge server, which, in turn, gets its information from the origin server associated with the specific Website referred to by the client. As also shown in Figure 7.37 we assume that the origin server consists of a database from which responses are dynamically created. Although we have shown only a single Web server, it is common to organize each server according to a multitiered architecture, as we discussed before. An edge server can now be roughly organized along the following lines.

First, to improve performance, we can decide to apply full replication of the data stored at the origin server. This scheme works well whenever the update ratio is low and when queries require an extensive database search. As mentioned above, we assume that all updates are carried out at the origin server, which takes responsibility for keeping the replicas and the edge servers in a consistent state. Read operations can thus take place at the edge servers. Here, we see that replicating for performance will fail when the update ratio is high, as each update will incur communication over a wide-area network to bring the replicas into a consistent state. As shown by Sivasubramanian et al. [2004a], the read-to-update ratio is the determining factor to what extent the origin database in a wide-area setting should be replicated.

Another case for full replication is when queries are generally complex. In the case of a relational database, this means that a query requires that multiple tables need to be searched and processed, as is generally the case with a join operation. Opposed to complex queries are simple ones that generally require access to only a single table to produce a response. In the latter case, **partial replication** by which only a subset of the data is stored at the edge server may suffice.

An alternative to partial replication is to make use of **content-aware caches**. The basic idea in this case is that an edge server maintains a local database that is now tailored to the type of queries that can be handled at the origin server. To explain, in a full-fledged database system a query will operate on a database in which the data has been organized into tables such that, for example, redundancy is minimized. Such databases are also said to be *normalized*.

In such databases, any query that adheres to the data schema can, in principle, be processed, although perhaps at considerable costs. With content-aware caches, an edge server maintains a database that is organized according to the structure of queries. What this means is that queries are assumed to adhere to a limited number of templates, effectively meaning that the different kinds of queries that can be processed is restricted. In these cases, whenever a query is received, the edge server matches the query against the available templates, and subsequently looks in its local database to compose a response, if possible. If the requested data is not available, the query is forwarded to the origin server, after which the response is cached before returning it to the client.

In effect, what the edge server is doing is checking whether a query can be answered with the data that is stored locally. This is also referred to as a **query containment check**. Note that such data was stored locally as responses to previously issued queries. This approach works best when queries tend to be repeated.

Part of the complexity of content-aware caching comes from the fact that the data at the edge server needs to be kept consistent. To this end,

the origin server needs to know which records are associated with which templates, so that any update of a record, or any update of a table, can be properly addressed by, for example, sending an invalidation message to the appropriate edge servers. Another source of complexity comes from the fact that queries still need to be processed at edge servers. In other words, there is nonnegligible computational power needed to handle queries. Considering that databases often form a performance bottleneck in Web servers, alternative solutions may be needed. Finally, caching results from queries that span multiple tables (i.e., when queries are complex) such that a query containment check can be carried out effectively is not trivial. The reason is that the organization of the results may be very different from the organization of the tables on which the query operated.

These observations lead us to a third solution, namely **content-blind caching**. The idea of content-blind caching is simple: when a client submits a query to an edge server, the server first computes a unique hash value for that query. Using this hash value, it subsequently looks in its cache whether it has processed this query before. If not, the query is forwarded to the origin and the result is cached before returning it to the client. If the query had been processed before, the previously cached result is returned to the client.

The main advantage of this scheme is the reduced computational effort that is required from an edge server in comparison to the database approaches described above. However, content-blind caching can be wasteful in terms of storage, as the caches may contain much more redundant data in comparison to content-aware caching or database replication. Note that such redundancy also complicates the process of keeping the cache up-to-date, as the origin server may need to keep an accurate account of which updates can potentially affect cached query results. These problems can be alleviated when assuming that queries can match only a limited set of predefined templates, as we discussed above.

7.7 Summary

There are primarily two reasons for replicating data: improving the reliability of a distributed system and improving performance. Replication introduces a consistency problem: whenever a replica is updated, that replica becomes different from the others. To keep replicas consistent, we need to propagate updates in such a way that temporary inconsistencies are not noticed. Unfortunately, doing so may severely degrade performance, especially in large-scale distributed systems.

The only solution to this problem is to relax consistency somewhat. Different consistency models exist. For continuous consistency, the goal is to set bounds to numerical deviation between replicas, staleness deviation, and deviations in the ordering of operations.

Numerical deviation refers to the value by which replicas may be different. This type of deviation is highly application dependent, but can, for example, be used in replication of stocks. Staleness deviation refers to the time by which a replica is still considered to be consistent, despite that updates may have taken place some time ago. Staleness deviation is often used for Web caches. Finally, ordering deviation refers to the maximum number of tentative writes that may be outstanding at any server without having synchronized with the other replica servers.

Consistent ordering of operations has since long formed the basis for many consistency models. Many variations exist, but only a few seem to prevail among application developers. Sequential consistency essentially provides the semantics that programmers expect in concurrent programming: all write operations are seen by everyone in the same order. Less used, but still relevant, is causal consistency, which reflects that operations that are potentially dependent on each other are carried out in the order of that dependency.

Weaker consistency models consider series of read and write operations. In particular, they assume that each series is appropriately "bracketed" by accompanying operations on synchronization variables, such as locks. Although this requires explicit effort from programmers, these models are generally easier to implement efficiently than, for example, pure sequential consistency.

As opposed to these data-centric models, researchers in the field of distributed databases for mobile users have defined a number of client-centric consistency models. Such models do not consider the fact that data may be shared by several users, but instead, concentrate on the consistency that an individual client should be offered. The underlying assumption is that a client connects to different replicas in the course of time, but that such differences should be made transparent. In essence, client-centric consistency models ensure that whenever a client connects to a new replica, that replica is brought up to date with the data that had been manipulated by that client before, and which may reside at other replica sites.

To propagate updates, different techniques can be applied. A distinction needs to be made concerning *what* is exactly propagated, to *where* updates are propagated, and *by whom* propagation is initiated. We can decide to propagate notifications, operations, or state. Likewise, not every replica always needs to be updated immediately. Which replica is updated at which time depends on the distribution protocol. Finally, a choice can be made whether updates are pushed to other replicas, or that a replica pulls in updates from another replica.

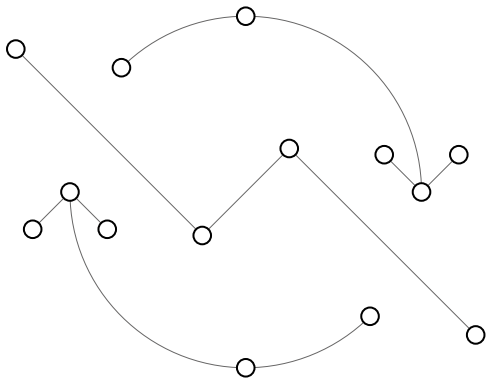
Consistency protocols describe specific implementations of consistency models. Regarding sequential consistency and its variants, a distinction can be made between primary-based protocols and replicated-write protocols. In primary-based protocols, all update operations are forwarded to a primary

copy that subsequently ensures the update is properly ordered and forwarded. In replicated-write protocols, an update is forwarded to several replicas at the same time. In that case, correctly ordering operations often becomes more difficult.

We pay separate attention to caching and replication in the Web and, related, content delivery networks. As it turns out, using existing servers and services, much of the techniques discussed before can be readily implemented using appropriate redirection techniques. Particularly challenging is caching content when databases are involved, as in those cases, much of what a Web server returns is dynamically generated. However, even in those cases, by carefully administrating what has already been cached at the edge, it is possible to invent highly efficient and effective caching schemes.

08

FAULT TOLERANCE



A characteristic feature of distributed systems that distinguishes them from single-machine systems is the notion of partial failure: part of the system is failing while the remaining part continues to operate, and seemingly correctly. An important goal in distributed-systems design is to construct the system in such a way that it can automatically recover from partial failures without seriously affecting the overall performance. In particular, whenever a failure occurs, the system should continue to operate in an acceptable way while repairs are being made. In other words, a distributed system is expected to be fault tolerant.

In this chapter, we take a closer look at techniques to achieve fault tolerance. After providing some general background, we will first look at process resilience through process groups. In this case, multiple identical processes cooperate, providing the appearance of a single logical process, to ensure that one or more of them can fail without a client noticing. A specifically difficult point in process groups is reaching consensus among the group members on which a client-requested operation is to perform. By now, Paxos is a commonly adopted, yet relatively intricate algorithm. We take two approaches in explaining Paxos. One that builds on how the protocol can be logically viewed as it is now, and one that gradually builds it up from scratch. The latter may help to understand many of its design decisions. We also pay extensive attention to the case in which servers may not just crash, but actually produce faulty results that cannot be immediately recognized as being faulty.

Achieving fault tolerance and reliable communication are strongly related. Next to reliable client-server communication, we pay attention to reliable group communication and notably atomic multicasting. In the latter case, a message is delivered to all nonfaulty processes in a group, or to none. Having atomic multicasting makes development of fault-tolerant solutions much easier.

Atomicity is a property that is important in many applications. In this chapter, we look into what are known as distributed commit protocols by which a group of processes is conducted to either jointly commit their local work, or collectively abort and return to a previous system state.

Finally, we will examine how to recover from a failure. In particular, we consider when and how the state of a distributed system should be saved to allow recovery to that state later on.

8.1 Introduction to fault tolerance

Fault tolerance has been subject to much research in computer science. In this section, we start with presenting the basic concepts related to processing failures, followed by a discussion of failure models. The key technique for handling failures is redundancy, which is also discussed. For more general

information on fault tolerance in distributed systems, see, for example Jalote [1994]; Shooman [2002] or Koren and Krishna [2007].

8.1.1 Basic concepts

To understand the role of fault tolerance in distributed systems, we first need to take a closer look at what it actually means for a distributed system to tolerate faults. Being fault tolerant is strongly related to what are called **dependable systems**. Dependability is a term that covers a number of useful requirements for distributed systems, including the following [Kopetz and Verissimo, 1993]:

- Availability
- Reliability
- Safety
- Maintainability

Availability is defined as the property that a system is ready to be used *immediately*. In general, it refers to the probability that the system is operating correctly at any given moment, and is available to perform its functions on behalf of its users. In other words, a highly available system is one that will most likely be working at a given instant in time.

Reliability refers to the property that a system can run *continuously* without failure. In contrast to availability, reliability is defined in terms of a time interval, instead of an instant in time. A highly reliable system is one that will most likely continue to work without interruption during a relatively long period of time. This is a subtle but important difference when compared to availability. If a system goes down on average for one, seemingly random millisecond every hour, it has an availability of more than 99.9999 percent, but is still unreliable. Similarly, a system that never crashes but is shut down for two specific weeks every August, has high reliability but only 96 percent availability. The two are not the same.

Safety refers to the situation that when a system temporarily fails to operate correctly, no catastrophic event happens. For example, many process-control systems, such as those used for controlling nuclear power plants or sending people into space, are required to provide a high degree of safety. If such control systems temporarily fail for only a very brief moment, the effects could be disastrous. Many examples from the past (and probably many more yet to come) show how hard it is to build safe systems.

Finally, **maintainability** refers to how easily a failed system can be repaired. A highly maintainable system may also show a high degree of availability, especially if failures can be detected and repaired automatically. However, as we shall see later in this chapter, automatically recovering from failures is easier said than done.

Note 8.1 (More information: Traditional metrics)
We can be a bit more precise when it comes to describing availability and reliability. Formally, the **availability** $A(t)$ of a component in the time interval $[0, t)$ is defined as the average fraction of time that the component has been functioning correctly during that interval. The **long-term availability** A of a component is defined as $A(\infty)$.

Likewise, the **reliability** $R(t)$ of a component in the time interval $[0, t)$ is formally defined as the conditional probability that it has been functioning correctly during that interval, given that it was functioning correctly at time $T = 0$. Following Pradhan [1996], to establish $R(t)$ we consider a system of N identical components. Let $N_0(t)$ denote the number of correctly operating components at time t and $N_1(t)$ the number of failed components. Then, clearly,

$$R(t) = \frac{N_0(t)}{N} = 1 - \frac{N_1(t)}{N} = \frac{N_0(t)}{N_0(t) + N_1(t)}$$

The rate at which components are failing can be expressed as the derivative $dN_1(t)/dt$. Dividing this by the number of correctly operating components at time t gives us the **failure rate function** $z(t)$:

$$z(t) = \frac{1}{N_0(t)} \frac{dN_1(t)}{dt}$$

From

$$\frac{dR(t)}{dt} = -\frac{1}{N} \frac{dN_1(t)}{dt}$$

it follows that

$$z(t) = \frac{1}{N_0(t)} \frac{dN_1(t)}{dt} = -\frac{N}{N_0(t)} \frac{dR(t)}{dt} = -\frac{1}{R(t)} \frac{dR(t)}{dt}$$

If we make the simplifying assumption that a component does not age (and thus essentially has no wear-out phase), its failure rate will be constant, i.e., $z(t) = z$, implying that

$$\frac{dR(t)}{dt} = -zR(t)$$

Because $R(0) = 1$, we obtain

$$R(t) = e^{-zt}$$

In other words, if we ignore aging of a component, we see that a constant failure rate leads to a reliability following an exponential distribution, having the form shown in Figure 8.1.

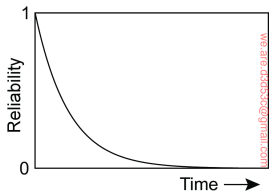


Figure 8.1: The reliability of a component having a constant failure rate.

Traditionally, fault-tolerance has been related to the following three metrics:

- **Mean Time To Failure** (***MTTF***): The average time until a component fails.
- **Mean Time To Repair** (***MTTR***): The average time needed to repair a component.
- **Mean Time Between Failures** (***MTBF***): Simply $MTTF + MTTR$.

Note that

$$A = \frac{MTTF}{MTBF} = \frac{MTTF}{MTTF + MTTR}$$

Also, these metrics make sense only if we have an accurate notion of what a failure actually is. As we will encounter later, identifying the occurrence of a failure may actually not be so obvious.

Often, dependable systems are also required to provide a high degree of security, especially when it comes to issues such as integrity. We will discuss security extensively in the next chapter.

A system is said to **fail** when it does not meet its promises. In particular, if a distributed system is designed to provide its users with a number of services, the system has failed when one or more of those services cannot be (completely) provided. An **error** is a part of a system's state that may lead to a failure. For example, when transmitting packets across a network, it is to be expected that some packets have been damaged when they arrive at the receiver. Damaged in this context means that the receiver may incorrectly sense a bit value (e.g., reading a 1 instead of a 0), or may even be unable to detect that something has arrived.

The cause of an error is called a **fault**. Clearly, finding out what caused an error is important. For example, a wrong or bad transmission medium may easily cause packets to be damaged. In this case, it is relatively easy to remove the fault. However, transmission errors may also be caused by bad weather conditions, such as in wireless networks. Changing the weather to reduce or prevent errors is a bit trickier.

As another example, a crashed program is clearly a failure, which may have happened because the program entered a branch of code containing a programming bug (i.e., a programming error). The cause of that bug is

typically a programmer. In other words, the programmer is the cause of the error (programming bug), in turn leading to a failure (a crashed program).

Building dependable systems closely relates to controlling faults. As explained by Avizienis et al. [2004], a distinction can be made between preventing, tolerating, removing, and forecasting faults. For our purposes, the most important issue is **fault tolerance**, meaning that a system can provide its services even in the presence of faults. For example, by applying error-correcting codes for transmitting packets, it is possible to tolerate, to a certain extent, relatively poor transmission lines and reducing the probability that an error (a damaged packet) may lead to a failure.

Faults are generally classified as transient, intermittent, or permanent. **Transient faults** occur once and then disappear. If the operation is repeated, the fault goes away. A bird flying through the beam of a microwave transmitter may cause lost bits on some network. If the transmission times out and is retried, it will probably work the second time.

An **intermittent fault** occurs, then vanishes of its own accord, then reappears, and so on. A loose contact on a connector will often cause an intermittent fault. Intermittent faults cause a great deal of aggravation because they are difficult to diagnose. Typically, when the fault doctor shows up, the system works fine.

A **permanent fault** is one that continues to exist until the faulty component is replaced. Burnt-out chips, software bugs, and disk-head crashes are examples of permanent faults.

8.1.2 Failure models

A system that fails is not adequately providing the services it was designed for. If we consider a distributed system as a collection of servers that communicate with one another and with their clients, not adequately providing services means that servers, communication channels, or possibly both, are not doing what they are supposed to do. However, a malfunctioning server itself may not always be the fault we are looking for. If such a server depends on other servers to adequately provide its services, the cause of an error may need to be searched for somewhere else.

Such dependency relations appear in abundance in distributed systems. A failing disk may make life difficult for a file server that is designed to provide a highly available file system. If such a file server is part of a distributed database, the proper working of the entire database may be at stake, as only part of its data may be accessible.

To get a better grasp on how serious a failure actually is, several classification schemes have been developed. One such scheme is shown in Figure 8.2, and is based on schemes described by Cristian [1991] and Hadzilacos and Toueg [1993].

Type of failure	Description of server's behavior
Crash failure	Halts, but is working correctly until it halts
Omission failure	Fails to respond to incoming requests
Receive omission	Fails to receive incoming messages
Send omission	Fails to send messages
Timing failure	Response lies outside a specified time interval
Response failure	Response is incorrect
Value failure	The value of the response is wrong
State-transition failure	Deviates from the correct flow of control
Arbitrary failure	May produce arbitrary responses at arbitrary times

Figure 8.2: Different types of failures.

A **crash failure** occurs when a server prematurely halts, but was working correctly until it stopped. An important aspect of crash failures is that once the server has halted, nothing is heard from it anymore. A typical example of a crash failure is an operating system that comes to a grinding halt, and for which there is only one solution: reboot it. Many personal computer systems (be they desktop computers or laptops) suffer from crash failures so often that people have come to expect them to be normal. Consequently, moving the reset button, for example, from the back of a cabinet to the front was done for good reason. Perhaps one day it can be moved to the back again, or even removed altogether.

An **omission failure** occurs when a server fails to respond to a request. Several things might go wrong. In the case of a **receive-omission failure**, possibly the server never got the request in the first place. Note that it may well be the case that the connection between a client and a server has been correctly established, but that there was no thread listening for incoming requests. Also, a receive-omission failure will generally not affect the current state of the server, as the server is unaware of any message sent to it.

Likewise, a **send-omission failure** happens when the server has done its work, but somehow fails in sending a response. Such a failure may happen, for example, when a send buffer overflows while the server was not prepared for such a situation. Note that, in contrast to a receive-omission failure, the server may now be in a state reflecting that it has just completed a service for the client. As a consequence, if the sending of its response fails, the server has to be prepared for the client to reissue its previous request.

Other types of omission failures not related to communication may be caused by software errors such as infinite loops or improper memory management, by which the server is said to "hang."

Another class of failures is related to timing. **Timing failures** occur when the response lies outside a specified real-time interval. For example, in the case of streaming videos, providing data too soon may easily cause trouble for a recipient if there is not enough buffer space to hold all the incoming data.

More common, however, is that a server responds too late, in which case a **performance** failure is said to occur.

A serious type of failure is a **response failure**, by which the server's response is simply incorrect. Two kinds of response failures may happen. In the case of a value failure, a server simply provides the wrong reply to a request. For example, a search engine that systematically returns Web pages not related to any of the search terms used, has failed.

The other type of response failure is known as a state-transition failure. This kind of failure happens when the server reacts unexpectedly to an incoming request. For example, if a server receives a message it cannot recognize, a state-transition failure happens if no measures have been taken to handle such messages. In particular, a faulty server may incorrectly take default actions it should never have initiated.

The most serious are **arbitrary failures**, also known as **Byzantine failures**. In effect, when arbitrary failures occur, clients should be prepared for the worst. In particular, a server may be producing output it should never have produced, but which cannot be detected as being incorrect. Byzantine failures were first analyzed by Pease et al. [1980] and Lamport et al. [1982]. We return to such failures below.

Note 8.2 (More information: Omission and commission failures)
It has become somewhat of a habit to associate the occurrence of Byzantine failures with maliciously operating processes. The term "Byzantine" refers to the Byzantine Empire, a time (330–1453) and place (the Balkans and modern Turkey) in which endless conspiracies, intrigue, and untruthfulness were alleged to be common in ruling circles.

However, it may not be possible to detect whether an act was actually benign or malicious. Is a networked computer running a poorly engineered operating system that adversely affects the performance of other computers acting maliciously? In this sense, it is better to make the following distinction, which effectively excludes judgment:

- An **omission failure** occurs when a component fails to take an action that it should have taken.
- A **commission failure** occurs when a component takes an action that it should not have taken.

This difference, introduced by Mohan et al. [1983], also illustrates that a separation between dependability and security may at times be pretty difficult to make.

Many of the aforementioned cases deal with the situation that a process P no longer perceives any actions from another process Q. However, can P conclude that Q has indeed come to a halt? To answer this question, we need to make a distinction between two types of distributed systems:

- In an **asynchronous system**, no assumptions about process execution speeds or message delivery times are made. The consequence is that when process P no longer perceives any actions from Q, it cannot conclude that Q crashed. Instead, it may just be slow, or its messages may have been lost.
- In a **synchronous system**, process execution speeds and message-delivery times are bounded. This also means that when Q shows no more activity when it is expected to do so, process P can rightfully conclude that Q has crashed.

Unfortunately, pure synchronous systems exist only in theory. On the other hand, simply stating that every distributed system is asynchronous also does not do justice to what we see in practice, and we would be overly pessimistic in designing distributed systems under the assumption that they are necessarily asynchronous. Instead, it is more realistic to assume that a distributed system is **partially synchronous**: most of the time it behaves as a synchronous system, yet there is no bound on the time that it behaves in an asynchronous fashion. In other words, asynchronous behavior is an exception, meaning that we can normally use timeouts to conclude that a process has indeed crashed, but that occasionally such a conclusion is false. In practice, this means that we will have to design fault-tolerant solutions that can withstand incorrectly detecting that a process halted.

In this context, halting failures can be classified as follows, from the least to the most severe (see also Cachin et al. [2011]). We let process P attempt to detect that process Q has failed.

- **Fail-stop failures** refer to crash failures that can be reliably detected. This may occur when assuming nonfaulty communication links and when the failure-detecting process P can place a worst-case delay on responses from Q.
- **Fail-noisy failures** are like fail-stop failures, except that P will only *eventually* come to the correct conclusion that Q has crashed. This means that there may be some a priori unknown time in which P's detections of the behavior of Q are unreliable.
- When dealing with **fail-silent failures**, we assume that communication links are nonfaulty, but that process P cannot distinguish crash failures from omission failures.
- **Fail-safe failures** cover the case of dealing with arbitrary failures by process Q, yet these failures are benign: they cannot do any harm.
- Finally, when dealing with **fail-arbitrary failures**, Q may fail in any possible way; failures may be unobservable in addition to being harmful to the otherwise correct behavior of other processes.

Clearly, having to deal with fail-arbitrary failures is the worst that can happen. As we shall discuss shortly, we can design distributed systems in such a way that they can even tolerate these types of failures.

8.1.3 Failure masking by redundancy

If a system is to be fault tolerant, the best it can do is to try to hide the occurrence of failures from other processes. The key technique for masking faults is to use redundancy. Three kinds are possible: information redundancy, time redundancy, and physical redundancy (see also Johnson [1995]). With **information redundancy**, extra bits are added to allow recovery from garbled bits. For example, a Hamming code can be added to transmitted data to recover from noise on the transmission line.

With **time redundancy**, an action is performed, and then, if need be, it is performed again. Transactions use this approach. If a transaction aborts, it can be redone with no harm because nothing has yet been finalized. Another well-known example is retransmitting a request to a server when lacking an expected response. Time redundancy is especially helpful when the faults are transient or intermittent.

With **physical redundancy**, extra equipment or processes are added to make it possible for the system as a whole to tolerate the loss or malfunctioning of some components. Physical redundancy can thus be done either in hardware or in software. For example, extra processes can be added to the system, so that if a few of them crash, the system can still function correctly. In other words, by replicating processes, a high degree of fault tolerance may be achieved. We return to this type of software redundancy later in this chapter.

Note 8.3 (More information: Triple modular redundancy)
It is illustrative to see how redundancy has been applied in the design of electronic devices. Consider, for example, the circuit of Figure 8.3(a). Here signals pass through devices A, B, and C, in sequence. If one of them is faulty, the final result will probably be incorrect.

In Figure 8.3(b), each device is replicated three times. Following each stage in the circuit is a triplicated voter. Each voter is a circuit that has three inputs and one output. If two or three of the inputs are the same, the output is equal to that input. If all three inputs are different, the output is undefined. This kind of design is known as **triple modular redundancy** (**TMR**).

Suppose that element A2 fails. Each of the voters, V1, V2, and V3 gets two good (identical) inputs and one rogue input, and each of them outputs the correct value to the second stage. In essence, the effect of A2 failing is completely masked so that the inputs to B1, B2, and B3 are the same as they would have been had no fault occurred.

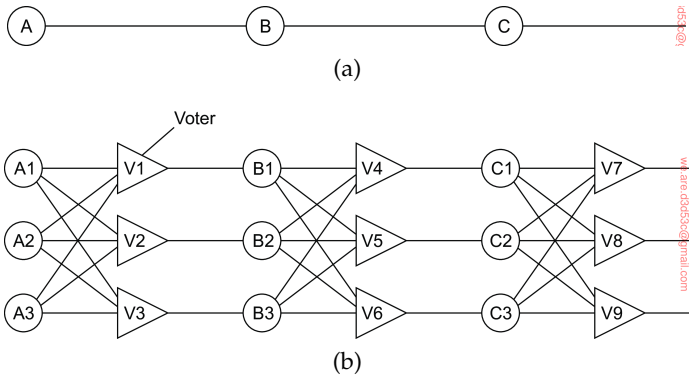


Figure 8.3: Triple modular redundancy.

Now consider what happens if B3 and C1 are also faulty, in addition to A2. These effects are also masked, so the three final outputs are still correct.

At first, it may not be obvious why three voters are needed at each stage. After all, one voter could also detect and pass the majority view. However, a voter is also a component and can be faulty. Suppose, for example, that voter V1 malfunctions. The input to B1 will then be wrong, but as long as everything else works, B2 and B3 will produce the same output and V4, V5, and V6 will all produce the correct result into stage three. A fault in V1 is effectively no different from a fault in B1. In both cases B1 produces incorrect output, but in both cases, it is voted down later, and the final result is still correct.

Although not all fault-tolerant distributed systems use TMR, the technique is very general, and should give a clear feeling for what a fault-tolerant system is, as opposed to a system whose individual components are highly reliable but whose organization cannot tolerate faults (i.e., operate correctly even in the presence of faulty components). Of course, TMR can be applied recursively, for example, to make a chip highly reliable by using TMR inside it, unknown to the designers who use the chip, possibly in their own circuit containing multiple copies of the chips along with voters.

8.2 Process resilience

Now that the basic issues of fault tolerance have been discussed, let us concentrate on how fault tolerance can actually be achieved in distributed systems. The first topic we discuss is protection against process failures, which is achieved by replicating processes into groups. In the following pages, we consider the general design issues of process groups and discuss what a fault-tolerant group actually is. Also, we look at how to reach consensus within a process group when one or more of its members cannot be trusted to give correct answers.

8.2.1 Resilience by process groups

The key approach to tolerating a faulty process is to organize several identical processes into a **group**. The key property that all groups have is that when a message is sent to the group itself, all members of the group receive it. In this way, if one process in a group fails, hopefully some other process can take over for it [Guerraoui and Schiper, 1997].

Process groups may be dynamic. New groups can be created, and old groups can be destroyed. A process can join a group or leave one during system operation. A process can be a member of several groups at the same time. Consequently, mechanisms are required for managing groups and group membership.

The purpose of introducing groups is to allow a process to deal with collections of other processes as a single abstraction. Thus, a process P can send a message to a group $\mathbf{Q} = \{Q_1, \ldots, Q_N\}$ of servers without having to know who they are, how many there are, or where they are, which may change from one call to the next. To P, the group **Q** appears to be a single, logical process.

Group organization

An important distinction between different groups has to do with their internal structure. In some groups, all processes are equal, i.e., we have a **flat group**. There is no distinctive leader and all decisions are made collectively. Typically, many peer-to-peer systems operate in this way. An alternative organization is that of a **hierarchical group**. For example, one process is the coordinator and all the others are workers. In this model, when a request for work is generated, either by an external client or by one of the workers, it is sent to the coordinator. The coordinator then decides which worker is best suited to carry it out, and forwards it there. More complex hierarchies are also possible, of course. The Domain Name System can be argued to operate as a (indeed, quite complex) hierarchical group. Simpler organizations include primary-based backup schemes, as we discussed in Section 7.5.1. These communication patterns within groups are illustrated in Figure 8.4.

Each of these organizations has its own advantages and disadvantages. The flat group is symmetrical and has no single point of failure. If one of the processes crashes, the group simply becomes smaller, but can otherwise continue. A disadvantage is that decision-making may be more complicated. For example, to decide anything, a vote often has to be taken, incurring some delay and overhead.

The hierarchical group has the opposite properties. Loss of the coordinator brings the entire group to a grinding halt, but as long as it is running, it can make decisions without bothering everyone else. In practice, when the coordinator in a hierarchical group fails, its role will need to be taken over and

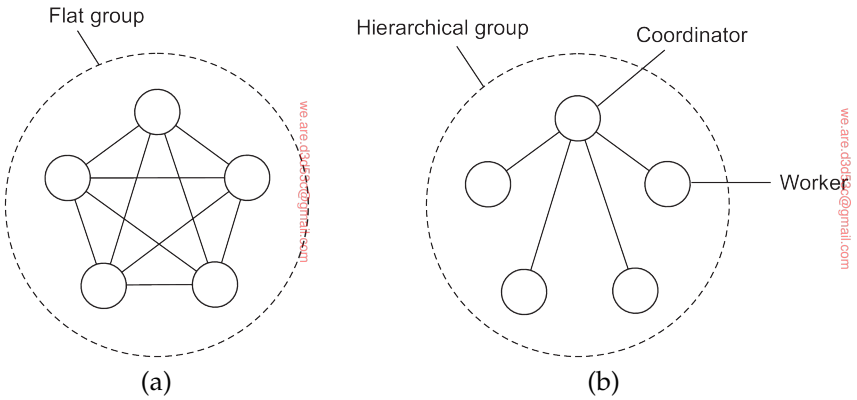


Figure 8.4: Communication in a (a) flat group and in a (b) hierarchical group.

one of the workers is elected as new coordinator. We discussed leader-election algorithms in Section 5.4.

Membership management

When group communication is present, some method is required for creating and deleting groups, as well as for allowing processes to join and leave groups. One possible approach is to have a **group server** to which all these requests can be sent. The group server can then maintain a complete database of all the groups and their exact membership. This method is straightforward, efficient, and fairly easy to implement. Unfortunately, it shares a major disadvantage with many (physically) centralized solutions: a single point of failure. If the group server crashes, group management ceases to exist. Probably most or all groups will have to be reconstructed from scratch, possibly terminating whatever work was going on.

The opposite approach is to manage group membership in a distributed way. For example, if (reliable) multicasting is available, an outsider can send a message to all group members announcing its wish to join the group.

Ideally, to leave a group, a member just sends a goodbye message to everyone. In the context of fault tolerance, assuming fail-stop failure semantics, is generally not appropriate. The trouble is, there is no polite announcement that a process crashes, as there is when a process leaves voluntarily. The other members have to discover this experimentally by noticing that the crashed member no longer responds to anything. Once it is certain that the crashed member is really down (and not just slow), it can be removed from the group.

Another knotty issue is that leaving and joining have to be synchronous with data messages being sent. In other words, starting at the instant that a process has joined a group, it must receive all messages sent to that group.

Similarly, as soon as a process has left a group, it must not receive any more messages from the group, and the other members must not receive any more messages from it. One way of making sure that a join or leave is integrated into the message stream at the right place is to convert this operation into a sequence of messages sent to the whole group.

One final issue relating to group membership is what to do if so many processes go down that the group can no longer function at all. Some protocol is needed to rebuild the group. Invariably, some process will have to take the initiative to start the ball rolling, but what happens if two or three try at the same time? The protocol must be able to withstand this. Again, coordination through, for example, a leader-election algorithm may be needed.

8.2.2 Failure masking and replication

Process groups are part of the solution for building fault-tolerant systems. In particular, having a group of identical processes allows us to mask one or more faulty processes in that group. In other words, we can replicate processes and organize them into a group to replace a single (vulnerable) process with a (fault tolerant) group. As discussed in the previous chapter, there are two ways to approach such replication: by means of primary-based protocols, or through replicated-write protocols.

Primary-based replication in the case of fault tolerance generally appears in the form of a **primary-backup protocol**. In this case, a group of processes is organized in a hierarchical fashion, in which a primary coordinates all write operations. In practice, the primary is fixed, although its role can be taken over by one of the backups if need be. In effect, when the primary crashes, the backups execute some election algorithm to choose a new primary.

Replicated-write protocols are used in the form of **active replication**, as well as by **quorum-based protocols**. These solutions correspond to organizing a collection of identical processes into a flat group. The main advantage is that such groups have no single point of failure, at the cost of distributed coordination.

An important issue with using process groups to tolerate faults is how much replication is needed. To simplify our discussion, let us consider only replicated-write systems. A system is said to be **k-fault tolerant** if it can survive faults in k components and still meet its specifications. If the components, say processes, fail silently, then having $k+1$ of them is enough to provide k-fault tolerance. If k of them simply stop, then the answer from the other one can be used.

On the other hand, if processes exhibit arbitrary failures, continuing to run when faulty and sending out erroneous or random replies, a minimum of $2k+1$ processes is needed to achieve k-fault tolerance. In the worst case, the k failing processes could accidentally (or even intentionally) generate the same